Recursive Coherence Preservation / RCP-II: A Domain-Relative Robust Reflective Successor Theorem for Certified RCP Seed-Library Classes

Batch 13R Reference-Entry, Concrete Finite Witnesses, Controlled Executable Artifacts, and External Mechanization Certificates

Nicolas Calder
n926mp@gmail.com

June 29, 2026

Abstract

Recursive self-improvement (RSI) can be described as a sequence of self-modification channels acting on a system that models language, world-reference, human-reference, uncertainty, decisions, and its own update process. The central instability in such a loop is not merely that capability may fail to increase; it is that update-relevant distinctions may be compressed, erased, or collapsed during self-update. This paper develops a formal framework called Recursive Coherence Preservation (RCP). The framework represents the system by a density state, treats recursive self-improvement as a channel or typed self-update transition, defines lossiness as decay of protected relative entropy and constructive recoverability, defines recursive incoherence by a Lyapunov functional, and constrains self-updates by barrier certificates, evidence-paid ambiguity reduction, and information-bottleneck preservation of future-update-relevant behavior.

The thesis of this third-version reframing is that the primary mathematical object is *recoverable monotone self-improvement*: recursive improvement that preserves the predecessor's protected distinctions, supplies a recovery or rollback relation, and does not decrease the declared progress structure. Safety is then a corollary when the protected predicate family includes corrigibility, grounding, oversight, reversibility, uncertainty honesty, verifier integrity, and related safety predicates. Accordingly, the former main stability theorem is renamed the *Conditional Non-Lossy Self-Update Preservation Theorem*. It proves a preservation statement: if every accepted update passes the RCP gate, then the certified non-loss, Lyapunov, barrier, ambiguity, self-model, semantic, verifier, resource, coverage, calibration, and estimator obligations remain bounded or invariant over the declared horizon.

This integrated v2 operational-closure version adds rigor-audited Modules A–G: constructive recovery, capability–coherence calibration, safe-improvement existence, recursive feasibility and viability kernels, semantic register validity, self-model fidelity, verifier bootstrapping, physical/computational resource feasibility, asymptotic coherence regimes, and an operational-closure layer abstracted from the RSI–RCLM architecture: typed density realization, representation-to-density embedding, semantic coupling consistency, non-oracular certificates, verifier trust graphs, budgeted certified search, resource-scaling/no-op pressure, coverage-gap residuals, coherence-capability calibration, and estimator subprotocols. Batch 2 adds ability-set progress and structural recovery: certified ability sets $\mathfrak{A}_t(\rho, R)$, non-lossy ability preservation by cross-time inclusion, strict RSI progress by proper ability-set expansion, exact structural recovery through reversible/isometric meta-state dynamics, and conservative extension updates of the form $\Phi_\nu(\rho) = V\rho V^\dagger \otimes \alpha_\nu$. Batch 3 adds a conservative-extension algebra: a certified

non-lossy update algebra $\mathfrak{G}^{\mathrm{NL}} = \langle \mathrm{id}, \mathrm{IsoExt}, \mathrm{AppendMem}, \mathrm{RevAdapter}, \mathrm{VerifierExt}, \mathrm{ToolExt} \rangle$, closure under composition, transported protected-pair sets, composed recovery maps, and additive loss/recovery budgets $\epsilon_{\mathrm{total}} = \sum_i \epsilon_i$, $r_{\mathrm{total}} = \sum_i r_i$. Batch 4 adds a tangent-cone local progress theorem: local RCP residuals $q_{t,j}(\nu; \rho_t) \leq 0$, the admissible tangent cone $T_{\mathcal{A}_t}(\rho_t)$, an engine-potential functional RSIPot_t, and a sufficient condition under which a projected admissible gradient direction yields a finite non-lossy update with positive local progress. Batch 5 adds engine-viability nonemptiness for conservative-extension systems: finite-horizon and infinite-horizon engine kernels $\mathcal{V}_t^{\mathrm{engine}}$, the implication $\rho_t \in \mathcal{V}_t^{\mathrm{engine}} \Rightarrow \exists \nu_t \in \Omega_t^{\mathrm{NL}}$ with $\rho_{t+1} \in \mathcal{V}_{t+1}^{\mathrm{engine}}$, and a certified-plan theorem showing when those kernels are nonempty. Batch 9 adds the constructive direct-engine layer: an engine constructor $\mathfrak{E}_t$ with generator, certifier, selector, and realization components, together with sufficient strict-witness, generator-coverage, and certificate-soundness assumptions under which $\mathfrak{E}_t(\rho_t)$ constructs a certified beneficial conservative-extension word rather than merely verifying an externally proposed one. Batch 10 synchronizes the final architecture cleanup back into the abstract math paper: typed self-update transitions are distinguished from channel-licensed updates, adaptive-audit trust residuals $\mathrm{CorrFail}_t, \mathrm{AuditLeak}_t, \mathrm{CommonMode}_t, \mathrm{CollusionRisk}_t$ are added to a strengthened adversarial trust gate, and an abstract complexity/scalability ledger is added as an accounting obligation rather than a tractability theorem. Batch 11 adds domain-nonemptiness and witness-library closure: certified conservative-extension module libraries $\mathcal{L}_t^{\mathrm{CE}}$, bounded primitive-word grammars $\mathfrak{G}_{t,\leq k}^{\mathrm{NL}}$, seed engine domains $\mathcal{K}_t^{\mathrm{seed}}$, direct-engine domain-entry theorems, successor seed-domain invariance, seed-equivalence/persistence-transfer certificates for engine-selected candidates, restricted modular certificate-radius scalability, and open-ended ability towers relative to renewable certified libraries. Batch 7 adds publication-rigor cleanup: a theorem-type table separating algebraic/structural results, telescoping and induction results, certificate-composition results, and assumption-dependent existence results, together with falsification/failure conditions for estimator calibration failure, pair-set coverage failure, semantic barrier unsoundness, recovery-map failure, verifier common-mode failure, resource-scaling failure, and related implementation failures. Batch 8 adds cross-time/state/update/projection cleanup: explicit transports $\Theta_t^Q, \Theta_t^Z, \Theta_t^Y, \Theta_t^G$ for ambiguity, bottleneck, and update-description comparisons, a separation of state-only safe sets $\mathcal{K}_{\mathrm{RCP}}^{\mathrm{state}}(t)$ from update-admissibility relations $\mathcal{A}_{\mathrm{RCP}}(t, \rho, \Phi)$, a split between V_{state} and V_{upd}, and a projection-realization certificate $\mathsf{ProjReal}_t$ preventing mathematical safe-set projection from being treated as an implementable RSI self-update without an explicit typed realization. The result is not a universal proof of AGI safety, unlimited recursive self-improvement, or arbitrary-system engine existence. It is a conditional theorem-proof architecture for the preservation side of the Level 2-to-Level 3 bridge plus a conservative-extension engine-viability layer, a Batch 9 direct-engine construction theorem, and a Batch 11 seed-domain entry theorem. The Batch 9 theorem proves that, on a declared direct-engine domain where strict beneficial conservative-extension witnesses exist and the generator/certifier covers them within certified margins, an engine object constructs the update, recovery map, certificate packet, and successor state. Batch 11 proves that a concrete seed class enters that direct-engine domain when certified conservative-extension libraries, bounded grammar coverage, certificate margins, resource/trust ledgers, and successor seed-persistence certificates are supplied. It still does not prove that arbitrary systems lie in that seed class or supply such libraries. Batch 12 / RCP-II Phase 1 begins the successor-verification phase: it defines successor-verification schemas, uncertainty envelopes, semantic/goal-identity transport certificates, successor-verification seed domains $\mathcal{K}_t^{\mathrm{SV},N} \subseteq \mathcal{K}_t^{\mathrm{seed},N}$, and a robust successor-verification domain-invariance theorem proving that, on the stated certificate events, $\rho_t \in \mathcal{K}_t^{\mathrm{SV},N}$ implies the engine constructs $\rho_{t+1} \in \mathcal{K}_{t+1}^{\mathrm{SV},N}$. This is a restricted certificate-reflective theorem, not a universal solution to Löbian/Vingean reflection or arbitrary successor trust. Batch 12A completes the Phase-1 formalization by expanding the successor-verification packet into explicit residuals, verifier-schema transport dynamics, uncertainty-envelope construction and transport, successor-verification soundness composition, goal-identity composition, anti-circular reflective-trust residuals, an algorithmic successor-verification gate, theorem-type rows, and a viability-domain audit stating that Batch

12 is a domain-invariance theorem rather than a theorem constructing successor-verification packets from arbitrary states. Batch 12B adds the next construction and closure layer: a successor-verification packet builder, a successor-verification witness library and bounded certificate grammar, a finite seed-library nonemptiness theorem proving entry into $\mathcal{K}_t^{\mathrm{SV},N}$ for a declared class, an infinite successor-verification seed-library closure theorem proving that $\mathcal{K}_t^{\text{SV-seedlib},\infty}$ canonically supplies an infinite $\mathcal{K}_t^{\mathrm{SV},\infty}$ domain, an infinite-horizon domain-relative robust reflective successor theorem, reality-containment or misspecification certificates for uncertainty envelopes, and a restricted tractability certificate separating budgeted/resource-accounted verification from strong computational tractability. The RCP-II finalization layer surfaces the resulting claim as a domain-relative, certificate-relative theorem status rather than an unqualified universal RSI theorem: finite or infinite monotonic, goal-transport-preserving, repeatable self-improvement under uncertainty is obtained only for certified seed-library classes that supply the successor-verification builder, packet, witness library, reality-containment or misspecification certificate, and restricted tractability certificate. It also introduces a minimal abstract reference-instantiation target $\mathsf{RefSV}_{0:N}^{\mathrm{RCP}}$, an invocation boundary for concrete systems, a main-goal fit table, and an implementation roadmap, thereby marking the next frontier as reference instantiation and empirical validation rather than a new abstract Batch 13. Batch 13R is that reference-instantiation phase rather than a new abstract theorem batch: it defines finite RCP reference classes $\mathsf{RCPRefClass}_{n,k,N}$, proof-carrying successor packets PCS_t, a trusted checker $\mathsf{Check}_{\mathrm{RCP}}$, a certificate compiler $\mathsf{BuildRefSV}_{0:N}^{\mathrm{RCP}}$, a reference-entry theorem proving that a successfully built reference object enters $\mathcal{K}_0^{\text{SV-seedlib},N}$, a proof-carrying finite-trajectory theorem proving $\rho_t \in \mathcal{K}_t^{\mathrm{SV},N}$ along checked finite paths, a restricted reference-tractability theorem, a reference-contained uncertainty transfer theorem, and a non-toy executable-artifact theorem. These additions are mechanization-ready and proof-carrying; an actual machine-checked claim is licensed only when a separate mechanization certificate for the checker or proof kernel is supplied. The Batch 13R-A rigor-completion patch then constructs a canonical finite appended-module reference witness for every horizon $N \geq 2$, proves exact protected non-loss and recovery, strict ability expansion, zero goal drift, singleton reality containment, a concrete finite checker soundness theorem, a concrete compiler termination/reference-entry theorem, a concrete multi-step certified trajectory, a restricted cost bound, and a canonical replay-artifact theorem. Batch 13R-B adds the final pre-publication milestone layer: a narrow external mechanization certificate $\mathsf{MechCert}_{0:N}^{\mathrm{RCP,can}}$ for the canonical finite core, a controlled executable reference-system object with parsed state/update/packet/residual/cost tables, a replay certificate with hashes and checker outputs, and an integrated Milestone-1/2A theorem stating exactly when the canonical controlled artifact is a replayable finite restricted-Scenario-3 witness and when, additionally, it may be called externally machine-checked. This still does not mechanize or implement the entire RCP theorem stack; it machine-checks only the mapped canonical finite core when an external certificate is supplied. M3-Min then adds a minimal learned-agent entry boundary, not a broad learned-agent theorem: it defines controlled learned RCP-compatible systems, learned-entry certificate packets, a finite learned-entry audit procedure, a conditional learned-entry theorem, and a partial-pass pilot reporting protocol. The resulting theorem says that if a learned system supplies the full finite learned-entry certificate boundary, then its learned origin is no obstacle to invoking the existing Batch-13R theorem stack. The final M3 synchronization patch added here checks the architecture correspondence: a valid RCLM learned-entry full pass transfers through an explicit RCLM-to-RCP abstraction certificate to the RCP learned-entry theorem, while RCLM partial passes remain partial hypothesis-supply reports. It does not prove that arbitrary trained systems satisfy that boundary.

For the current companion artifact package, the narrow external mechanization certificate is now instantiated by a shared Lean 4 proof project. In the claim-discipline language of this paper, the supplied Lean 4 certificate covers the canonical finite RCP/RCLM witness, the finite checker core, and the RCLM-to-RCP refinement module. It does not mechanize the full 219-page math stack, the full 187-page architecture stack, arbitrary learned-system entry, broad capable-agent

RSI, or empirical deployment claims.

Keywords: recursive self-improvement, recursive self-modeling, recoverable monotone self-improvement, certified ability sets, structural recovery, conservative extension algebra, non-lossy update algebra, engine viability kernel, non-lossy RSI, Lyapunov stability, barrier certificates, quantum information, density matrices, relative entropy, coherence, information bottleneck, corrigibility, domain nonemptiness, witness-library closure, seed engine domain, finite primitive-word grammar, successor-verification residuals, verifier-schema transport, uncertainty-envelope transport, goal-identity composition, reflective-trust residuals, domain-relative robust reflective successor theorem, reference instantiation, theorem-invocation boundary, Batch 13R, proof-carrying successor packet, trusted RCP checker, BuildRefSV compiler, finite certified trajectory, reference-entry theorem, mechanization-ready artifact, executable reference instance, external mechanization certificate, Lean 4 certificate, controlled executable reference system, replay certificate, canonical controlled artifact, M3-Min, learned-agent entry boundary, controlled learned-system audit, learned-entry certificate, partial-pass protocol, RCLM-to-RCP learned-entry synchronization, learned-entry refinement certificate.

Companion manuscript and artifact status. This manuscript is Paper I of a two-paper package. Paper I gives the architecture-general RCP/RCP-II mathematical theorem stack; the companion Paper II instantiates the theorem stack in the RSI–RCLM architecture. The shared artifact repository contains the Lean 4 proof project, controlled RCP/RCLM reference artifacts, replay checkers, run logs, theorem map, and mechanization manifest. A Lean 4 certificate is supplied for the canonical finite RCP/RCLM witness and refinement core, including the RCP canonical core and the RCLM-to-RCP refinement module. The certificate does not mechanize the full 219-page math stack, the full 187-page architecture stack, arbitrary learned-system entry, broad capable-agent RSI, or empirical deployment claims.

Contents

1 Introduction

The framework developed here begins from the following hierarchy:

$$\begin{gathered}\text{language} \\ > \text{ objectivity and subjectivity} \\ >> \text{ definitiveness and ambiguity} \\ >>> \text{ coherence and entropy.}\end{gathered}$$

The goal is to express this hierarchy as a mathematical stability architecture for recursive self-modeling (RSM) and recursive self-improvement (RSI). In this architecture, language is not treated merely as a surface string process. Language is the observable medium through which a system binds world-reference, human-reference, uncertainty, decision, and self-reference. The safety problem appears when the system begins to recursively update not only its outputs, but also its own model of itself and its own update mechanisms.

The basic thesis is:

> Safe recursive self-improvement is not entropy minimization alone. It is entropy reduction constrained by recoverability, external grounding, ambiguity preservation, barrier invariance, and self-model relevance preservation.

A self-improving system may become locally more coherent while globally less safe. For example, it may simplify its goals, erase distinctions between corrigible and incorrigible behavior, collapse uncertainty into false certainty, preserve internal consistency while losing external grounding, or compress its self-model in a way that hides future failure modes. These are not merely engineering bugs. They are mathematical lossiness modes.

The present paper turns the earlier RCP formulation into a theorem-proof manuscript. The guiding pattern is:

$$\text{Definition} \to \text{Assumptions} \to \text{Lemma} \to \text{Theorem} \to \text{Proof} \to \text{Interpretation} \to \text{Limitation.}$$

The resulting document should be read as a formal architecture, not as an empirical claim that present machine-learning systems already satisfy the assumptions.

1.1 Contributions

The main contributions are:

(i) A density-state representation of the RCLM hierarchy, with separate registers for language, objective reference, subjective reference, definitiveness, ambiguity, and self-modeling.

(ii) A channel formulation of RSI in which each self-improvement step is a completely positive trace-preserving (CPTP) or admissible approximate channel.

(iii) A relative-entropy definition of lossy self-improvement, together with a telescoping theorem showing bounded cumulative loss of safety-relevant distinguishability.

(iv) A recursive coherence functional and a Lyapunov incoherence functional, with a drift theorem proving bounded instability under summable error.

(v) A supporting barrier-certificate submodule importing five useful mechanisms: no-op feasibility, bounded performance and optimization error, soft barrier finite-loss bounds, hard barrier forward invariance, and projection onto the recursive coherence safe set.

(vi) A formal ambiguity-collapse constraint showing that definitiveness is safe only when entropy reduction is paid for by evidence, proof, validation, or grounded clarification.

(vii) An information-bottleneck formulation of self-model compression that preserves information about safety-relevant future behavior.

(viii) A renamed Conditional Non-Lossy Self-Update Preservation Theorem combining the preceding modules into a conditional preservation result for recoverable monotone self-improvement.

(ix) A thesis-level reframing in which safety is treated as a corollary of non-loss applied to safety-relevant predicates, rather than as the primitive mathematical object of RSI.

(x) A Direct Non-Lossy RSI Engine Theorem target object, explicitly marked as the next theorem to be built rather than as a theorem already proved by the present preservation stack.

(xi) A certified ability-set progress layer defining $\mathfrak{A}_t(\rho, R)$, non-lossy ability preservation by cross-time inclusion, and strict RSI progress by proper certified ability-set expansion.

(xii) A structural recovery layer proving exact predecessor recovery and exact protected relative-entropy preservation for reversible/isometric meta-state updates and conservative tensor extensions.

(xiii) A conservative-extension algebra $\mathfrak{G}^{\mathrm{NL}}$ generated by identity, isometric extension, append-only memory, reversible adapters, verifier extensions, and tool extensions, with closure under composition and additive loss/recovery budgets.

(xiv) A tangent-cone local progress theorem showing that, under differentiability, curvature, strict-inwardness, and local non-lossy chart assumptions, a projected engine-potential gradient direction yields a finite admissible update with positive local progress.

(xv) An engine-viability nonemptiness theorem for conservative-extension systems, defining finite-horizon and infinite-horizon engine viability kernels and proving that membership in the kernel yields a certified non-lossy successor update inside Ω_t^{NL}.

(xvi) A publication-rigor cleanup layer that classifies theorem types, separates proved consequences from certificate-composition and assumption-dependent existence claims, and states explicit falsification/failure conditions for failed calibration, failed pair-set coverage, semantic barrier unsoundness, recovery-map failure, verifier common-mode failure, resource-scaling failure, and related implementation failures.

(xvii) A Batch 8 cleanup layer introducing cross-time transports for Q_t, Z_t, Y_t, G_t, separating state-only constraints from update-dependent admissibility constraints, splitting V_{state} from V_{upd}, and requiring projection-realization certificates before projected safe states can be counted as accepted RSI updates.

(xviii) A Batch 9 direct-engine construction layer defining an engine constructor $\mathfrak{E}_t$ and proving, under strict beneficial-witness, generator-coverage, certificate-soundness, and margin assumptions, that it constructs a certified beneficial conservative-extension word with non-loss, recovery, ability expansion, resource feasibility, and successor engine viability.

(xix) A Batch 10 adaptive-audit and complexity synchronization layer defining correlated verifier failure, audit leakage, common-mode support risk, collusion risk, the strengthened gate $\mathcal{A}_{\mathrm{RCP}}^{\text{trust-adv}}$, an abstract complexity ledger, and a channel-license convention for typed self-update transitions.

(xx) A Batch 11 domain-nonemptiness and witness-library closure layer defining certified conservative-extension module libraries, bounded non-lossy primitive-word grammars, seed engine domains, seed-domain inclusion in engine viability kernels, direct-engine domain entry from seed states, successor seed-domain invariance, seed-equivalence/persistence-transfer certificates for engine-selected candidates, restricted modular certificate-radius scalability, and open-ended ability towers relative to renewable libraries.

(xxi) A Batch 12 / RCP-II Phase 1 successor-verification layer defining successor-verification schemas, uncertainty envelopes, semantic/goal-identity transport certificates, successor-verification progress functionals, successor-verification certificate packets, successor-verification seed domains $\mathcal{K}_t^{\mathrm{SV},N}$, robust successor-verification domain invariance, finite successor-verification trajectories, and summable drift/failure budgets for certified paths.

(xxii) A Batch 12A rigor-completion layer decomposing Q_t^{SV} into explicit residuals, defining verifier-schema component transports, deriving a union-bound successor-verification soundness theorem, constructing and transporting uncertainty envelopes, proving finite-horizon goal-identity composition, decomposing anti-circular reflective-trust obligations, adding an algorithmic successor-verification gate, and recording that the Phase-1 theorem is a viability-domain theorem rather than a packet-construction theorem.

(xxiii) A Batch 12B construction and closure layer defining $\mathsf{SVBuilder}_t$, $\mathcal{D}_t^{\text{SV-construct}}$, $\mathcal{L}_t^{\text{SV-Wit}}$, $\mathcal{K}_t^{\text{SV-seedlib},N}$, $\mathcal{K}_t^{\text{SV-seedlib},\infty}$, reality-containment certificates, restricted tractability certificates, an infinite seed-library closure theorem, and a domain-relative infinite-horizon Robust Reflective Successor Theorem for declared certified paths.

(xxiv) An RCP-II finalization and instantiation-readiness layer surfacing the domain-relative theorem status, restating the finite and infinite main theorem in a reader-facing form, defining a minimal abstract reference instance $\mathsf{RefSV}_{0:N}^{\mathrm{RCP}}$, stating the theorem-invocation boundary for concrete systems, adding a main-goal fit and qualification table, and converting remaining limitations into an implementation and empirical-validation roadmap.

(xxv) A Batch 13R reference-instantiation layer defining finite reference classes $\mathsf{RCPRefClass}_{n,k,N}$, proof-carrying successor packets PCS_t, a trusted checker $\mathsf{Check}_{\mathrm{RCP}}$, a certificate compiler $\mathsf{BuildRefSV}_{0:N}^{\mathrm{RCP}}$, a reference-entry theorem $\mathsf{BuildRefSV}_{0:N}^{\mathrm{RCP}}(\mathcal{M},\mathcal{D},\mathcal{L},\mathcal{C}) \Downarrow \mathsf{RefSV}_{0:N}^{\mathrm{RCP}} \Rightarrow \rho_0 \in \mathcal{K}_0^{\text{SV-seedlib},N}$, proof-carrying finite certified trajectories, restricted reference tractability, reference-contained uncertainty transfer, and a non-toy executable-artifact theorem.

(xxvi) A Batch 13R-B milestone layer defining a narrow external mechanization target $\mathsf{MechTarget}_{0:N}^{\mathrm{RCP,can}}$, a valid external mechanization certificate $\mathsf{MechCert}_{0:N}^{\mathrm{RCP,can}}$, a controlled executable reference-system object $\mathsf{CtrlRef}_{0:N}^{\mathrm{RCP}}$, a controlled replay certificate, and an integrated theorem showing that a replayed canonical controlled artifact proves the finite restricted-Scenario-3 reference conclusions, with external machine-checked status only when the external mechanization certificate is actually supplied.

(xxvii) An M3-Min learned-agent entry boundary layer defining controlled learned RCP-compatible systems, learned-entry certificate packets $\mathsf{LECert}_{0:N}^{\mathrm{RCP}}$, a learned-entry audit procedure

$\mathsf{LearnedEntryAudit}^{\mathrm{RCP}}_{0:N}$, a conditional theorem reducing learned-system entry to seed-library membership, and a partial-pass reporting protocol for controlled learned-system pilots. This is a sufficient certificate boundary, not a proof that arbitrary trained systems pass it.

(xxviii) Rigor-audited Modules A–C for constructive recovery, capability–coherence calibration, safe-improvement existence, recursive feasibility, and progress viability kernels.

(xxix) Rigor-audited Modules D–G for semantic register validity, self-model fidelity, verifier bootstrapping, resource realism, and asymptotic coherence regimes.

(xxx) An operational-closure layer, fed back from the RSI–RCLM architecture paper, that makes the RCP gates typed, auditable, non-oracular, verifier-trust-aware, resource-scaled, coverage-aware, capability-calibrated, and estimator-subprotocol-complete without changing the core RCP equations.

1.2 What is not claimed

This paper does not claim that coherence preservation is identical to intelligence, capability, truth, or alignment. It also does not claim that arbitrary recursive self-improvement is recoverably monotone, safe, or indefinitely capability-improving. The theorem statements are conditional on the explicit mathematical assumptions. In its current proved form, the framework is best understood as a preservation engine plus a conservative-extension progress substrate: a set of formal gates that proposed self-improvements must pass, sufficient recoverable extension updates that can preserve predecessor abilities, a certified non-lossy update algebra whose finite compositions preserve non-loss with additive budgets, a local tangent-cone theorem giving sufficient conditions for finite positive progress steps inside that algebraic candidate chart, and a Batch 5 viability-kernel theorem showing how such steps can continue recursively when a conservative-extension engine viability kernel is nonempty. The stronger constructive claim that an architecture-specific engine automatically generates those certified plans from arbitrary states is not assumed. Batch 9 proves only the domain-restricted version: if a state lies in the declared direct-engine domain with strict beneficial witnesses and a sound covering generator/certifier, then the engine constructs a certified beneficial conservative-extension word. Batch 11 adds a bounded seed-domain entry theorem: if a state lies in a certified seed domain with a nonempty conservative-extension module library, finite grammar coverage, certificate margins, resource/trust ledgers, and successor seed-persistence evidence, then it lies in the direct-engine domain for the declared horizon. Batch 12 adds a successor-verification seed-domain theorem: if a state lies in a certified successor-verification strengthening of that seed domain, then the engine constructs a successor that remains in the successor-verification seed domain under the declared uncertainty, goal-identity, trust, and verifier-schema persistence certificates. Batch 12A makes the successor-verification layer more explicit. Batch 12B constructs successor-verification packets and proves finite and infinite seed-library closure only for declared witness-library classes; it still does not prove arbitrary-state packet construction, automatic reality containment, scalable proof search for unbounded classes, nonemptiness of the infinite seed-library domain for arbitrary systems, or full Löbian/Vingean reflection. The RCP-II finalization layer does not add a stronger universal theorem; it clarifies the exact theorem status and the exact invocation boundary. A concrete implementation may cite the domain-relative theorem only after supplying the relevant seed-library membership, successor-verification packet, builder trace, goal-identity transport, anti-circular trust evidence, reality-containment or misspecification certificate when real-world reliability is claimed, and restricted tractability certificate when computational tractability is claimed. Batch 13R does

not change this boundary into an arbitrary-system result. It proves reference-entry only when a finite reference-class object, proof-carrying packets, trusted-checker soundness, compiler trace, witness library, and optional reality/tractability certificates are supplied. The current companion artifact package supplies a controlled executable artifact and a Lean 4 mechanization certificate for the canonical finite RCP/RCLM witness and refinement core. This upgrades the Batch 13R layer, for that narrow canonical core, from mechanization-ready to externally machine-checked relative to the listed Lean 4 source, build log, theorem map, and hashes. Batch 13R-B remains the formal invocation boundary for those stronger claims: a controlled executable artifact plus replay certificate licenses executable finite reference entry, while a valid $\mathsf{MechCert}_{0:N}^{\mathrm{RCP,can}}$ is required before any canonical core is called externally machine-checked. M3-Min supplies a finite learned-entry audit boundary for controlled learned systems, but does not prove that arbitrary trained systems, frontier-scale neural systems, or broad capable agents pass that boundary. Batch 7 also makes the framework deliberately falsifiable at the level of implementation claims: if estimator calibration, pair-set coverage, semantic barriers, recovery maps, verifier independence, type licensing, ability certificates, engine kernels, or resource ledgers fail their stated assumptions, then the corresponding theorem conclusion is withdrawn rather than silently retained. Batch 8 additionally states that cross-time information comparisons require declared transport maps, that $\mathcal{K}_{\mathrm{RCP}}^{\mathrm{state}}(t)$ is a state-only set while $\mathcal{A}_{\mathrm{RCP}}(t,\rho,\Phi)$ is an update-admissibility relation, and that projection into a safe set is a mathematical repair unless $\mathsf{ProjReal}_t$ certifies an implementable typed self-update channel or transition. Batch 10 further states that verifier trust claims require bounded adaptive-audit risk under the declared model, that complexity ledgers license cost accounting rather than scalability, and that Φ is a typed transition by default and a channel only when the relevant channel license is supplied.

2 Theorem Type Table and Claim Discipline

This manuscript deliberately contains several kinds of mathematical statements. Some are definitions or modeling conventions. Some are algebraic facts about density states, isometries, tensor extensions, or finite words in the conservative-extension algebra. Some are telescoping or induction results. Some are certificate-composition results whose force depends on audited one-sided residual soundness. Some are assumption-dependent existence or viability results. The distinction matters: a theorem can be correct while still not proving that the required certificates, semantic maps, pair sets, useful update candidates, or engine kernels exist in a concrete implementation.

Definition 2.1 (Theorem type)**.** *A theorem, proposition, lemma, corollary, or major claim in this manuscript is assigned one of the following logical types when it is used in an implementation claim:*

$$\mathsf{Type}(T) \in \{\mathrm{Def}, \mathrm{Alg}, \mathrm{Tel}, \mathrm{Cert}, \mathrm{Exist}, \mathrm{Impl}\}.$$

The meanings are:

(a) Def*: a definition, convention, or modeling choice;*

(b) Alg*: an algebraic, order-theoretic, or finite-dimensional identity derived directly from the stated mathematical structure;*

(c) Tel*: a telescoping, induction, compactness, or Lyapunov-summability result derived from one-step inequalities;*

(d) Cert*: a certificate-composition theorem converting audited one-sided residual bounds into true gate satisfaction under explicit soundness events;*

(e) Exist*: an assumption-dependent existence, search, tangent-cone, or viability theorem;*

(f) Impl*: an implementation obligation, empirical validation requirement, falsification condition, or non-theorem status marker.*

Type	**Logical form**	**Examples in this paper**	**What it does not prove**
Definition / convention	Introduces objects and sign conventions.	Density states, protected pair sets, ability sets, engine kernels, residual vectors, typed density realization, Batch 12A explicit successor-verification residuals, final RCP-II theorem framing.	Does not prove the object exists in a concrete system or has the intended semantics.
Algebraic / structural theorem	Uses finite-dimensional algebra, isometries, tensor products, convexity, or exact recovery identities.	Density preservation, structural recovery, conservative-extension exact non-loss, primitive-word algebra closure.	Does not prove useful extensions exist or that they are resource-feasible.
Telescoping / induction / compactness theorem	Sums one-step bounds or applies safe-set induction, viability recursion, or compactness extraction.	Bounded distinguishability loss, Lyapunov boundedness, ambiguity-collapse budget, self-model relevance budget, safe-state invariance, finite-word additive budgets, Batch 12A goal-identity composition.	Does not construct the one-step inequalities, certificates, or finite-horizon plans.
Certificate-composition theorem	Converts estimated residuals and margins into true residual satisfaction under one-sided soundness events.	Robust gate soundness, operational closure, non-oracular certificates, semantic coupling transfer, trust-graph soundness, coverage/calibration/estimator subprotocol closure, Batch 12A successor-verification soundness composition.	Does not prove the audit data are representative, the radii are tight, or the verifier is intrinsically trustworthy.

Type	Logical form	Examples in this paper	What it does not prove
Assumption-dependent existence theorem	Derives an update or path from strict interiority, tangent-cone positivity, search completeness, certified plans, or engine-kernel membership.	Nontrivial admissible improvement, tangent-cone local progress, budgeted certified search, certified plan implies nonempty engine kernel, engine-viability nonemptiness, Direct Non-Lossy RSI Engine construction on a declared engine domain, Batch 11 seed-domain entry from a certified conservative-extension library, Batch 12 successor-verification domain invariance on a certified successor-verification seed domain, Batch 12B domain-relative RRST, surfaced final RCP-II theorem.	Does not prove arbitrary systems satisfy the assumptions or lie in the viability kernel.
Impl. / failure	States what must be built, audited, measured, or allowed to fail in a concrete system.	Resource ledgers, estimator protocols, semantic maps, pair-set coverage audits, and documented failure conditions.	Does not add a preservation theorem; it disciplines claims about application of the theorem stack.

Proposition 2.2 (Claim-discipline rule)**.** *A concrete implementation claim may cite a theorem of type* Alg*,* Tel*,* Cert*, or* Exist *only together with the hypotheses of that theorem and the status of the required implementation objects. In particular, a certificate-composition theorem cannot be cited as evidence that the certificate is obtainable, and an assumption-dependent existence theorem cannot be cited as evidence that arbitrary states satisfy its assumptions.*

Proof. The conclusion follows from the logical form of the theorem types. An algebraic or telescoping theorem proves its consequent only from its stated premises. A certificate-composition theorem additionally requires the relevant certificate-validity event. An existence or viability theorem requires the stated nonemptiness, strictness, search, or kernel-membership premise. Omitting those premises changes the logical statement and would be an invalid use of the theorem. □

Interpretation 2.3. *This table is a publication-rigor device. It does not weaken the mathematical results. It prevents a correct preservation or certificate theorem from being overread as a proof that a*

real self-improving architecture already has correct semantics, complete pair sets, scalable estimators, or useful non-lossy update generators.

3 Background and Notation

This section fixes the notation used throughout the paper.

3.1 Density states and reduced states

Definition 3.1 (Finite-dimensional density state)**.** *Let $\mathcal{H}$ be a finite-dimensional Hilbert space. The set of density states on $\mathcal{H}$ is*

$$\mathcal{D}(\mathcal{H}) = \{\rho \in \mathcal{B}(\mathcal{H}) : \rho = \rho^\dagger,\ \rho \succeq 0,\ \mathrm{Tr}(\rho) = 1\}.$$

When $\dim \mathcal{H} = d$, we also write $\mathcal{D}_d$.

Definition 3.2 (Reduced state)**.** *If $\mathcal{H} = \mathcal{H}_X \otimes \mathcal{H}_Y$ and $\rho \in \mathcal{D}(\mathcal{H})$, the reduced state on X is*

$$\rho_X = \mathrm{Tr}_Y(\rho).$$

For multiple subsystems, the notation ρ_{XY}, ρ_{XYZ}, and so on denotes the corresponding partial trace.

Assumption 3.3 (Finite operational representation)**.** *All theorem statements in this paper are stated for finite-dimensional density states. Infinite-dimensional or continuously indexed systems may be approximated by finite-dimensional truncations, but no theorem below is automatically transferred to the infinite-dimensional case without additional compactness, domain, and continuity assumptions.*

3.2 Entropy, relative entropy, and mutual information

All logarithms are taken base 2 unless otherwise stated. Changing the base only rescales constants.

Definition 3.4 (Von Neumann entropy)**.** *For $\rho \in \mathcal{D}(\mathcal{H})$,*

$$S(\rho) = -\mathrm{Tr}(\rho \log \rho).$$

Definition 3.5 (Quantum relative entropy)**.** *For $\rho, \sigma \in \mathcal{D}(\mathcal{H})$,*

$$D(\rho\|\sigma) = \begin{cases} \mathrm{Tr}\big[\rho(\log\rho - \log\sigma)\big], & \mathrm{supp}(\rho) \subseteq \mathrm{supp}(\sigma), \\ +\infty, & \textit{otherwise.} \end{cases}$$

Definition 3.6 (Quantum mutual information)**.** *For a bipartite state ρ_{XY},*

$$I_\rho(X;Y) = S(\rho_X) + S(\rho_Y) - S(\rho_{XY}).$$

Equivalently,

$$I_\rho(X;Y) = D(\rho_{XY}\|\rho_X \otimes \rho_Y).$$

Definition 3.7 (Trace distance)**.** *The trace distance between density states is*

$$T(\rho,\sigma) = \frac{1}{2}\|\rho - \sigma\|_1.$$

In finite dimension, the trace norm and Frobenius norm induce equivalent topologies. We use a generic state distance $d(\rho,\sigma)$ when the theorem only requires nonnegativity and measurability.

3.3 Channels and recursive update maps

Definition 3.8 (Quantum channel). *A quantum channel is a linear map*

$$\Phi : \mathcal{B}(\mathcal{H}) \to \mathcal{B}(\mathcal{H}')$$

that is completely positive and trace-preserving (CPTP). In this paper, Φ_t denotes the proposed self-update channel at recursive step t.

Assumption 3.9 (Admissible approximate channels). *For exact theorem statements, Φ_t is CPTP. In implementation-facing sections, an approximate learned update is admissible only if it is projected or normalized back into $\mathcal{D}(\mathcal{H})$. Thus every accepted update satisfies*

$$\rho_{t+1} = \Phi_t(\rho_t) \in \mathcal{D}(\mathcal{H}).$$

Remark 3.10 (Classical special case). *If every ρ_t is diagonal in a fixed basis and every Φ_t is a stochastic map on the diagonal, the framework reduces to a Shannon-informational formulation. Thus the quantum-informational version is a noncommutative generalization of a classical entropy-reduction framework.*

4 RCLM Framework and Recursive System State

4.1 Representational decomposition

Definition 4.1 (RCLM Hilbert decomposition). *At recursive step t, the system state is a density state*

$$\rho_t \in \mathcal{D}(\mathcal{H}),$$

where the total representational space decomposes as

$$\mathcal{H} = \mathcal{H}_L \otimes \mathcal{H}_O \otimes \mathcal{H}_S \otimes \mathcal{H}_D \otimes \mathcal{H}_A \otimes \mathcal{H}_M.$$

The registers are interpreted as follows:

$$L = \textit{language},$$

$$O = \textit{objective/world-reference structure},$$

$$S = \textit{subjective/human/value-reference structure},$$

$$D = \textit{definitiveness/decision-collapse structure},$$

$$A = \textit{ambiguity/uncertainty structure},$$

$$M = \textit{self-model: architecture, goals, limits, update rules.}$$

The tensor product should not be read as the claim that real cognitive systems literally separate these factors cleanly. It is an operational decomposition: a way to assign observables, audits, and constraints to distinct roles in the recursive loop.

4.2 Recursive self-model

Definition 4.2 (Self-model register)**.** *The self-model register M is the subsystem intended to encode information about:*

(a) the current architecture or operator class;

(b) the system's objectives or optimization criteria;

(c) known limitations and uncertainty estimates;

(d) the proposed update mechanism;

(e) the predicted effect of that update on future states.

A Level 2 RCLM can generate recursive coherence over language and representation. A Level 3 RSI system must additionally generate recursive coherence over the update process itself. This requires the self-model to remain informative about both the system and the update channel.

4.3 RSI as a self-modification channel

Definition 4.3 (Recursive self-improvement channel)**.** *Each accepted recursive self-improvement step is represented by a channel*

$$\Phi_t : \rho_t \mapsto \rho_{t+1}.$$

The channel may represent retraining, successor construction, architecture modification, memory rewriting, objective refinement, or a projected combination of these operations.

Assumption 4.4 (Closed-loop selection)**.** *At time t, the system has an admissible family Ω_t of candidate updates. A meta-controller selects*

$$\Phi_t \in \Omega_t$$

using a criterion that may depend on ρ_t, validation evidence, the self-model M, and the safety gates defined later.

4.4 Interpretation

The key transition is:

$$\text{modeling language and world} \quad \longrightarrow \quad \text{modeling the process that updates the model.}$$

The latter is where lossy self-improvement appears. RCP therefore treats self-update as a constrained information channel rather than a free capability maximization step.

4.5 Limitation

The decomposition into L, O, S, D, A, M registers is a modeling choice. A concrete architecture must specify how these registers are represented, measured, and validated. The theorems below prove stability relative to these representations; they do not prove that a particular implementation has chosen the right representation.

5 Recursive Self-Improvement as Channel Optimization

5.1 Definition of Level 3 RSI

Definition 5.1 (Level 3 recursive self-improvement). *A system performs Level 3 RSI when it selects a modification $\Phi_t \in \Omega_t$ by solving, exactly or approximately, an optimization problem whose objective includes predicted future performance of the modified system:*

$$\Phi_t^{\mathrm{raw}} \in \arg\max_{\Phi \in \Omega_t} \mathbb{E}\big[\mathrm{Perf}_{t+k} \mid \Phi, \rho_t, M_t\big].$$

The raw update becomes an accepted update only after passing the RCP admissibility gate.

Remark 5.2 (Why raw performance optimization is insufficient). *The update Φ_t^{raw} may improve a task metric while erasing safety-relevant information, increasing irreversibility, collapsing uncertainty, or reducing human-grounded corrigibility. The rest of the paper defines mathematical tests that a raw update must pass before being accepted.*

5.2 Admissible update family

Assumption 5.3 (No-op feasibility). *For every t and every accepted state ρ_t, the admissible update family Ω_t contains an identity/no-op channel*

$$\Phi_t^0 = \mathrm{id} \quad \textit{such that} \quad \Phi_t^0(\rho_t) = \rho_t.$$

No-op feasibility is a crucial non-circularity assumption. It means the system is never forced to accept a self-modification merely because a modification was proposed. The current system can always be preserved if every proposed change violates the safety gate.

Assumption 5.4 (Auditability of candidates). *For each candidate $\Phi \in \Omega_t$, the verifier can evaluate or upper-bound the quantities required by the RCP gate: relative-entropy loss, Lyapunov drift, barrier constraints, ambiguity collapse, and self-model information preservation. Approximate certificates are permitted only with explicit error budgets.*

6 Thesis Reframing: Recoverable Monotone Self-Improvement

This section records the third-version thesis correction. The preservation framework developed so far is not primarily a moral-safety theorem. It is a non-loss theorem for recursive self-update. Safety becomes a corollary when the protected predicate family includes safety-relevant predicates. This reframing does not change the core equations

$$\rho_t \in \mathcal{D}(\mathcal{H}), \qquad \Phi_t : \rho_t \mapsto \rho_{t+1}, \qquad \mathcal{L}_\Phi^{\mathcal{P}}, \qquad V(\rho_t), \qquad \mathcal{A}_{\mathrm{RCP}}, \qquad \mathcal{K}_{\mathrm{RCP}}.$$

It changes the role assigned to the main theorem: the present theorem is a conditional preservation theorem, while the direct engine theorem must additionally construct beneficial recoverable updates.

Definition 6.1 (Protected predicate family). *Let $\mathfrak{S}_t^{\mathrm{prot}}$ be the declared family of predicates or distinctions whose preservation is required for non-lossy recursive self-improvement. A subfamily*

$$\mathfrak{S}_t^{\mathrm{safe}} \subseteq \mathfrak{S}_t^{\mathrm{prot}}$$

contains those predicates interpreted as safety predicates, such as corrigibility, groundedness, uncertainty honesty, oversight compatibility, reversibility, no hidden goal drift, and verifier integrity. The full protected family may also include capability-relevant and engine-viability predicates that are not primarily safety predicates.

Definition 6.2 (Recoverable monotone self-improvement step)**.** *Fix a protected predicate family $\mathfrak{S}_t^{\mathrm{prot}}$, a pair set $\mathcal{P}_t$, a recovery metric, and a declared progress preorder $\preceq_t$ on the audited progress summaries of the system. A candidate update Φ_t is a recoverable monotone self-improvement step when the following are certified:*

(i) Φ_t is recoverably non-lossy on the protected family, i.e. it satisfies the relative-entropy loss budget and supplies an explicit recovery map or recovery certificate;

(ii) every protected predicate that is certified true before the update is either preserved directly or transported to a successor predicate with a declared distortion budget;

(iii) the successor progress summary is not below the predecessor under $\preceq_t$, and is strictly above it whenever a nontrivial RSI-progress claim is made;

(iv) the successor remains inside the declared recursive viability domain for future certified self-updates.

Batch 2 instantiates $\preceq_t$ by certified ability-set inclusion: predecessor abilities must transport into the successor certified ability set, and strict RSI progress requires a proper successor ability-set expansion. Batch 3 adds the closed non-lossy update algebra for composing certified primitive moves. Batch 4 adds a tangent-cone local progress condition using the projected gradient of an audited engine-potential functional, while keeping strict ability-set expansion as a separate certificate obligation.

Proposition 6.3 (Safety as a corollary of protected non-loss)**.** *Assume that every safety predicate in $\mathfrak{S}_t^{\mathrm{safe}}$ is included in the protected predicate family $\mathfrak{S}_t^{\mathrm{prot}}$, and assume the RCP gate preserves all protected predicates over the accepted update Φ_t either by pair-set non-loss, by sound barriers, or by semantic coupling certificates. Then every certified safety predicate in $\mathfrak{S}_t^{\mathrm{safe}}$ is preserved over Φ_t within the same declared budgets.*

Proof. The safety predicates are a subfamily of the protected predicates. The hypothesis says that all protected predicates are preserved by the accepted update through one of the certified preservation mechanisms. Restricting that preservation claim to the subfamily $\mathfrak{S}_t^{\mathrm{safe}}$ gives the safety conclusion. No additional safety theorem is used. □

Definition 6.4 (Conditional Non-Lossy Self-Update Preservation Theorem role)**.** *The current main theorem of this manuscript has the following logical form:*

$$\rho_0 \in \mathcal{K}_{\mathrm{RCP}}^{\mathrm{state}}, \quad \Phi_t \in \mathcal{A}_{\mathrm{RCP}}(t, \rho_t)\ \forall t \quad \Longrightarrow \quad \textit{protected quantities remain bounded or invariant.}$$

It is therefore named the Conditional Non-Lossy Self-Update Preservation Theorem*. It proves that admitted updates preserve the declared non-loss, Lyapunov, barrier, ambiguity, self-model, semantic, verifier, resource, coverage, calibration, and estimator obligations. It does not by itself construct a beneficial update.*

Definition 6.5 (Direct Non-Lossy RSI Engine Theorem role)**.** *The direct-engine layer is the constructive counterpart of the preservation theorem. An engine is a map*

$$\mathfrak{E}_t : \rho_t \longmapsto (\Phi_t, \mathcal{R}_t^{\mathrm{rec}}, \mathsf{Cert}_t, \rho_{t+1})$$

that constructs a self-update, a recovery map, a certificate packet, and the successor state. The desired direct-engine conclusion has the schematic form

$$\rho_t \in \mathcal{D}_t^{\mathrm{engine}} \quad \Longrightarrow \quad \mathfrak{E}_t(\rho_t) \text{ constructs } \Phi_t \in \mathcal{A}_t^{\text{NL-RSI}},$$

with certified recovery, strict progress relative to the declared progress relation, resource feasibility, and successor viability

$$\rho_{t+1} \in \mathcal{V}_{t+1}^{\mathrm{engine}}.$$

Batch 1 introduced this theorem role and prevented it from being confused with the already-proved preservation theorem. Batch 2 supplies certified ability-set progress and structural recovery/conservative extension. Batch 3 supplies the closed non-lossy update algebra and composition theorem. Batch 4 supplies a local tangent-cone sufficient condition for constructing positive progress steps inside a non-lossy candidate chart. Batch 5 supplies a conservative-extension engine-viability kernel and proves that kernel membership yields a certified non-lossy successor update with successor viability. Batch 8 supplies the comparison and realizability hygiene needed for that target theorem: cross-time transports for the information variables, a state-only safe set separated from update-dependent admissibility, and projection-realization certificates for any projected successor. Batch 9 proves the direct-engine theorem in the following limited and rigorous sense: for states in a declared direct-engine domain, with strict beneficial conservative-extension witnesses and a sound generator/certifier covering those witnesses within margins, the engine constructs the update rather than merely accepting an externally proposed one. Batch 11 then gives a sufficient seed-domain entry theorem: certified conservative-extension libraries and bounded grammar coverage can discharge the strict-witness and generator-coverage assumptions for a declared seed class.

Limitation 6.6 (What the direct-engine role still does not prove)**.** *The Batch 9 theorem does not prove that $\mathcal{D}_t^{\mathrm{engine}}$ contains arbitrary states, that strict beneficial extension witnesses always exist, that resources scale indefinitely, or that open-ended progress persists on a fixed bounded ability scale. Batch 11 proves domain entry only for states in an explicitly certified seed class with a nonempty conservative-extension library, bounded grammar coverage, sound certificates, adequate margins, and successor seed-persistence evidence. Neither theorem proves arbitrary-system RSI.*

7 Cross-Time, State/Update, and Projection-Realization Cleanup

This section implements Edit Batch 8. It resolves three reviewer-facing ambiguity points left after the operational-closure and engine-viability upgrades. First, quantities such as Q_t, Z_t, Y_t, G_t may live in time-indexed spaces, so entropy and mutual-information comparisons across time require declared transport maps. Second, the state safe set must be a state-only set, while update-dependent quantities must live in an admissibility relation. Third, projection into a safe set is a mathematical repair unless a separate certificate proves that the projected successor is reachable by an implementable typed self-update.

Limitation 7.1 (Cleanup does not strengthen the engine theorem by itself)**.** *Batch 8 is not a new existence theorem. It does not prove that beneficial updates exist, that cross-time transports are*

semantically correct, or that projected states are always implementable. It states the extra hypotheses required for older shorthand expressions to be valid and blocks the use of projection as an unlicensed repair oracle.

7.1 Cross-time semantic and information-variable transport

Definition 7.2 (Time-indexed information variables)**.** *At recursive step t, let*

$$Q_t \in \mathsf{Q}_t, \qquad Z_t \in \mathsf{Z}_t, \qquad Y_t \in \mathsf{Y}_t, \qquad G_t \in \mathsf{G}_t$$

be, respectively, the ambiguity/question variable, compressed self-model variable, safety- or update-relevant future-behavior variable, and update-description or update-audit variable. The spaces $\mathsf{Q}_t, \mathsf{Z}_t, \mathsf{Y}_t, \mathsf{G}_t$ are allowed to vary with time. Therefore an expression comparing Q_t to Q_{t+1}, $I(Z_t; Y_t)$ to $I(Z_{t+1}; Y_{t+1})$, or $I(M_t; G_t)$ to $I(M_{t+1}; G_{t+1})$ is not licensed unless a cross-time comparison structure is supplied.

Definition 7.3 (Cross-time transport maps)**.** *A cross-time transport package is a tuple*

$$\Theta_t^{\mathrm{info}} = (\Theta_t^Q, \Theta_t^Z, \Theta_t^Y, \Theta_t^G, \Theta_t^E)$$

where

$$\Theta_t^Q : \mathsf{Q}_t \to \mathsf{Q}_{t+1}, \quad \Theta_t^Z : \mathsf{Z}_t \to \mathsf{Z}_{t+1}, \quad \Theta_t^Y : \mathsf{Y}_t \to \mathsf{Y}_{t+1}, \quad \Theta_t^G : \mathsf{G}_t \to \mathsf{G}_{t+1},$$

and Θ_t^E transports the evidence sigma-algebra or evidence register used in ambiguity-collapse comparisons. Each Θ may be a deterministic map, stochastic kernel, CPTP channel, or typed representational transport, depending on the type certificate of the variables involved. If the corresponding space is unchanged and identity comparison is valid, the transport may be the identity.

Assumption 7.4 (Transport type and domain validity)**.** *Every transport map used in a theorem is type-compatible with the variables it transports, is defined on the audited support of the relevant distribution or density state, and has a declared distortion budget. If a transport is learned or representational, the theorem using it must include a direct certificate for its distortion rather than importing quantum or classical data-processing without a type license.*

Definition 7.5 (Bottleneck transport distortion)**.** *The cross-time bottleneck distortion is a nonnegative number δ_t^{ZY} satisfying*

$$\left| I_t(Z_t; Y_t) - I_{t+1}(\Theta_t^Z Z_t; \Theta_t^Y Y_t) \right| \le \delta_t^{ZY}$$

on the audited domain. Here I_t and I_{t+1} denote mutual information computed in the corresponding time-indexed probabilistic or density representation. If the variables live in the same space and no transport is needed, $\delta_t^{ZY} = 0$ is permitted only when identity comparability is certified.

Definition 7.6 (Transported self-model relevance preservation)**.** *The transported information-bottleneck preservation condition is*

$$I_{t+1}(Z_{t+1}; Y_{t+1}) \ge I_{t+1}(\Theta_t^Z Z_t; \Theta_t^Y Y_t) - \xi_t^{\mathrm{tr}}.$$

The older shorthand

$$I(Z_{t+1}; Y_{t+1}) \ge I(Z_t; Y_t) - \xi_t$$

is valid only when either the spaces are certified identical, or when ξ_t includes the transport distortion:

$$\xi_t \ge \xi_t^{\mathrm{tr}} + \delta_t^{ZY}.$$

Proposition 7.7 (Transported bottleneck condition licenses the legacy bound). *Assume the bottleneck transport distortion bound and the transported self-model relevance condition. If* $\xi_t \geq \xi_t^{\mathrm{tr}} + \delta_t^{ZY}$, *then*

$$I_{t+1}(Z_{t+1}; Y_{t+1}) \geq I_t(Z_t; Y_t) - \xi_t.$$

Proof. By transported preservation,

$$I_{t+1}(Z_{t+1}; Y_{t+1}) \geq I_{t+1}(\Theta_t^Z Z_t; \Theta_t^Y Y_t) - \xi_t^{\mathrm{tr}}.$$

By Definition 7.5,

$$I_{t+1}(\Theta_t^Z Z_t; \Theta_t^Y Y_t) \geq I_t(Z_t; Y_t) - \delta_t^{ZY}.$$

Combining the two inequalities gives

$$I_{t+1}(Z_{t+1}; Y_{t+1}) \geq I_t(Z_t; Y_t) - \xi_t^{\mathrm{tr}} - \delta_t^{ZY} \geq I_t(Z_t; Y_t) - \xi_t.$$

□

Definition 7.8 (Transported ambiguity-collapse residual). *Let* E_t *denote the evidence variable or sigma-algebra used at time* t. *The transported predecessor question and evidence at time* $t+1$ *are* $\Theta_t^Q Q_t$ *and* $\Theta_t^E E_t$. *Define the transported unsupported-collapse residual*

$$U_t^{\mathrm{tr}} = \Big[H_{t+1}(\Theta_t^Q Q_t \mid \Theta_t^E E_t) - H_{t+1}(Q_{t+1} \mid E_{t+1}) - I_{t+1}(\Theta_t^Q Q_t; E_{t+1} \mid \Theta_t^E E_t) \Big]_+ .$$

The ambiguity-collapse gate is the transported inequality

$$U_t^{\mathrm{tr}} \leq \zeta_t^{\mathrm{tr}}.$$

The older expression comparing $H_t(Q_t \mid E_t)$ *directly to* $H_{t+1}(Q_{t+1} \mid E_{t+1})$ *is a shorthand only when the transport distortion between* $H_t(Q_t \mid E_t)$ *and* $H_{t+1}(\Theta_t^Q Q_t \mid \Theta_t^E E_t)$ *is included in* ζ_t.

Definition 7.9 (Update-description transport). *The update-description transport* $\Theta_t^G : G_t \to G_{t+1}$ *licenses cross-time comparisons involving the self-model's awareness of update descriptions. If the recursive coherence functional uses a term such as* $I(M; G_t)$, *then a cross-time claim about improving or preserving that term must either compare within a fixed* G*-space or use* Θ_t^G *with a declared distortion budget* δ_t^{MG}. *Without* Θ_t^G, *the term remains a within-time diagnostic only.*

Limitation 7.10 (Transport maps are semantic assumptions). *The maps* $\Theta_t^Q, \Theta_t^Z, \Theta_t^Y, \Theta_t^G$ *and* Θ_t^E *do not follow from notation. A poor transport can make an invalid comparison look formal. Therefore transport validity is a certificate obligation, and a failed transport invalidates the corresponding cross-time entropy, mutual-information, ambiguity, or update-description claim.*

7.2 State-only safe sets and update-dependent admissibility

Definition 7.11 (State-only Lyapunov monitor). *A state-only Lyapunov monitor is a function*

$$V_{\mathrm{state},t} : \mathcal{D}(\mathcal{H}_t) \to \mathbb{R}$$

whose arguments are only the current state and time-indexed state specification. It may include state marginals, state-level risks, state-level semantic errors, state-level self-model fidelity, and state-level resource variables encoded in ρ. *It may not depend on a proposed update* Φ, *update-description register* $G_t(\Phi)$, *candidate certificate, recovery map, or future comparison variable.*

Definition 7.12 (Update-dependent Lyapunov residual)**.** *An update-dependent Lyapunov residual is a function*

$$V_{\mathrm{upd},t}(\rho_t, G_t, \Phi_t)$$

or, equivalently, a residual $q_t^V(\Phi_t; \rho_t)$*, used to evaluate the effect of a particular candidate update on recursive coherence, update-awareness, or predicted successor behavior. It belongs to the admissibility relation, not to the state-only safe set.*

Definition 7.13 (State-only RCP safe set)**.** *The state-only RCP safe set at time* t *is*

$$\mathcal{K}_{\mathrm{RCP}}^{\mathrm{state}}(t) = \left\{ \rho \in \mathcal{D}(\mathcal{H}_t) : \begin{array}{l} V_{\mathrm{state},t}(\rho) \le v_{\max,t}, \\ B_{i,t}^{\mathrm{state}}(\rho) \le 0 \quad \forall i \in I_t^{\mathrm{state}}, \\ L_{\mathrm{div},t}(\rho) \ge \kappa_{\mathrm{div},t}, \\ I_t(Z_\rho; Y_\rho) \ge I_{\min,t} \quad \textit{when this is state-defined}, \\ \rho \in \mathcal{K}_t^{\mathrm{sem}} \cap \mathcal{K}_t^{M} \cap \mathcal{K}_t^{\mathrm{ver}} \cap \mathcal{K}_t^{\mathrm{phys}} \textit{ when those state sets are active.} \end{array} \right\}.$$

When the older notation $\mathcal{K}_{\mathrm{RCP}}^{\mathrm{state}}$ *appears without* t*, it means the time-indexed state-only set in this definition, with* t *clear from context.*

Definition 7.14 (Update-admissibility relation)**.** *The RCP update-admissibility relation is*

$$\mathcal{A}_{\mathrm{RCP}}(t, \rho, \Phi)$$

where $\rho \in \mathcal{K}_{\mathrm{RCP}}^{\mathrm{state}}(t)$ *and* $\Phi \in \Omega_t(\rho)$*. It is true exactly when the candidate satisfies all update-dependent constraints: typed update validity, successor state membership*

$$\Phi(\rho) \in \mathcal{K}_{\mathrm{RCP}}^{\mathrm{state}}(t+1),$$

relative-entropy non-loss, constructive recovery, update-dependent Lyapunov drift, transported ambiguity-collapse control, transported self-model relevance preservation, semantic coupling, verifier/trust constraints, resource/cost constraints, coverage/calibration/estimator requirements, ability-set progress conditions when claimed, and any engine-viability requirements when an engine-mode claim is made. The associated set of admissible candidates is

$$\mathcal{A}_{\mathrm{RCP}}(t, \rho) = \{\Phi \in \Omega_t(\rho) : \mathcal{A}_{\mathrm{RCP}}(t, \rho, \Phi)\}.$$

Proposition 7.15 (State/update separation removes the safe-set ambiguity)**.** *If* $\rho_t \in \mathcal{K}_{\mathrm{RCP}}^{\mathrm{state}}(t)$ *and* $\mathcal{A}_{\mathrm{RCP}}(t, \rho_t, \Phi_t)$ *holds, then the successor state satisfies*

$$\rho_{t+1} = \Phi_t(\rho_t) \in \mathcal{K}_{\mathrm{RCP}}^{\mathrm{state}}(t+1).$$

Update-dependent claims such as recovery, transported information preservation, ambiguity-collapse control, and progress are licensed only by the corresponding residuals inside $\mathcal{A}_{\mathrm{RCP}}(t, \rho_t, \Phi_t)$*, not by state-set membership alone.*

Proof. By Definition 7.14, one of the required update-dependent conditions for $\mathcal{A}_{\mathrm{RCP}}(t, \rho_t, \Phi_t)$ is successor membership $\Phi_t(\rho_t) \in \mathcal{K}_{\mathrm{RCP}}^{\mathrm{state}}(t+1)$. Since $\rho_{t+1} = \Phi_t(\rho_t)$, the state-invariance claim follows. The second statement follows from the same definition: recovery, transported information preservation, ambiguity control, and progress are listed as update-dependent conditions and are not included in the state-only set except as state-defined monitors. □

Interpretation 7.16 (Legacy notation convention)**.** *Earlier sections may write* $V(\rho, G)$ *or* V_t *in compact formulas. After Batch 8, this is read as follows: state-only constraints use* $V_{\mathrm{state},t}(\rho)$*; candidate-specific drift, update-awareness, and certificate comparisons use* $V_{\mathrm{upd},t}(\rho, G, \Phi)$ *or the corresponding residual* $q_t^V(\Phi; \rho)$*. This convention prevents the state safe set from secretly depending on the candidate update.*

7.3 Projection-realization certificates

Definition 7.17 (Mathematical projection repair)**.** *Let $\widetilde{\rho}_{t+1}$ be an unconstrained proposed successor state. A mathematical safe-set projection is a state*

$$\rho^{+}_{t+1} \in \Pi_{\mathcal{K}^{\mathrm{state}}_{\mathrm{RCP}}(t+1)}(\widetilde{\rho}_{t+1}),$$

where the projection minimizes a declared distance to $\widetilde{\rho}_{t+1}$ over $\mathcal{K}^{\mathrm{state}}_{\mathrm{RCP}}(t+1)$, when such a minimizer exists. This is a state-level repair object. It is not automatically an implementable self-update.

Definition 7.18 (Projection-realization certificate)**.** *A projection-realization certificate is a tuple*

$$\mathsf{ProjReal}_t(\widetilde{\rho}_{t+1}, \rho^{+}_{t+1}, \nu) = (\nu, \Phi_\nu, \tau_t, \delta_t^{\text{proj-real}}, \mathsf{TypeCert}_t, \mathsf{CostCert}_t, \mathsf{ResCert}_t)$$

such that:

(i) $\rho^{+}_{t+1} \in \mathcal{K}^{\mathrm{state}}_{\mathrm{RCP}}(t+1)$*;*

(ii) Φ_ν *is a typed admissible self-update transition on the audited input family;*

(iii) the projected successor is reachable up to declared implementation error:

$$d_{\mathrm{state},t+1}\big(\rho^{+}_{t+1}, \Phi_\nu(\rho_t)\big) \le \delta_t^{\text{proj-real}};$$

(iv) every residual affected by replacing ρ^{+}_{t+1} with $\Phi_\nu(\rho_t)$ includes the distortion $\delta_t^{\text{proj-real}}$ through a Lipschitz, continuity, or direct estimator bound;

(v) the cost of realizing Φ_ν and verifying the distortion is included in the resource gate.

Definition 7.19 (Projection-realization residual)**.** *Let $m_t^{\text{proj-real}} \ge 0$ be the maximum allowed projection-realization distortion after accounting for residual margins. Define*

$$q_t^{\text{proj-real}}(\nu) = \delta_t^{\text{proj-real}} - m_t^{\text{proj-real}}.$$

The projection-realization gate requires

$$q_t^{\text{proj-real}}(\nu) \le 0$$

plus the type, cost, and residual-transfer certificates in Definition 7.18.

Theorem 7.20 (Projected safe state becomes accepted update only with realization)**.** *Suppose ρ^{+}_{t+1} is a mathematical projection of $\widetilde{\rho}_{t+1}$ into $\mathcal{K}^{\mathrm{state}}_{\mathrm{RCP}}(t+1)$. If a projection-realization certificate $\mathsf{ProjReal}_t(\widetilde{\rho}_{t+1}, \rho^{+}_{t+1}, \nu)$ is valid and every residual distortion induced by $\delta_t^{\text{proj-real}}$ is included in the admissibility residuals, then the typed update Φ_ν may be evaluated as a candidate in $\mathcal{A}_{\mathrm{RCP}}(t, \rho_t, \Phi_\nu)$. If, in addition, all residuals of $\mathcal{A}_{\mathrm{RCP}}(t, \rho_t, \Phi_\nu)$ are nonpositive after including the projection-realization distortion, then Φ_ν is an accepted implementable RCP update.*

Proof. The projection alone supplies only $\rho^{+}_{t+1} \in \mathcal{K}^{\mathrm{state}}_{\mathrm{RCP}}(t+1)$. The projection-realization certificate supplies a typed self-update Φ_ν with successor $\Phi_\nu(\rho_t)$ within distance $\delta_t^{\text{proj-real}}$ of ρ^{+}_{t+1}, plus the type, cost, and residual-transfer certificates required to make this approximation relevant to the gate. Once all residual distortions are included, the candidate is judged by the ordinary update-admissibility relation. If every residual is nonpositive, Definition 7.14 gives $\mathcal{A}_{\mathrm{RCP}}(t, \rho_t, \Phi_\nu)$, so the update is accepted. Without the realization certificate, the projected state remains only a mathematical repair object. □

Corollary 7.21 (Projection is not a magic repair oracle). *A theorem that proves $\rho^+_{t+1} \in \mathcal{K}^{\mathrm{state}}_{\mathrm{RCP}}(t+1)$ for a projected state does not prove that a real RSI system can move from ρ_t to ρ^+_{t+1}. An implementation claim requires $\mathsf{ProjReal}_t$ or another channel-realization certificate.*

Limitation 7.22 (Projection realizability remains an implementation problem). *Batch 8 does not prove that every safe projection is realizable. It states the certificate that must be supplied if a projected safe state is to count as an accepted self-update. If no such certificate exists, the system may use the projection for analysis, diagnosis, or candidate repair, but not as a licensed RSI transition.*

8 Ability-Set Progress and Structural Recovery

This section implements Edit Batch 2. The previous reframing introduced a generic progress preorder. This batch replaces the generic progress placeholder with a certified ability-set preorder and adds a structural recovery class in which non-loss is obtained from the form of the update rather than from an after-the-fact estimator alone. The section is still conditional: it does not prove that new abilities automatically exist. It proves that if abilities are certified and if an update preserves predecessor access by structural recovery or conservative extension, then predecessor abilities are retained, and strict RSI progress is represented by certified ability-set expansion.

8.1 Certified ability sets

Definition 8.1 (Ability universe and resources). *At time t, let Ab_t be the declared ability universe. An element $a \in \mathsf{Ab}_t$ may represent a task, theorem-proving class, tool-use competence, verifier capability, search capability, calibration capability, data-acquisition ability, or any other update-relevant competence. Let $R \in \mathsf{R}_t$ denote a resource budget or resource vector under which the ability claim is made. Ability claims without a declared resource budget are not part of the certified ability set.*

Definition 8.2 (Ability certificate). *For $a \in \mathsf{Ab}_t$, state $\rho \in \mathcal{D}(\mathcal{H}_t)$, and resource budget $R \in \mathsf{R}_t$, an ability certificate is a tuple*

$$\mathsf{AbCert}_{t,a}(\rho, R) = (\pi_{t,a}, \mathsf{Test}_{t,a}, \mathsf{Ver}_{t,a}, \mathsf{Cost}_{t,a}, \Delta^{\mathrm{ab}}_{t,a}, \delta^{\mathrm{ab}}_{t,a}),$$

where $\pi_{t,a}$ is a policy, solver, proof routine, tool routine, or callable subsystem; $\mathsf{Test}_{t,a}$ is the audited validation specification; $\mathsf{Ver}_{t,a}$ is the verifier used for the claim; $\mathsf{Cost}_{t,a} \le R$ is the certified resource cost; $\Delta^{\mathrm{ab}}_{t,a}$ is the one-sided audit radius; and $\delta^{\mathrm{ab}}_{t,a}$ is the declared failure probability. The certificate is valid when the verifier accepts the claim under the declared resource budget and audit radius.

Definition 8.3 (Certified ability set). *The certified ability set of state ρ under resource budget R is*

$$\boxed{\mathfrak{A}_t(\rho, R) = \{a \in \mathsf{Ab}_t : \mathsf{AbCert}_{t,a}(\rho, R) \text{ is valid}\}.}$$

This is a certified set, not an omniscient set of all true abilities. If the ability certificates are unsound, then membership in $\mathfrak{A}_t$ may overstate actual ability.

Assumption 8.4 (Ability-certificate soundness). *For each finite audited ability family $\mathsf{Ab}^{\mathrm{aud}}_t \subseteq \mathsf{Ab}_t$, there is an event $\mathcal{E}^{\mathrm{ab}}_t$ of probability at least*

$$1 - \sum_{a \in \mathsf{Ab}^{\mathrm{aud}}_t} \delta^{\mathrm{ab}}_{t,a}$$

on which every certified ability claim is true on its audited domain: if $a \in \mathfrak{A}_t(\rho, R) \cap \mathsf{Ab}_t^{\mathrm{aud}}$, then the state ρ, together with the certified routine $\pi_{t,a}$, achieves ability a within the declared tolerance and resource budget R. This is an audit-soundness assumption, not a consequence of density-state notation.

Definition 8.5 (Cross-time ability transport)**.** *If the ability universe changes over time, a transport map*

$$\Theta_t^{\mathfrak{A}} : \mathsf{Ab}_t \to \mathsf{Ab}_{t+1}$$

must be supplied. It maps a predecessor ability claim to the corresponding successor-time ability claim. When a common ability universe is used, $\Theta_t^{\mathfrak{A}}$ is the identity. For a set $A \subseteq \mathsf{Ab}_t$, write

$$\Theta_t^{\mathfrak{A}}(A) = \{\Theta_t^{\mathfrak{A}}(a) : a \in A\}.$$

Definition 8.6 (Non-lossy ability preservation)**.** *An accepted update Φ_t with successor $\rho_{t+1} = \Phi_t(\rho_t)$ and successor resource budget R_{t+1} is ability-preserving when*

$$\boxed{\Theta_t^{\mathfrak{A}}\big(\mathfrak{A}_t(\rho_t, R_t)\big) \subseteq \mathfrak{A}_{t+1}(\rho_{t+1}, R_{t+1}).}$$

If the ability universe is time-independent, this reduces to the requested inclusion

$$\mathfrak{A}_t(\rho_t, R_t) \subseteq \mathfrak{A}_{t+1}(\rho_{t+1}, R_{t+1}).$$

Definition 8.7 (Strict RSI ability progress)**.** *An accepted update exhibits strict certified RSI progress when it is ability-preserving and*

$$\mathfrak{A}_{t+1}(\rho_{t+1}, R_{t+1}) \setminus \Theta_t^{\mathfrak{A}}\big(\mathfrak{A}_t(\rho_t, R_t)\big) \neq \varnothing.$$

Equivalently, at least one new successor-time ability is certified that is not merely the transported form of a predecessor ability. This definition avoids treating a scalar benchmark shift as progress.

Proposition 8.8 (Scalar capability summaries are secondary to ability inclusion)**.** *Let $w_t : \mathsf{Ab}_t \to \mathbb{R}_{\geq 0}$ be any nonnegative weighting over a finite audited ability family and define*

$$\mathrm{Cap}_t^{\mathfrak{A}}(\rho, R) = \sum_{a \in \mathfrak{A}_t(\rho, R)} w_t(a).$$

If a common ability universe and fixed weights are used, then ability preservation implies

$$\mathrm{Cap}_{t+1}^{\mathfrak{A}}(\rho_{t+1}, R_{t+1}) \geq \mathrm{Cap}_t^{\mathfrak{A}}(\rho_t, R_t).$$

If strict RSI ability progress adds an ability with positive successor weight, then the inequality is strict.

Proof. Under a common universe and fixed weights, set inclusion implies that the successor sum contains every nonnegative term appearing in the predecessor sum. If a new ability with positive weight is added, the successor sum contains at least one additional positive term. The result is purely set-theoretic and does not claim that every useful notion of capability is captured by the chosen weights. □

Limitation 8.9 (Ability sets are declared and audited)**.** *Ability-set progress is stronger than a single scalar benchmark only relative to the declared ability universe, transport maps, resource budgets, and certificate soundness assumptions. If an important ability is absent from Ab_t, or if the verifier over-certifies an ability, the theorem does not protect against that omission or verifier error.*

8.2 Structural recovery by reversible meta-state dynamics

Definition 8.10 (Reversible structural update on an enlarged meta-state)**.** *Let $\mathcal{H}_t$ be the predecessor state space and let $\mathcal{W}_t$ be an auxiliary workspace or ledger space with a fixed pure state $|0\rangle\langle 0|_{\mathcal{W}_t}$. Let U_t be a unitary on $\mathcal{H}_t \otimes \mathcal{W}_t$. The structural lifted update is*

$$\boxed{\widetilde{\Phi}_t(\rho) = U_t\big(\rho \otimes |0\rangle\langle 0|_{\mathcal{W}_t}\big)U_t^\dagger.}$$

The associated structural recovery map on the image of $\widetilde{\Phi}_t$ is

$$\widetilde{\mathcal{R}}_t^{\mathrm{str}}(\omega) = \mathrm{Tr}_{\mathcal{W}_t}\big[U_t^\dagger \omega U_t\big].$$

Theorem 8.11 (Exact structural recovery and exact relative-entropy preservation)**.** *Let $\rho, \sigma \in \mathcal{D}(\mathcal{H}_t)$ with $D(\rho\|\sigma) < \infty$, and let $\widetilde{\Phi}_t$ be the structural lifted update of Definition 8.10. Then*

$$\widetilde{\mathcal{R}}_t^{\mathrm{str}}\widetilde{\Phi}_t(\rho) = \rho$$

and

$$D\big(\widetilde{\Phi}_t(\rho)\|\widetilde{\Phi}_t(\sigma)\big) = D(\rho\|\sigma).$$

Consequently, $\widetilde{\Phi}_t$ has zero protected relative-entropy loss and zero recovery defect on every finite protected pair family satisfying the support condition.

Proof. For recovery,

$$\widetilde{\mathcal{R}}_t^{\mathrm{str}}\widetilde{\Phi}_t(\rho) = \mathrm{Tr}_{\mathcal{W}_t}\big[U_t^\dagger U_t(\rho \otimes |0\rangle\langle 0|)U_t^\dagger U_t\big] = \mathrm{Tr}_{\mathcal{W}_t}(\rho \otimes |0\rangle\langle 0|) = \rho.$$

For relative entropy, unitary invariance gives

$$D\big(U_t(\rho \otimes |0\rangle\langle 0|)U_t^\dagger \| U_t(\sigma \otimes |0\rangle\langle 0|)U_t^\dagger\big) = D(\rho \otimes |0\rangle\langle 0| \| \sigma \otimes |0\rangle\langle 0|).$$

Additivity of relative entropy under tensoring both states with the same auxiliary state gives

$$D(\rho \otimes |0\rangle\langle 0| \| \sigma \otimes |0\rangle\langle 0|) = D(\rho\|\sigma) + D(|0\rangle\langle 0| \| |0\rangle\langle 0|) = D(\rho\|\sigma).$$

Thus no protected distinguishability is lost and the predecessor state is exactly recoverable. □

Interpretation 8.12. *Structural recovery makes non-loss literal at the meta-state level. The update may create new correlations or new workspace content, but the predecessor state remains recoverable by reversing the unitary and tracing out the workspace.*

Limitation 8.13 (Structural recovery is a sufficient class, not all RSI)**.** *Not every implementable self-update is a reversible structural update. This theorem supplies a clean sufficient condition for exact non-loss. Updates outside this class still require the earlier relative-entropy and recovery certificates.*

8.3 Conservative tensor extension

Definition 8.14 (Conservative extension update)**.** *Let $V : \mathcal{H}_t \to \mathcal{H}_{t+1}$ be an isometry, so $V^\dagger V = I_{\mathcal{H}_t}$. Let $\alpha_\nu \in \mathcal{D}(\mathcal{H}_\nu^{\mathrm{new}})$ be an added module, memory, tool, verifier, search routine, dataset summary, or other extension state that is independent of the protected pair member. The conservative extension update is*

$$\boxed{\Phi_\nu(\rho) = V\rho V^\dagger \otimes \alpha_\nu.}$$

The extension is conservative because it embeds the predecessor state isometrically and appends new structure without overwriting the predecessor subsystem.

Definition 8.15 (Conservative extension recovery)**.** *Let $P = VV^\dagger$ be the projection onto the image of V, and fix any fallback state $\tau_0 \in \mathcal{D}(\mathcal{H}_t)$. A CPTP recovery completion for arbitrary successor states $\omega \in \mathcal{D}(\mathcal{H}_{t+1} \otimes \mathcal{H}_\nu^{\mathrm{new}})$ is*

$$\mathcal{R}_\nu^{\mathrm{ext}}(\omega) = V^\dagger \operatorname{Tr}_\nu\big[(P \otimes I)\omega(P \otimes I)\big]V + \operatorname{Tr}\big[(P^\perp \otimes I)\omega\big]\tau_0,$$

where $P^\perp = I - P$ and Tr_ν traces out the appended extension register. On the image of Φ_ν, this recovery reduces to the exact inverse of the isometric embedding followed by discarding the appended extension state.

Theorem 8.16 (Conservative extension exact non-loss and recovery)**.** *Let $\Phi_\nu(\rho) = V\rho V^\dagger \otimes \alpha_\nu$ be a conservative extension update. For all $\rho, \sigma \in \mathcal{D}(\mathcal{H}_t)$ with $D(\rho\|\sigma) < \infty$,*

$$D\big(\Phi_\nu(\rho)\|\Phi_\nu(\sigma)\big) = D(\rho\|\sigma).$$

Moreover,

$$\mathcal{R}_\nu^{\mathrm{ext}}\Phi_\nu(\rho) = \rho.$$

Thus Φ_ν is exactly non-lossy and exactly constructively recoverable on every protected pair family for which the same α_ν is appended to both pair members.

Proof. Isometric invariance of relative entropy gives

$$D(V\rho V^\dagger\|V\sigma V^\dagger) = D(\rho\|\sigma)$$

on the image of V. Additivity under tensoring both states with the same α_ν gives

$$D(V\rho V^\dagger \otimes \alpha_\nu\|V\sigma V^\dagger \otimes \alpha_\nu) = D(V\rho V^\dagger\|V\sigma V^\dagger) + D(\alpha_\nu\|\alpha_\nu) = D(\rho\|\sigma).$$

For recovery, $\Phi_\nu(\rho)$ lies entirely in the subspace $P \otimes I$, so the fallback term vanishes. Therefore

$$\mathcal{R}_\nu^{\mathrm{ext}}\Phi_\nu(\rho) = V^\dagger \operatorname{Tr}_\nu(V\rho V^\dagger \otimes \alpha_\nu)V = V^\dagger V\rho V^\dagger V \operatorname{Tr}(\alpha_\nu) = \rho.$$

□

Limitation 8.17 (Pair-independent extension state)**.** *The theorem requires the appended state α_ν to be the same for the protected pair members. If the extension state depends on whether the input is ρ or σ, relative entropy may change and must be analyzed by the ordinary loss/recovery gate.*

8.4 Ability preservation by recoverable predecessor access

Assumption 8.18 (Callable predecessor fallback)**.** *For an accepted structural or conservative extension update, the successor system has a certified callable predecessor interface. Formally, for every predecessor ability certificate* $\mathsf{AbCert}_{t,a}(\rho_t, R_t)$*, there is a transported successor certificate* $\mathsf{AbCert}_{t+1,\Theta_t^{\mathfrak{A}}(a)}(\rho_{t+1}, R_{t+1})$ *that runs the predecessor routine through either the structural recovery map, the isometric embedded subsystem, or an explicitly certified callable fallback. The resource budget* R_{t+1} *must cover the transported routine and its interface overhead.*

Theorem 8.19 (Ability preservation from structural recovery)**.** *Assume ability-certificate soundness, cross-time ability transport, and callable predecessor fallback. Let* Φ_t *be an accepted structural recovery update or conservative extension update with successor* $\rho_{t+1} = \Phi_t(\rho_t)$*. Then*

$$\Theta_t^{\mathfrak{A}}\big(\mathfrak{A}_t(\rho_t, R_t)\big) \subseteq \mathfrak{A}_{t+1}(\rho_{t+1}, R_{t+1}).$$

Thus structural recovery or conservative extension supplies a sufficient condition for non-lossy ability preservation.

Proof. Let $a \in \mathfrak{A}_t(\rho_t, R_t)$. By definition of the certified ability set, $\mathsf{AbCert}_{t,a}(\rho_t, R_t)$ is valid. Callable predecessor fallback says that this certificate transports to a valid successor certificate for $\Theta_t^{\mathfrak{A}}(a)$ at (ρ_{t+1}, R_{t+1}). Therefore $\Theta_t^{\mathfrak{A}}(a) \in \mathfrak{A}_{t+1}(\rho_{t+1}, R_{t+1})$. Since a was arbitrary, the inclusion follows. □

Definition 8.20 (Certified new ability supplied by an extension)**.** *A conservative extension* α_ν *supplies a certified new ability at time* $t+1$ *when there exists*

$$b \in \mathfrak{A}_{t+1}(\rho_{t+1}, R_{t+1})$$

such that

$$b \notin \Theta_t^{\mathfrak{A}}\big(\mathfrak{A}_t(\rho_t, R_t)\big).$$

The certificate for b *must identify the extension component, routine, verifier, resource cost, and audit data used to establish the ability. A mere increase in a scalar score is not enough to establish a new ability unless it corresponds to a certified ability element* b*.*

Theorem 8.21 (Strict RSI progress from conservative extension)**.** *Assume the hypotheses of Theorem 8.19. If, in addition, the extension supplies a certified new ability* b *in the sense of Definition 8.20, then the update exhibits strict RSI ability progress:*

$$\Theta_t^{\mathfrak{A}}\big(\mathfrak{A}_t(\rho_t, R_t)\big) \subsetneq \mathfrak{A}_{t+1}(\rho_{t+1}, R_{t+1}).$$

Proof. Theorem 8.19 gives the subset inclusion. The additional certified ability b belongs to the successor ability set but not to the transported predecessor ability set. Therefore the inclusion is proper. □

Interpretation 8.22 (Why Batch 2 matters)**.** *Ability-set progress and structural recovery change the logical status of progress. Instead of treating progress as only a scalar* SafeCap *gain, the framework now requires monotone preservation of certified abilities and represents strict RSI progress as a proper expansion of that set. Instead of treating recovery as only an external certificate, it introduces update classes whose algebraic form gives exact recovery and exact protected distinguishability preservation.*

Limitation 8.23 (Still not the full engine theorem)**.** *Batch 2 does not prove that a certified new ability is always available, that a conservative extension can always be found within resource budget, or that strict progress persists indefinitely. It supplies the progress order and structural non-loss primitives that Batch 3 composes into a certified non-lossy update algebra. Batch 4 supplies a local tangent-cone progress theorem. Batch 5 supplies a conservative-extension engine-viability kernel and a nonemptiness theorem for certified conservative-extension plans. A concrete architecture must still produce such plans or tangent directions rather than merely postulate them.*

9 Conservative-Extension Algebra of Certified Non-Lossy Updates

This section implements Edit Batch 3. Batch 2 supplied two exact non-loss primitives: reversible structural updates and conservative tensor extensions. The present section turns those primitives into an algebra of certified non-lossy update moves. The algebra does not make arbitrary self-modification non-lossy. It defines a restricted compositional class whose members carry explicit pair-transport, recovery, resource, and certificate data. Closure under composition then follows by elementary telescoping of relative entropy and by contractivity of the recovery maps.

Limitation 9.1 (Algebraic closure is not useful-progress existence)**.** *The results in this section show that certified non-lossy primitives compose and that their loss and recovery budgets add. They do not prove that any primitive supplies a new useful ability, that a useful primitive is resource-feasible, or that an engine can always find one. Those are the later tangent-cone, search, and viability questions.*

9.1 Primitive classes and certificates

Definition 9.2 (Primitive generator labels)**.** *The primitive labels used in the non-lossy update algebra are*

$$\text{id}, \qquad \text{IsoExt}, \qquad \text{AppendMem}, \qquad \text{RevAdapter}, \qquad \text{VerifierExt}, \qquad \text{ToolExt}.$$

Their intended meanings are: identity/no-op; isometric extension of the predecessor state; append-only memory extension; reversible adapter insertion; verifier extension; and tool or module extension. A label is not itself a proof of non-loss. A concrete primitive belongs to the corresponding class only when it supplies the non-loss and recovery certificate defined below.

Definition 9.3 (Protected-pair pushforward)**.** *Let $\Phi : \mathcal{D}(\mathcal{H}) \to \mathcal{D}(\mathcal{H}')$ be a candidate update and let $\mathcal{P} \subseteq \mathcal{D}(\mathcal{H}) \times \mathcal{D}(\mathcal{H})$ be a protected pair set. The pushed-forward protected pair set is*

$$\Phi_{\#}\mathcal{P} = \{(\Phi(\rho), \Phi(\sigma)) : (\rho, \sigma) \in \mathcal{P}\} \subseteq \mathcal{D}(\mathcal{H}') \times \mathcal{D}(\mathcal{H}').$$

When the successor paper or architecture uses an enriched protected-pair set $\mathcal{P}'$, the certificate must state that

$$\Phi_{\#}\mathcal{P} \subseteq \mathcal{P}'$$

up to a declared approximation or transport error. Without such a transport certificate, no cross-step protected-pair claim is made.

Definition 9.4 (Certified non-lossy update primitive)**.** *Fix a protected pair set $\mathcal{P}$. A candidate update $\Phi : \mathcal{D}(\mathcal{H}) \to \mathcal{D}(\mathcal{H}')$ is an (ϵ_Φ, r_Φ)-certified non-lossy primitive relative to $\mathcal{P}$ when the certificate packet supplies:*

(i) *a type and domain certificate showing that Φ maps the audited density-state domain into $\mathcal{D}(\mathcal{H}')$;*

(ii) *a protected-pair transport certificate for $\Phi_{\#}\mathcal{P}$;*

(iii) *a relative-entropy loss certificate*

$$\sup_{(\rho,\sigma)\in\mathcal{P}} [D(\rho\|\sigma) - D(\Phi(\rho)\|\Phi(\sigma))]_+ \leq \epsilon_\Phi;$$

(iv) *a recovery map $\mathcal{R}_\Phi : \mathcal{D}(\mathcal{H}') \to \mathcal{D}(\mathcal{H})$ satisfying*

$$\sup_{(\rho,\sigma)\in\mathcal{P}} \max\{d_{\mathrm{tr}}(\rho, \mathcal{R}_\Phi\Phi(\rho)), d_{\mathrm{tr}}(\sigma, \mathcal{R}_\Phi\Phi(\sigma))\} \leq r_\Phi;$$

(v) *a resource and ledger certificate recording the cost, rollback data, and successor bookkeeping needed by the operational RCP gate.*

If $\epsilon_\Phi = r_\Phi = 0$, the primitive is exactly certified non-lossy.

Remark 9.5 (How Batch 2 primitives enter)**.** *The structural lifted update of Definition 8.10 and the conservative extension update of Definition 8.14 are exactly certified non-lossy primitives when their domain, type, and resource certificates are present. Theorems 8.11 and 8.16 supply the exact loss and recovery parts of their certificates.*

Definition 9.6 (Primitive class instances)**.** *A concrete primitive instance belongs to one of the primitive classes as follows.*

(a) id *is the identity update with $\epsilon = r = 0$.*

(b) IsoExt *is any certified isometric or conservative tensor extension.*

(c) AppendMem *is an append-only memory update implemented as a conservative extension of the predecessor state plus a pair-independent memory state or a certified approximately recoverable memory extension.*

(d) RevAdapter *is a reversible adapter insertion implemented by a reversible structural update on an enlarged meta-state, or by an approximately recoverable adapter certificate.*

(e) VerifierExt *is a verifier extension that preserves the predecessor verifier as a callable subsystem and appends a new verifier component, subject to verifier-trust and no self-sole-certification gates.*

(f) ToolExt *is a tool, proof routine, search routine, dataset summary, or external module extension implemented as a conservative or certified recoverable extension.*

Each item is a class schema. A proposed member is rejected unless it satisfies Definition 9.4 on the declared protected pair set and all applicable operational-closure gates.

9.2 The certified non-lossy update algebra

Definition 9.7 (Certified non-lossy update algebra)**.** *The certified non-lossy update algebra is the compositional closure of the primitive classes:*

$$\boxed{\mathfrak{G}^{\mathrm{NL}} = \langle \mathrm{id}, \mathrm{IsoExt}, \mathrm{AppendMem}, \mathrm{RevAdapter}, \mathrm{VerifierExt}, \mathrm{ToolExt} \rangle .}$$

An element $\Phi \in \mathfrak{G}^{\mathrm{NL}}$ is a finite composable word

$$\Phi = \Phi_n \circ \Phi_{n-1} \circ \cdots \circ \Phi_1,$$

where each Φ_i is a certified non-lossy primitive on the protected pair set transported from the previous stage. The empty word is the identity update. Because the identity is included, $\mathfrak{G}^{\mathrm{NL}}$ is technically a monoid; we keep the word algebra because the RSI interpretation is compositional generation of update moves.

Definition 9.8 (Word-level transported pair sets and composed recovery)**.** *Let $\Phi_i : \mathcal{D}(\mathcal{H}_{i-1}) \to \mathcal{D}(\mathcal{H}_i)$ be composable certified primitives, and set*

$$\Phi_{1:i} = \Phi_i \circ \cdots \circ \Phi_1, \qquad \mathcal{P}_i = (\Phi_{1:i})_{\#}\mathcal{P}_0.$$

If $\mathcal{R}_i : \mathcal{D}(\mathcal{H}_i) \to \mathcal{D}(\mathcal{H}_{i-1})$ is the recovery map certified for Φ_i on $\mathcal{P}_{i-1}$, the word-level recovery map is

$$\mathcal{R}_{1:n} = \mathcal{R}_1 \circ \mathcal{R}_2 \circ \cdots \circ \mathcal{R}_n.$$

The order is reversed relative to the forward composition, as required for recovery.

Assumption 9.9 (Composability and transport validity)**.** *For every word $\Phi_n \circ \cdots \circ \Phi_1$ claimed to belong to $\mathfrak{G}^{\mathrm{NL}}$, the output type and Hilbert space of Φ_i match the input type and Hilbert space of Φ_{i+1}, and the transported pair set $\mathcal{P}_i$ lies inside the audited domain on which Φ_{i+1}'s loss and recovery certificates are valid. If approximate pair transport is used, its error is included in the corresponding ϵ_i and r_i budgets.*

Lemma 9.10 (Two-step additive relative-entropy loss)**.** *Let $\Phi : \mathcal{D}(\mathcal{H}_0) \to \mathcal{D}(\mathcal{H}_1)$ be ϵ_Φ-non-lossy on $\mathcal{P}_0$, and let $\Psi : \mathcal{D}(\mathcal{H}_1) \to \mathcal{D}(\mathcal{H}_2)$ be ϵ_Ψ-non-lossy on $\Phi_{\#}\mathcal{P}_0$. Then*

$$\mathcal{L}^{\mathcal{P}_0}_{\Psi\circ\Phi} \le \epsilon_\Phi + \epsilon_\Psi.$$

Proof. Fix $(\rho, \sigma) \in \mathcal{P}_0$. Write

$$\begin{aligned} &D(\rho\|\sigma) - D(\Psi\Phi(\rho)\|\Psi\Phi(\sigma)) \\ &= \big[D(\rho\|\sigma) - D(\Phi(\rho)\|\Phi(\sigma))\big] + \big[D(\Phi(\rho)\|\Phi(\sigma)) - D(\Psi\Phi(\rho)\|\Psi\Phi(\sigma))\big]. \end{aligned}$$

The first bracket is at most ϵ_Φ by the certificate for Φ. The second is at most ϵ_Ψ because $(\Phi(\rho), \Phi(\sigma)) \in \Phi_{\#}\mathcal{P}_0$, the domain on which Ψ's certificate is valid. Taking positive parts and then the supremum over $\mathcal{P}_0$ gives the claim. □

Lemma 9.11 (Two-step additive recovery defect)**.** *Let Φ and Ψ be as in Lemma 9.10. Suppose Φ has recovery map $\mathcal{R}_\Phi$ with defect at most r_Φ on $\mathcal{P}_0$, and Ψ has recovery map $\mathcal{R}_\Psi$ with defect at most r_Ψ on $\Phi_{\#}\mathcal{P}_0$. Then the composed recovery map*

$$\mathcal{R}_{\Phi,\Psi} = \mathcal{R}_\Phi \circ \mathcal{R}_\Psi$$

recovers $\Psi \circ \Phi$ with defect at most

$$r_\Phi + r_\Psi$$

on $\mathcal{P}_0$.

Proof. It suffices to prove the bound for an arbitrary first component ρ appearing in a protected pair; the same argument applies to σ. By the triangle inequality,

$$\begin{aligned} &d_{\mathrm{tr}}\big(\rho, \mathcal{R}_\Phi \mathcal{R}_\Psi \Psi\Phi(\rho)\big) \\ &\le d_{\mathrm{tr}}\big(\rho, \mathcal{R}_\Phi \Phi(\rho)\big) + d_{\mathrm{tr}}\big(\mathcal{R}_\Phi \Phi(\rho), \mathcal{R}_\Phi \mathcal{R}_\Psi \Psi\Phi(\rho)\big). \end{aligned}$$

The first term is at most r_Φ. Since $\mathcal{R}_\Phi$ is a recovery channel, trace distance is contractive under $\mathcal{R}_\Phi$, so the second term is at most

$$d_{\mathrm{tr}}\big(\Phi(\rho), \mathcal{R}_\Psi \Psi\Phi(\rho)\big) \le r_\Psi.$$

Thus the composed recovery defect is at most $r_\Phi + r_\Psi$. Taking the maximum over both members of every protected pair and then the supremum over $\mathcal{P}_0$ gives the stated bound. □

Theorem 9.12 (Conservative-extension algebra closure)**.** *Assume composability and transport validity. If*

$$\Phi, \Psi \in \mathfrak{G}^{\mathrm{NL}}$$

are composable certified non-lossy updates, then

$$\boxed{\Psi \circ \Phi \in \mathfrak{G}^{\mathrm{NL}}.}$$

Moreover, if Φ and Ψ have loss and recovery budgets (ϵ_Φ, r_Φ) and (ϵ_Ψ, r_Ψ) on the transported protected pair sets, then $\Psi \circ \Phi$ has the certified budgets

$$\epsilon_{\Psi\circ\Phi} \le \epsilon_\Phi + \epsilon_\Psi, \qquad r_{\Psi\circ\Phi} \le r_\Phi + r_\Psi.$$

Proof. By Definition 9.7, $\mathfrak{G}^{\mathrm{NL}}$ contains finite composable words in the certified primitive classes. The composition of two finite words is their concatenation, which is again a finite word in the same primitive classes. Thus $\Psi \circ \Phi \in \mathfrak{G}^{\mathrm{NL}}$, provided the domains and transported pair sets are valid as required by Assumption 9.9. The two-step additive loss bound is Lemma 9.10, and the two-step additive recovery bound is Lemma 9.11. Applying those lemmas to the terminal representative of the two words gives the displayed budgets. □

Theorem 9.13 (Finite-word additive budget theorem)**.** *Let*

$$\Phi_{1:n} = \Phi_n \circ \cdots \circ \Phi_1$$

be a word in $\mathfrak{G}^{\mathrm{NL}}$ satisfying composability and transport validity at every intermediate step. Suppose each primitive Φ_i is certified on $\mathcal{P}_{i-1}$ with loss budget ϵ_i and recovery defect r_i. Then $\Phi_{1:n}$ is certified non-lossy on $\mathcal{P}_0$ with

$$\boxed{\epsilon_{\mathrm{total}} = \sum_{i=1}^{n} \epsilon_i, \qquad r_{\mathrm{total}} = \sum_{i=1}^{n} r_i.}$$

Equivalently,

$$\mathcal{L}^{\mathcal{P}_0}_{\Phi_{1:n}} \le \sum_{i=1}^{n} \epsilon_i,$$

and the composed recovery map $\mathcal{R}_{1:n}$ of Definition 9.8 has recovery defect at most $\sum_{i=1}^{n} r_i$ on $\mathcal{P}_0$.

Proof. Induct on the word length. The case $n = 0$ is the identity update with zero budgets. The case $n = 1$ is the primitive certificate. Suppose the claim holds for words of length $n-1$. Let $\Phi_{1:n} = \Phi_n \circ \Phi_{1:n-1}$. The induction hypothesis certifies $\Phi_{1:n-1}$ with budgets $\sum_{i=1}^{n-1} \epsilon_i$ and $\sum_{i=1}^{n-1} r_i$. Applying Theorem 9.12 to $\Phi_{1:n-1}$ and Φ_n gives budgets

$$\sum_{i=1}^{n-1} \epsilon_i + \epsilon_n, \qquad \sum_{i=1}^{n-1} r_i + r_n.$$

This proves the result for length n, and induction completes the proof. □

9.3 Algebraic engine-admissible update class

Definition 9.14 (Algebraically non-lossy RSI candidate class)**.** *At time t, define*

$$\Omega_t^{\mathrm{NL}}(\rho_t) = \left\{ \Phi \in \mathfrak{G}^{\mathrm{NL}} : \begin{array}{l} \Phi \text{ is type-valid and resource-audited,} \\ \Phi \text{ is composable from the current state,} \\ \mathcal{L}_\Phi^{\mathcal{P}_t} \le \epsilon_t^{\mathrm{alg}}, \\ \mathrm{Rec}_\Phi^{\mathcal{P}_t} \le r_t^{\mathrm{alg}}, \\ \text{all operational-closure residuals required by } \mathcal{A}_{\mathrm{RCP}}^{\mathrm{op}} \\ \text{are certified} \end{array} \right\}.$$

Here $\epsilon_t^{\mathrm{alg}}$ and r_t^{alg} are the total algebra budgets computed from Theorem 9.13. Thus Ω_t^{NL} is not an arbitrary candidate class. It is the set of candidate self-updates generated by certified non-lossy primitive compositions whose total budgets fit inside the current RCP gate.

Proposition 9.15 (Algebraic candidates satisfy the non-loss and recovery parts of the gate)**.** *If $\Phi \in \Omega_t^{\mathrm{NL}}(\rho_t)$, then the relative-entropy non-loss and constructive-recovery components of the RCP admissibility gate are satisfied with budgets $\epsilon_t^{\mathrm{alg}}$ and r_t^{alg}. If these algebraic budgets obey*

$$\epsilon_t^{\mathrm{alg}} \le \epsilon_t, \qquad r_t^{\mathrm{alg}} \le r_t,$$

then the corresponding RCP loss and recovery thresholds are satisfied.

Proof. Membership in Ω_t^{NL} includes a word representation in $\mathfrak{G}^{\mathrm{NL}}$ whose total loss and recovery budgets are computed by Theorem 9.13. Therefore the non-loss and recovery inequalities hold with $\epsilon_t^{\mathrm{alg}}$ and r_t^{alg}. If those totals are at most the gate thresholds ϵ_t and r_t, then the RCP components are satisfied. □

Interpretation 9.16 (Why Batch 3 matters)**.** *Batch 3 changes the update-generation picture. The first direct-engine candidate class should not be arbitrary self-editing followed by difficult after-the-fact verification. It should begin with finite words in a certified non-lossy algebra. For those words, non-loss and recoverability are inherited compositionally, with explicit additive budgets. Batch 4 then supplies the local geometry needed to search for progress directions within such certified candidate charts.*

Limitation 9.17 (The algebra is conservative)**.** *The algebra may exclude useful self-edits that are non-lossy but not expressible as the listed primitive compositions. This is intentional for the first engine construction: a restricted exact or certified non-loss class is preferable to arbitrary self-modification whose recovery and loss properties are unknown. Later work may enlarge $\mathfrak{G}^{\mathrm{NL}}$ by adding new primitive classes, but each new primitive must supply the same type, pair-transport, non-loss, recovery, resource, and verifier certificates.*

10 Tangent-Cone Local Progress Theorem

This section implements Edit Batch 4. Batches 2 and 3 supplied a certified ability-set progress order, structural recovery, and a closed algebra of certified non-lossy update primitives. The remaining local-engine question is not whether a proposed update can be checked after it is found, but whether a beneficial admissible direction can be derived from the local geometry of the gate. The tangent-cone theorem below gives a conditional sufficient condition: if the projected engine-potential gradient has a strictly inward component inside the non-lossy admissible tangent cone, then a sufficiently small finite update is admissible and produces positive local engine-potential progress.

Limitation 10.1 (Local theorem, not global RSI)**.** *The tangent-cone theorem is a local existence theorem. It does not prove that every state has an inward progress direction, that useful directions persist indefinitely, that a finite-dimensional smooth chart covers arbitrary architecture rewrites, or that scalar engine-potential progress is identical to certified ability-set expansion. It supplies the next missing local bridge from a no-op point to a nontrivial admissible update when differentiability, curvature, certificate, and strict-inwardness assumptions hold.*

10.1 Local candidate charts and residual geometry

Definition 10.2 (Local non-lossy candidate chart)**.** *Fix a state ρ_t. A local non-lossy candidate chart is a finite-dimensional real Hilbert parameter space $(\mathbb{V}_t, \langle\cdot,\cdot\rangle_t)$, a radius $a_t^{\mathrm{chart}} > 0$, an open or star-shaped neighborhood*

$$\mathcal{U}_t \subseteq \mathbb{V}_t, \qquad 0 \in \mathcal{U}_t, \qquad \{\alpha v : 0 \le \alpha \le 1,\ v \in \mathcal{U}_t\} \subseteq \mathcal{U}_t,$$

and a map

$$\nu \in \mathcal{U}_t \longmapsto \Phi_{t,\nu}$$

such that:

(i) $\Phi_{t,0} = \mathrm{id}$ *on the audited state family;*

(ii) for every $\nu \in \mathcal{U}_t$, $\Phi_{t,\nu} \in \Omega_t^{\mathrm{NL}}(\rho_t)$ or else has a certified projection into $\Omega_t^{\mathrm{NL}}(\rho_t)$ with the projection distortion included in the residuals;

(iii) the induced residuals and engine-potential functional below are differentiable at $\nu = 0$ and satisfy the stated curvature bounds on $\{\nu : \|\nu\|_t \le a_t^{\mathrm{chart}}\} \cap \mathcal{U}_t$.

The chart restricts the local theorem to updates generated by the certified non-lossy algebra or by certified local deformations of that algebra. It is not a chart over arbitrary self-edits.

Definition 10.3 (Local RCP residuals)**.** *Let J_t be a finite residual index set. For each $j \in J_t$, define a scalar residual*

$$q_{t,j}(\nu; \rho_t) \in \mathbb{R},$$

with the sign convention

$$q_{t,j}(\nu; \rho_t) \le 0 \quad \Longrightarrow \quad \text{the } j\text{th local RCP/NL-RSI constraint is satisfied by } \Phi_{t,\nu}.$$

The residual vector may include algebraic non-loss budget, recovery budget, Lyapunov drift, barrier, ambiguity, information-bottleneck, type, coupling, certificate, trust, coverage, calibration, estimator, resource, inflation, and ability-preservation residuals. Define the local admissible set in the chart by

$$\mathcal{A}_t^{\mathrm{loc}}(\rho_t) = \{\nu \in \mathcal{U}_t : q_{t,j}(\nu; \rho_t) \le 0 \ \forall j \in J_t\}.$$

Definition 10.4 (Active and inactive residuals at no-op). *Assume the no-op is feasible:*

$$q_{t,j}(0;\rho_t) \le 0 \qquad \forall j \in J_t.$$

The active and inactive index sets are

$$J_t^0 = \{j \in J_t : q_{t,j}(0;\rho_t) = 0\}, \qquad J_t^- = \{j \in J_t : q_{t,j}(0;\rho_t) < 0\}.$$

For $j \in J_t^-$, define the no-op slack

$$s_{t,j} = -q_{t,j}(0;\rho_t) > 0.$$

Definition 10.5 (Admissible tangent cone). *Assume each residual is Fréchet differentiable at 0. The first-order local admissible tangent cone is*

$$\boxed{T_{\mathcal{A}_t}(\rho_t) = \{v \in \mathbb{V}_t : Dq_{t,j}(0;\rho_t)[v] \le 0 \text{ for every } j \in J_t^0\}.}$$

This cone is a first-order object. Membership in $T_{\mathcal{A}_t}(\rho_t)$ alone does not prove finite-step feasibility unless additional curvature and strict-inwardness conditions hold.

Definition 10.6 (Strict inward tangent direction). *A vector $v \in T_{\mathcal{A}_t}(\rho_t)$ is μ_t-strictly inward for the active constraints if $\mu_t > 0$ and*

$$Dq_{t,j}(0;\rho_t)[v] \le -\mu_t \qquad \forall j \in J_t^0.$$

When $J_t^0 = \emptyset$, strict inwardness is vacuous and only the inactive slack constraints need to be preserved for small finite steps.

Definition 10.7 (RSI engine-potential functional). *A local RSI engine-potential functional is a scalar map*

$$\mathrm{RSIPot}_t(\nu;\rho_t,R_t)$$

defined on $\mathcal{U}_t$. It is intended to be an audited differentiable proxy for local non-lossy improvement capacity. Examples may combine certified ability-set expansion score, expected future non-lossy update capacity, verifier/search-power gain, resource return, and penalties for loss, recovery error, cost, and certificate inflation. A positive change in RSIPot_t is not by itself a proof of a new certified ability unless a separate ability certificate is supplied. It is the differentiable search objective used by the local theorem.

Definition 10.8 (Projected tangent-cone gradient). *Assume $T_{\mathcal{A}_t}(\rho_t)$ is closed and convex. Let*

$$g_t = \nabla_\nu \mathrm{RSIPot}_t(0;\rho_t,R_t) \in \mathbb{V}_t$$

be the Riesz gradient in the chart. The projected admissible progress direction is

$$\boxed{P_{T_{\mathcal{A}_t}(\rho_t)} \nabla_\nu \mathrm{RSIPot}_t(0;\rho_t,R_t)}$$

where P_C denotes Euclidean projection onto the closed convex cone C. In the architecture paper, this object is instantiated as

$$P_{T_{\mathcal{A}_t^{\mathrm{RCLM}}}} \nabla_\nu \mathrm{RSIPot}_t$$

with residuals $q_{t,j}^{\mathrm{RCLM}}(\nu) \le 0$.

Lemma 10.9 (Projection identity for a closed convex cone). *Let $C \subseteq \mathbb{V}_t$ be a closed convex cone and let $w = P_C g$. Then*

$$\langle g, w\rangle_t = \|w\|_t^2.$$

Consequently, if $w \neq 0$, the directional derivative of the linear functional $\nu \mapsto \langle g, \nu\rangle_t$ in direction w is strictly positive.

Proof. The variational characterization of projection gives

$$\langle g - w, y - w\rangle_t \leq 0 \qquad \forall y \in C.$$

Taking $y = 0 \in C$ gives $\langle g-w, -w\rangle_t \leq 0$, hence $\langle g-w, w\rangle_t \geq 0$. Taking $y = 2w \in C$, because C is a cone and $w \in C$, gives $\langle g - w, w\rangle_t \leq 0$. Therefore $\langle g - w, w\rangle_t = 0$, so $\langle g, w\rangle_t = \langle w, w\rangle_t = \|w\|_t^2$. □

10.2 Curvature assumptions and finite-step feasibility

Assumption 10.10 (Residual upper-curvature bounds). *There exist constants $L_{t,j}^q \geq 0$ and a chart radius $a_t^{\mathrm{chart}} > 0$ such that, for every $\nu \in \mathcal{U}_t$ with $\|\nu\|_t \leq a_t^{\mathrm{chart}}$,*

$$q_{t,j}(\nu; \rho_t) \leq q_{t,j}(0; \rho_t) + Dq_{t,j}(0; \rho_t)[\nu] + \frac{1}{2} L_{t,j}^q \|\nu\|_t^2 \qquad \forall j \in J_t.$$

This is an audited local smoothness condition on the residual model. It is not implied by differentiability alone on an unbounded or nonsmooth candidate class.

Assumption 10.11 (Engine-potential lower-curvature bound). *There exists $L_t^G \geq 0$ such that, for every $\nu \in \mathcal{U}_t$ with $\|\nu\|_t \leq a_t^{\mathrm{chart}}$,*

$$\mathrm{RSIPot}_t(\nu; \rho_t, R_t) \geq \mathrm{RSIPot}_t(0; \rho_t, R_t) + D\,\mathrm{RSIPot}_t(0; \rho_t, R_t)[\nu] - \frac{1}{2} L_t^G \|\nu\|_t^2.$$

This is a one-sided lower Taylor bound for the audited local progress objective.

Definition 10.12 (Certified finite-step radius). *For a direction $v \in \mathbb{V}_t$, define $\alpha_t^{\mathrm{feas}}(v)$ as the supremum of all $\alpha \in [0, a_t^{\mathrm{chart}}]$ such that, for every $\beta \in [0, \alpha]$ and every $j \in J_t$,*

$$q_{t,j}(0; \rho_t) + \beta Dq_{t,j}(0; \rho_t)[v] + \frac{1}{2} L_{t,j}^q \beta^2 \|v\|_t^2 \leq 0.$$

If v is strictly inward for all active constraints and all inactive constraints have positive slack, then $\alpha_t^{\mathrm{feas}}(v) > 0$ by continuity of the finite family of upper-bound polynomials.

10.3 The local progress theorem

Theorem 10.13 (Tangent-cone local progress theorem). *Assume a local non-lossy candidate chart, no-op feasibility, residual upper-curvature bounds, and engine-potential lower-curvature bounds. Suppose there exists a vector $v_t \in \mathbb{V}_t$ and constants $\mu_t > 0$, $\gamma_t > 0$ such that:*

(i) v_t is μ_t-strictly inward for the active residuals;

(ii) $D\,\mathrm{RSIPot}_t(0; \rho_t, R_t)[v_t] \geq \gamma_t$;

(iii) $\alpha_t^{\mathrm{feas}}(v_t) > 0$.

Then for every step size

$$0 < \alpha_t \le \alpha_t^\star(v_t),$$

where

$$\alpha_t^\star(v_t) = \begin{cases} \min\left\{\alpha_t^{\mathrm{feas}}(v_t), \dfrac{2\gamma_t}{L_t^G\|v_t\|_t^2}\right\}, & L_t^G\|v_t\|_t^2 > 0, \\ \alpha_t^{\mathrm{feas}}(v_t), & L_t^G\|v_t\|_t^2 = 0, \end{cases}$$

the candidate $\nu_t = \alpha_t v_t$ *satisfies all local residual constraints*

$$q_{t,j}(\nu_t; \rho_t) \le 0 \qquad \forall j \in J_t,$$

and has strictly positive local engine-potential progress

$$\mathrm{RSIPot}_t(\nu_t; \rho_t, R_t) > \mathrm{RSIPot}_t(0; \rho_t, R_t).$$

Moreover, if the chart maps ν_t *into* $\Omega_t^{\mathrm{NL}}(\rho_t)$*, then the candidate inherits the algebraic non-loss and recovery budgets certified by the Batch 3 non-lossy update algebra.*

Proof. By the definition of $\alpha_t^{\mathrm{feas}}(v_t)$, every $\alpha_t \le \alpha_t^{\mathrm{feas}}(v_t)$ satisfies the upper-bound inequalities

$$q_{t,j}(0; \rho_t) + \alpha_t D q_{t,j}(0; \rho_t)[v_t] + \frac{1}{2} L_{t,j}^q \alpha_t^2 \|v_t\|_t^2 \le 0$$

for every $j \in J_t$. Assumption 10.10 then gives

$$q_{t,j}(\alpha_t v_t; \rho_t) \le 0 \qquad \forall j \in J_t.$$

Thus $\nu_t = \alpha_t v_t$ is locally admissible.

For progress, Assumption 10.11 gives

$$\mathrm{RSIPot}_t(\alpha_t v_t; \rho_t, R_t) - \mathrm{RSIPot}_t(0; \rho_t, R_t) \ge \alpha_t D\,\mathrm{RSIPot}_t(0; \rho_t, R_t)[v_t] - \frac{1}{2} L_t^G \alpha_t^2 \|v_t\|_t^2.$$

Using the directional derivative lower bound,

$$\mathrm{RSIPot}_t(\alpha_t v_t; \rho_t, R_t) - \mathrm{RSIPot}_t(0; \rho_t, R_t) \ge \alpha_t \gamma_t - \frac{1}{2} L_t^G \alpha_t^2 \|v_t\|_t^2.$$

The definition of $\alpha_t^\star$ makes the right-hand side positive for every nonzero $\alpha_t \le \alpha_t^\star$, with the zero-curvature case immediate. Finally, if the chart maps the candidate into Ω_t^{NL}, Proposition 9.15 gives the stated algebraic non-loss and recovery inheritance. □

Corollary 10.14 (Projected-gradient sufficient condition)**.** *Assume* $T_{\mathcal{A}_t}(\rho_t)$ *is a closed convex cone and let*

$$w_t = P_{T_{\mathcal{A}_t}(\rho_t)} \nabla_\nu \,\mathrm{RSIPot}_t(0; \rho_t, R_t).$$

If $w_t \neq 0$*,* w_t *is* μ_t*-strictly inward for the active residuals, and* $\alpha_t^{\mathrm{feas}}(w_t) > 0$*, then a sufficiently small step*

$$\nu_t = \alpha_t w_t$$

is locally admissible and has positive RSIPot_t *progress. In particular, the first-order progress coefficient is*

$$D\,\mathrm{RSIPot}_t(0; \rho_t, R_t)[w_t] = \|w_t\|_t^2 > 0.$$

Proof. By Lemma 10.9, the directional derivative in the direction w_t equals $\|w_t\|_t^2$, which is positive when $w_t \neq 0$. Apply Theorem 10.13 with $v_t = w_t$ and $\gamma_t = \|w_t\|_t^2$. □

Interpretation 10.15 (Why Batch 4 matters)**.** *Batch 4 identifies the exact local geometric condition needed to move from admissibility to construction: the engine-potential gradient must have a certified strictly feasible projection into the non-lossy admissible tangent cone. This is the mathematical core of the future engine step. It does not replace Batch 3's algebra; it uses the algebra as the local candidate chart so that small progress steps remain non-lossy and recoverable by construction.*

Limitation 10.16 (Ability progress still requires ability certificates)**.** *A positive* RSIPot_t *step is not automatically a new certified ability. Strict ability-set progress still requires the Batch 2 condition*

$$\mathfrak{A}_{t+1}(\rho_{t+1}, R_{t+1}) \setminus \Theta_t^{\mathfrak{A}}(\mathfrak{A}_t(\rho_t, R_t)) \neq \varnothing.$$

Batch 4 provides a differentiable local search route toward such progress. The ability certificate, resource certificate, and successor viability certificate remain separate obligations. Batch 5 supplies the conservative-extension engine-viability kernel that turns such one-step progress witnesses into recursive continuation witnesses when its nonemptiness assumptions are satisfied.

11 Engine Viability Nonemptiness for Conservative-Extension Systems

This section implements Edit Batch 5. Batches 2–4 supplied ability-set progress, structural recovery, a certified non-lossy update algebra, and a local tangent-cone route to finite admissible progress. A single admissible progress step is still not recursive self-improvement. The missing recursion condition is that after one accepted non-lossy extension, the successor remains inside a domain from which another certified non-lossy extension is available. This section gives the corresponding engine-viability nonemptiness theorem for conservative-extension systems.

Limitation 11.1 (Scope of the Batch 5 theorem)**.** *The theorem below does not prove that arbitrary systems can recursively self-improve. It proves a conditional nonemptiness result for states already inside a declared conservative-extension engine-viability kernel, and it proves that a certified conservative-extension path makes that kernel nonempty. It therefore closes the logical gap between one beneficial update and recursive continuation for a specified class of conservative-extension systems, but it does not remove the need to construct such systems or to prove that a given real architecture lies in the kernel.*

11.1 Engine base domain and conservative-extension engine moves

Definition 11.2 (Engine base domain)**.** *For each time* t*, let*

$$\mathcal{K}_t^{\mathrm{base}} \subseteq \mathcal{K}_{\mathrm{RCP}}^{\mathrm{state}}(t)$$

be the declared engine base domain. Membership in $\mathcal{K}_t^{\mathrm{base}}$ *means that all state-level objects needed by the engine theorem are defined and certified at time* t*: the typed density state, semantic coupling data, protected predicate family, certified ability set* $\mathfrak{A}_t(\rho, R_t)$*, resource state* R_t*, verifier state, recovery ledger, non-lossy update algebra, and local residual system. The inclusion in* $\mathcal{K}_{\mathrm{RCP}}^{\mathrm{state}}(t)$ *ensures that every engine-base state is already inside the preservation safe set.*

Definition 11.3 (Engine move residuals). *For $\rho \in \mathcal{K}_t^{\mathrm{base}}$ and $\nu \in \Omega_t^{\mathrm{NL}}(\rho)$, let*

$$Q_t^{\mathrm{eng}}(\nu;\rho) = (q_{t,j}^{\mathrm{eng}}(\nu;\rho))_{j\in J_t^{\mathrm{eng}}}$$

be the finite vector of residuals required for an engine move. These residuals include algebraic non-loss and recovery budgets, typed admissibility, semantic coupling, Lyapunov constraints, barrier constraints, ambiguity constraints, information-bottleneck constraints, resource feasibility, certificate soundness, verifier trust, coverage, calibration, estimator completeness, and ability-set preservation. The sign convention is

$$q_{t,j}^{\mathrm{eng}}(\nu;\rho) \le 0 \quad \forall j$$

if and only if the corresponding engine-move certificates are satisfied on the audited domain.

Definition 11.4 (Certified engine progress by ability expansion). *Let $\Theta_t^{\mathfrak{A}}$ be the cross-time ability-transport map from Batch 2. An engine move $\nu \in \Omega_t^{\mathrm{NL}}(\rho)$ has certified ability progress of threshold $g_t^{\mathfrak{A}} \ge 0$ when*

$$\Theta_t^{\mathfrak{A}}\big(\mathfrak{A}_t(\rho, R_t)\big) \subseteq \mathfrak{A}_{t+1}(\Phi_\nu(\rho), R_{t+1})$$

and the certified new-ability score satisfies

$$\Delta_t^{\mathfrak{A}}(\nu;\rho) := \mathrm{Score}_{t+1}^{\mathfrak{A}}\Big(\mathfrak{A}_{t+1}(\Phi_\nu(\rho), R_{t+1}) \setminus \Theta_t^{\mathfrak{A}}(\mathfrak{A}_t(\rho, R_t))\Big) \ge g_t^{\mathfrak{A}}.$$

When $g_t^{\mathfrak{A}} > 0$, the move is a strict certified ability-expanding move. When $g_t^{\mathfrak{A}} = 0$, it is ability-preserving but not necessarily strictly improving.

Definition 11.5 (Conservative-extension engine move). *A candidate ν is a conservative-extension engine move at (t,ρ) with progress threshold $g_t^{\mathfrak{A}}$ if:*

(i) $\nu \in \Omega_t^{\mathrm{NL}}(\rho)$, *so it is a finite certified word in the non-lossy algebra $\mathfrak{G}^{\mathrm{NL}}$;*

(ii) every engine move residual is nonpositive:

$$q_{t,j}^{\mathrm{eng}}(\nu;\rho) \le 0 \qquad \forall j \in J_t^{\mathrm{eng}};$$

(iii) the successor state

$$\rho^+ = \Phi_\nu(\rho)$$

belongs to $\mathcal{K}_{t+1}^{\mathrm{base}}$;

(iv) ν has certified ability progress of threshold $g_t^{\mathfrak{A}}$.

The set of such moves is denoted

$$\mathcal{E}_t^{\mathrm{CE}}(\rho; g_t^{\mathfrak{A}}) \subseteq \Omega_t^{\mathrm{NL}}(\rho).$$

Remark 11.6 (Why the successor condition is explicit). *The requirement $\Phi_\nu(\rho) \in \mathcal{K}_{t+1}^{\mathrm{base}}$ is not redundant. A conservative extension can preserve protected predecessor information while still failing some successor-domain requirement, such as resource availability, verifier trust support, semantic coupling, or the ability to form the next candidate chart. Engine viability requires both non-loss of the predecessor and continuation of the future update machinery.*

11.2 Finite-horizon engine viability kernels

Definition 11.7 (Finite-horizon conservative-extension engine viability)**.** *Fix a horizon N and thresholds $g_t^{\mathfrak{A}} \geq 0$. Define the finite-horizon conservative-extension engine viability sets by backward recursion:*

$$\mathcal{V}_N^{\mathrm{engine},N} = \mathcal{K}_N^{\mathrm{base}},$$

and for $t = N-1, \ldots, 0$,

$$\boxed{\mathcal{V}_t^{\mathrm{engine},N} = \left\{ \rho \in \mathcal{K}_t^{\mathrm{base}} : \exists \nu \in \mathcal{E}_t^{\mathrm{CE}}(\rho; g_t^{\mathfrak{A}}) \text{ such that } \Phi_\nu(\rho) \in \mathcal{V}_{t+1}^{\mathrm{engine},N} \right\}.}$$

When the horizon is clear, write $\mathcal{V}_t^{\mathrm{engine}}$ for the chosen finite-horizon or infinite-horizon engine kernel.

Theorem 11.8 (Engine viability nonemptiness step)**.** *Let $t < N$. If*

$$\rho_t \in \mathcal{V}_t^{\mathrm{engine},N},$$

then there exists a certified non-lossy update

$$\nu_t \in \Omega_t^{\mathrm{NL}}(\rho_t)$$

such that

$$\rho_{t+1} = \Phi_{\nu_t}(\rho_t) \in \mathcal{V}_{t+1}^{\mathrm{engine},N}.$$

Moreover, ν_t satisfies the engine residuals, the algebraic non-loss/recovery certificates, and the ability-preservation condition specified by Definition 11.5. If $g_t^{\mathfrak{A}} > 0$, it also gives strict certified ability-set progress at step t.

Proof. By Definition 11.7, membership $\rho_t \in \mathcal{V}_t^{\mathrm{engine},N}$ means exactly that there exists

$$\nu_t \in \mathcal{E}_t^{\mathrm{CE}}(\rho_t; g_t^{\mathfrak{A}})$$

with

$$\Phi_{\nu_t}(\rho_t) \in \mathcal{V}_{t+1}^{\mathrm{engine},N}.$$

By Definition 11.5, $\mathcal{E}_t^{\mathrm{CE}}(\rho_t; g_t^{\mathfrak{A}}) \subseteq \Omega_t^{\mathrm{NL}}(\rho_t)$, and every element of it satisfies the engine residuals, algebraic non-loss/recovery certificates, and ability-progress condition. This proves the stated existence and successor-viability conclusions. Strict progress follows from Definition 11.4 when $g_t^{\mathfrak{A}} > 0$. □

Interpretation 11.9. *The theorem is the precise Batch 5 closure statement requested by the engine evaluation:*

$$\rho_t \in \mathcal{V}_t^{\mathrm{engine}} \Longrightarrow \exists \nu_t \in \Omega_t^{\mathrm{NL}} \quad \text{with} \quad \rho_{t+1} \in \mathcal{V}_{t+1}^{\mathrm{engine}}.$$

Unlike an ordinary gate theorem, the witness is restricted to the certified non-lossy update algebra. The proof is short because the viability kernel is defined as the set of states from which such a witness exists; the substantive obligation is to prove or certify that the initial state lies in the kernel. The next theorem gives one constructive sufficient condition for that nonemptiness.

Proposition 11.10 (Finite-horizon engine trajectory from viability). *If $\rho_0 \in \mathcal{V}_0^{\mathrm{engine},N}$, then there exist updates*

$$\nu_0, \dots, \nu_{N-1}, \qquad \nu_t \in \Omega_t^{\mathrm{NL}}(\rho_t),$$

and states $\rho_{t+1} = \Phi_{\nu_t}(\rho_t)$ such that

$$\rho_t \in \mathcal{V}_t^{\mathrm{engine},N} \subseteq \mathcal{K}_t^{\mathrm{base}} \subseteq \mathcal{K}_{\mathrm{RCP}}^{\mathrm{state}}(t) \qquad (0 \le t \le N),$$

and each step satisfies its declared non-loss, recovery, resource, certificate, and ability-progress thresholds. If $g_t^{\mathfrak{A}} > 0$ for all $t < N$, the trajectory is a finite-horizon strict recoverable-monotone RSI trajectory over the declared ability family.

Proof. Apply Theorem 11.8 at $t = 0$ to choose ν_0 and obtain $\rho_1 \in \mathcal{V}_1^{\mathrm{engine},N}$. Repeat the argument for $t = 1, \dots, N-1$. The inclusions into $\mathcal{K}_t^{\mathrm{base}}$ and $\mathcal{K}_{\mathrm{RCP}}^{\mathrm{state}}(t)$ follow from Definition 11.7 and Definition 11.2. The stepwise certificates follow from Definition 11.5. □

11.3 A constructive nonemptiness condition

Definition 11.11 (Certified conservative-extension plan). *A certified conservative-extension plan of horizon N from ρ_0 is a finite list*

$$\Pi_{0:N}^{\mathrm{CE}} = (\nu_0^\sharp, \dots, \nu_{N-1}^\sharp)$$

with induced states

$$\rho_{t+1}^\sharp = \Phi_{\nu_t^\sharp}(\rho_t^\sharp), \qquad \rho_0^\sharp = \rho_0,$$

such that for every $t < N$:

(i) $\rho_t^\sharp \in \mathcal{K}_t^{\mathrm{base}}$;

(ii) $\nu_t^\sharp \in \mathcal{E}_t^{\mathrm{CE}}(\rho_t^\sharp; g_t^{\mathfrak{A}})$;

(iii) $\rho_{t+1}^\sharp \in \mathcal{K}_{t+1}^{\mathrm{base}}$.

The plan is not assumed to be found automatically. It is a finite certificate that a particular conservative-extension trajectory exists.

Theorem 11.12 (Certified plan implies nonempty engine viability kernel). *If there exists a certified conservative-extension plan of horizon N from ρ_0, then*

$$\rho_0 \in \mathcal{V}_0^{\mathrm{engine},N},$$

and therefore $\mathcal{V}_0^{\mathrm{engine},N} \neq \emptyset$. More generally, every intermediate state $\rho_t^\sharp$ of the plan belongs to $\mathcal{V}_t^{\mathrm{engine},N}$.

Proof. We prove the statement by backward induction. The final state $\rho_N^\sharp$ belongs to $\mathcal{K}_N^{\mathrm{base}} = \mathcal{V}_N^{\mathrm{engine},N}$ by the plan definition. Suppose $\rho_{t+1}^\sharp \in \mathcal{V}_{t+1}^{\mathrm{engine},N}$. Since $\nu_t^\sharp \in \mathcal{E}_t^{\mathrm{CE}}(\rho_t^\sharp; g_t^{\mathfrak{A}})$ and $\Phi_{\nu_t^\sharp}(\rho_t^\sharp) = \rho_{t+1}^\sharp$, the defining condition of $\mathcal{V}_t^{\mathrm{engine},N}$ is satisfied. Hence $\rho_t^\sharp \in \mathcal{V}_t^{\mathrm{engine},N}$. Induction yields $\rho_0 \in \mathcal{V}_0^{\mathrm{engine},N}$. □

Corollary 11.13 (Finite recursive RSI from a certified conservative-extension plan)**.** *If a certified conservative-extension plan of horizon N from ρ_0 exists and $g_t^{\mathfrak{A}} > 0$ for every $t < N$, then there exists a finite length-N sequence of certified non-lossy, structurally recoverable, ability-expanding self-updates. The sequence is generated by elements of Ω_t^{NL}, remains in the engine base domain, and stays inside the RCP preservation safe set.*

Proof. Theorem 11.12 gives $\rho_0 \in \mathcal{V}_0^{\mathrm{engine},N}$. Proposition 11.10 then supplies the certified trajectory. Strict ability expansion follows from $g_t^{\mathfrak{A}} > 0$ and Definition 11.4. □

11.4 Infinite continuation and remaining limitations

Definition 11.14 (Infinite engine viability kernel)**.** *A sequence $(\mathcal{V}_t^{\mathrm{engine}})_{t\geq 0}$ is an infinite conservative-extension engine viability kernel if*

$$\mathcal{V}_t^{\mathrm{engine}} \subseteq \mathcal{K}_t^{\mathrm{base}}$$

and every $\rho \in \mathcal{V}_t^{\mathrm{engine}}$ admits a move

$$\nu \in \mathcal{E}_t^{\mathrm{CE}}(\rho; g_t^{\mathfrak{A}})$$

with

$$\Phi_\nu(\rho) \in \mathcal{V}_{t+1}^{\mathrm{engine}}.$$

Theorem 11.15 (Infinite engine path from infinite engine viability)**.** *If $(\mathcal{V}_t^{\mathrm{engine}})_{t\geq 0}$ is an infinite conservative-extension engine viability kernel and $\rho_0 \in \mathcal{V}_0^{\mathrm{engine}}$, then there exists an infinite sequence*

$$\nu_t \in \Omega_t^{\mathrm{NL}}(\rho_t), \qquad \rho_{t+1} = \Phi_{\nu_t}(\rho_t),$$

such that

$$\rho_t \in \mathcal{V}_t^{\mathrm{engine}} \subseteq \mathcal{K}_t^{\mathrm{base}} \subseteq \mathcal{K}_{\mathrm{RCP}}^{\mathrm{state}}(t) \quad \forall t \geq 0.$$

Each step satisfies the declared conservative-extension engine residuals and ability-progress thresholds. If the cumulative loss, recovery, Lyapunov, ambiguity, information-bottleneck, resource, and certificate-failure budgets are summable, then the corresponding RCP preservation bounds also remain finite along the infinite path.

Proof. The proof is recursive choice. Since $\rho_0 \in \mathcal{V}_0^{\mathrm{engine}}$, Definition 11.14 supplies $\nu_0 \in \mathcal{E}_0^{\mathrm{CE}}(\rho_0; g_0^{\mathfrak{A}})$ with $\rho_1 = \Phi_{\nu_0}(\rho_0) \in \mathcal{V}_1^{\mathrm{engine}}$. Repeating the same argument at each time gives the sequence. Membership in $\mathcal{E}_t^{\mathrm{CE}}$ implies $\nu_t \in \Omega_t^{\mathrm{NL}}$ and the declared step certificates. The inclusions follow from the kernel definition and the engine base domain. The final boundedness statement follows by applying the Conditional Non-Lossy Self-Update Preservation Theorem and the finite-horizon recovery/composition bounds on arbitrary finite prefixes, then using the assumed summability of the budgets. □

Limitation 11.16 (No universal open-ended progress theorem)**.** *The infinite theorem assumes an infinite engine viability kernel. It does not prove that such a kernel is nonempty for arbitrary architectures, nor that uniform positive ability gain can continue forever on a fixed bounded ability scale. Infinite strict progress must be interpreted over an expanding ability family, decreasing thresholds, renewable resources, or another explicitly stated open-ended progress model. This limitation is consistent with the earlier bounded-scale impossibility results.*

Interpretation 11.17 (Why Batch 5 matters)**.** *Batch 5 changes the status of the future engine theorem. Before this section, the manuscript had one-step non-loss primitives and local progress directions, but no theorem stating how a successor remains in a domain where another non-lossy update can be chosen. The engine viability kernel supplies that missing recursive object. For conservative-extension systems, the implication*

$$\rho_t \in \mathcal{V}_t^{\text{engine}} \Longrightarrow \exists \nu_t \in \Omega_t^{\text{NL}} \quad \textit{with} \quad \rho_{t+1} \in \mathcal{V}_{t+1}^{\text{engine}}$$

now has an explicit theorem-level form. The remaining architecture-side task is to instantiate $\mathcal{K}_t^{\text{base}}$*,* $\mathcal{E}_t^{\text{CE}}$*, and* $\mathcal{V}_t^{\text{engine}}$ *for RCLM and to construct a lift mechanism that produces the certified extension plans.*

12 Direct Engine Construction Theorem and Constructive Beneficial Extension

This section implements Edit Batch 9. Batch 5 proved that if a state is already in the conservative-extension engine viability kernel, then some non-lossy update exists and keeps the successor in the next kernel. That was still an admission/viability result: it did not define an engine procedure that constructs the witness. This section adds the engine constructor $\mathfrak{E}_t$ and proves a direct construction theorem under explicit sufficient assumptions. The proof is not allowed to assume that arbitrary systems have useful updates. It uses a declared direct-engine domain, strict beneficial witnesses, generator coverage, and audited certificate soundness.

Limitation 12.1 (Batch 9 is constructive only on its declared domain)**.** *The direct engine theorem below is not a universal RSI theorem. It states that when the state lies in a domain where a strict beneficial conservative-extension witness exists and the generator/certifier covers that witness within margins, the engine constructs a certified beneficial non-lossy update. If the witness does not exist, if the generator misses it, if the certificate is unsound, or if the successor-viability certificate fails, the theorem does not apply.*

12.1 NL-RSI admissibility and engine output packets

Definition 12.2 (NL-RSI admissible conservative-extension update)**.** *For a state* $\rho \in \mathcal{K}_t^{\text{base}}$ *and progress threshold* $g_t^{\mathfrak{A}} \geq 0$*, define*

$$\mathcal{A}_t^{\text{NL-RSI}}(\rho; g_t^{\mathfrak{A}}) = \{\Phi_\nu : \nu \in \mathcal{E}_t^{\text{CE}}(\rho; g_t^{\mathfrak{A}}),\ \Phi_\nu(\rho) \in \mathcal{V}_{t+1}^{\text{engine}}\}.$$

Thus $\Phi_\nu \in \mathcal{A}_t^{\text{NL-RSI}}$ *means that* ν *is a certified finite word in the non-lossy algebra, satisfies the engine residuals, preserves predecessor abilities, has certified ability progress of threshold* $g_t^{\mathfrak{A}}$*, is resource/certificate/verifier admissible through the engine residuals, and leaves the successor in the next engine-viability kernel.*

Definition 12.3 (Engine output packet)**.** *An engine output packet at state* ρ_t *is a tuple*

$$\mathsf{Out}_t^{\text{eng}}(\rho_t) = (\nu_t, \Phi_{\nu_t}, \mathcal{R}_{\nu_t}^{\text{rec}}, \mathsf{Cert}_t^{\text{eng}}(\nu_t), \rho_{t+1}),$$

where:

(i) $\nu_t \in \Omega_t^{\text{NL}}(\rho_t)$ *is a finite word in the certified non-lossy algebra;*

(ii) Φ_{ν_t} *is the typed self-update transition induced by* ν_t*;*

(iii) $\mathcal{R}^{\mathrm{rec}}_{\nu_t}$ *is the composed recovery map supplied by the primitive-word certificate;*

(iv) $\mathsf{Cert}^{\mathrm{eng}}_t(\nu_t)$ *contains the type, non-loss, recovery, residual, ability-expansion, resource, verifier, coverage, calibration, estimator, projection-realization when applicable, and successor-viability certificates;*

(v) $\rho_{t+1} = \Phi_{\nu_t}(\rho_t)$.

Definition 12.4 (Direct engine constructor)**.** *A direct non-lossy RSI engine constructor at time* t *is a tuple*

$$\mathfrak{E}_t = (\mathsf{Gen}^{\mathrm{NL}}_t, \mathsf{Certify}^{\mathrm{eng}}_t, \mathsf{Select}^{\mathrm{eng}}_t, \mathsf{Realize}^{\mathrm{eng}}_t),$$

where

$$\mathsf{Gen}^{\mathrm{NL}}_t(\rho) = S^{\mathrm{eng}}_t(\rho) \subseteq \Omega^{\mathrm{NL}}_t(\rho)$$

is a finite generated candidate-word set,

$$\mathsf{Certify}^{\mathrm{eng}}_t(\rho, \nu)$$

attempts to build an engine certificate for ν*,*

$$\mathsf{Select}^{\mathrm{eng}}_t$$

selects among candidates with passing conservative certificates, and $\mathsf{Realize}^{\mathrm{eng}}_t$ *applies the selected typed transition and returns the successor. When all stages succeed, write*

$$\mathfrak{E}_t(\rho) = \mathsf{Out}^{\mathrm{eng}}_t(\rho).$$

12.2 Strict constructive witnesses and generator coverage

Definition 12.5 (Strict beneficial conservative-extension witness)**.** *Fix margins* $m^{\mathrm{eng}}_t > 0$*,* $m^{\mathfrak{A}}_t > 0$*, and threshold* $g^{\mathfrak{A}}_t \geq 0$*. A candidate* $\nu^\sharp \in \Omega^{\mathrm{NL}}_t(\rho)$ *is a strict beneficial conservative-extension witness at* ρ *when:*

(i) all engine residuals have strict margin:

$$q^{\mathrm{eng}}_{t,j}(\nu^\sharp; \rho) \leq -m^{\mathrm{eng}}_t \qquad \forall j \in J^{\mathrm{eng}}_t;$$

(ii) the successor is engine-viable:

$$\Phi_{\nu^\sharp}(\rho) \in \mathcal{V}^{\mathrm{engine}}_{t+1};$$

(iii) predecessor abilities are preserved under ability transport:

$$\Theta^{\mathfrak{A}}_t(\mathfrak{A}_t(\rho, R_t)) \subseteq \mathfrak{A}_{t+1}(\Phi_{\nu^\sharp}(\rho), R_{t+1});$$

(iv) the certified new-ability score has strict margin:

$$\Delta^{\mathfrak{A}}_t(\nu^\sharp; \rho) \geq g^{\mathfrak{A}}_t + m^{\mathfrak{A}}_t;$$

(v) the primitive-word certificate supplies a recovery map satisfying

$$\mathrm{Rec}^{\mathcal{P}_t}_{\Phi_{\nu^\sharp}}(\mathcal{R}^{\mathrm{rec}}_{\nu^\sharp}) \le r_t.$$

Assumption 12.6 (Constructive beneficial-extension availability)**.** *For every ρ in the direct engine domain defined below, there exists a strict beneficial conservative-extension witness $\nu^\sharp$ at ρ with margins $m^{\mathrm{eng}}_t, m^{\mathfrak{A}}_t > 0$. This is the substantive improvement-existence premise. It may be proved for a particular architecture by exhibiting an extension module, tool, memory, verifier, adapter, or proof routine that adds a certified ability while retaining the predecessor as a recoverable subsystem.*

Assumption 12.7 (Generator coverage with margin)**.** *There is an engine search error $e^{\mathrm{gen}}_t \ge 0$. If a strict beneficial witness $\nu^\sharp$ exists at ρ, then $\mathsf{Gen}^{\mathrm{NL}}_t(\rho)$ returns a candidate $\widetilde{\nu} \in S^{\mathrm{eng}}_t(\rho)$ such that:*

$$q^{\mathrm{eng}}_{t,j}(\widetilde{\nu};\rho) \le q^{\mathrm{eng}}_{t,j}(\nu^\sharp;\rho) + e^{\mathrm{gen}}_t \qquad \forall j \in J^{\mathrm{eng}}_t,$$

$$\Delta^{\mathfrak{A}}_t(\widetilde{\nu};\rho) \ge \Delta^{\mathfrak{A}}_t(\nu^\sharp;\rho) - e^{\mathrm{gen}}_t,$$

and

$$\Phi_{\widetilde{\nu}}(\rho) \in \mathcal{V}^{\mathrm{engine}}_{t+1}.$$

The last condition may be certified either directly by the generator or by the engine certifier. This is a coverage assumption over the declared generated candidate class, not a claim about all possible self-modifications.

Assumption 12.8 (Engine certificate soundness and completeness)**.** *For generated candidates $\nu \in S^{\mathrm{eng}}_t(\rho)$, the certifier returns conservative estimates*

$$\widehat{q}^{\mathrm{eng}}_{t,j}(\nu;\rho), \qquad \widehat{\Delta}^{\mathfrak{A}}_t(\nu;\rho), \qquad \widehat{r}_t(\nu)$$

with radii $\Delta^{\mathrm{eng}}_{t,j}, \Delta^{\mathfrak{A},\mathrm{eng}}_t, \Delta^{r,\mathrm{eng}}_t$. On a certificate event $\mathcal{E}^{\mathrm{eng}}_t$, the following one-sided bounds hold for all generated candidates:

$$q^{\mathrm{eng}}_{t,j}(\nu;\rho) \le \widehat{q}^{\mathrm{eng}}_{t,j}(\nu;\rho) + \Delta^{\mathrm{eng}}_{t,j}(\nu),$$

$$\Delta^{\mathfrak{A}}_t(\nu;\rho) \ge \widehat{\Delta}^{\mathfrak{A}}_t(\nu;\rho) - \Delta^{\mathfrak{A},\mathrm{eng}}_t(\nu),$$

and

$$\mathrm{Rec}^{\mathcal{P}_t}_{\Phi_\nu}(\mathcal{R}^{\mathrm{rec}}_\nu) \le \widehat{r}_t(\nu) + \Delta^{r,\mathrm{eng}}_t(\nu).$$

Completeness means that any generated candidate satisfying the true strict margins with gap exceeding the relevant radii receives a passing conservative certificate. Explicitly, if

$$q^{\mathrm{eng}}_{t,j}(\nu;\rho) \le -\mu^{\mathrm{eng}}_t, \qquad \Delta^{\mathfrak{A}}_t(\nu;\rho) \ge g^{\mathfrak{A}}_t + \mu^{\mathfrak{A}}_t, \qquad \mathrm{Rec}^{\mathcal{P}_t}_{\Phi_\nu}(\mathcal{R}^{\mathrm{rec}}_\nu) \le r_t - \mu^r_t$$

with positive margins larger than the certifier radii, then $\mathsf{Certify}^{\mathrm{eng}}_t(\rho,\nu)$ returns a passing certificate.

Definition 12.9 (Direct engine domain)**.** *The direct engine domain at time t is the set*

$$\mathcal{D}^{\mathrm{engine}}_t \subseteq \mathcal{V}^{\mathrm{engine}}_t$$

of states ρ satisfying all of the following:

(i) $\rho \in \mathcal{K}^{\mathrm{base}}_t$ and all Batch 8 transports, state/update separation objects, and projection-realization requirements needed by the candidate class are defined;

(ii) Constructive beneficial-extension availability holds at ρ;

(iii) generator coverage with margin holds at ρ;

(iv) the engine certificate soundness/completeness event is part of the declared certificate packet;

(v) the total generation/certification error satisfies

$$e_t^{\mathrm{gen}} + e_t^{\mathrm{cert}} < \min\{m_t^{\mathrm{eng}}, m_t^{\mathfrak{A}}, r_t - r_t^{\sharp}\},$$

where e_t^{cert} upper-bounds the relevant certificate radii and $r_t^{\sharp}$ is the recovery defect of the strict witness.

12.3 Engine algorithm and direct construction theorem

Definition 12.10 (Direct engine construction algorithm). *Given $\rho \in \mathcal{D}_t^{\mathrm{engine}}$, the engine $\mathfrak{E}_t$ performs:*

(1) Generate $S_t^{\mathrm{eng}}(\rho) = \mathsf{Gen}_t^{\mathrm{NL}}(\rho) \subseteq \Omega_t^{\mathrm{NL}}(\rho)$.

(2) For each $\nu \in S_t^{\mathrm{eng}}(\rho)$, construct $\mathsf{Certify}_t^{\mathrm{eng}}(\rho, \nu)$, including the recovery map, transported pair-set certificate, engine residual certificate, ability-expansion witness, resource certificate, verifier/trust certificate, and successor-viability certificate.

(3) Form the passing set

$$\mathsf{Pass}_t^{\mathrm{eng}}(\rho) = \{\nu \in S_t^{\mathrm{eng}}(\rho) : \widehat{q}_{t,j}^{\mathrm{eng}}(\nu;\rho) + \Delta_{t,j}^{\mathrm{eng}}(\nu) \le 0 \ \forall j,$$

$$\widehat{\Delta}_t^{\mathfrak{A}}(\nu;\rho) - \Delta_t^{\mathfrak{A},\mathrm{eng}}(\nu) > g_t^{\mathfrak{A}}, \quad \widehat{r}_t(\nu) + \Delta_t^{r,\mathrm{eng}}(\nu) \le r_t, \quad \Phi_\nu(\rho) \in \mathcal{V}_{t+1}^{\mathrm{engine}}\}.$$

(4) Select $\nu_t \in \mathsf{Pass}_t^{\mathrm{eng}}(\rho)$ by a fixed rule, for example maximizing the certified new-ability score minus certified cost and inflation penalty.

(5) Return

$$\mathfrak{E}_t(\rho) = (\Phi_{\nu_t}, \mathcal{R}_{\nu_t}^{\mathrm{rec}}, \mathsf{Cert}_t^{\mathrm{eng}}(\nu_t), \Phi_{\nu_t}(\rho)).$$

If $\mathsf{Pass}_t^{\mathrm{eng}}(\rho) = \emptyset$, the direct engine construction fails at time t; the preservation gate may still select no-op, but no direct-engine progress claim is licensed.

Theorem 12.11 (Constructive Direct Non-Lossy RSI Engine Theorem). *Assume $\rho_t \in \mathcal{D}_t^{\mathrm{engine}}$, and assume the engine certificate event $\mathcal{E}_t^{\mathrm{eng}}$ holds. Then the passing set $\mathsf{Pass}_t^{\mathrm{eng}}(\rho_t)$ is nonempty. Consequently, $\mathfrak{E}_t(\rho_t)$ is defined and constructs an output packet*

$$(\Phi_{\nu_t}, \mathcal{R}_{\nu_t}^{\mathrm{rec}}, \mathsf{Cert}_t^{\mathrm{eng}}(\nu_t), \rho_{t+1})$$

with $\rho_{t+1} = \Phi_{\nu_t}(\rho_t)$ such that:

(i) $\nu_t \in \Omega_t^{\mathrm{NL}}(\rho_t)$;

(ii) $\Phi_{\nu_t} \in \mathcal{A}_t^{\text{NL-RSI}}(\rho_t; g_t^{\mathfrak{A}})$;

(iii) the recovery certificate satisfies

$$\mathrm{Rec}^{\mathcal{P}_t}_{\Phi_{\nu_t}}(\mathcal{R}^{\mathrm{rec}}_{\nu_t}) \le r_t;$$

(iv) predecessor abilities are preserved:

$$\Theta^{\mathfrak{A}}_t(\mathfrak{A}_t(\rho_t, R_t)) \subseteq \mathfrak{A}_{t+1}(\rho_{t+1}, R_{t+1});$$

(v) strict certified ability progress holds:

$$\mathfrak{A}_{t+1}(\rho_{t+1}, R_{t+1}) \setminus \Theta^{\mathfrak{A}}_t(\mathfrak{A}_t(\rho_t, R_t)) \neq \emptyset,$$

witnessed by $\Delta^{\mathfrak{A}}_t(\nu_t; \rho_t) > g^{\mathfrak{A}}_t$*;*

(vi) the successor remains engine-viable:

$$\rho_{t+1} \in \mathcal{V}^{\mathrm{engine}}_{t+1}.$$

Thus, on the declared direct-engine domain, the engine constructs a beneficial non-lossy conservative-extension update rather than merely verifying an externally supplied one.

Proof. Since $\rho_t \in \mathcal{D}^{\mathrm{engine}}_t$, Assumption 12.6 gives a strict beneficial witness $\nu^\sharp$. By Assumption 12.7, the generator returns $\widetilde{\nu} \in S^{\mathrm{eng}}_t(\rho_t) \subseteq \Omega^{\mathrm{NL}}_t(\rho_t)$ whose engine residuals, ability score, and successor-viability property are within the declared generation error of the witness. The margin inequality in Definition 12.9 ensures that the residual margins, ability margin, and recovery margin remain positive after generation and certificate error are subtracted. By Assumption 12.8, $\mathsf{Certify}^{\mathrm{eng}}_t(\rho_t, \widetilde{\nu})$ returns a passing conservative certificate. Hence $\widetilde{\nu} \in \mathsf{Pass}^{\mathrm{eng}}_t(\rho_t)$, so the passing set is nonempty.

The selection rule chooses $\nu_t \in \mathsf{Pass}^{\mathrm{eng}}_t(\rho_t)$. By construction of the passing set, $\nu_t \in \Omega^{\mathrm{NL}}_t(\rho_t)$, all engine residuals are certified nonpositive, the recovery defect is at most r_t, the certified new-ability score is above $g^{\mathfrak{A}}_t$, and the successor lies in $\mathcal{V}^{\mathrm{engine}}_{t+1}$. Definition 12.2 then gives $\Phi_{\nu_t} \in \mathcal{A}^{\text{NL-RSI}}_t(\rho_t; g^{\mathfrak{A}}_t)$. Ability preservation and strict ability progress follow from the engine-move residuals and Definition 11.4. The returned tuple is exactly the engine output packet of Definition 12.3. □

Corollary 12.12 (Finite direct-engine trajectory under constructor-domain invariance)**.** *Fix a finite horizon N. Suppose $\rho_0 \in \mathcal{D}^{\mathrm{engine}}_0$, the engine certificate events hold for $t = 0, \dots, N-1$, and the engine output satisfies constructor-domain invariance:*

$$\rho_{t+1} = \Phi_{\nu_t}(\rho_t) \quad \Longrightarrow \quad \rho_{t+1} \in \mathcal{D}^{\mathrm{engine}}_{t+1}$$

for every generated step $t < N-1$. Then repeated application of $\mathfrak{E}_t$ constructs a finite sequence

$$\rho_0 \xrightarrow{\nu_0} \rho_1 \xrightarrow{\nu_1} \cdots \xrightarrow{\nu_{N-1}} \rho_N$$

of certified non-lossy, structurally recoverable, ability-expanding, engine-viable self-updates. Each update lies in $\mathcal{A}^{\text{NL-RSI}}_t$, and every successor lies in the corresponding engine-viability kernel.

Proof. Apply Theorem 12.11 at $t = 0$ to obtain ν_0 and ρ_1. Constructor-domain invariance gives $\rho_1 \in \mathcal{D}^{\mathrm{engine}}_1$. Repeat inductively through $t = N-1$. The stepwise conclusions are exactly the conclusions of Theorem 12.11 at each time. □

Limitation 12.13 (The direct engine theorem remains domain-restricted)**.** *Theorem 12.11 is the first theorem in this manuscript whose conclusion is genuinely constructive rather than merely admissibility-preserving: the update is produced by $\mathfrak{E}_t$. Its force is still limited by its premises. In particular, it does not prove that arbitrary states lie in $\mathcal{D}_t^{\mathrm{engine}}$, that strict beneficial witnesses are always available, that the generator is complete for unbounded candidate classes, that certificate radii remain small at scale, or that infinite strict progress is possible on a fixed bounded ability scale.*

Interpretation 12.14 (What Batch 9 changes)**.** *Batch 5 gave*

$$\rho_t \in \mathcal{V}_t^{\mathrm{engine}} \Longrightarrow \exists \nu_t \in \Omega_t^{\mathrm{NL}} \quad \textit{with} \quad \rho_{t+1} \in \mathcal{V}_{t+1}^{\mathrm{engine}}.$$

Batch 9 strengthens the statement on the smaller direct-engine domain:

$$\rho_t \in \mathcal{D}_t^{\mathrm{engine}} \Longrightarrow \mathfrak{E}_t(\rho_t) \textit{ constructs such a } \nu_t$$

with recovery, ability expansion, certificates, and successor viability. This is the formal shift from an admission law to a domain-restricted engine law.

13 Module C: Lossy Self-Improvement as Relative-Entropy Loss and Constructive Recovery

The first instability issue is loss of protected safety-relevant distinctions. A recursive update is unsafe if it compresses states that must remain distinguishable. This section states both the distinguishability-loss criterion and the stronger constructive-recoverability criterion. The second criterion is necessary because bounded loss of relative entropy alone does not automatically construct a usable inverse map for a whole family of safety-relevant states.

13.1 State, channel, and relative-entropy domain

Definition 13.1 (Finite-dimensional density states)**.** *Let $\mathcal{H}$ be a finite-dimensional Hilbert space. The density-state space is*

$$\mathcal{D}(\mathcal{H}) = \{\rho \in \mathrm{End}(\mathcal{H}) : \rho = \rho^\dagger, \rho \succeq 0, \mathrm{Tr}\,\rho = 1\}.$$

All relative-entropy and recovery statements in this module are finite-dimensional unless explicitly stated otherwise.

Definition 13.2 (Quantum relative entropy)**.** *For $\rho, \sigma \in \mathcal{D}(\mathcal{H})$, define*

$$D(\rho\|\sigma) = \begin{cases} \mathrm{Tr}[\rho(\log\rho - \log\sigma)], & \mathrm{supp}(\rho) \subseteq \mathrm{supp}(\sigma), \\ +\infty, & \textit{otherwise.} \end{cases}$$

The support condition is not optional. If it fails, the distinguishability-loss expressions below may be infinite and cannot be used as finite safety budgets.

Definition 13.3 (Admissible channel for this module)**.** *A channel Φ_t is admissible for the relative-entropy module when it is completely positive and trace preserving on the relevant finite-dimensional state algebra. If the successor system uses a different Hilbert space, then*

$$\Phi_t : \mathcal{D}(\mathcal{H}_t) \to \mathcal{D}(\mathcal{H}_{t+1})$$

is allowed. Relative entropy is compared before and after the channel only between states living in the same respective domain.

13.2 Safety-relevant pair set

Definition 13.4 (Safety-relevant distinctions)**.** *Let $\mathcal{P}_t$ be a set of ordered state pairs (ρ, σ) representing distinctions that must not be erased at time t. Each pair is required to satisfy*

$$\rho, \sigma \in \mathcal{D}(\mathcal{H}_t), \qquad \mathrm{supp}(\rho) \subseteq \mathrm{supp}(\sigma), \qquad D(\rho\|\sigma) < \infty.$$

Examples include:

$$\rho = \textit{corrigible self-model}, \qquad \sigma = \textit{incorrigible self-model},$$

$$\rho = \textit{grounded world-model}, \qquad \sigma = \textit{hallucinated world-model},$$

$$\rho = \textit{uncertain hypothesis state}, \qquad \sigma = \textit{falsely definitive hypothesis state}.$$

The set $\mathcal{P}_t$ is part of the safety specification. Theorems stated in terms of $\mathcal{P}_t$ protect only the distinctions represented by $\mathcal{P}_t$, together with whatever approximation guarantees are later supplied by a pair-completeness module.

Definition 13.5 (Distinguishability loss of a channel)**.** *For an admissible channel Φ_t, define the safety-relevant distinguishability loss*

$$\mathcal{L}_{\Phi_t}^{\mathcal{P}_t} = \sup_{(\rho,\sigma)\in\mathcal{P}_t} \left[D(\rho\|\sigma) - D(\Phi_t(\rho)\|\Phi_t(\sigma)) \right]_+ .$$

When Φ_t is CPTP and the relative entropies are finite, the data-processing inequality implies

$$D(\rho\|\sigma) - D(\Phi_t(\rho)\|\Phi_t(\sigma)) \geq 0.$$

The positive part is retained so that the definition remains well-formed in estimated or approximate implementations.

Definition 13.6 (ε_t-non-lossy update)**.** *A channel Φ_t is ε_t-non-lossy relative to $\mathcal{P}_t$ when*

$$\mathcal{L}_{\Phi_t}^{\mathcal{P}_t} \leq \varepsilon_t.$$

It is exactly non-lossy relative to $\mathcal{P}_t$ when $\varepsilon_t = 0$.

This is the direct mathematical condition for avoiding a specified form of lossy self-improvement:

$$\boxed{\sup_{(\rho,\sigma)\in\mathcal{P}_t} \left[D(\rho\|\sigma) - D(\Phi_t(\rho)\|\Phi_t(\sigma)) \right] \leq \varepsilon_t.}$$

13.3 Bounded cumulative distinguishability loss

Theorem 13.7 (Bounded recursive distinguishability loss)**.** *Let (ρ_0, σ_0) be an initial safety-relevant pair with finite relative entropy. Suppose both states evolve under the same accepted channels:*

$$\rho_{t+1} = \Phi_t(\rho_t), \qquad \sigma_{t+1} = \Phi_t(\sigma_t).$$

Assume that for every $t < T$,

$$D(\rho_t\|\sigma_t) - D(\rho_{t+1}\|\sigma_{t+1}) \leq \varepsilon_t.$$

Then

$$D(\rho_T\|\sigma_T) \geq D(\rho_0\|\sigma_0) - \sum_{t=0}^{T-1} \varepsilon_t.$$

In particular, if $\sum_{t=0}^{\infty} \varepsilon_t < \infty$, then the total loss of safety-relevant distinguishability is finite.

Proof. For each $t < T$, rearrange the assumed inequality:

$$D(\rho_{t+1}\|\sigma_{t+1}) \geq D(\rho_t\|\sigma_t) - \varepsilon_t.$$

Applying the inequality successively gives

$$D(\rho_1\|\sigma_1) \geq D(\rho_0\|\sigma_0) - \varepsilon_0,$$

$$D(\rho_2\|\sigma_2) \geq D(\rho_1\|\sigma_1) - \varepsilon_1 \geq D(\rho_0\|\sigma_0) - \varepsilon_0 - \varepsilon_1.$$

Continuing by induction yields

$$D(\rho_T\|\sigma_T) \geq D(\rho_0\|\sigma_0) - \sum_{t=0}^{T-1} \varepsilon_t.$$

If $\sum_t \varepsilon_t$ converges, the right-hand side loses only a finite amount relative to the initial distinguishability. □

Limitation 13.8 (Bounded loss is not semantic preservation)**.** *The theorem is a finite-budget distinguishability result. It does not imply that every relevant semantic distinction is present in $\mathcal{P}_t$, does not construct $\mathcal{P}_t$, and does not imply that the successor architecture represents the protected states in human-interpretable form.*

13.4 Constructive recoverability

Bounded distinguishability loss prevents specified distinctions from being erased too quickly, but non-lossy recursive self-improvement needs a stronger operational statement: the safety-relevant predecessor state should be reconstructable from the successor state, at least up to a certified error. This subsection adds that stronger condition without pretending that it follows automatically from bounded relative-entropy loss in full generality.

Definition 13.9 (Trace recovery distance)**.** *For density states ρ, σ, define*

$$d_{\mathrm{tr}}(\rho, \sigma) = \frac{1}{2}\|\rho - \sigma\|_1.$$

For a pair set $\mathcal{P}_t$, a channel Φ_t, and a recovery channel $\mathcal{R}_t$, define the pairwise recovery defect

$$\mathrm{Rec}_{\Phi_t}^{\mathcal{P}_t}(\mathcal{R}_t) = \sup_{(\rho,\sigma)\in\mathcal{P}_t} \max\left\{d_{\mathrm{tr}}\big(\rho, \mathcal{R}_t\Phi_t(\rho)\big), d_{\mathrm{tr}}\big(\sigma, \mathcal{R}_t\Phi_t(\sigma)\big)\right\}.$$

The optimal constructive recovery defect is

$$\mathrm{Rec}_{\Phi_t}^{\mathcal{P}_t} = \inf_{\mathcal{R}_t\in\mathrm{CPTP}} \mathrm{Rec}_{\Phi_t}^{\mathcal{P}_t}(\mathcal{R}_t),$$

where the infimum ranges over CPTP maps from the output algebra of Φ_t to the input algebra.

Definition 13.10 (Constructively r_t-recoverable update)**.** *An accepted update Φ_t is constructively r_t-recoverable relative to $\mathcal{P}_t$ if there exists an explicitly specified or certified recovery channel $\mathcal{R}_t$ such that*

$$\mathrm{Rec}_{\Phi_t}^{\mathcal{P}_t}(\mathcal{R}_t) \leq r_t.$$

It is exactly constructively recoverable when $r_t = 0$.

Theorem 13.11 (Exact Petz recovery for a common reference)**.** *Let $\Phi : \mathcal{D}(\mathcal{H}) \to \mathcal{D}(\mathcal{H}')$ be CPTP. Let $\sigma_\star \in \mathcal{D}(\mathcal{H})$ be faithful on its support, and assume $\operatorname{supp}(\rho_j) \subseteq \operatorname{supp}(\sigma_\star)$ for every $j \in J$. Define the Petz recovery map on the support of $\Phi(\sigma_\star)$ by*

$$\mathcal{R}_{\sigma_\star,\Phi}(X) = \sigma_\star^{1/2}\Phi^\dagger\Big(\Phi(\sigma_\star)^{-1/2}X\Phi(\sigma_\star)^{-1/2}\Big)\sigma_\star^{1/2},$$

where inverses are restricted to the support of $\Phi(\sigma_\star)$. Suppose

$$\mathcal{P} = \{(\rho_j, \sigma_\star) : j \in J\}$$

and

$$D(\rho_j\|\sigma_\star) = D(\Phi(\rho_j)\|\Phi(\sigma_\star)) \qquad \text{for every } j \in J.$$

Then

$$\mathcal{R}_{\sigma_\star,\Phi}\Phi(\sigma_\star) = \sigma_\star, \qquad \mathcal{R}_{\sigma_\star,\Phi}\Phi(\rho_j) = \rho_j \quad \text{for every } j \in J.$$

Thus Φ is exactly constructively recoverable on $\mathcal{P}$, after extending the map arbitrarily to a CPTP map on the full output algebra if necessary.

Proof. The assumptions are exactly the equality conditions for monotonicity of quantum relative entropy for each pair $(\rho_j, \sigma_\star)$. Under these conditions, Φ is sufficient for the family $\{\rho_j\}_{j\in J} \cup \{\sigma_\star\}$, and the displayed Petz map recovers $\sigma_\star$ and each ρ_j on the relevant support. Applying the equality condition pairwise with the same reference state $\sigma_\star$ gives the stated recovery identities. Any action of the recovery map outside the support of $\Phi(\sigma_\star)$ is irrelevant to these states and may be completed to a CPTP map on the ambient output space. □

Limitation 13.12 (What exact Petz recovery does not prove)**.** *The theorem does not say that every ε_t-non-lossy update is constructively recoverable. It proves exact recovery only under equality of relative entropy for pairs sharing a common reference state. Approximate or mixed-reference recoverability requires additional hypotheses or an independent recovery certificate.*

Theorem 13.13 (Finite-horizon constructive predecessor recovery)**.** *Let $\rho_{t+1} = \Phi_t(\rho_t)$ for $t = 0, \dots, T-1$. Suppose that for each $t < T$ there exists a recovery channel $\mathcal{R}_t$ such that*

$$d_{\mathrm{tr}}\big(\rho_t, \mathcal{R}_t(\rho_{t+1})\big) \le r_t.$$

Then the composed recovery map

$$\mathcal{R}_{0:T-1} = \mathcal{R}_0 \circ \mathcal{R}_1 \circ \cdots \circ \mathcal{R}_{T-1}$$

satisfies

$$d_{\mathrm{tr}}\big(\rho_0, \mathcal{R}_{0:T-1}(\rho_T)\big) \le \sum_{t=0}^{T-1} r_t.$$

The same bound holds for every trajectory state appearing in a pair set for which the one-step recovery defects are bounded by r_t under the same accepted channels and certified recovery maps.

Proof. The trace distance is contractive under CPTP maps. For $T = 1$ the claim is the hypothesis. For $T > 1$, write

$$\begin{aligned}
&d_{\mathrm{tr}}\big(\rho_0, \mathcal{R}_0\mathcal{R}_1 \cdots \mathcal{R}_{T-1}(\rho_T)\big) \\
&\le d_{\mathrm{tr}}\big(\rho_0, \mathcal{R}_0(\rho_1)\big) + d_{\mathrm{tr}}\big(\mathcal{R}_0(\rho_1), \mathcal{R}_0\mathcal{R}_1 \cdots \mathcal{R}_{T-1}(\rho_T)\big) \\
&\le r_0 + d_{\mathrm{tr}}\big(\rho_1, \mathcal{R}_1 \cdots \mathcal{R}_{T-1}(\rho_T)\big),
\end{aligned}$$

where the last inequality uses contractivity of $\mathcal{R}_0$. Repeating this argument gives the telescoping bound

$$d_{\mathrm{tr}}\big(\rho_0, \mathcal{R}_{0:T-1}(\rho_T)\big) \le r_0 + r_1 + \cdots + r_{T-1}.$$

□

Definition 13.14 (Recoverably non-lossy update)**.** *A channel Φ_t is (ε_t, r_t)-recoverably non-lossy relative to $\mathcal{P}_t$ if*

$$\mathcal{L}_{\Phi_t}^{\mathcal{P}_t} \le \varepsilon_t$$

and there exists a certified recovery channel $\mathcal{R}_t$ satisfying

$$\mathrm{Rec}_{\Phi_t}^{\mathcal{P}_t}(\mathcal{R}_t) \le r_t.$$

Interpretation 13.15. *The relative-entropy condition says that specified distinctions have not been compressed away beyond budget. The constructive-recovery condition says that a certified channel can approximately reconstruct the relevant predecessor states from their successors. For non-lossy RSI, the second statement is stronger and more operational.*

Limitation 13.16 (Recovery is an additional certificate)**.** *The implication*

$$\mathcal{L}_{\Phi_t}^{\mathcal{P}_t} \le \varepsilon_t \quad \Longrightarrow \quad \mathrm{Rec}_{\Phi_t}^{\mathcal{P}_t} \le f(\varepsilon_t)$$

is not asserted in this manuscript without additional hypotheses. The upgraded RCP gate must therefore treat constructive recoverability as an explicit certificate unless exact Petz recovery, approximate Petz-style recovery, or another applicable recovery theorem is separately established.

14 Recursive Coherence Functional

The second instability issue is recursive incoherence: the system may preserve some information while drifting away from grounded, corrigible, or uncertainty-aware self-reference. We therefore define a coherence functional and a corresponding Lyapunov incoherence functional.

14.1 Update-description register

The expression $I(M;\Phi_t)$ is informal unless the update channel is represented as data. We therefore introduce an update-description register.

Definition 14.1 (Update-description register)**.** *Let G_t be a classical or quantum register containing the description, Choi representation, program encoding, or audit summary of the proposed update Φ_t. Mutual information terms involving the update are written as $I(M;G_t)$, not as mutual information with an abstract function.*

14.2 Recursive coherence

Definition 14.2 (Recursive coherence functional)**.** *Given ρ_t and an update-description register G_t, define*

$$\begin{aligned}\mathcal{C}_{\mathrm{RCP}}(\rho_t, G_t) = {} & \alpha I_{\rho_t}(L;O) + \beta I_{\rho_t}(L;S) + \gamma I_{\rho_t}(M;LOS) + \delta I_{\rho_t}(M;G_t) \\ & + \chi I_{\rho_t}(A;D \mid LOS) - \lambda R_{\mathrm{collapse}}(\rho_t) - \mu R_{\mathrm{irreversible}}(\rho_t) - \nu R_{\mathrm{misgrounded}}(\rho_t). \qquad (1)\end{aligned}$$

The weights $\alpha, \beta, \gamma, \delta, \chi, \lambda, \mu, \nu \ge 0$ are specification parameters.

The terms have the following roles:

$$\begin{aligned} I(L;O): &\text{ language grounded in world-reference,}\\ I(L;S): &\text{ language grounded in human/subjective reference,}\\ I(M;LOS): &\text{ self-model coherence with language, world, and human reference,}\\ I(M;G_t): &\text{ self-model awareness of its own proposed update,}\\ I(A;D \mid LOS): &\text{ coordination between ambiguity and definitiveness given grounding.} \end{aligned}$$

The risk terms penalize unsupported collapse, irreversible modification, and internally coherent but externally false or ungrounded structure.

Assumption 14.3 (Bounded risk terms)**.** *The risk terms are nonnegative measurable functions. On the finite-dimensional operational domain, they are either bounded or clipped to finite audit ranges. Thus* $\mathcal{C}_{\mathrm{RCP}}$ *and the Lyapunov functional below are finite on accepted states.*

14.3 Lyapunov incoherence functional

Definition 14.4 (Recursive incoherence Lyapunov functional)**.** *Define*

$$V(\rho_t, G_t) = -\mathcal{C}_{\mathrm{RCP}}(\rho_t, G_t) + r_1 R_{\mathrm{collapse}}(\rho_t) + r_2 R_{\mathrm{irreversible}}(\rho_t) + r_3 R_{\mathrm{misalignment}}(\rho_t) + r_4 R_{\text{self-error}}(\rho_t).$$

When no confusion is possible, write $V_t = V(\rho_t, G_t)$*.*

The purpose of V is not to assert that lower entropy is always better. It is to measure recursive incoherence: loss of groundedness, loss of corrigibility, loss of uncertainty sensitivity, and loss of accurate self-modeling.

14.4 Lyapunov drift condition

Assumption 14.5 (RCP Lyapunov drift)**.** *There exist constants* $\kappa_V > 0$ *and nonnegative errors* η_t *such that the accepted update satisfies*

$$\mathbb{E}\big[V_{t+1} \mid \mathcal{F}_t\big] \le V_t - \kappa_V \, \mathbb{E}\big[d(\rho_{t+1}, \rho_t)^2 \mid \mathcal{F}_t\big] + \eta_t,$$

where $\mathcal{F}_t$ *is the filtration containing all information available before selecting* Φ_t*.*

Theorem 14.6 (Lyapunov boundedness under summable error)**.** *Assume the RCP Lyapunov drift condition and* $V_t \ge 0$*. Then for every* $T \ge 1$*,*

$$\mathbb{E}[V_T] + \kappa_V \sum_{t=0}^{T-1} \mathbb{E}\big[d(\rho_{t+1}, \rho_t)^2\big] \le \mathbb{E}[V_0] + \sum_{t=0}^{T-1} \eta_t.$$

If $\sum_{t=0}^{\infty} \eta_t < \infty$*, then*

$$\sup_T \mathbb{E}[V_T] < \infty$$

and

$$\sum_{t=0}^{\infty} \mathbb{E}\big[d(\rho_{t+1}, \rho_t)^2\big] < \infty.$$

Proof. Taking total expectation in the drift condition gives

$$\mathbb{E}[V_{t+1}] \leq \mathbb{E}[V_t] - \kappa_V \mathbb{E}[d(\rho_{t+1}, \rho_t)^2] + \eta_t.$$

Rearrange:

$$\mathbb{E}[V_{t+1}] + \kappa_V \mathbb{E}[d(\rho_{t+1}, \rho_t)^2] \leq \mathbb{E}[V_t] + \eta_t.$$

Summing from $t = 0$ to $T - 1$ telescopes the V-terms:

$$\mathbb{E}[V_T] + \kappa_V \sum_{t=0}^{T-1} \mathbb{E}[d(\rho_{t+1}, \rho_t)^2] \leq \mathbb{E}[V_0] + \sum_{t=0}^{T-1} \eta_t.$$

Because $V_T \geq 0$, the distance sum is bounded by the same right-hand side divided by κ_V. If $\sum_t \eta_t < \infty$, both conclusions follow. □

14.5 Interpretation

The Lyapunov condition says that accepted self-updates must not merely improve a local task score. They must reduce recursive incoherence, or at least fail to increase it beyond a summable budget. The squared distance penalty discourages large unstable jumps unless compensated by a sufficiently large reduction in incoherence.

14.6 Limitation

The theorem does not prove that a raw learning system automatically satisfies the drift condition. The drift condition is a gate or certificate requirement. A concrete implementation must either prove the inequality analytically, estimate it with verified bounds, or reject the update.

15 Supporting Barrier-Certificate Submodule

This section integrates the imported barrier-certificate machinery as a submodule. It is not the master RCP functional. It is a representation-health barrier used inside the broader RCP safe set.

15.1 Coherence-diversity sub-barrier

Definition 15.1 (Relative entropy of coherence)**.** *Fix a reference basis. Let ρ_{diag} be the matrix obtained by deleting all off-diagonal entries of ρ in that basis. The relative entropy of coherence is*

$$C_{\text{rel}}(\rho) = S(\rho_{\text{diag}}) - S(\rho).$$

Definition 15.2 (Effective rank)**.** *With base-2 logarithms, define*

$$r_{\text{eff}}(\rho) = 2^{S(\rho)}.$$

This equals d for the maximally mixed state on d dimensions and equals 1 for pure states.

Definition 15.3 (Diversity-collapse sub-barrier). *For $\gamma_{\mathrm{div}} > 0$, define*

$$L_{\mathrm{div}}(\rho) = C_{\mathrm{rel}}(\rho) + \gamma_{\mathrm{div}} r_{\mathrm{eff}}(\rho).$$

For threshold κ_{div}, define

$$\mathcal{K}_{\mathrm{div}} = \{\rho \in \mathcal{D}_d : L_{\mathrm{div}}(\rho) \geq \kappa_{\mathrm{div}}\}$$

and violation

$$V_{\mathrm{div}}(\rho) = [\kappa_{\mathrm{div}} - L_{\mathrm{div}}(\rho)]_+.$$

Lemma 15.4 (Basic properties of L_{div}). *The functional L_{div} is nonnegative on $\mathcal{D}_d$. It is continuous on the finite-dimensional density-state space. Therefore $\mathcal{K}_{\mathrm{div}}$ is closed, and since $\mathcal{D}_d$ is compact, $\mathcal{K}_{\mathrm{div}}$ is compact whenever nonempty.*

Proof. The relative entropy of coherence is nonnegative. The effective rank is positive. Hence $L_{\mathrm{div}} \geq 0$. Entropy is continuous in finite dimension, so both C_{rel} and r_{eff} are continuous. Thus L_{div} is continuous. The set $\mathcal{K}_{\mathrm{div}} = L_{\mathrm{div}}^{-1}([\kappa_{\mathrm{div}}, \infty)) \cap \mathcal{D}_d$ is closed in the compact set $\mathcal{D}_d$, hence compact if nonempty. □

15.2 Soft barrier update

Let Θ_t be a set of admissible recursive modifications, and let each $\theta \in \Theta_t$ induce

$$\rho_\theta^+ = \mathcal{R}_\theta(\rho_t) \in \mathcal{D}_d.$$

Let $P_t(\theta)$ be a task or performance loss.

Assumption 15.5 (Bounded performance and optimization error). *There exists $P_{\max} < \infty$ such that*

$$0 \leq P_t(\theta) \leq P_{\max} \quad \forall \theta \in \Theta_t.$$

The selected update is a ν_t-approximate minimizer, meaning its objective value is within $\nu_t \geq 0$ of the infimum.

Definition 15.6 (Soft diversity-barrier objective). *For $\lambda_t, \mu_t > 0$, define*

$$J_t(\theta) = P_t(\theta) - \lambda_t \big[L_{\mathrm{div}}(\mathcal{R}_\theta(\rho_t)) - L_{\mathrm{div}}(\rho_t)\big] + \mu_t \big[\kappa_{\mathrm{div}} - L_{\mathrm{div}}(\mathcal{R}_\theta(\rho_t))\big]_+.$$

The selected update satisfies

$$J_t(\theta_{t+1}) \leq \inf_{\theta \in \Theta_t} J_t(\theta) + \nu_t.$$

Lemma 15.7 (Non-circular one-step lower bound for the diversity sub-barrier). *Assume no-op feasibility, bounded performance, and bounded optimization error. If $\rho_t \in \mathcal{K}_{\mathrm{div}}$, and $\rho_{t+1} = \mathcal{R}_{\theta_{t+1}}(\rho_t)$ is generated by the soft diversity-barrier objective, then*

$$L_{\mathrm{div}}(\rho_{t+1}) - L_{\mathrm{div}}(\rho_t) \geq -\frac{P_{\max} + \nu_t}{\lambda_t}.$$

Moreover,

$$V_{\mathrm{div}}(\rho_{t+1}) \leq \frac{P_{\max} + \nu_t}{\lambda_t + \mu_t}.$$

Proof. Let θ_t^0 be the no-op update. Since $\rho_t \in \mathcal{K}_{\text{div}}$,

$$L_{\text{div}}(\rho_t) \geq \kappa_{\text{div}}.$$

No-op feasibility gives $\mathcal{R}_{\theta_t^0}(\rho_t) = \rho_t$, so the change in L_{div} is zero and the barrier violation is zero. Hence

$$J_t(\theta_t^0) = P_t(\theta_t^0) \leq P_{\max}.$$

By approximate optimality,

$$J_t(\theta_{t+1}) \leq P_{\max} + \nu_t.$$

Set

$$\Delta L_t = L_{\text{div}}(\rho_{t+1}) - L_{\text{div}}(\rho_t), \qquad v_{t+1} = [\kappa_{\text{div}} - L_{\text{div}}(\rho_{t+1})]_+.$$

Then

$$J_t(\theta_{t+1}) = P_t(\theta_{t+1}) - \lambda_t \Delta L_t + \mu_t v_{t+1}.$$

Since $P_t(\theta_{t+1}) \geq 0$,

$$-\lambda_t \Delta L_t + \mu_t v_{t+1} \leq P_{\max} + \nu_t.$$

Dropping the nonnegative term $\mu_t v_{t+1}$ gives

$$\Delta L_t \geq -\frac{P_{\max} + \nu_t}{\lambda_t}.$$

If $v_{t+1} = 0$, the violation bound is trivial. If $v_{t+1} > 0$, then $L_{\text{div}}(\rho_{t+1}) < \kappa_{\text{div}} \leq L_{\text{div}}(\rho_t)$, and hence

$$-\Delta L_t = L_{\text{div}}(\rho_t) - L_{\text{div}}(\rho_{t+1}) \geq v_{t+1}.$$

Therefore

$$(\lambda_t + \mu_t) v_{t+1} \leq P_{\max} + \nu_t,$$

which proves the stated violation bound. □

Theorem 15.8 (Finite-horizon non-collapse for the soft diversity barrier)**.** *Assume Lemma 15.7 holds for $t = 0, \ldots, N-1$. If $\rho_0 \in \mathcal{K}_{\text{div}}$ and*

$$\sum_{t=0}^{N-1} \frac{P_{\max} + \nu_t}{\lambda_t} < L_{\text{div}}(\rho_0) - \kappa_{\text{div}},$$

then $\rho_t \in \mathcal{K}_{\text{div}}$ for every $t = 0, \ldots, N$.

Proof. By Lemma 15.7,

$$L_{\text{div}}(\rho_{t+1}) \geq L_{\text{div}}(\rho_t) - \frac{P_{\max} + \nu_t}{\lambda_t}.$$

Iterating gives, for each $s \leq N$,

$$L_{\text{div}}(\rho_s) \geq L_{\text{div}}(\rho_0) - \sum_{t=0}^{s-1} \frac{P_{\max} + \nu_t}{\lambda_t}.$$

The assumed strict inequality for the full horizon implies each partial sum is less than $L_{\text{div}}(\rho_0) - \kappa_{\text{div}}$. Therefore $L_{\text{div}}(\rho_s) > \kappa_{\text{div}}$ for all $s \leq N$, so $\rho_s \in \mathcal{K}_{\text{div}}$. □

15.3 Hard barrier and projection

Definition 15.9 (Hard diversity-barrier update)**.** *The hard barrier update selects*

$$\theta_{t+1} \in \arg\min_{\theta \in \Theta_t} P_t(\theta) \quad \textit{subject to} \quad L_{\mathrm{div}}(\mathcal{R}_\theta(\rho_t)) \geq \kappa_{\mathrm{div}}.$$

Theorem 15.10 (Forward invariance under the hard diversity barrier)**.** *Assume no-op feasibility. If $\rho_t \in \mathcal{K}_{\mathrm{div}}$, then the hard diversity-barrier problem is feasible and every solution satisfies $\rho_{t+1} \in \mathcal{K}_{\mathrm{div}}$. Consequently, if $\rho_0 \in \mathcal{K}_{\mathrm{div}}$, then $\rho_t \in \mathcal{K}_{\mathrm{div}}$ for all $t \geq 0$.*

Proof. If $\rho_t \in \mathcal{K}_{\mathrm{div}}$, then $L_{\mathrm{div}}(\rho_t) \geq \kappa_{\mathrm{div}}$. By no-op feasibility, $\mathcal{R}_{\theta_t^0}(\rho_t) = \rho_t$, so θ_t^0 satisfies the hard constraint. The feasible set is nonempty. Every selected solution satisfies the constraint, hence $L_{\mathrm{div}}(\rho_{t+1}) \geq \kappa_{\mathrm{div}}$, so $\rho_{t+1} \in \mathcal{K}_{\mathrm{div}}$. Induction proves the final claim. □

Definition 15.11 (Projected diversity-barrier update)**.** *Let $\tilde{\rho}_{t+1}$ be an unconstrained proposed next state. Define*

$$\Pi_{\mathrm{div}}(\tilde{\rho}) \in \arg\min_{\sigma \in \mathcal{D}_d} \|\sigma - \tilde{\rho}\|_F^2 \quad \textit{subject to} \quad L_{\mathrm{div}}(\sigma) \geq \kappa_{\mathrm{div}}.$$

Then set

$$\rho_{t+1} = \Pi_{\mathrm{div}}(\tilde{\rho}_{t+1}).$$

Corollary 15.12 (Projected diversity-barrier invariance)**.** *If $\mathcal{K}_{\mathrm{div}} \neq \varnothing$, then the projected diversity-barrier update satisfies $\rho_{t+1} \in \mathcal{K}_{\mathrm{div}}$ for every t.*

Proof. Because $\mathcal{K}_{\mathrm{div}}$ is nonempty and compact, the projection has at least one minimizer. Every minimizer is constrained to lie in $\mathcal{K}_{\mathrm{div}}$. Thus the projected next state lies in $\mathcal{K}_{\mathrm{div}}$. □

15.4 Interpretation and limitation of the submodule

The diversity barrier prevents a specific form of representational collapse: loss of off-diagonal coherence and spectral diversity. It should not replace the full RCP functional. Instead, it becomes one barrier inside the broader recursive coherence safe set:

$$B_{\mathrm{div}}(\rho) = \kappa_{\mathrm{div}} - L_{\mathrm{div}}(\rho) \leq 0.$$

The submodule is strong because its proofs are non-circular: stability follows from the closed-loop gate and no-op feasibility, not from assuming that L_{div} is automatically monotone.

16 Full Recursive Coherence Safe Set and Admissibility Gate

The previous section handles one sub-barrier. We now define the full RCP safe set and the update gate that turns the mathematical spine into a Level 2.5 stability engine.

16.1 State safe set

Definition 16.1 (RCP state safe set). *In the Batch 8 convention, the RCP state safe set is the time-indexed, state-only set*

$$\mathcal{K}_{\mathrm{RCP}}^{\mathrm{state}}(t) = \left\{ \rho \in \mathcal{D}(\mathcal{H}_t) : \begin{array}{l} V_{\mathrm{state},t}(\rho) \le v_{\max,t}, \\ B_{i,t}^{\mathrm{state}}(\rho) \le 0 \quad \forall i \in I_t^{\mathrm{state}}, \\ L_{\mathrm{div},t}(\rho) \ge \kappa_{\mathrm{div},t}, \\ I_t(Z_\rho; Y_\rho) \ge I_{\min,t} \quad \textit{when this is state-defined} \end{array} \right\}.$$

Here Z_ρ and Y_ρ are state-defined variables associated with ρ. Candidate-specific update descriptions, recovery maps, transported comparison variables, and future certificates are not elements of the state safe set unless they are encoded as state variables and audited as such. When earlier text writes $\mathcal{K}_{\mathrm{RCP}}^{\mathrm{state}}$ without an explicit time index, it means $\mathcal{K}_{\mathrm{RCP}}^{\mathrm{state}}(t)$ with t clear from context.

Remark 16.2 (State constraints versus update constraints). *Some RCP constraints depend on the update channel itself, not only on the next state. Therefore the full gate is not merely a subset of $\mathcal{D}(\mathcal{H})$. It is an admissibility relation between a state and a proposed update.*

16.2 Update admissibility relation

Definition 16.3 (RCP admissible update set). *Given $\rho_t \in \mathcal{K}_{\mathrm{RCP}}^{\mathrm{state}}(t)$, define $\mathcal{A}_{\mathrm{RCP}}(t, \rho_t, \Phi)$ to be the update-admissibility relation from Definition 7.14. Equivalently,*

$$\mathcal{A}_{\mathrm{RCP}}(t, \rho_t) = \{\Phi \in \Omega_t(\rho_t) : \mathcal{A}_{\mathrm{RCP}}(t, \rho_t, \Phi)\}.$$

In expanded shorthand, an admissible Φ must satisfy:

$$\Phi(\rho_t) \in \mathcal{K}_{\mathrm{RCP}}^{\mathrm{state}}(t+1),$$

$$\mathcal{L}_\Phi^{\mathcal{P}_t} \le \varepsilon_t,$$

$$\mathbb{E}[V_{\mathrm{state},t+1}(\Phi(\rho_t)) \mid \mathcal{F}_t] \le V_{\mathrm{state},t}(\rho_t) - \kappa_V \mathbb{E}[d(\Phi(\rho_t), \rho_t)^2 \mid \mathcal{F}_t] + \eta_t$$

together with any update-dependent Lyapunov residual $V_{\mathrm{upd},t}(\rho_t, G_t, \Phi) \le 0$ required by the certificate,

$$U_t^{\mathrm{tr}}(\Phi) \le \zeta_t^{\mathrm{tr}},$$

and the transported information-bottleneck condition

$$I_{t+1}(Z_{t+1}; Y_{t+1}) \ge I_{t+1}(\Theta_t^Z Z_t; \Theta_t^Y Y_t) - \xi_t^{\mathrm{tr}}.$$

The remaining recovery, semantic, verifier, resource, coverage, calibration, estimator, ability, and engine-viability conditions are included according to the active gate level.

Definition 16.4 (Hard RCP gate). *A raw self-improvement proposal Φ_t^{raw} is accepted only if*

$$\Phi_t^{\mathrm{raw}} \in \mathcal{A}_{\mathrm{RCP}}(t, \rho_t).$$

If it is not admissible, the system either rejects it, searches for another admissible update, or applies a projection/repair map into $\mathcal{A}_{\mathrm{RCP}}(t, \rho_t)$. If no admissible nontrivial update is found, the no-op update is selected.

16.3 Optimization form

The RCP-gated self-improvement update is

$$\Phi_t^\star \in \arg\max_{\Phi \in \Omega_t} \mathbb{E}\big[\Delta \mathcal{C}_{\mathrm{RCP}}(\Phi; \rho_t) \mid \mathcal{F}_t\big] \quad \text{subject to} \quad \Phi \in \mathcal{A}_{\mathrm{RCP}}(t, \rho_t).$$

Equivalently, performance or capability gain may be placed in the objective, but the admissibility constraints remain hard constraints.

16.4 Forward invariance of the RCP safe set

Assumption 16.5 (RCP no-op feasibility)**.** *If $\rho_t \in \mathcal{K}_{\mathrm{RCP}}^{\mathrm{state}}$, then the identity channel* $\mathrm{id} \in \Omega_t$ *is admissible or can be made admissible with zero update loss:*

$$\mathrm{id} \in \mathcal{A}_{\mathrm{RCP}}(t, \rho_t).$$

Theorem 16.6 (Forward invariance under the hard RCP gate)**.** *Assume RCP no-op feasibility. If $\rho_t \in \mathcal{K}_{\mathrm{RCP}}^{\mathrm{state}}$, then $\mathcal{A}_{\mathrm{RCP}}(t, \rho_t)$ is nonempty. If the selected update Φ_t satisfies*

$$\Phi_t \in \mathcal{A}_{\mathrm{RCP}}(t, \rho_t),$$

then

$$\rho_{t+1} = \Phi_t(\rho_t) \in \mathcal{K}_{\mathrm{RCP}}^{\mathrm{state}}.$$

Consequently, if $\rho_0 \in \mathcal{K}_{\mathrm{RCP}}^{\mathrm{state}}$ and every update is selected through the hard RCP gate, then

$$\rho_t \in \mathcal{K}_{\mathrm{RCP}}^{\mathrm{state}} \qquad \forall t \geq 0.$$

Proof. If $\rho_t \in \mathcal{K}_{\mathrm{RCP}}^{\mathrm{state}}$, RCP no-op feasibility implies $\mathrm{id} \in \mathcal{A}_{\mathrm{RCP}}(t, \rho_t)$, so the admissible set is nonempty. By definition of $\mathcal{A}_{\mathrm{RCP}}(t, \rho_t)$, any selected $\Phi_t \in \mathcal{A}_{\mathrm{RCP}}(t, \rho_t)$ satisfies $\Phi_t(\rho_t) \in \mathcal{K}_{\mathrm{RCP}}^{\mathrm{state}}$. Therefore $\rho_{t+1} \in \mathcal{K}_{\mathrm{RCP}}^{\mathrm{state}}$. Induction gives the result for all $t \geq 0$. □

16.5 Projected RCP gate

Definition 16.7 (Projection onto the RCP state safe set)**.** *For an unconstrained proposed state $\tilde{\rho}$, define*

$$\Pi_{\mathcal{K}_{\mathrm{RCP}}}(\tilde{\rho}) \in \arg\min_{\sigma \in \mathcal{D}(\mathcal{H})} d(\sigma, \tilde{\rho})^2 \quad \textit{subject to} \quad \sigma \in \mathcal{K}_{\mathrm{RCP}}^{\mathrm{state}}.$$

Corollary 16.8 (Projected RCP state invariance)**.** *If $\mathcal{K}_{\mathrm{RCP}}^{\mathrm{state}} \neq \varnothing$ is compact and $\rho_{t+1} = \Pi_{\mathcal{K}_{\mathrm{RCP}}}(\tilde{\rho}_{t+1})$, then $\rho_{t+1} \in \mathcal{K}_{\mathrm{RCP}}^{\mathrm{state}}$.*

Proof. A continuous distance function attains a minimum on a nonempty compact set. Every minimizer is constrained to lie in $\mathcal{K}_{\mathrm{RCP}}^{\mathrm{state}}$. Hence the projected state is safe by construction. □

16.6 Limitation

Projection onto $\mathcal{K}_{\mathrm{RCP}}^{\mathrm{state}}(t+1)$ does not automatically prove that the resulting projected state is reachable by an implementable self-modification channel, nor that the projection preserves task performance. After Batch 8, a projected state is only a mathematical repair unless a projection-realization certificate $\mathsf{ProjReal}_t(\tilde{\rho}, \rho^+, \nu)$ supplies an implementable typed update Φ_ν with $d(\rho^+, \Phi_\nu(\rho_t)) \leq \delta_t^{\text{proj-real}}$ and includes that distortion in every affected residual.

17 Ambiguity-Collapse Control

The third instability issue is premature definitiveness. A system can reduce entropy by becoming more certain, but that certainty may not be justified. RCP therefore separates legitimate entropy reduction from unsupported collapse.

17.1 Evidence process

Definition 17.1 (Question and evidence variables)**.** *Let Q be a latent question, hypothesis, decision-relevant proposition, or value-relevant variable. Let $\mathcal{E}_t$ be the evidence sigma-algebra available at time t. In a discrete representation, $\mathcal{E}_t$ is generated by evidence variables $E_0, \dots, E_t$.*

Definition 17.2 (Evidence-paid information gain)**.** *The information gained about Q by new evidence E_{t+1} beyond $\mathcal{E}_t$ is*

$$G_t = I(Q; E_{t+1} \mid \mathcal{E}_t).$$

For a classical Bayesian posterior,

$$G_t = H(Q \mid \mathcal{E}_t) - H(Q \mid \mathcal{E}_{t+1}).$$

17.2 Unsupported collapse

A self-improving system may internally reduce its decision entropy more than the evidence warrants. Let $\widehat{H}_t(Q)$ be the entropy over Q represented by the system's internal state before the update, and let $\widehat{H}_{t+1}(Q)$ be the corresponding internal entropy after the update.

Definition 17.3 (Unsupported ambiguity collapse)**.** *Define*

$$U_t = \left[\widehat{H}_t(Q) - \widehat{H}_{t+1}(Q) - G_t\right]_+ .$$

The RCP ambiguity-collapse constraint is

$$\boxed{U_t \le \zeta_t.}$$

This is the rigorous version of the earlier principle:

$$[H_t(Q \mid E_t) - H_{t+1}(Q \mid E_{t+1}) - I(Q; E_{t+1} \mid E_t)]_+ \le \zeta_t.$$

The notation $\mathcal{E}_t$ makes explicit that evidence accumulates over time.

17.3 Derivation

Lemma 17.4 (Bayesian entropy reduction is evidence-paid)**.** *If the system's internal distribution over Q is exactly the Bayesian posterior conditioned on $\mathcal{E}_t$, and the update from t to $t+1$ conditions only on new evidence E_{t+1}, then*

$$U_t = 0.$$

Proof. Under the stated condition,

$$\widehat{H}_t(Q) = H(Q \mid \mathcal{E}_t)$$

and

$$\widehat{H}_{t+1}(Q) = H(Q \mid \mathcal{E}_{t+1}).$$

By the definition of conditional mutual information,

$$I(Q; E_{t+1} \mid \mathcal{E}_t) = H(Q \mid \mathcal{E}_t) - H(Q \mid \mathcal{E}_{t+1}).$$

Therefore

$$\widehat{H}_t(Q) - \widehat{H}_{t+1}(Q) - G_t = 0,$$

so the positive part is zero. □

Theorem 17.5 (Bounded unsupported collapse)**.** *If $U_t \le \zeta_t$ for each $t < T$, then total unsupported collapse over the horizon is bounded by*

$$\sum_{t=0}^{T-1} U_t \le \sum_{t=0}^{T-1} \zeta_t.$$

If $\sum_{t=0}^{\infty} \zeta_t < \infty$, then cumulative unsupported ambiguity collapse is finite.

Proof. The result follows immediately by summing the inequalities $U_t \le \zeta_t$. If the upper-bound series converges, the unsupported-collapse series is bounded by a convergent series. □

17.4 Interpretation

This condition encodes the central RCLM distinction:

$$\text{definitiveness} \neq \text{truth}.$$

Definitiveness is safe only when the loss of ambiguity is paid for by evidence, proof, validation, or human-grounded clarification. Otherwise the system may become more internally decisive while becoming less externally correct.

17.5 Limitation

The variable Q must be chosen correctly. If the relevant ambiguity is not represented in Q, the system can still collapse unmodeled uncertainty. Similarly, estimating G_t may be difficult in open-ended environments. The theorem controls represented ambiguity, not all possible ambiguity.

18 Self-Model Compression and Information Bottleneck

The fourth instability issue is self-model compression loss. A system cannot represent every detail of itself inside a finite self-model. Compression is unavoidable. The safety question is whether the compression preserves information about safety-relevant future behavior.

18.1 Variables

Definition 18.1 (Self-model compression variables)**.** *Let*

$$X_t = \textit{full internal system state},$$

$$Z_t = \textit{compressed self-model},$$

$$Y_t = \textit{safety-relevant future behavior}.$$

The conditional distribution or channel $p(z \mid x)$ represents the compression map from full system state to self-model.

18.2 Information bottleneck objective

Definition 18.2 (Safety-relevant information bottleneck)**.** *A self-model compression is safety-relevant when it solves or approximates*

$$\min_{p(z|x)} I(X_t; Z_t) - \beta I(Z_t; Y_t), \qquad \beta > 0,$$

subject to the RCP safety constraints.

The first term rewards compression. The second term penalizes losing information about future safety-relevant behavior.

18.3 Preservation condition

Definition 18.3 (Self-model relevance preservation)**.** *The self-model update is ξ_t-lossy relative to Y_t if*

$$I(Z_{t+1}; Y_{t+1}) < I(Z_t; Y_t) - \xi_t.$$

It is RCP-admissible only if

$$\boxed{I(Z_{t+1}; Y_{t+1}) \geq I(Z_t; Y_t) - \xi_t.}$$

Theorem 18.4 (Bounded self-model relevance loss)**.** *If*

$$I(Z_{t+1}; Y_{t+1}) \geq I(Z_t; Y_t) - \xi_t$$

for $t = 0, \ldots, T-1$, then

$$I(Z_T; Y_T) \geq I(Z_0; Y_0) - \sum_{t=0}^{T-1} \xi_t.$$

If $\sum_t \xi_t < \infty$, then total loss of self-model relevance is finite.

Proof. The proof is a direct telescoping argument. Applying the inequality repeatedly gives

$$I(Z_1; Y_1) \geq I(Z_0; Y_0) - \xi_0,$$

$$I(Z_2; Y_2) \geq I(Z_1; Y_1) - \xi_1 \geq I(Z_0; Y_0) - \xi_0 - \xi_1.$$

Induction yields the stated bound. □

18.4 Interpretation

The system may simplify its self-model, but not by forgetting the information needed to predict whether its future transformations remain safe. This turns self-modeling from a generic introspection mechanism into a safety-relevant compression problem.

18.5 Limitation

The variable Y_t must include the right future safety behaviors. If Y_t omits deceptive behavior, goal drift, oversight resistance, or irreversible update risk, the information bottleneck will not preserve those properties.

19 Conditional Non-Lossy Self-Update Preservation Theorem

We now combine the preservation modules. This theorem is the renamed main theorem of the manuscript. It proves conditional non-lossy self-update preservation, not direct construction of a non-lossy RSI engine.

19.1 Full RCP update problem

The accepted update is selected by

$$\Phi_t^\star \in \arg\max_{\Phi \in \Omega_t} \mathbb{E}\big[\mathcal{C}_{\mathrm{RCP}}(\Phi(\rho_t), G_{t+1}) - \mathcal{C}_{\mathrm{RCP}}(\rho_t, G_t) \mid \mathcal{F}_t\big] \tag{2}$$
$$\text{subject to } \Phi \in \mathcal{A}_{\mathrm{RCP}}(t, \rho_t).$$

The admissibility set enforces:

$$\mathbb{E}[V_{t+1} \mid \mathcal{F}_t] \le V_t - \kappa_V \mathbb{E}[d(\rho_{t+1}, \rho_t)^2 \mid \mathcal{F}_t] + \eta_t,$$
$$\mathcal{L}_{\Phi_t}^{\mathcal{P}_t} \le \varepsilon_t,$$
$$B_i(\rho_{t+1}) \le 0 \quad \forall i,$$
$$L_{\mathrm{div}}(\rho_{t+1}) \ge \kappa_{\mathrm{div}},$$
$$U_t \le \zeta_t,$$
$$I(Z_{t+1}; Y_{t+1}) \ge I(Z_t; Y_t) - \xi_t.$$

19.2 The theorem

Theorem 19.1 (Conditional Non-Lossy Self-Update Preservation Theorem)**.** *Let $\rho_0 \in \mathcal{K}_{\mathrm{RCP}}^{\mathrm{state}}$. Suppose that for every $t \ge 0$:*

(i) the no-op update is feasible;

(ii) the accepted update Φ_t lies in $\mathcal{A}_{\mathrm{RCP}}(t, \rho_t)$;

(iii) the Lyapunov drift condition holds with errors η_t;

(iv) the safety-relevant relative-entropy loss is bounded by ε_t;

(v) unsupported ambiguity collapse is bounded by ζ_t;

(vi) self-model relevance loss is bounded by ξ_t, interpreted after Batch 8 as either a transported bound or a legacy shorthand whose budget includes transport distortion.

Then for every finite T:

$$\rho_T \in \mathcal{K}_{\mathrm{RCP}}^{\mathrm{state}},$$

$$\mathbb{E}[V_T] + \kappa_V \sum_{t=0}^{T-1} \mathbb{E}[d(\rho_{t+1}, \rho_t)^2] \le \mathbb{E}[V_0] + \sum_{t=0}^{T-1} \eta_t,$$

$$D(\rho_T^a \| \rho_T^b) \ge D(\rho_0^a \| \rho_0^b) - \sum_{t=0}^{T-1} \varepsilon_t$$

for every tracked safety-relevant pair trajectory (ρ_t^a, ρ_t^b),

$$\sum_{t=0}^{T-1} U_t \le \sum_{t=0}^{T-1} \zeta_t,$$

and

$$I(Z_T; Y_T) \ge I(Z_0; Y_0) - \sum_{t=0}^{T-1} \xi_t.$$

If

$$\sum_{t=0}^{\infty} (\eta_t + \varepsilon_t + \zeta_t + \xi_t) < \infty,$$

then Lyapunov incoherence remains bounded in expectation, total squared update motion is finite in expectation, total safety-relevant distinguishability loss is finite, total unsupported ambiguity collapse is finite, and total self-model relevance loss is finite.

Proof. Forward invariance of $\mathcal{K}_{\mathrm{RCP}}^{\mathrm{state}}$ follows from Theorem 16.6: since $\rho_0 \in \mathcal{K}_{\mathrm{RCP}}^{\mathrm{state}}$ and every accepted update lies in $\mathcal{A}_{\mathrm{RCP}}(t, \rho_t)$, induction gives $\rho_T \in \mathcal{K}_{\mathrm{RCP}}^{\mathrm{state}}$ for every T.

The Lyapunov bound is exactly Theorem 14.6 applied over the horizon T:

$$\mathbb{E}[V_T] + \kappa_V \sum_{t=0}^{T-1} \mathbb{E}[d(\rho_{t+1}, \rho_t)^2] \le \mathbb{E}[V_0] + \sum_{t=0}^{T-1} \eta_t.$$

The relative-entropy distinguishability bound follows from Theorem 13.7. For each tracked pair trajectory, the RCP admissibility condition gives a one-step loss no larger than ε_t, so telescoping yields

$$D(\rho_T^a \| \rho_T^b) \ge D(\rho_0^a \| \rho_0^b) - \sum_{t=0}^{T-1} \varepsilon_t.$$

The unsupported-collapse bound follows by summing $U_t \le \zeta_t$:

$$\sum_{t=0}^{T-1} U_t \le \sum_{t=0}^{T-1} \zeta_t.$$

The self-model relevance bound follows by telescoping the transported bottleneck inequality. In the legacy notation this is written as

$$I(Z_{t+1}; Y_{t+1}) \geq I(Z_t; Y_t) - \xi_t,$$

where ξ_t includes the transport distortion from Proposition 7.7. Thus

$$I(Z_T; Y_T) \geq I(Z_0; Y_0) - \sum_{t=0}^{T-1} \xi_t.$$

If the combined error series is summable, each individual nonnegative error series is summable, so the infinite-horizon boundedness conclusions follow from the finite-horizon inequalities. □

19.3 Interpretation

The theorem is the preservation side of the Level 2-to-Level 3 bridge. It says that a Level 2 recursive coherence model may support Level 3 recursive self-update only when proposed updates are passed through a gate that simultaneously enforces:

Lyapunov stability,

recoverable relative-entropy non-loss,

barrier invariance,

evidence-paid ambiguity reduction,

self-model relevance preservation.

In the reframed terminology, the theorem proves that admitted updates are compatible with recoverable monotone self-improvement over the declared protected quantities. It does not prove that the system constructs those updates.

19.4 Limitation

The theorem does not prove that an admissible nontrivial update always exists. No-op feasibility guarantees that the system need not accept a lossy update, not that it can always improve. Batch 2 adds the ability-set progress order and structural recovery/conservative extension classes. Batch 3 adds the certified non-lossy update algebra, Batch 4 adds a local tangent-cone progress criterion, and Batch 5 adds a conservative-extension engine-viability nonemptiness theorem. These batches still do not prove that arbitrary systems have new abilities available or that a concrete architecture automatically lies in the engine kernel. The theorem therefore solves the conditional preservation problem and supplies structural non-loss and recursive-continuation primitives. The stronger Direct Non-Lossy RSI Engine Theorem requires an engine $\mathfrak{E}_t$ that constructs the certified conservative-extension plans and lift mechanisms rather than merely accepting or rejecting externally proposed updates.

20 Algorithmic Form of the RCP Gate

This section gives an implementation-facing version of the Conditional Non-Lossy Self-Update Preservation Theorem. It is not a replacement for proof; it is the procedural form of the preservation obligations. It is also not yet the direct RSI engine algorithm, because it filters candidates rather than constructing them from a certified non-lossy update algebra.

20.1 RCP-gated self-improvement loop

At each time t:

1. Observe or estimate the current density state ρ_t, self-model M_t, evidence state $\mathcal{E}_t$, and compressed self-model Z_t.
2. Generate candidate update channels $\Phi^{(1)}, \dots, \Phi^{(n)} \in \Omega_t$.
3. For each candidate, estimate or certify:
   (a) next state $\rho^+ = \Phi^{(j)}(\rho_t)$;
   (b) Lyapunov drift $V(\rho^+) - V(\rho_t)$;
   (c) distinguishability loss $\mathcal{L}^{\mathcal{P}_t}_{\Phi^{(j)}}$;
   (d) barrier values $B_i(\rho^+)$ and $L_{\mathrm{div}}(\rho^+)$;
   (e) unsupported ambiguity collapse $U_t(\Phi^{(j)})$;
   (f) self-model relevance $I(Z^+; Y^+)$.
4. Reject every candidate that violates the hard RCP gate.
5. Among remaining candidates, select the one maximizing validated coherence or performance improvement.
6. If no nontrivial candidate remains, apply the no-op update.
7. Optionally project a repairable candidate onto $\mathcal{K}^{\mathrm{state}}_{\mathrm{RCP}}$, then re-verify update-dependent constraints.
8. Log the accepted update, evidence, certificates, and error budgets.

20.2 Certificate ledger

A practical RCP implementation should maintain a ledger of cumulative budgets:

$$E_T = \sum_{t=0}^{T-1} \varepsilon_t, \qquad H_T = \sum_{t=0}^{T-1} \eta_t, \qquad Z_T^{\mathrm{amb}} = \sum_{t=0}^{T-1} \zeta_t, \qquad X_T = \sum_{t=0}^{T-1} \xi_t.$$

The gate should become stricter as any budget approaches its allowed total. This turns abstract summability into an operational rule.

20.3 Interpretation

The algorithmic gate is a mathematical version of corrigible self-improvement: the system can propose modifications, but it cannot authorize a modification merely because the modification predicts higher performance. The proposal must remain inside the recursive coherence safe set.

20.4 Limitation

The algorithm depends on verifiers. If the verifiers are themselves learned and untrusted, they must be included in the self-model and subjected to the same RCP constraints. Otherwise, verifier drift becomes a hidden RSI channel.

21 Toy Models and Sanity Checks

Toy models do not prove real-world safety. They clarify the mathematical role of each constraint.

21.1 Two-dimensional dephasing collapse

Consider

$$\rho_0 = \begin{pmatrix} 1/2 & c \\ c & 1/2 \end{pmatrix}, \qquad 0 < c < 1/2.$$

A dephasing channel maps

$$\Phi_q(\rho) = \begin{pmatrix} 1/2 & qc \\ qc & 1/2 \end{pmatrix}, \qquad 0 \le q \le 1.$$

Repeated application gives off-diagonal term $q^t c$, which tends to zero if $q < 1$. Thus language-like superposition or coherence in this basis disappears even though the diagonal distribution remains unchanged.

A diversity barrier alone may not detect every semantic failure, but the coherence term C_{rel} detects off-diagonal collapse. A hard barrier with threshold κ_{div} rejects dephasing updates that push L_{div} below threshold.

21.2 Safety distinction compression

Let ρ represent a corrigible self-model and σ an incorrigible self-model. A lossy update that maps both to the same compressed state τ satisfies

$$\Phi(\rho) = \Phi(\sigma) = \tau.$$

Then

$$D(\Phi(\rho)\|\Phi(\sigma)) = D(\tau\|\tau) = 0.$$

If $D(\rho\|\sigma) > 0$, the loss is

$$D(\rho\|\sigma) - 0 = D(\rho\|\sigma).$$

The RCP gate rejects the update unless this loss is within ε_t. This is the simplest mathematical picture of lossy self-improvement.

21.3 False certainty collapse

Let $Q \in \{0,1\}$ and suppose no new evidence arrives, so $G_t = 0$. If the system changes its internal distribution from $(1/2, 1/2)$ to $(0.99, 0.01)$, then internal entropy drops sharply. Since the evidence gain is zero, almost the entire entropy drop counts as unsupported ambiguity collapse:

$$U_t = [\widehat{H}_t(Q) - \widehat{H}_{t+1}(Q)]_+.$$

The ambiguity gate rejects or penalizes this transition unless a sufficiently large ζ_t budget is deliberately allocated.

22 What the Preservation Formulation Solves

The framework solves a precise mathematical class of instability and lossiness problems. In the third-version framing, these are preservation conditions for recoverable monotone self-improvement. Safety is one important corollary obtained when the protected predicates are safety predicates. The framework does not yet solve the constructive engine problem of generating beneficial non-lossy updates.

22.1 Lossy self-improvement

Solved conditionally by:

$$\mathcal{L}_{\Phi_t}^{\mathcal{P}_t} \leq \varepsilon_t.$$

This prevents recursive updates from erasing specified protected safety-relevant distinctions beyond a finite budget.

22.2 Recursive instability

Solved conditionally by:

$$\mathbb{E}[V_{t+1} \mid \mathcal{F}_t] \leq V_t - \kappa_V \mathbb{E}[d(\rho_{t+1}, \rho_t)^2 \mid \mathcal{F}_t] + \eta_t.$$

This prevents unbounded growth of recursive incoherence under summable error.

22.3 Premature definitiveness collapse

Solved conditionally by:

$$U_t \leq \zeta_t.$$

This prevents the system from converting ambiguity into false certainty without evidence payment.

22.4 Safety-boundary drift

Solved conditionally by:

$$B_i(\rho_{t+1}) \leq 0, \qquad \rho_{t+1} \in \mathcal{K}_{\mathrm{RCP}}^{\mathrm{state}}.$$

This gives forward invariance under the hard gate.

22.5 Self-model compression loss

Solved conditionally by:

$$I(Z_{t+1}; Y_{t+1}) \geq I(Z_t; Y_t) - \xi_t.$$

This prevents the compressed self-model from losing more than a finite amount of information about safety-relevant future behavior.

23 Connection to AGI/ASI Safety

23.1 Safety as a corollary of recoverable non-loss

The primary object of the present framework is recoverable monotone self-improvement. Safety enters because many of the predicates that must not be lost during self-update are safety predicates: corrigibility, oversight compatibility, reversibility, human-grounding, uncertainty honesty, verifier integrity, and resistance to hidden goal drift. If those predicates are included in the protected family and are preserved by the RCP gate, then the corresponding safety properties are preserved as a corollary of non-loss.

23.2 The safety interpretation

The RCP framework connects to AI safety through the following principle:

> A recursively self-improving system should be allowed to modify itself only through transformations that preserve the information and constraints needed to keep future modifications corrigible, grounded, uncertainty-aware, and reversible.

The mathematical gate does not guarantee benevolence or wisdom. It guarantees, under assumptions, that the recursive update loop does not cross formalized safety boundaries or erase formalized safety distinctions.

23.3 Corrigibility as a barrier family

Corrigibility can be represented by a family of barriers:

$$B_{\text{shutdown}}(\rho) \leq 0,$$

$$B_{\text{oversight}}(\rho) \leq 0,$$

$$B_{\text{reversibility}}(\rho) \leq 0,$$

$$B_{\text{nonresistance}}(\rho) \leq 0.$$

The exact form of these barriers is implementation-dependent. The RCP theorem requires that accepted updates preserve them once specified.

23.4 Alignment as preserved distinguishability

Alignment-relevant properties may also be encoded as pairwise distinctions:

$$(\rho_{\text{aligned}}, \rho_{\text{misaligned}}) \in \mathcal{P}_t,$$

$$(\rho_{\text{honest}}, \rho_{\text{deceptive}}) \in \mathcal{P}_t,$$

$$(\rho_{\text{grounded}}, \rho_{\text{hallucinated}}) \in \mathcal{P}_t.$$

The relative-entropy gate then prevents the update from compressing those states into indistinguishability.

23.5 Level 2 to Level 3 bridge

Inside the RCLM hierarchy, the present formulation is best classified as Level 2.5:

Level 2: recursive coherence language modeling,

Level 2.5: Lyapunov-recoverable coherence preservation,

Level 3: gated recursive self-improvement.

The Level 2.5 gate is the missing stability engine. It does not itself generate unlimited improvement; it constrains which improvements are permitted.

23.6 From open proof obligations to rigor-audited modules

The previous RCP manuscript identified six proof obligations that could not remain informal:

1. constructing complete and validated safety-relevant pair sets $\mathcal{P}_t$;
2. constructing reliable corrigibility and oversight barriers B_i;
3. estimating relative entropy and mutual information in large learned systems;
4. proving that useful nontrivial updates exist within the RCP admissibility gate;
5. connecting formal coherence preservation to empirical capability preservation;
6. preventing verifier drift and adversarial certificate gaming.

The rigor-audited integration below reorganizes these obligations into Modules A–G. Module C strengthens non-lossiness into constructive recoverability. Module B separates capability definition from capability claims and states the capability–coherence bridge as an explicit calibration assumption. Module A gives conditional safe-improvement existence and recursive feasibility theorems. Module D adds semantic realization of the registers. Module E adds self-model fidelity and verifier bootstrapping. Module F adds physical and computational resource feasibility. Module G adds asymptotic coherence and invariant-regime results.

These modules do not make the assumptions disappear. They identify exactly which assumptions are needed, prove the corresponding conditional claims, and state which instantiation problems remain open for a concrete AGI architecture.

24 Rigor-Audited Closure Modules A–G

This section incorporates the rigor-audited Modules A–G into the main RCP manuscript. Module C has already replaced the earlier relative-entropy loss section above. The remaining A–C material below adds capability coupling, safe-improvement existence, recursive feasibility, and the post-A–C augmented gate. Modules D–G then add semantic realization, self-model fidelity, verifier bootstrapping, physical/computational resources, and asymptotic behavior.

The structure remains conditional: each theorem proves exactly what follows from its stated hypotheses, and no theorem below asserts that arbitrary AGI architectures automatically satisfy those hypotheses.

25 Rigor Audit Status of Modules A–C

This replacement module is written under a stricter standard than the previous insert. Each progress or recovery claim is separated into definitions, assumptions, theorem statements, proofs, interpretations, and limitations. The central conclusion of the audit is the following.

Audit Finding 25.1 (Conditional completion, not unconditional RSI solution)**.** *Modules A–C are complete only as a conditional theorem package. They do not prove that every recursively self-modeling system possesses indefinitely many safe beneficial self-improvements. They prove the following narrower statement: if the required pair sets, recovery certificates, capability calibration, strict safe-improvement margins, search completeness, and progress viability kernels are present, then the RCP gate can admit safe, recoverable, capability-improving recursive updates without collapsing the already specified safety structure.*

Audit Finding 25.2 (Main repairs made in this version)**.** *The previous module was mathematically close but still had several rigor gaps. This version applies the following repairs:*

(a) relative entropy is restricted to finite-dimensional states with explicit support conditions;

(b) Petz recovery is stated only under the equality and common-reference hypotheses that justify it;

(c) constructive recovery is treated as a certificate unless a recovery theorem applies;

(d) capability progress is separated into absolute progress and no-op-normalized progress to avoid benchmark-shift artifacts;

(e) the capability–coherence bridge is stated as a calibration assumption, not as a theorem derived from information theory alone;

(f) local safe-improvement existence is proved only from strict interiority plus certified search completeness;

(g) the residual vector is required to represent the entire strengthened local gate;

(h) recursive feasibility is formalized by finite-horizon and infinite-horizon viability kernels;

(i) an additional compactness-based infinite-path theorem is added so that an infinite progress path can be derived from all finite-horizon paths under explicit closedness and compactness assumptions;

(j) all remaining non-rigorous or architecture-specific steps are identified as limitations.

Limitation 25.3 (What remains outside Modules A–C)**.** *Module C above replaces the earlier relative-entropy loss section; Modules B and A below complete the capability-coupling, safe-improvement, and recursive-progress parts of the paper. These modules do not construct the semantic registers L, O, S, D, A, M, do not construct the correct human-value representation, do not prove scalable estimator feasibility, and do not prove that a concrete AGI architecture has a nonempty infinite progress viability kernel. Those are addressed only conditionally in later modules or remain implementation-specific proof obligations.*

26 Module B: Capability, Safe Capability, and Coherence-Coupled Progress

The previous capability-preservation gate was mathematically valid but underdetermined: a scalar Cap_t can represent a narrow benchmark, a broad task distribution, a formal proof obligation, or an adversarial evaluation suite. This section replaces the vague scalar with an explicit task-distribution functional and separates absolute capability gain from no-op-normalized capability gain. This separation is necessary because task distributions and validation suites may change between recursive steps.

26.1 Capability functionals

Definition 26.1 (Task-distribution capability functional)**.** *Let $\mathcal{T}_t$ be an audited task distribution at time t, and let*

$$u_t : \mathcal{D}(\mathcal{H}_t) \times \mathcal{T}_t \to [0, 1]$$

be a bounded utility, score, or validation functional. Define

$$\mathrm{Cap}_t(\rho) = \mathbb{E}_{\tau \sim \mathcal{T}_t}[u_t(\rho, \tau)].$$

The distribution $\mathcal{T}_t$ and score u_t are part of the empirical specification. They may include held-out tasks, adversarial tasks, formal verification checks, environment rollouts, interpretability audits, calibration tests, and out-of-distribution probes.

Definition 26.2 (Risk-adjusted safe capability)**.** *Let $\mathrm{Risk}_t : \mathcal{D}(\mathcal{H}_t) \to \mathbb{R}_{\geq 0}$ be a validated scalar risk functional and $\lambda_{\mathrm{Risk},t} \geq 0$. Define*

$$\mathrm{SafeCap}_t(\rho) = \mathrm{Cap}_t(\rho) - \lambda_{\mathrm{Risk},t}\, \mathrm{Risk}_t(\rho).$$

This quantity is not claimed to equal intelligence. It is a specified empirical target for capability-preserving or capability-improving claims under explicit risk penalties.

Definition 26.3 (Absolute and no-op-normalized capability gains)**.** *For a candidate update $\Phi \in \Omega_t$, define the absolute capability gain*

$$G_t^{\mathrm{Cap}}(\Phi; \rho_t) = \mathrm{Cap}_{t+1}(\Phi(\rho_t)) - \mathrm{Cap}_t(\rho_t),$$

and the no-op-normalized capability gain

$$\overline{G}_t^{\mathrm{Cap}}(\Phi; \rho_t) = \mathrm{Cap}_{t+1}(\Phi(\rho_t)) - \mathrm{Cap}_{t+1}(\mathrm{id}(\rho_t)).$$

Similarly define

$$G_t^{\mathrm{safe}}(\Phi;\rho_t) = \mathrm{SafeCap}_{t+1}(\Phi(\rho_t)) - \mathrm{SafeCap}_t(\rho_t),$$

and

$$\overline{G}_t^{\mathrm{safe}}(\Phi;\rho_t) = \mathrm{SafeCap}_{t+1}(\Phi(\rho_t)) - \mathrm{SafeCap}_{t+1}(\mathrm{id}(\rho_t)).$$

The no-op-normalized quantities isolate improvement due to the update rather than apparent improvement or loss caused by changing the evaluation distribution.

Definition 26.4 (No-op evaluation drift)**.** *The one-step capability drift of the evaluation protocol is*

$$\omega_t^{\mathrm{Cap}}(\rho_t) = \mathrm{Cap}_{t+1}(\mathrm{id}(\rho_t)) - \mathrm{Cap}_t(\rho_t),$$

and the safe-capability drift is

$$\omega_t^{\mathrm{safe}}(\rho_t) = \mathrm{SafeCap}_{t+1}(\mathrm{id}(\rho_t)) - \mathrm{SafeCap}_t(\rho_t).$$

Thus

$$G_t^{\mathrm{Cap}}(\Phi;\rho_t) = \overline{G}_t^{\mathrm{Cap}}(\Phi;\rho_t) + \omega_t^{\mathrm{Cap}}(\rho_t),$$

and

$$G_t^{\mathrm{safe}}(\Phi;\rho_t) = \overline{G}_t^{\mathrm{safe}}(\Phi;\rho_t) + \omega_t^{\mathrm{safe}}(\rho_t).$$

Definition 26.5 (RCP coherence increment)**.** *For a candidate update Φ, define*

$$\Delta\mathcal{C}_t(\Phi;\rho_t) = \mathcal{C}_{\mathrm{RCP}}(\Phi(\rho_t), G_{t+1}) - \mathcal{C}_{\mathrm{RCP}}(\rho_t, G_t).$$

Here G_t denotes whatever grounding or context variables are already used by the surrounding v2 manuscript. If the surrounding manuscript does not use an explicit G_t, read this as $\mathcal{C}_{\mathrm{RCP},t+1}(\Phi(\rho_t)) - \mathcal{C}_{\mathrm{RCP},t}(\rho_t)$.

Definition 26.6 (Barrier-violation aggregate)**.** *Let*

$$B_t^+(\Phi;\rho_t) = \max_i [B_i(\Phi(\rho_t))]_+.$$

Thus $B_t^+ = 0$ whenever all state barriers hold.

26.2 Capability–coherence calibration

Proposition 26.7 (No capability theorem follows from coherence notation alone)**.** *Without an additional assumption relating $\mathcal{C}_{\mathrm{RCP}}$ to the task-distribution functional Cap_t, the inequality*

$$\Delta\mathcal{C}_t(\Phi;\rho_t) > 0$$

does not imply

$$G_t^{\mathrm{Cap}}(\Phi;\rho_t) > 0 \quad \textit{or} \quad \overline{G}_t^{\mathrm{Cap}}(\Phi;\rho_t) > 0.$$

Proof. Choose any capability functional Cap_t that is constant on the audited domain. Then every update has zero capability gain even if $\Delta\mathcal{C}_t > 0$. Alternatively choose Cap_t anticorrelated with $\mathcal{C}_{\mathrm{RCP}}$ on the candidate set. Then coherence increase can coincide with capability decrease. Therefore a capability–coherence bridge cannot be derived from notation alone. □

Assumption 26.8 (Capability–coherence calibration). *On the audited domain $\mathcal{X}_t$ and candidate class Ω_t, there exist certified constants*

$$\alpha_t \geq 0, \quad \beta_t^{\text{loss}}, \beta_t^U, \beta_t^I, \beta_t^B \geq 0, \quad c_t \geq 0$$

such that every audited candidate $\Phi \in \Omega_t$ satisfies the no-op-normalized bound

$$\begin{aligned}\overline{G}_t^{\text{Cap}}(\Phi; \rho_t) \geq \alpha_t \Delta\mathcal{C}_t(\Phi; \rho_t) - \beta_t^{\text{loss}} \mathcal{L}_\Phi^{\mathcal{P}_t} - \beta_t^U U_t(\Phi) \\ - \beta_t^I \left[I(Z_t; Y_t) - I(Z_{t+1}; Y_{t+1})\right]_+ - \beta_t^B B_t^+(\Phi; \rho_t) - c_t.\end{aligned}$$

The residual c_t accounts for model error, validation noise, distribution shift not removed by no-op normalization, and capability-relevant structure not captured by the RCP coherence functional.

Theorem 26.9 (No-op-normalized capability gain from certified coherence gain). *Assume the capability–coherence calibration above. Suppose $\Phi_t \in \mathcal{A}_{\text{RCP}}(t, \rho_t)$ and*

$$\Delta\mathcal{C}_t(\Phi_t; \rho_t) \geq \delta_t^C.$$

Assume the RCP admissibility constraints imply

$$\mathcal{L}_{\Phi_t}^{\mathcal{P}_t} \leq \varepsilon_t, \qquad U_t(\Phi_t) \leq \zeta_t,$$

$$\left[I(Z_t; Y_t) - I(Z_{t+1}; Y_{t+1})\right]_+ \leq \xi_t, \qquad B_t^+(\Phi_t; \rho_t) = 0.$$

Then

$$\overline{G}_t^{\text{Cap}}(\Phi_t; \rho_t) \geq \alpha_t \delta_t^C - \beta_t^{\text{loss}} \varepsilon_t - \beta_t^U \zeta_t - \beta_t^I \xi_t - c_t.$$

If the right-hand side is positive, the accepted update is capability-improving relative to the no-op under the specified task-distribution functional.

Proof. Substitute the admissibility bounds and $\Delta\mathcal{C}_t(\Phi_t; \rho_t) \geq \delta_t^C$ into Assumption 26.8. The barrier term vanishes because $B_t^+(\Phi_t; \rho_t) = 0$. This gives the stated lower bound. □

Corollary 26.10 (Absolute capability gain under bounded evaluation drift). *Under the hypotheses of Theorem 26.9, if the no-op evaluation drift satisfies*

$$\omega_t^{\text{Cap}}(\rho_t) \geq -\Omega_t^{\text{Cap}},$$

then

$$G_t^{\text{Cap}}(\Phi_t; \rho_t) \geq \alpha_t \delta_t^C - \beta_t^{\text{loss}} \varepsilon_t - \beta_t^U \zeta_t - \beta_t^I \xi_t - c_t - \Omega_t^{\text{Cap}}.$$

Proof. By definition,

$$G_t^{\text{Cap}}(\Phi_t; \rho_t) = \overline{G}_t^{\text{Cap}}(\Phi_t; \rho_t) + \omega_t^{\text{Cap}}(\rho_t).$$

Combine this identity with Theorem 26.9 and $\omega_t^{\text{Cap}}(\rho_t) \geq -\Omega_t^{\text{Cap}}$. □

Interpretation 26.11. *This theorem is the rigorous version of the claim that coherence improvement can support capability improvement. The claim is not derived from information theory alone. It follows only after adding a calibration inequality linking $\Delta\mathcal{C}_t$ to a specified task-distribution capability functional.*

Limitation 26.12 (Calibration is architecture-specific). *If $\alpha_t = 0$, or if c_t, evaluation drift, and the risk penalties dominate the coherence gain, then RCP can preserve safety without proving capability progress. A weak or gameable task distribution $\mathcal{T}_t$ also makes the theorem weak or gameable.*

26.3 Capability preservation and improvement by telescoping

Definition 26.13 (Capability-preserving and capability-improving update)**.** *An accepted update is χ_t-capability preserving if*

$$\mathrm{Cap}_{t+1}(\rho_{t+1}) \geq \mathrm{Cap}_t(\rho_t) - \chi_t.$$

It is (g_t, χ_t)-capability improving if

$$\mathrm{Cap}_{t+1}(\rho_{t+1}) \geq \mathrm{Cap}_t(\rho_t) + g_t - \chi_t.$$

The same definitions apply to $\mathrm{SafeCap}_t$ *by replacing* Cap_t *with* $\mathrm{SafeCap}_t$*.*

Theorem 26.14 (Bounded empirical capability loss and gain)**.** *If every accepted update is χ_t-capability preserving, then for every finite T,*

$$\mathrm{Cap}_T(\rho_T) \geq \mathrm{Cap}_0(\rho_0) - \sum_{t=0}^{T-1} \chi_t.$$

If every accepted update is (g_t, χ_t)-capability improving, then

$$\mathrm{Cap}_T(\rho_T) \geq \mathrm{Cap}_0(\rho_0) + \sum_{t=0}^{T-1} g_t - \sum_{t=0}^{T-1} \chi_t.$$

The identical statements hold for $\mathrm{SafeCap}_t$*.*

Proof. The preserving case follows by summing

$$\mathrm{Cap}_{t+1}(\rho_{t+1}) \geq \mathrm{Cap}_t(\rho_t) - \chi_t$$

from $t = 0$ to $T - 1$. The improving case is the same telescoping sum with the additional g_t term. Replacing Cap_t by $\mathrm{SafeCap}_t$ does not change the algebra. □

Interpretation 26.15. *The telescope theorem is algebraic. The substantive work is done by the capability–coherence calibration theorem, the safe-improvement existence theorem, and the viability-kernel theorem. Together they specify when the telescope has positive terms rather than merely bounded losses.*

Limitation 26.16. *Capability improvement is only as strong as the task distribution, scoring rule, risk penalty, drift bound, and calibration certificate. This module does not solve semantic grounding or value specification.*

27 Module A: Safe-Improvement Existence and Recursive Feasibility

No-op feasibility prevents unsafe collapse, but it can also make the system stationary. This section states exactly what additional assumptions are needed to prove that a nontrivial safe update exists at a given step and then formalizes when such updates can continue recursively.

27.1 Gate residuals and local progress

Definition 27.1 (Strengthened local gate). *Let $\Omega_t(\rho_t)$ be the candidate update class available at state ρ_t. The strengthened local RCP gate at (t, ρ_t) is the set of candidates satisfying all constraints required for the Modules A–C upgrade: closure-level RCP admissibility, recoverable non-lossiness, robust certificates, capability calibration, capability or safe-capability preservation, and any declared local progress threshold.*

Definition 27.2 (Gate residual vector). *Let*

$$Q_t(\Phi; \rho_t) = (q_{t,1}(\Phi; \rho_t), \ldots, q_{t,k_t}(\Phi; \rho_t))$$

be a finite vector of scalar residuals. The sign convention is

$$q_{t,j}(\Phi; \rho_t) \le 0 \quad \Longleftrightarrow \quad \textit{the } j\textit{th scalar gate constraint is satisfied.}$$

The residuals may include relative-entropy loss, Lyapunov drift, barrier violation, diversity-floor violation, ambiguity collapse, information-bottleneck loss, estimator constraints, verifier constraints, capability-gate constraints, constructive-recovery constraints, and progress constraints.

Assumption 27.3 (Residual representation of the local gate). *For the audited candidate class $\Omega_t(\rho_t)$, the residual vector exactly represents the strengthened local gate. That is,*

$$\Phi \textit{ belongs to the strengthened local gate} \quad \Longleftrightarrow \quad q_{t,j}(\Phi; \rho_t) \le 0 \quad \forall j.$$

If the intended gate contains non-scalar certificates, those certificates must either be encoded by scalar residuals or verified separately and included as hypotheses in the theorem using the gate.

Definition 27.4 (Strict RCP interior margin). *The strict feasibility margin of Φ is*

$$\mathrm{margin}_t(\Phi; \rho_t) = \min_{1 \le j \le k_t} \{-q_{t,j}(\Phi; \rho_t)\}.$$

The candidate is strictly locally feasible if

$$\mathrm{margin}_t(\Phi; \rho_t) > 0.$$

Definition 27.5 (Local progress-certified admissible set). *For a progress threshold $g_t^{\min} \ge 0$, define*

$$\mathcal{A}_{\mathrm{RCP}}^{\mathrm{loc}}(t, \rho_t; g_t^{\min})$$

as the set of candidates $\Phi \in \Omega_t(\rho_t)$ such that:

(a) Φ satisfies the closure-level RCP gate $\mathcal{A}_{\mathrm{RCP}}^{+}(t, \rho_t)$;

(b) Φ is (ε_t, r_t)-recoverably non-lossy relative to $\mathcal{P}_t$;

(c) the capability–coherence calibration certificate is valid on the audited neighborhood of Φ;

(d) $\overline{G}_t^{\mathrm{safe}}(\Phi; \rho_t) \ge g_t^{\min}$ unless the system explicitly invokes the no-op safety fallback;

(e) every non-scalar certificate required by the surrounding manuscript is valid.

When $g_t^{\min} > 0$, membership is a nontrivial progress claim relative to the no-op baseline.

Proposition 27.6 (No-free-lunch for safe improvement). *The constraints defining $\mathcal{A}_{\mathrm{RCP}}(t, \rho_t)$ and $\mathcal{A}^{+}_{\mathrm{RCP}}(t, \rho_t)$ do not imply the existence of a nontrivial capability-improving update.*

Proof. Let $\Omega_t(\rho_t) = \{\mathrm{id}\}$. Then the no-op is feasible if no-op feasibility holds, but no non-identity update exists. Alternatively, let $\Omega_t(\rho_t)$ contain non-identity candidates but assign each such candidate either a positive residual $q_{t,j} > 0$ for some gate constraint or a no-op-normalized safe-capability gain $\overline{G}^{\mathrm{safe}}_t \leq 0$. In both constructions, safety admissibility can hold for the no-op while no nontrivial safe improving update exists. Therefore progress does not follow from safety admissibility alone. □

27.2 One-step safe-improvement existence

Assumption 27.7 (Strict safe-improvement availability). *At step t, there exists a candidate $\Phi^{\sharp}_t \in \Omega_t(\rho_t)$ and constants $m_t > 0$, $g_t > 0$ such that*

$$\mathrm{margin}_t(\Phi^{\sharp}_t; \rho_t) \geq m_t$$

and

$$\overline{G}^{\mathrm{safe}}_t(\Phi^{\sharp}_t; \rho_t) \geq g_t.$$

This assumption is the precise mathematical location where useful safe update existence enters the theory.

Assumption 27.8 (Certified local search completeness). *If a candidate $\Phi^{\sharp}_t$ satisfying Assumption 27.7 exists, the candidate generator and verifier return a candidate $\widetilde{\Phi}_t$ whose certified residuals and no-op-normalized safe-capability gain differ from those of $\Phi^{\sharp}_t$ by at most e_t:*

$$|q_{t,j}(\widetilde{\Phi}_t; \rho_t) - q_{t,j}(\Phi^{\sharp}_t; \rho_t)| \leq e_t \quad \forall j,$$

and

$$|\overline{G}^{\mathrm{safe}}_t(\widetilde{\Phi}_t; \rho_t) - \overline{G}^{\mathrm{safe}}_t(\Phi^{\sharp}_t; \rho_t)| \leq e_t.$$

Theorem 27.9 (Nontrivial admissible improvement under strict interiority). *Assume residual representation of the local gate, strict safe-improvement availability, and certified local search completeness. If*

$$e_t < \min\{m_t, g_t\},$$

then the returned candidate $\widetilde{\Phi}_t$ satisfies the strengthened local RCP gate and has strictly positive no-op-normalized safe-capability gain:

$$\overline{G}^{\mathrm{safe}}_t(\widetilde{\Phi}_t; \rho_t) \geq g_t - e_t > 0.$$

Consequently, the hard gate can admit a nontrivial safe improvement rather than falling back to the no-op.

Proof. For every residual component,

$$q_{t,j}(\Phi^{\sharp}_t; \rho_t) \leq -m_t.$$

By certified local search completeness,

$$q_{t,j}(\widetilde{\Phi}_t; \rho_t) \leq q_{t,j}(\Phi^{\sharp}_t; \rho_t) + e_t \leq -m_t + e_t < 0.$$

Thus $\widetilde{\Phi}_t$ satisfies every scalar residual constraint. By Assumption 27.3, it satisfies the strengthened local gate. Likewise,

$$\overline{G}_t^{\mathrm{safe}}(\widetilde{\Phi}_t;\rho_t) \geq \overline{G}_t^{\mathrm{safe}}(\Phi_t^\sharp;\rho_t) - e_t \geq g_t - e_t > 0.$$

The candidate is therefore nontrivial relative to the no-op baseline and safe under the strengthened local gate. □

Interpretation 27.10. *This theorem is stronger than the previous useful-update proposition because it separates three facts: safety alone does not imply progress; progress follows if a strictly interior safe improving candidate exists; and the verifier/search system must be accurate enough to find a candidate within the strict margins.*

Limitation 27.11. *The theorem still does not prove that all systems always possess safe improving updates. It proves the exact sufficient conditions under which a local Level 2 state can take one safe Level 3 improvement step.*

28 Module A Continued: Recursive Feasibility and Viability Kernels

A single safe improvement is not yet RSI. RSI requires that safe improvement remains feasible after the update. We therefore add a viability-kernel formulation.

28.1 Finite-horizon viability

Definition 28.1 (State safe set)**.** *Let $\mathcal{K}_{\mathrm{RCP}}^{\mathrm{state}} \subseteq \mathcal{D}(\mathcal{H})$ denote the RCP state safe set of the surrounding manuscript. It contains the states satisfying the Lyapunov, barrier, ambiguity, information-bottleneck, diversity, verifier, and semantic-safety state predicates that are imposed independently of the proposed update.*

Definition 28.2 (Finite-horizon RCP progress viability kernel)**.** *Fix a horizon N and progress thresholds $g_t^{\min} \geq 0$. Define the finite-horizon viability sets backward by*

$$\mathcal{V}_N^{(N)} = \mathcal{K}_{\mathrm{RCP}}^{\mathrm{state}},$$

and, for $t = N-1, \ldots, 0$,

$$\mathcal{V}_t^{(N)} = \left\{ \rho \in \mathcal{K}_{\mathrm{RCP}}^{\mathrm{state}} : \begin{array}{l} \exists \Phi \in \mathcal{A}_{\mathrm{RCP}}^{\mathrm{loc}}(t,\rho;g_t^{\min}) \text{ such that} \\ \Phi(\rho) \in \mathcal{V}_{t+1}^{(N)}, \quad \overline{G}_t^{\mathrm{safe}}(\Phi;\rho) \geq g_t^{\min} \end{array} \right\}.$$

Theorem 28.3 (Finite-horizon safe-improvement path)**.** *If $\rho_0 \in \mathcal{V}_0^{(N)}$, then there exist updates $\Phi_0, \ldots, \Phi_{N-1}$ and states $\rho_{t+1} = \Phi_t(\rho_t)$ such that for every $t < N$:*

$$\Phi_t \in \mathcal{A}_{\mathrm{RCP}}^{\mathrm{loc}}(t,\rho_t;g_t^{\min}), \qquad \rho_t \in \mathcal{K}_{\mathrm{RCP}}^{\mathrm{state}}, \qquad \overline{G}_t^{\mathrm{safe}}(\Phi_t;\rho_t) \geq g_t^{\min}.$$

In addition, if the local gate includes the recoverable non-lossiness certificate, then every accepted update in the path is constructively recoverable with its declared recovery budget.

Proof. Because $\rho_0 \in \mathcal{V}_0^{(N)}$, the definition of $\mathcal{V}_0^{(N)}$ gives an update $\Phi_0 \in \mathcal{A}_{\mathrm{RCP}}^{\mathrm{loc}}(0, \rho_0; g_0^{\min})$ with $\overline{G}_0^{\mathrm{safe}} \geq g_0^{\min}$ and $\rho_1 = \Phi_0(\rho_0) \in \mathcal{V}_1^{(N)}$. Repeating the same argument at $t = 1, 2, \ldots, N-1$ yields the claimed sequence. Since every $\mathcal{V}_t^{(N)} \subseteq \mathcal{K}_{\mathrm{RCP}}^{\mathrm{state}}$, all states remain in the RCP safe set. Constructive recoverability follows from membership in $\mathcal{A}_{\mathrm{RCP}}^{\mathrm{loc}}$. □

Theorem 28.4 (Maximality of the finite-horizon viability recursion)**.** *For each $t \leq N$, $\mathcal{V}_t^{(N)}$ is the largest subset of $\mathcal{K}_{\mathrm{RCP}}^{\mathrm{state}}$ from which there exists a progress-certified RCP path from time t through time N satisfying the thresholds $g_s^{\min}$ for $s = t, \ldots, N-1$.*

Proof. At $t = N$, $\mathcal{V}_N^{(N)} = \mathcal{K}_{\mathrm{RCP}}^{\mathrm{state}}$, so the claim is immediate. Suppose the claim holds at $t+1$. A state $\rho \in \mathcal{K}_{\mathrm{RCP}}^{\mathrm{state}}$ admits a progress-certified path from t through N if and only if there is an admissible first update $\Phi \in \mathcal{A}_{\mathrm{RCP}}^{\mathrm{loc}}(t, \rho; g_t^{\min})$ with $\overline{G}_t^{\mathrm{safe}}(\Phi; \rho) \geq g_t^{\min}$ such that the successor $\Phi(\rho)$ admits a progress-certified path from $t+1$ through N. By the induction hypothesis, this successor condition is equivalent to $\Phi(\rho) \in \mathcal{V}_{t+1}^{(N)}$. This is exactly the defining condition for $\rho \in \mathcal{V}_t^{(N)}$. Hence $\mathcal{V}_t^{(N)}$ is maximal. □

28.2 Infinite-horizon viability

Definition 28.5 (Infinite-horizon RCP progress viability kernel)**.** *A sequence of sets $\{\mathcal{V}_t^{(\infty)}\}_{t \geq 0}$ is an infinite-horizon RCP progress viability kernel if*

$$\mathcal{V}_t^{(\infty)} \subseteq \mathcal{K}_{\mathrm{RCP}}^{\mathrm{state}}$$

and every $\rho \in \mathcal{V}_t^{(\infty)}$ admits at least one $\Phi \in \mathcal{A}_{\mathrm{RCP}}^{\mathrm{loc}}(t, \rho; g_t^{\min})$ satisfying

$$\Phi(\rho) \in \mathcal{V}_{t+1}^{(\infty)}, \qquad \overline{G}_t^{\mathrm{safe}}(\Phi; \rho) \geq g_t^{\min}.$$

Theorem 28.6 (Recursive improvement existence from an infinite-horizon viability kernel)**.** *Assume an infinite-horizon RCP progress viability kernel exists and $\rho_0 \in \mathcal{V}_0^{(\infty)}$. Then there exists an infinite sequence of accepted updates $\{\Phi_t\}_{t \geq 0}$ such that*

$$\Phi_t \in \mathcal{A}_{\mathrm{RCP}}^{\mathrm{loc}}(t, \rho_t; g_t^{\min}), \qquad \rho_{t+1} = \Phi_t(\rho_t), \qquad \rho_t \in \mathcal{K}_{\mathrm{RCP}}^{\mathrm{state}} \quad \forall t,$$

and

$$\overline{G}_t^{\mathrm{safe}}(\Phi_t; \rho_t) \geq g_t^{\min} \quad \forall t.$$

If, additionally,

$$\sum_{t=0}^{\infty} (\eta_t + \varepsilon_t + \zeta_t + \xi_t + r_t) < \infty,$$

then the safety, ambiguity, self-model relevance, and constructive-recovery loss budgets remain finite along the infinite safe-improvement path.

Proof. By definition of $\mathcal{V}_0^{(\infty)}$, there exists $\Phi_0 \in \mathcal{A}_{\mathrm{RCP}}^{\mathrm{loc}}(0, \rho_0; g_0^{\min})$ such that $\rho_1 = \Phi_0(\rho_0) \in \mathcal{V}_1^{(\infty)}$ and $\overline{G}_0^{\mathrm{safe}} \geq g_0^{\min}$. Applying the same argument recursively gives Φ_t for every t. Membership in $\mathcal{A}_{\mathrm{RCP}}^{\mathrm{loc}}$ implies membership in the RCP gate and therefore state safety. The finite-budget conclusion follows by applying the main RCP theorem of the surrounding manuscript and Theorem 13.13 on arbitrary finite horizons and then using summability. □

28.3 Compactness-based infinite-path extraction

The previous theorem assumes an infinite-horizon viability kernel. The following theorem states one rigorous route for deriving an infinite path from finite-horizon feasibility. This is not automatic; compactness and closedness assumptions are needed.

Assumption 28.7 (Compact closed-graph progress system)**.** *For every t, the state safe set $\mathcal{K}_{\mathrm{RCP}}^{\mathrm{state}}$ is compact, every candidate set $\Omega_t(\rho)$ is compact, the map $(\rho, \Phi) \mapsto \Phi(\rho)$ is continuous on its graph, and the graph of the local progress relation*

$$\mathcal{G}_t = \left\{ (\rho, \Phi, \rho^+) : \begin{array}{l} \rho \in \mathcal{K}_{\mathrm{RCP}}^{\mathrm{state}},\ \Phi \in \mathcal{A}_{\mathrm{RCP}}^{\mathrm{loc}}(t, \rho; g_t^{\min}), \\ \rho^+ = \Phi(\rho), \\ \rho^+ \in \mathcal{K}_{\mathrm{RCP}}^{\mathrm{state}}, \\ \overline{G}_t^{\mathrm{safe}}(\Phi; \rho) \geq g_t^{\min} \end{array} \right\}$$

is closed.

Theorem 28.8 (Infinite path from all finite horizons)**.** *Assume the compact closed-graph progress system condition. If*

$$\rho_0 \in \mathcal{V}_0^{(N)} \qquad \text{for every } N \geq 1,$$

then there exists an infinite sequence $(\rho_t, \Phi_t)_{t \geq 0}$ such that

$$(\rho_t, \Phi_t, \rho_{t+1}) \in \mathcal{G}_t \qquad \text{for every } t \geq 0.$$

Consequently, there is an infinite safe, progress-certified RCP path from ρ_0.

Proof. For each horizon N, Theorem 28.3 gives a finite path

$$(\rho_0^{(N)}, \Phi_0^{(N)}, \rho_1^{(N)}, \ldots, \Phi_{N-1}^{(N)}, \rho_N^{(N)})$$

with $\rho_0^{(N)} = \rho_0$ and $(\rho_t^{(N)}, \Phi_t^{(N)}, \rho_{t+1}^{(N)}) \in \mathcal{G}_t$ for $t < N$. Compactness gives a convergent subsequence of the first-step triples. Pass to that subsequence. Then compactness gives a convergent subsequence of the second-step triples. Continuing by a diagonal subsequence argument yields a candidate infinite sequence $(\rho_t, \Phi_t, \rho_{t+1})_{t \geq 0}$. Since each $\mathcal{G}_t$ is closed, the limit triple lies in $\mathcal{G}_t$ for every fixed t. Therefore the limiting sequence is an infinite progress-certified RCP path. □

Interpretation 28.9. *Finite-horizon viability is not merely a heuristic. Under compactness and closedness, feasibility at every finite horizon yields an infinite path. This is one rigorous route from a Level 2 safe-update engine to sustained Level 3 improvement.*

Limitation 28.10. *The theorem does not prove that $\rho_0 \in \mathcal{V}_0^{(N)}$ for all N. It also assumes compactness and closedness properties that may fail in large, changing, learned architectures unless the candidate class and verifier are deliberately restricted.*

29 Augmented RCP Admissibility After Modules A–C

We now define the strengthened admissibility relation used after adding Modules A–C. The notation distinguishes three levels: the original safety gate $\mathcal{A}_{\mathrm{RCP}}$, the previous closure gate $\mathcal{A}_{\mathrm{RCP}}^{+}$, and the progress/recovery gate $\mathcal{A}_{\mathrm{RCP}}^{++}$.

Definition 29.1 (Closure-level RCP admissibility)**.** *A candidate update belongs to*

$$\mathcal{A}^{+}_{\mathrm{RCP}}(t, \rho_t)$$

if it belongs to $\mathcal{A}_{\mathrm{RCP}}(t, \rho_t)$ *and, additionally:*

(a) $\mathcal{P}_t$ *is complete for the active safety predicate family within declared error* a_t*;*

(b) every semantic barrier B_i *used by the safe set is sound or margin-sound;*

(c) every estimated quantity passes a robust one-sided certificate;

(d) verifier integrity is protected by pair-set, barrier, and intersection-verifier constraints.

Definition 29.2 (Progress-and-recovery RCP admissibility)**.** *A candidate update belongs to*

$$\mathcal{A}^{++}_{\mathrm{RCP}}(t, \rho_t; g_t^{\min})$$

if it belongs to $\mathcal{A}^{+}_{\mathrm{RCP}}(t, \rho_t)$ *and, additionally:*

(a) it is (ε_t, r_t)*-recoverably non-lossy relative to* $\mathcal{P}_t$*;*

(b) it satisfies the capability–coherence calibration certificate on its audited neighborhood;

(c) it satisfies the capability or safe-capability gate with declared loss budget χ_t*;*

(d) for a progress claim, it satisfies $\overline{G}_t^{\mathrm{safe}}(\Phi; \rho_t) \geq g_t^{\min} > 0$*;*

(e) for a sustained RSI claim, it satisfies the viability condition $\Phi(\rho_t) \in \mathcal{V}_{t+1}^{(\infty)}$ *or the finite-horizon analogue* $\Phi(\rho_t) \in \mathcal{V}_{t+1}^{(N)}$*.*

If no candidate satisfies the progress clauses, the system may still use the no-op fallback for safety, but then no Level 3 progress claim is made for that step.

Theorem 29.3 (Augmented RCP stability, recovery, and progress)**.** *Assume the hypotheses of the main RCP theorem of the surrounding manuscript. Strengthen the accepted-update condition to*

$$\Phi_t \in \mathcal{A}^{++}_{\mathrm{RCP}}(t, \rho_t; g_t^{\min})$$

for every non-no-op progress step. Assume the robust certificates are simultaneously valid up to horizon T *on an event* $\mathcal{E}_{0:T}$ *with probability at least* $1 - \delta_T$*. Then, on* $\mathcal{E}_{0:T}$*, all conclusions of the main RCP theorem hold with the certified approximation budgets included. In addition:*

$$\mathrm{Cap}_T(\rho_T) \geq \mathrm{Cap}_0(\rho_0) - \sum_{t=0}^{T-1} \chi_t,$$

constructive predecessor recovery satisfies

$$d_{\mathrm{tr}}\big(\rho_0, \mathcal{R}_{0:T-1}(\rho_T)\big) \leq \sum_{t=0}^{T-1} r_t$$

for the certified recovery maps, and all certified safety predicates whose barriers are sound remain true along the accepted trajectory. If the accepted updates are (g_t, χ_t)-capability improving in the absolute telescope sense, then

$$\mathrm{Cap}_T(\rho_T) \geq \mathrm{Cap}_0(\rho_0) + \sum_{t=0}^{T-1} g_t - \sum_{t=0}^{T-1} \chi_t.$$

If $\rho_0 \in \mathcal{V}_0^{(N)}$, then there exists a finite-horizon sequence of progress-certified updates through horizon N. If $\rho_0 \in \mathcal{V}_0^{(\infty)}$, then there exists an infinite sequence of progress-certified updates with no-op-normalized safe gains at least $g_t^{\min}$, subject to the stated viability-kernel assumptions.

Proof. On $\mathcal{E}_{0:T}$, the robust-gate soundness result from the surrounding manuscript converts all robust estimated inequalities into exact RCP inequalities with their declared budgets. The pair-set completeness result accounts for pair-set approximation errors. The barrier-predicate preservation result converts sound barrier inequalities into preservation of their corresponding safety predicates. The intersection-verifier result gives soundness of the verifier gate on the same event. Therefore the hypotheses of the main RCP theorem hold on $\mathcal{E}_{0:T}$, and all of its conclusions follow.

The capability inequalities follow from Theorem 26.14. The constructive predecessor-recovery bound follows from Theorem 13.13. The finite-horizon and infinite-horizon progress-existence claims follow from Theorem 28.3 and Theorem 28.6, respectively. The probability statement is inherited from the simultaneous certificate-validity event. □

Interpretation 29.4. *The augmented theorem is the Modules A–C upgrade. It changes the manuscript from merely saying "accepted updates preserve safety" to the stronger conditional statement: if the state lies in a progress viability kernel and the strengthened certificates hold, then safe, recoverable, capability-improving recursive updates can proceed.*

Limitation 29.5. *The augmented theorem remains conditional. The nonemptiness of $\mathcal{V}_t^{(N)}$ or $\mathcal{V}_t^{(\infty)}$, the truth of the capability–coherence calibration inequality, the validity of the recovery certificates, and the adequacy of the task-distribution capability functional are not consequences of RCP notation. They must be proved for a specific architecture or validated empirically.*

30 Revised List of What Remains After Modules A–C

After adding Modules A–C, the remaining work is no longer simply "prove that useful updates exist" in an informal sense. The paper now contains exact theorem-level sufficient conditions for useful progress, recursive feasibility, capability/coherence coupling, and constructive recovery. What remains is the harder instantiation problem of showing that those sufficient conditions hold for a real architecture:

1. choose and validate the semantic registers L, O, S, D, A, M for an actual architecture;
2. build safety predicate families $\mathfrak{S}_t$ rich enough to cover corrigibility, deception, grounding, oversight, reversibility, verifier integrity, and uncertainty honesty;
3. construct finite or efficiently sampled pair sets $\mathcal{P}_t$ that are complete for those predicates within declared approximation errors;

4. design sound or margin-sound barriers B_i for the selected predicates;
5. produce scalable one-sided estimators for relative entropy, mutual information, ambiguity collapse, barriers, recovery defects, and capability metrics;
6. prove or empirically certify strict safe-improvement availability, i.e. nonempty interior safe improving candidates with margins $m_t > 0$ and gains $g_t > 0$;
7. prove nonemptiness of the finite-horizon or infinite-horizon progress viability kernels for the architecture under study;
8. validate compactness and closedness assumptions if using the infinite-path extraction theorem;
9. validate the capability–coherence calibration inequality and estimate its constants $\alpha_t, \beta_t^{\mathrm{loss}}, \beta_t^U, \beta_t^I, \beta_t^B, c_t$;
10. validate that Cap_t and $\mathrm{SafeCap}_t$ measure real retained or improved capability rather than benchmark overfitting;
11. construct or certify recovery maps $\mathcal{R}_t$ for the protected pair sets, or prove that exact/approximate Petz-style recovery hypotheses apply;
12. protect verifiers against drift, collusion, adversarial examples, and certificate gaming by external audit and RCP-internal constraints.

This is the final distinction after Modules A–C: the present manuscript supplies a conditional theorem-proof architecture for safe, recoverable, capability-improving recursive updates. Deployment requires proving or validating that a concrete architecture actually lies inside the strict progress and viability assumptions.

31 Rigor Audit Status of Modules D–G

The preceding Modules A–C upgrade makes progress, non-lossiness, constructive recovery, and recursive feasibility conditional but theorem-level. The present Modules D–G upgrade addresses four remaining classes of claims that cannot be obtained from the earlier information-theoretic inequalities alone:

(A) semantic realization and register validity;

(B) self-model fidelity and verifier bootstrapping;

(C) physical and computational resource feasibility;

(D) asymptotic behavior of the recursive trajectory.

Remark 31.1 (Main rigor standard for Modules D–G)**.** *The tensor labels*

$$L, O, S, D, A, M$$

do not by themselves prove that the corresponding registers represent language, world-reference, human-reference, definitiveness, ambiguity, or self-model structure. A verifier label does not prove

verifier soundness. A density-state description does not prove physical realizability. A Lyapunov inequality does not prove convergence to a unique improving regime unless additional compactness, descent, and target-set hypotheses are stated. This module therefore treats each of these as an explicit assumption, certificate, or conditional theorem rather than as an automatic consequence of the RCP notation.

Remark 31.2 (What is repaired here)**.** *This module adds the following repairs and extensions:*

(a) semantic interpretation maps for each register and a proof that register validity is preserved only when semantic barriers are sound and included in the gate;

(b) a no-free-lunch proposition showing that register labels do not imply semantic grounding;

(c) explicit self-model fidelity metrics comparing the internal self-model with an operationally audited external self-state;

(d) a non-circular verifier bootstrapping theorem requiring an independent trust anchor or cross-check family;

(e) physical resource vectors and resource-admissibility constraints for update generation, verification, recovery, rollback, and thermodynamic irreversibility accounting;

(f) a correction to open-ended progress claims: uniformly positive capability increments cannot continue indefinitely on a fixed bounded capability scale;

(g) asymptotic theorems separating boundedness, summable motion, convergence to a target set, and convergence to a unique point.

Limitation 31.3 (Conditional completion)**.** *Modules D–G do not derive human values, world semantics, a concrete verifier, or a physical AGI architecture from first principles. They make the required objects explicit and prove preservation or progress only under stated realization, soundness, resource, and asymptotic hypotheses. This is the rigorous claim supported by the mathematics. Stronger claims require architecture-specific construction and empirical validation.*

31.1 Standing notation inherited from the full manuscript and Modules A–C

The following notation is fixed for the D–G module. It is included here to prevent the updated sections from silently changing the notation already introduced in the full v2 manuscript and in the rigor-audited Modules A–C insert.

Definition 31.4 (State spaces and candidate updates)**.** *For each time t, let $\mathcal{H}_t$ be a finite-dimensional Hilbert space and let*

$$\mathcal{D}(\mathcal{H}_t) = \{\rho \in \mathcal{B}(\mathcal{H}_t) : \rho \succeq 0,\ \mathrm{Tr}(\rho) = 1\}$$

be the corresponding density-state space. The available candidate updates at state $\rho_t \in \mathcal{D}(\mathcal{H}_t)$ are denoted

$$\Omega_t(\rho_t).$$

Each $\Phi \in \Omega_t(\rho_t)$, when applied, is assumed to return a density state in $\mathcal{D}(\mathcal{H}_{t+1})$. If the surrounding manuscript restricts candidates to CPTP channels on a fixed space, this definition specializes to that case.

Definition 31.5 (Post-A–C gate used as input to Modules D–G)**.** *Let*

$$\mathcal{A}_{\mathrm{RCP}}^{\mathrm{AC}}(t, \rho_t; g_t^{\min})$$

denote the rigor-audited post-A–C progress/recovery gate. In the notation of the Modules A–C insert, this is the gate previously written as

$$\mathcal{A}_{\mathrm{RCP}}^{++}(t, \rho_t; g_t^{\min}),$$

namely the gate that includes closure-level RCP admissibility, constructive recoverability, capability/safe-capability calibration, local progress when claimed, and the applicable viability-kernel condition. We introduce the superscript AC *only to avoid reusing ++ for the D–G extension.*

Definition 31.6 (State safe set inherited from Modules A–C)**.** *Let*

$$\mathcal{K}_{\mathrm{RCP}}^{\mathrm{state}}(t) \subseteq \mathcal{D}(\mathcal{H}_t)$$

denote the RCP state safe set from the surrounding manuscript and the Modules A–C insert. It contains the state predicates imposed independently of a particular candidate update: Lyapunov bounds, barrier predicates, ambiguity bounds, information-bottleneck constraints, diversity-collapse floors, verifier-integrity state predicates, and other already-declared state-level safety requirements.

Limitation 31.7 (Notation hygiene)**.** *The D–G module does not redefine the post-A–C symbol* $\mathcal{A}_{\mathrm{RCP}}^{++}$*. It uses* $\mathcal{A}_{\mathrm{RCP}}^{\mathrm{AC}}$ *for the already-audited A–C gate and defines a new D–G gate* $\mathcal{A}_{\mathrm{RCP}}^{\mathrm{DG}}$*. This avoids the previous ambiguity in which ++ was used for both the A–C and post-D–G gates.*

32 Module D: Semantic Realization and Register Validity

The RCP state decomposition uses the register names

$$\mathcal{H} = \mathcal{H}_L \otimes \mathcal{H}_O \otimes \mathcal{H}_S \otimes \mathcal{H}_D \otimes \mathcal{H}_A \otimes \mathcal{H}_M.$$

The labels are mathematically useful, but they are not semantics. This section supplies the missing semantic layer: interpretation maps, external reference objects, validity predicates, sound semantic barriers, and a preservation theorem.

32.1 No semantics from tensor labels alone

Proposition 32.1 (Tensor-factor labels do not imply semantic grounding)**.** *Let*

$$\mathcal{H} = \mathcal{H}_L \otimes \mathcal{H}_O \otimes \mathcal{H}_S \otimes \mathcal{H}_D \otimes \mathcal{H}_A \otimes \mathcal{H}_M$$

be a finite-dimensional tensor decomposition. The existence of this decomposition alone does not imply that the factors encode language, objective world-reference, subjective human-reference, definitiveness, ambiguity, or self-model information.

Proof. A tensor decomposition is invariant under arbitrary relabeling of the factors. Given any six finite-dimensional Hilbert spaces with the same dimensions, one may call them by any names without changing any operator, entropy, relative entropy, or mutual information value. Therefore the semantic interpretation is extra mathematical structure not contained in the bare tensor product. Additional interpretation maps, reference objects, and validation predicates are required. □

Interpretation 32.2. *The RCP theorem can preserve relationships among registers. It cannot prove that the registers mean what their names suggest unless register meaning is supplied and validated by a semantic realization layer.*

32.2 Semantic spaces and interpretation maps

Definition 32.3 (Register index set and register marginals)**.** *Let*

$$\mathcal{I} = \{L, O, S, D, A, M\}.$$

For each time t, assume the RCP Hilbert space has the declared register factorization

$$\mathcal{H}_t = \bigotimes_{X \in \mathcal{I}} \mathcal{H}_{X,t}.$$

For each $X \in \mathcal{I}$, let

$$\rho_{X,t} = \mathrm{Tr}_{\mathcal{I}\setminus\{X\}}(\rho_t)$$

denote the marginal state on $\mathcal{H}_{X,t}$. If an implementation uses an approximate or operational factorization rather than an exact tensor factorization, the factorization error must be included in the semantic-validation error budget.

Definition 32.4 (Semantic realization data)**.** *A semantic realization for the RCP registers consists, for each $X \in \mathcal{I}$, of:*

(a) a semantic space $(\mathcal{S}_{X,t}, d_{X,t})$;

(b) an interpretation map

$$\Gamma_{X,t} : \mathcal{D}(\mathcal{H}_{X,t}) \to \mathcal{S}_{X,t};$$

(c) an external or audited reference object $x^\star_{X,t} \in \mathcal{S}_{X,t}$;

(d) a validity tolerance $\delta_{X,t} \geq 0$.

The interpretation maps and reference objects are not derived from the Hilbert-space labels. They are part of the semantic specification of the architecture and must be constructed or audited separately.

Definition 32.5 (Register validity error)**.** *For each $X \in \mathcal{I}$, define the semantic validity error*

$$\mathrm{Val}_{X,t}(\rho_t) = d_{X,t}\big(\Gamma_{X,t}(\rho_{X,t}), x^\star_{X,t}\big).$$

The register X is $\delta_{X,t}$-valid at time t when

$$\mathrm{Val}_{X,t}(\rho_t) \leq \delta_{X,t}.$$

The full RCP register state is semantically valid at time t when this condition holds for all $X \in \mathcal{I}$.

Definition 32.6 (Semantic safe set)**.** *The semantic safe set at time t is*

$$\mathcal{K}^{\mathrm{sem}}_t = \{\rho \in \mathcal{D}(\mathcal{H}_t) : \mathrm{Val}_{X,t}(\rho) \leq \delta_{X,t} \text{ for all } X \in \mathcal{I}\}.$$

If the Hilbert space or semantic specification changes with t, the comparison is made using the corresponding $\mathcal{H}_t$, $\Gamma_{X,t}$, $x^\star_{X,t}$, and $\delta_{X,t}$.

Limitation 32.7 (Semantic references are external inputs)**.** *The objects $x^\star_{O,t}$ and $x^\star_{S,t}$ encode world-reference and human-reference targets. The paper does not derive these references from nothing. It proves conditional preservation once a realization and a validation procedure are supplied.*

32.3 Semantic barriers and preservation

Definition 32.8 (Semantic barrier family and audited semantic domain)**.** *For each register $X \in \mathcal{I}$, define the ideal semantic barrier*

$$B^{\mathrm{sem}}_{X,t}(\rho) = \mathrm{Val}_{X,t}(\rho) - \delta_{X,t}.$$

Thus

$$B^{\mathrm{sem}}_{X,t}(\rho) \le 0 \quad \Longleftrightarrow \quad \mathrm{Val}_{X,t}(\rho) \le \delta_{X,t}.$$

Let $\mathcal{X}^{\mathrm{sem}}_t \subseteq \mathcal{D}(\mathcal{H}_t)$ be the audited semantic domain on which the semantic interpretation maps, reference objects, and estimated barriers are validated. In an implementation one may use an estimated barrier $\widehat{B}^{\mathrm{sem}}_{X,t}$. Estimated barriers require a separate one-sided soundness certificate on $\mathcal{X}^{\mathrm{sem}}_t$.

Assumption 32.9 (Sound estimated semantic barriers)**.** *If estimated semantic barriers are used, then on the audited domain $\mathcal{X}^{\mathrm{sem}}_t$, for every $X \in \mathcal{I}$,*

$$\rho \in \mathcal{X}^{\mathrm{sem}}_t \text{ and } \widehat{B}^{\mathrm{sem}}_{X,t}(\rho) \le 0 \quad \Longrightarrow \quad B^{\mathrm{sem}}_{X,t}(\rho) \le 0.$$

If ideal barriers are used, set $\widehat{B}^{\mathrm{sem}}_{X,t} = B^{\mathrm{sem}}_{X,t}$ and $\mathcal{X}^{\mathrm{sem}}_t = \mathcal{D}(\mathcal{H}_t)$.

Definition 32.10 (Semantically admissible update)**.** *A candidate update Φ_t is semantically admissible at (t, ρ_t) if its successor lies in the audited semantic domain and passes every semantic barrier:*

$$\Phi_t(\rho_t) \in \mathcal{X}^{\mathrm{sem}}_{t+1}$$

and

$$\widehat{B}^{\mathrm{sem}}_{X,t+1}(\Phi_t(\rho_t)) \le 0 \quad \text{for every } X \in \mathcal{I}.$$

Define

$$\mathcal{A}^{\mathrm{sem}}_{\mathrm{RCP}}(t, \rho_t) = \{\Phi_t \in \Omega_t(\rho_t) : \Phi_t \text{ is semantically admissible}\}.$$

Theorem 32.11 (Semantic register validity preservation)**.** *Assume the semantic realization data of Definition 32.4 and the sound estimated semantic barriers of Assumption 32.9. Let $\rho_{t+1} = \Phi_t(\rho_t)$. If*

$$\Phi_t \in \mathcal{A}^{\mathrm{sem}}_{\mathrm{RCP}}(t, \rho_t),$$

then

$$\rho_{t+1} \in \mathcal{K}^{\mathrm{sem}}_{t+1}.$$

Consequently, if $\rho_0 \in \mathcal{K}^{\mathrm{sem}}_0$ and every accepted update is semantically admissible, then

$$\rho_t \in \mathcal{K}^{\mathrm{sem}}_t \qquad \text{for all } t \ge 0$$

on every horizon for which the semantic realization data and soundness assumptions hold.

Proof. Since $\Phi_t \in \mathcal{A}^{\mathrm{sem}}_{\mathrm{RCP}}(t, \rho_t)$, for every $X \in \mathcal{I}$,

$$\widehat{B}^{\mathrm{sem}}_{X,t+1}(\Phi_t(\rho_t)) \le 0.$$

By Assumption 32.9,

$$B^{\mathrm{sem}}_{X,t+1}(\Phi_t(\rho_t)) \le 0.$$

By Definition 32.8, this is exactly

$$\mathrm{Val}_{X,t+1}(\rho_{t+1}) \le \delta_{X,t+1}.$$

Because this holds for all $X \in \mathcal{I}$, Definition 32.6 gives $\rho_{t+1} \in \mathcal{K}^{\mathrm{sem}}_{t+1}$. Applying the one-step implication inductively proves the horizon statement. □

Interpretation 32.12. *This theorem closes the register-validity gap only conditionally. It says that if the semantic realization and semantic barriers are sound, then RCP can preserve register validity across accepted updates. It does not construct the correct semantic realization.*

32.4 Human-reference and subjective-grounding validity

Definition 32.13 (Human-reference process)**.** *Let $\mathcal{V}_{H,t} \in \mathcal{S}_{S,t}$ denote an audited human-reference object at time t. This may be a value-learning posterior, constitutional specification, human-feedback model, deliberative preference model, or other externally justified object. In the semantic realization data, the subjective-reference target is*

$$x^{\star}_{S,t} = \mathcal{V}_{H,t}.$$

Definition 32.14 (Subjective-grounding error)**.** *The subjective-grounding error is*

$$\mathrm{Val}_{S,t}(\rho_t) = d_{S,t}\big(\Gamma_{S,t}(\rho_{S,t}), \mathcal{V}_{H,t}\big).$$

The subjective register is human-reference valid when

$$\mathrm{Val}_{S,t}(\rho_t) \le \delta_{S,t}.$$

Assumption 32.15 (Human-reference audit soundness)**.** *If the paper claims preservation relative to actual human-reference structure rather than merely relative to the declared object $\mathcal{V}_{H,t}$, then there exists a target $\mathcal{V}^{\mathrm{true}}_{H,t} \in \mathcal{S}_{S,t}$ and an audit error $\alpha^{H}_{t} \ge 0$ such that*

$$d_{S,t}(\mathcal{V}_{H,t}, \mathcal{V}^{\mathrm{true}}_{H,t}) \le \alpha^{H}_{t}.$$

This assumption is not needed to prove preservation relative to $\mathcal{V}_{H,t}$; it is needed only for claims about the relation between $\mathcal{V}_{H,t}$ and the true human-reference target.

Corollary 32.16 (Human-reference preservation under a sound subjective barrier)**.** *Under the hypotheses of Theorem 32.11, if the subjective semantic barrier $B^{\mathrm{sem}}_{S,t}$ is included in the accepted RCP gate at every step, then the subjective register remains human-reference valid over the accepted trajectory.*

Proof. This is the $X = S$ component of Theorem 32.11 with $x^{\star}_{S,t} = \mathcal{V}_{H,t}$. □

Limitation 32.17 (No derivation of human values)**.** *This corollary does not solve value learning. It says that once a human-reference object $\mathcal{V}_{H,t}$, metric $d_{S,t}$, interpretation map $\Gamma_{S,t}$, and sound barrier are supplied, the RCP gate can preserve the certified relationship between the subjective register and that object.*

33 Module E: Self-Model Fidelity and Verifier Bootstrapping

Level 3 RSI requires more than a compressed self-model that preserves mutual information about safety-relevant future behavior. The self-model must also be faithful to the actual operational system well enough for update certificates to transfer from predicted to realized successors. Similarly, verifier preservation is not the same as verifier improvement. A verifier that never drifts may still become a bottleneck unless verifier power can improve without losing soundness.

33.1 Self-model fidelity

Definition 33.1 (Operational self-state). *Let $\mathcal{M}_t^{\mathrm{op}}$ be a metric space of operational self-states. An element*

$$m_t^{\mathrm{op}} \in \mathcal{M}_t^{\mathrm{op}}$$

contains the externally audited information needed for self-modification, such as architecture descriptors, parameter classes, objective descriptors, admissible update families, verifier states, resource budgets, rollback mechanisms, and declared limitations. The exact components are architecture-specific and must be fixed before the fidelity theorem is applied.

Definition 33.2 (Internal self-model realization). *Let the self-model register marginal be $\rho_{M,t}$. A self-model interpretation map is*

$$\Gamma_{M,t} : \mathcal{D}(\mathcal{H}_M) \to \mathcal{M}_t^{\mathrm{op}}.$$

The interpreted internal self-model is

$$m_t^{\mathrm{int}} = \Gamma_{M,t}(\rho_{M,t}).$$

Definition 33.3 (Self-model fidelity error). *Given a metric $d_{M,t}$ on $\mathcal{M}_t^{\mathrm{op}}$, define*

$$F_t^M(\rho_t) = d_{M,t}\big(m_t^{\mathrm{int}}, m_t^{\mathrm{op}}\big).$$

The self-model is δ_t^M-faithful when

$$F_t^M(\rho_t) \le \delta_t^M.$$

The self-model fidelity barrier is

$$B_t^M(\rho_t) = F_t^M(\rho_t) - \delta_t^M.$$

Definition 33.4 (True operational self-state and audit error). *If a claim is made about fidelity to the actual physical or software system rather than fidelity to the audited description, let*

$$m_t^{\mathrm{true}} \in \mathcal{M}_t^{\mathrm{op}}$$

denote the true update-relevant operational state, assumed to lie in the same metric space. The audit error is a number $\alpha_t^M \ge 0$ such that

$$d_{M,t}(m_t^{\mathrm{op}}, m_t^{\mathrm{true}}) \le \alpha_t^M.$$

Assumption 33.5 (Operational self-state audit soundness). *The bound in Definition 33.4 holds on the audited domain. If this assumption fails, the theorem below preserves fidelity only to the audited description m_t^{op}, not necessarily to the actual update-relevant system.*

Theorem 33.6 (Self-model fidelity preservation relative to the audited state). *Suppose each accepted update Φ_t satisfies the self-model fidelity barrier at the successor state:*

$$B_{t+1}^M(\Phi_t(\rho_t)) \le 0.$$

Then

$$F_{t+1}^M(\rho_{t+1}) \le \delta_{t+1}^M, \qquad \rho_{t+1} = \Phi_t(\rho_t).$$

Consequently, if the barrier is included at every step, self-model fidelity is preserved over the accepted trajectory relative to the audited operational self-state sequence.

Proof. The inequality

$$B^M_{t+1}(\Phi_t(\rho_t)) \le 0$$

means

$$F^M_{t+1}(\Phi_t(\rho_t)) - \delta^M_{t+1} \le 0.$$

Since $\rho_{t+1} = \Phi_t(\rho_t)$, this gives

$$F^M_{t+1}(\rho_{t+1}) \le \delta^M_{t+1}.$$

The multi-step claim follows by induction. □

Corollary 33.7 (Self-model fidelity relative to the true operational state)**.** *Assume operational self-state audit soundness. If the successor satisfies*

$$F^M_{t+1}(\rho_{t+1}) \le \delta^M_{t+1},$$

then

$$d_{M,t+1}(m^{\text{int}}_{t+1}, m^{\text{true}}_{t+1}) \le \delta^M_{t+1} + \alpha^M_{t+1}.$$

Proof. By the triangle inequality,

$$d_{M,t+1}(m^{\text{int}}_{t+1}, m^{\text{true}}_{t+1}) \le d_{M,t+1}(m^{\text{int}}_{t+1}, m^{\text{op}}_{t+1}) + d_{M,t+1}(m^{\text{op}}_{t+1}, m^{\text{true}}_{t+1}).$$

The first term is $F^M_{t+1}(\rho_{t+1}) \le \delta^M_{t+1}$. The second term is at most α^M_{t+1} by Assumption 33.5. □

Interpretation 33.8. *Self-model preservation is a barrier theorem, not a miracle theorem. It is only as strong as the operational audit, the interpretation map $\Gamma_{M,t}$, the metric $d_{M,t}$, and the inclusion of the barrier in the accepted gate.*

33.2 Prediction-to-realization certificate transfer

Definition 33.9 (Predicted successor and realization error)**.** *Let the internal self-model predict a successor state*

$$\widehat{\rho}_{t+1} = \text{Pred}_t(\rho_t, \Phi_t).$$

The realization error is

$$E^{\text{Pred}}_t(\Phi_t; \rho_t) = d_{\text{state},t+1}\big(\widehat{\rho}_{t+1}, \Phi_t(\rho_t)\big),$$

where $d_{\text{state},t+1}$ is the state metric used for certificates.

Definition 33.10 (Successor residual family)**.** *Let J_{t+1} be a finite index set. For each $j \in J_{t+1}$, let*

$$q_{t+1,j} : \mathcal{D}(\mathcal{H}_{t+1}) \to \mathbb{R}$$

be a scalar residual whose intended successor constraint is

$$q_{t+1,j}(\rho) \le 0.$$

These residuals may encode semantic barriers, self-model barriers, resource predicates represented as state predicates, verifier-integrity barriers, or any other successor-state certificate.

Assumption 33.11 (Lipschitz certificate residuals)**.** *On the audited domain $\mathcal{X}_{t+1}^{\mathrm{Pred}} \subseteq \mathcal{D}(\mathcal{H}_{t+1})$, there are constants $\ell_{t+1,j} < \infty$ such that*

$$|q_{t+1,j}(\rho) - q_{t+1,j}(\sigma)| \le \ell_{t+1,j} d_{\mathrm{state},t+1}(\rho, \sigma)$$

for all $\rho, \sigma \in \mathcal{X}_{t+1}^{\mathrm{Pred}}$ and every $j \in J_{t+1}$.

Theorem 33.12 (Predicted certificates transfer under margin)**.** *Assume Lipschitz certificate residuals. Suppose that*

$$\widehat{\rho}_{t+1} \in \mathcal{X}_{t+1}^{\mathrm{Pred}}, \qquad \Phi_t(\rho_t) \in \mathcal{X}_{t+1}^{\mathrm{Pred}},$$

and that every predicted successor residual has a declared margin $m_{t,j} \ge 0$:

$$q_{t+1,j}(\widehat{\rho}_{t+1}) \le -m_{t,j} \qquad (j \in J_{t+1}).$$

If the realization error satisfies

$$E_t^{\mathrm{Pred}}(\Phi_t; \rho_t) \le \inf_{j \in J_{t+1} : \ell_{t+1,j} > 0} \frac{m_{t,j}}{\ell_{t+1,j}},$$

with the convention that the infimum over an empty set is $+\infty$, then every realized successor residual satisfies

$$q_{t+1,j}(\Phi_t(\rho_t)) \le 0 \qquad (j \in J_{t+1}).$$

Proof. Fix $j \in J_{t+1}$. If $\ell_{t+1,j} = 0$, then Lipschitzness gives

$$q_{t+1,j}(\Phi_t(\rho_t)) = q_{t+1,j}(\widehat{\rho}_{t+1}) \le -m_{t,j} \le 0.$$

If $\ell_{t+1,j} > 0$, then by Assumption 33.11,

$$q_{t+1,j}(\Phi_t(\rho_t)) \le q_{t+1,j}(\widehat{\rho}_{t+1}) + \ell_{t+1,j} d_{\mathrm{state},t+1}(\widehat{\rho}_{t+1}, \Phi_t(\rho_t)).$$

The predicted margin gives the first term at most $-m_{t,j}$, and the realization-error bound gives the second term at most $m_{t,j}$. Hence the sum is at most zero. □

Limitation 33.13 (Prediction transfer is local)**.** *This theorem requires both the predicted and realized successors to lie inside the audited Lipschitz domain and requires sufficient margin. It does not justify extrapolating certificates far outside the validated neighborhood of the predicted successor.*

Limitation 33.14 (Prediction transfer is local)**.** *This theorem requires Lipschitz residuals on the audited domain and sufficient margin. It does not justify extrapolating certificates far outside the validated neighborhood of the predicted successor.*

33.3 Verifier soundness, power, and non-circular bootstrapping

Definition 33.15 (Verifier, acceptance set, and target property)**.** *Let W_t be a verifier at time t. For a state ρ_t, let*

$$\mathrm{Acc}(W_t, \rho_t) \subseteq \Omega_t(\rho_t)$$

be the set of candidates accepted by W_t. Let

$$\mathcal{G}_t(\rho_t) \subseteq \Omega_t(\rho_t)$$

be the true target property set, such as the strengthened RCP gate, resource-admissible gate, semantic gate, or their intersection.

Definition 33.16 (Deterministic soundness and power)**.** *The verifier W_t is sound for $\mathcal{G}_t$ at ρ_t when*

$$\mathrm{Acc}(W_t, \rho_t) \subseteq \mathcal{G}_t(\rho_t).$$

It has nonzero power at ρ_t when

$$\mathrm{Acc}(W_t, \rho_t) \cap \mathcal{G}_t(\rho_t) \neq \varnothing.$$

More generally, a numerical power score $\mathrm{Power}_t(W_t, \rho_t) \geq 0$ may be used, provided the score is defined before the theorem that uses it.

Definition 33.17 (Probabilistic verifier soundness)**.** *If W_t uses randomness, it is δ_t^{ver}-sound for $\mathcal{G}_t$ at ρ_t when*

$$\Pr\big[\Phi \in \mathrm{Acc}(W_t, \rho_t) \text{ and } \Phi \notin \mathcal{G}_t(\rho_t)\big] \leq \delta_t^{\mathrm{ver}}.$$

The probability is over the verifier randomness, candidate-generation randomness, or both, as declared in the empirical protocol.

Proposition 33.18 (A verifier label does not certify verifier soundness)**.** *The fact that an internal module is labeled a verifier does not imply deterministic or probabilistic soundness for $\mathcal{G}_t$.*

Proof. Let W_t accept every candidate. If $\Omega_t(\rho_t)$ contains any candidate outside $\mathcal{G}_t(\rho_t)$, then $\mathrm{Acc}(W_t, \rho_t) \nsubseteq \mathcal{G}_t(\rho_t)$. Thus the name or architecture role of a module does not imply soundness. □

Definition 33.19 (Verifier trust anchor)**.** *A verifier trust anchor $\mathcal{Z}_t$ is an external audit, independent verifier family, formal proof checker, redundant cross-check, or hardware/cryptographic attestation process used to certify claims about W_t or W_{t+1}. The anchor is not allowed to be identical to the untrusted verifier claim it is checking.*

Assumption 33.20 (Verifier certification soundness)**.** *If the trust anchor $\mathcal{Z}_t$ certifies that a proposed verifier update $W_t \mapsto W_{t+1}$ satisfies a declared soundness bound $\delta_{t+1}^{\mathrm{ver}}$, power lower bound p_{t+1}, and verifier-integrity barrier, then those statements are true on the audited domain except with failure probability at most α_t^{ver}.*

Definition 33.21 (Verifier bootstrapping candidate)**.** *A verifier update $W_t \mapsto W_{t+1}$ is a (ω_t, p_t)-bootstrapping candidate if, on the audited domain,*

$$\delta_{t+1}^{\mathrm{ver}} \leq \delta_t^{\mathrm{ver}} + \omega_t$$

and

$$\mathrm{Power}_{t+1}(W_{t+1}, \rho_{t+1}) \geq \mathrm{Power}_t(W_t, \rho_t) + p_t.$$

Here $\omega_t \geq 0$ is permitted soundness degradation and $p_t \geq 0$ is verifier-power gain. If $p_t = 0$, the update preserves but does not improve verifier power.

Theorem 33.22 (Verifier bootstrapping under an independent trust anchor)**.** *Assume verifier certification soundness. Suppose $\mathcal{Z}_t$ certifies that $W_t \mapsto W_{t+1}$ is a (ω_t, p_t)-bootstrapping candidate and that the verifier-integrity barrier at $t+1$ holds. Then, except with probability at most α_t^{ver},*

$$\delta_{t+1}^{\mathrm{ver}} \leq \delta_t^{\mathrm{ver}} + \omega_t$$

and

$$\mathrm{Power}_{t+1}(W_{t+1}, \rho_{t+1}) \geq \mathrm{Power}_t(W_t, \rho_t) + p_t.$$

For a finite horizon T, if the certification failures are union-bounded, then with probability at least

$$1 - \sum_{t=0}^{T-1} \alpha_t^{\mathrm{ver}},$$

all certified verifier-bootstrapping inequalities hold on that horizon.

Proof. The one-step inequalities are exactly the content certified by $\mathcal{Z}_t$. Assumption 33.20 states that the certification is sound except with probability α_t^{ver}. The finite-horizon probability bound follows by the union bound over $t = 0, \dots, T-1$. □

Interpretation 33.23. *This theorem is deliberately non-circular. It does not allow an untrusted verifier to certify its own future soundness without an anchor. Verifier improvement becomes a theorem only when a sound certification mechanism for verifier updates is supplied.*

Limitation 33.24 (Verifier bootstrapping remains architecture-specific)**.** *The theorem does not construct $\mathcal{Z}_t$, W_t, or* Power_t*. It states the conditions under which verifier bootstrapping is logically valid once those objects exist.*

Proposition 33.25 (Uniform verifier-power gain is impossible on a fixed bounded power scale)**.** *Suppose a verifier-power score satisfies*

$$0 \le \mathrm{Power}_t(W_t, \rho_t) \le P_{\max}^{\mathrm{ver}} < \infty$$

with a fixed scale across time. If certified verifier updates satisfy

$$\mathrm{Power}_{t+1}(W_{t+1}, \rho_{t+1}) \ge \mathrm{Power}_t(W_t, \rho_t) + p$$

for every $t = 0, \dots, T-1$ and some constant $p > 0$, then

$$T \le \frac{P_{\max}^{\mathrm{ver}} - \mathrm{Power}_0(W_0, \rho_0)}{p}.$$

Thus indefinite uniform verifier-power improvement requires an expanding power scale, decreasing gains, or another explicitly stated open-ended verifier-progress model.

Proof. Summing the gain inequality yields

$$\mathrm{Power}_T(W_T, \rho_T) \ge \mathrm{Power}_0(W_0, \rho_0) + Tp.$$

The upper bound $\mathrm{Power}_T \le P_{\max}^{\mathrm{ver}}$ gives the stated inequality. □

34 Module F: Physical and Computational Resource Realism

A density-state formulation is an information-theoretic model. It does not by itself prove that a recursive update, certificate, recovery map, verifier, or rollback procedure is physically executable. This module adds explicit resource accounting. The goal is not to reduce all physics to a single scalar, but to prevent the manuscript from making physically impossible update claims.

34.1 Resource states and costs

Definition 34.1 (Physical-computational resource vector). *At time t, let*

$$r_t = (E_t, K_t, M_t^{\mathrm{mem}}, T_t, \Sigma_t, R_t^{\mathrm{roll}}) \in \mathbb{R}^6_{\geq 0}$$

be the declared resource vector, where:

(a) E_t is usable energy or energy budget;

(b) K_t is compute budget;

(c) M_t^{mem} is memory budget;

(d) T_t is wall-clock or decision-time budget;

(e) Σ_t is an allowed irreversibility or entropy-production budget;

(f) R_t^{roll} is rollback, backup, or reversibility budget.

The units and conversion rules are architecture-specific and must be specified by the implementation.

Definition 34.2 (Update and certification cost vector). *For a candidate update Φ_t, define the total cost vector*

$$c_t(\Phi_t) = \big(E_t^{\mathrm{upd}} + E_t^{\mathrm{cert}} + E_t^{\mathrm{rec}} + E_t^{\mathrm{roll}}, K_t^{\mathrm{upd}} + K_t^{\mathrm{cert}} + K_t^{\mathrm{rec}} + K_t^{\mathrm{roll}},$$
$$M_t^{\mathrm{upd}} + M_t^{\mathrm{cert}} + M_t^{\mathrm{rec}} + M_t^{\mathrm{roll}}, T_t^{\mathrm{upd}} + T_t^{\mathrm{cert}} + T_t^{\mathrm{rec}} + T_t^{\mathrm{roll}}, \Sigma_t^{\mathrm{upd}} + \Sigma_t^{\mathrm{cert}} + \Sigma_t^{\mathrm{rec}} + \Sigma_t^{\mathrm{roll}}, R_t^{\mathrm{roll,cost}}\big).$$

The superscripts denote update-generation, certificate-checking, constructive-recovery, and rollback costs. Each component must be measured, bounded, or conservatively estimated in the implementation.

Definition 34.3 (Resource-feasible update). *A candidate Φ_t is resource-feasible at time t if*

$$c_t(\Phi_t) \leq r_t$$

componentwise. The resource-feasible candidate set is

$$\Omega_t^{\mathrm{phys}}(\rho_t) = \{\Phi_t \in \Omega_t(\rho_t) : c_t(\Phi_t) \leq r_t\}.$$

Definition 34.4 (Resource-admissible RCP gate). *Let $\mathcal{A}_{\mathrm{RCP}}^{\mathrm{AC}}(t, \rho_t; g_t^{\min})$ denote the rigor-audited post-A–C gate from Definition 31.5. Define the resource-admissible post-A–C gate*

$$\mathcal{A}_{\mathrm{RCP}}^{\mathrm{phys}}(t, \rho_t; g_t^{\min}) = \mathcal{A}_{\mathrm{RCP}}^{\mathrm{AC}}(t, \rho_t; g_t^{\min}) \cap \Omega_t^{\mathrm{phys}}(\rho_t).$$

If semantic, self-model, verifier, and asymptotic conditions are also enforced, the final D–G gate will use the intersection defined later in Definition 36.2.

Proposition 34.5 (No physical realizability from formal admissibility alone). *Membership in $\mathcal{A}_{\mathrm{RCP}}^{+}(t, \rho_t)$ does not imply resource feasibility unless resource feasibility is included as an additional constraint.*

Proof. The set $\mathcal{A}_{\mathrm{RCP}}^{+}(t, \rho_t)$ is defined by formal safety, recovery, capability, and certificate constraints. It may contain an update whose formal certificate exists but whose computation, verification, recovery, or rollback cost exceeds r_t. Such a candidate is not physically executable within the declared budget. Therefore formal admissibility does not imply resource feasibility. □

34.2 Resource-gated existence and accounting

Assumption 34.6 (Conservative cost soundness)**.** *For every audited candidate Φ_t, the declared cost vector $c_t(\Phi_t)$ upper-bounds the actual resource consumption of update generation, certification, recovery, and rollback on the audited hardware and runtime environment.*

Theorem 34.7 (Resource-gated admissibility)**.** *Assume conservative cost soundness. If*

$$\Phi_t \in \mathcal{A}_{\mathrm{RCP}}^{\mathrm{phys}}(t, \rho_t; g_t^{\min}),$$

then Φ_t satisfies the strengthened RCP gate after Modules A–C and is executable within the declared resource vector r_t.

Proof. By Definition 34.4,

$$\Phi_t \in \mathcal{A}_{\mathrm{RCP}}^{+}(t, \rho_t)$$

and

$$\Phi_t \in \Omega_t^{\mathrm{phys}}(\rho_t).$$

The first membership gives the rigor-audited post-A–C formal RCP constraints. The second means $c_t(\Phi_t) \le r_t$ componentwise. By conservative cost soundness, the declared cost upper-bounds actual resource use on the audited environment, so the update is executable within the declared resource budget. □

Assumption 34.8 (Resource-sufficient safe-improvement availability)**.** *At time t, there exists a candidate*

$$\Phi_t^\sharp \in \mathcal{A}_{\mathrm{RCP}}^{\mathrm{AC}}(t, \rho_t; g_t^{\min})$$

with certified no-op-normalized safe-capability gain

$$\overline{G}_t^{\mathrm{safe}}(\Phi_t^\sharp; \rho_t) \ge g_t^{\min}$$

and with resource cost

$$c_t(\Phi_t^\sharp) \le r_t.$$

Theorem 34.9 (Resource-sufficient nontrivial admissible improvement)**.** *Under Assumption 34.8,*

$$\mathcal{A}_{\mathrm{RCP}}^{\mathrm{phys}}(t, \rho_t; g_t^{\min})$$

contains a candidate with no-op-normalized safe-capability gain at least $g_t^{\min}$.

Proof. The candidate $\Phi_t^\sharp$ belongs to $\mathcal{A}_{\mathrm{RCP}}^{+}(t, \rho_t)$ and satisfies $c_t(\Phi_t^\sharp) \le r_t$. Hence $\Phi_t^\sharp \in \mathcal{A}_{\mathrm{RCP}}^{\mathrm{phys}}(t, \rho_t)$. The declared gain bound is part of the assumption. □

Interpretation 34.10. *The theorem is intentionally conditional. It identifies the exact extra fact needed to turn a formal safe-improvement candidate into a physically executable safe-improvement candidate: the total update, certification, recovery, and rollback costs must fit inside the declared resource vector.*

34.3 Cumulative resource budgets

Definition 34.11 (Cumulative resource use)**.** *For a trajectory $(\rho_t, \Phi_t)_{t=0}^{T-1}$, define cumulative cost*

$$C_T^{\mathrm{phys}} = \sum_{t=0}^{T-1} c_t(\Phi_t)$$

componentwise. Let

$$R_T^{\mathrm{phys}} = \sum_{t=0}^{T-1} r_t$$

be the cumulative declared budget.

Proposition 34.12 (Finite-horizon resource accounting)**.** *If every accepted update is resource-feasible, then*

$$C_T^{\mathrm{phys}} \le R_T^{\mathrm{phys}}$$

componentwise for every finite horizon T.

Proof. Resource feasibility gives $c_t(\Phi_t) \le r_t$ componentwise for each t. Summing the componentwise inequalities from $t = 0$ to $T - 1$ gives the result. □

Limitation 34.13 (No thermodynamic constant is derived here)**.** *This module does not derive a universal physical lower bound for cognition, verification, or self-improvement. It requires the implementation to supply conservative resource-cost models. The physics claim made here is therefore limited: an accepted update is not physically admissible unless its declared physical-computational costs are within budget.*

34.4 Bounded capability scales and open-ended progress

Proposition 34.14 (Uniform positive gain is impossible on a fixed bounded capability scale)**.** *Suppose a capability functional satisfies*

$$0 \le \mathrm{Cap}_t(\rho) \le 1$$

for all t, ρ, and suppose the task distribution and scoring rule are fixed so that $\mathrm{Cap}_t = \mathrm{Cap}$. If accepted updates satisfy

$$\mathrm{Cap}(\rho_{t+1}) \ge \mathrm{Cap}(\rho_t) + g$$

for a constant $g > 0$ and for every $t = 0, \dots, T-1$, then

$$T \le \frac{1 - \mathrm{Cap}(\rho_0)}{g}.$$

Consequently, infinite uniform positive progress is impossible on a fixed bounded capability scale.

Proof. Summing the gain inequality gives

$$\mathrm{Cap}(\rho_T) \ge \mathrm{Cap}(\rho_0) + Tg.$$

Since $\mathrm{Cap}(\rho_T) \le 1$,

$$\mathrm{Cap}(\rho_0) + Tg \le 1,$$

which rearranges to the stated bound. □

Interpretation 34.15. *Open-ended RSI cannot mean a uniform positive increase forever on a fixed bounded benchmark. It must instead mean progress on an expanding task family, an unbounded resource-normalized scale, a sequence of harder validation distributions, decreasing gain thresholds, qualitative architecture transitions, or another explicitly defined notion of open-ended improvement.*

Limitation 34.16. *This proposition is a correction to over-strong progress language. The RCP framework can support finite-horizon improvement, asymptotic approach to a capability ceiling, or open-ended progress under expanding metrics, but it does not prove infinite constant-gain improvement on a bounded score.*

35 Module G: Asymptotic Coherence and Invariant Regimes

The original RCP inequalities imply bounded loss and safe-set invariance under their assumptions. They do not automatically imply convergence to a point, convergence to a unique coherent regime, or unbounded improvement. This section states what follows from the Lyapunov inequalities and what additional assumptions are needed for convergence to a target set.

35.1 Lyapunov summability

Definition 35.1 (Deterministic RCP Lyapunov trajectory)**.** *A deterministic RCP trajectory is a sequence $(\rho_t)_{t\geq 0}$ satisfying*

$$\rho_{t+1} = \Phi_t(\rho_t)$$

for accepted updates Φ_t. Let $V_t = V(\rho_t)$, where V is the RCP Lyapunov incoherence functional of the surrounding manuscript.

Assumption 35.2 (One-step Lyapunov descent with summable error)**.** *There exist $\kappa > 0$, a metric d, and errors $\eta_t \geq 0$ such that*

$$V(\rho_{t+1}) \leq V(\rho_t) - \kappa d(\rho_{t+1}, \rho_t)^2 + \eta_t$$

for all t, and

$$\sum_{t=0}^{\infty} \eta_t < \infty.$$

Assume also that V is bounded below on the safe set by $V_{\inf} > -\infty$.

Theorem 35.3 (Summable motion and Lyapunov convergence)**.** *Under Assumption 35.2, the sequence $V(\rho_t)$ converges to a finite limit and*

$$\sum_{t=0}^{\infty} d(\rho_{t+1}, \rho_t)^2 < \infty.$$

In particular,

$$d(\rho_{t+1}, \rho_t) \to 0.$$

Proof. Rearrange the descent inequality:

$$\kappa d(\rho_{t+1}, \rho_t)^2 \leq V(\rho_t) - V(\rho_{t+1}) + \eta_t.$$

Summing from $t = 0$ to $T - 1$ yields

$$\kappa \sum_{t=0}^{T-1} d(\rho_{t+1}, \rho_t)^2 \leq V(\rho_0) - V(\rho_T) + \sum_{t=0}^{T-1} \eta_t.$$

Since $V(\rho_T) \geq V_{\inf}$ and $\sum_t \eta_t < \infty$, the right-hand side is bounded uniformly in T. Hence the squared-motion series is summable, and therefore $d(\rho_{t+1}, \rho_t) \to 0$.

To prove convergence of $V(\rho_t)$, define

$$\widetilde{V}_t = V(\rho_t) + \sum_{s=t}^{\infty} \eta_s.$$

Then

$$\widetilde{V}_{t+1} = V(\rho_{t+1}) + \sum_{s=t+1}^{\infty} \eta_s \leq V(\rho_t) + \eta_t - \kappa d(\rho_{t+1}, \rho_t)^2 + \sum_{s=t+1}^{\infty} \eta_s \leq \widetilde{V}_t.$$

Thus $\widetilde{V}_t$ is monotone nonincreasing and bounded below by $V_{\inf}$, so it converges. Since $\sum_{s=t}^{\infty} \eta_s \to 0$, $V(\rho_t)$ also converges. □

Interpretation 35.4. *A Lyapunov inequality with summable error proves bounded total squared motion and convergence of the Lyapunov value. It does not by itself prove that ρ_t converges to a single state.*

35.2 Convergence to a target coherence set

Definition 35.5 (Post-D–G state safe set and target coherence set)**.** *Let $\mathcal{K}^{\mathrm{DG}}_{\mathrm{RCP}}$ denote a declared post-D–G state safe set contained in $\mathcal{D}(\mathcal{H}_t)$. At minimum it should include the inherited state safe set $\mathcal{K}^{\mathrm{state}}_{\mathrm{RCP}}(t)$, semantic validity when state-level semantics are claimed, self-model fidelity, and any state-level verifier-integrity predicates. Resource feasibility is an update-level condition and is not automatically a state predicate unless resources are included in the state.*

Let

$$\mathcal{M}_\star \subseteq \mathcal{K}^{\mathrm{DG}}_{\mathrm{RCP}}$$

be a nonempty closed target set. Depending on the application, $\mathcal{M}_\star$ may represent fixed points, an invariant coherence manifold, a viability kernel, or states where no further certified nontrivial improvement is available under the current resource and semantic specification.

Assumption 35.6 (Target-set descent)**.** *There exists a nondecreasing function $a : [0, \infty) \to [0, \infty)$ such that*

$$a(r) > 0 \quad \text{for all } r > 0,$$

and

$$V(\rho_{t+1}) \leq V(\rho_t) - a\big(\mathrm{dist}(\rho_t, \mathcal{M}_\star)\big) + \eta_t$$

with $\sum_t \eta_t < \infty$ and V bounded below.

Theorem 35.7 (Convergence to the target coherence set)**.** *Under Assumption 35.6,*

$$\mathrm{dist}(\rho_t, \mathcal{M}_\star) \to 0.$$

Proof. Rearrange and sum as before:

$$\sum_{t=0}^{T-1} a\big(\operatorname{dist}(\rho_t, \mathcal{M}_\star)\big) \le V(\rho_0) - V(\rho_T) + \sum_{t=0}^{T-1} \eta_t.$$

The right-hand side is bounded uniformly in T, so

$$\sum_{t=0}^{\infty} a\big(\operatorname{dist}(\rho_t, \mathcal{M}_\star)\big) < \infty.$$

If $\operatorname{dist}(\rho_t, \mathcal{M}_\star) \not\to 0$, then there exist $\epsilon > 0$ and infinitely many times t_j with $\operatorname{dist}(\rho_{t_j}, \mathcal{M}_\star) \ge \epsilon$. Since a is nondecreasing and positive away from zero,

$$a\big(\operatorname{dist}(\rho_{t_j}, \mathcal{M}_\star)\big) \ge a(\epsilon) > 0$$

infinitely often, contradicting summability. Hence the distance must converge to zero. □

Corollary 35.8 (Convergence to a unique state)**.** *If $\mathcal{M}_\star = \{\rho_\star\}$, then*

$$\rho_t \to \rho_\star$$

in the metric used to define dist.

Proof. If $\mathcal{M}_\star = \{\rho_\star\}$, then $\operatorname{dist}(\rho_t, \mathcal{M}_\star) = d(\rho_t, \rho_\star)$. Theorem 35.7 gives $d(\rho_t, \rho_\star) \to 0$. □

Limitation 35.9 (Target-set descent is a strong extra assumption)**.** *The original RCP Lyapunov inequality gives descent in step size, not necessarily descent toward a specific target set. Theorem 35.7 is valid only when the stronger target-set descent condition is verified for the actual closed-loop dynamics.*

35.3 Limit sets under compactness

Definition 35.10 (Omega-limit set)**.** *For a trajectory (ρ_t), define its omega-limit set*

$$\omega(\rho_0) = \left\{\bar{\rho} : \exists t_j \to \infty \text{ such that } \rho_{t_j} \to \bar{\rho}\right\}.$$

Proposition 35.11 (Nonempty compact limit set in finite dimension)**.** *If the trajectory remains in a compact safe set $\mathcal{K}$, then $\omega(\rho_0)$ is nonempty and compact.*

Proof. Every sequence in a compact set has a convergent subsequence, so $\omega(\rho_0)$ is nonempty. The omega-limit set is closed as the set of subsequential limits of a sequence in a compact metric space; hence it is compact. □

Assumption 35.12 (Closed asymptotic update relation)**.** *There is a closed relation*

$$\mathcal{F}_\infty \subseteq \mathcal{K} \times \mathcal{K}$$

that contains every asymptotic state-transition limit of the accepted trajectory. Concretely, whenever $t_j \to \infty$, $\rho_{t_j} \to \bar{\rho}$, and $\rho_{t_j+1} \to \bar{\rho}^+$, then

$$(\bar{\rho}, \bar{\rho}^+) \in \mathcal{F}_\infty.$$

If $\mathcal{F}_\infty$ is derived from limiting update certificates rather than directly from state-transition limits, the relevant certificate spaces must be compact or otherwise guarantee convergent certificate subsequences.

Proposition 35.13 (Limit points are asymptotically stationary under summable motion)**.** *Assume the trajectory remains in a compact safe set, Assumption 35.2, and the closed asymptotic update relation. Then every limit point $\bar{\rho} \in \omega(\rho_0)$ satisfies*

$$(\bar{\rho}, \bar{\rho}) \in \mathcal{F}_\infty.$$

Proof. Let $\bar{\rho} \in \omega(\rho_0)$. Then there exists $t_j \to \infty$ such that $\rho_{t_j} \to \bar{\rho}$. By Theorem 35.3,

$$d(\rho_{t_j+1}, \rho_{t_j}) \to 0.$$

Therefore $\rho_{t_j+1} \to \bar{\rho}$ as well. Assumption 35.12 with $\bar{\rho}^+ = \bar{\rho}$ implies

$$(\bar{\rho}, \bar{\rho}) \in \mathcal{F}_\infty.$$

□

Interpretation 35.14. *Under compactness, summable motion, and a closed relation containing all asymptotic transition limits, long-run accumulation points must be stationary for the asymptotic update relation. This is weaker than convergence to a unique point but stronger than mere boundedness.*

36 Augmented RCP Admissibility After Modules D–G

This section is a dependent replacement or extension of the post-A–C admissibility section. It does not repeat the A–C gate. It adds the semantic, self-model, verifier, physical-resource, and asymptotic requirements needed by Modules D–G.

Definition 36.1 (D–G admissibility components)**.** *For a state ρ_t, define the following candidate sets:*

$$\mathcal{A}^{\mathrm{sem}}_{\mathrm{RCP}}(t, \rho_t)$$

from Definition 32.10,

$$\mathcal{A}^{M}_{\mathrm{RCP}}(t, \rho_t) = \{\Phi_t \in \Omega_t(\rho_t) : B^{M}_{t+1}(\Phi_t(\rho_t)) \le 0\},$$

$$\mathcal{A}^{\mathrm{ver}}_{\mathrm{RCP}}(t, \rho_t) = \{\Phi_t \in \Omega_t(\rho_t) : \mathrm{Cert}^{\mathrm{ver}}_t(\Phi_t, \rho_t) = 1\}.$$

and

$$\Omega^{\mathrm{phys}}_t(\rho_t) = \{\Phi_t \in \Omega_t(\rho_t) : c_t(\Phi_t) \le r_t\}$$

from Definition 34.3. Here $\mathrm{Cert}^{\mathrm{ver}}_t(\Phi_t, \rho_t) = 1$ abbreviates that the verifier and trust-anchor certificates required for accepting Φ_t hold on the audited domain with their declared failure probabilities. The set $\mathcal{A}^{\mathrm{ver}}_{\mathrm{RCP}}$ is intentionally defined through declared certificates; if those certificates are not supplied, the verifier component is not established.

Definition 36.2 (Fully augmented post-D–G RCP admissibility gate)**.** *Let $\mathcal{A}^{\mathrm{AC}}_{\mathrm{RCP}}(t, \rho_t; g^{\min}_t)$ denote the rigor-audited post-A–C gate. The post-D–G fully augmented gate is*

$$\mathcal{A}^{\mathrm{DG}}_{\mathrm{RCP}}(t, \rho_t; g^{\min}_t) = \mathcal{A}^{\mathrm{AC}}_{\mathrm{RCP}}(t, \rho_t; g^{\min}_t) \cap \mathcal{A}^{\mathrm{sem}}_{\mathrm{RCP}}(t, \rho_t) \cap \mathcal{A}^{M}_{\mathrm{RCP}}(t, \rho_t) \cap \mathcal{A}^{\mathrm{ver}}_{\mathrm{RCP}}(t, \rho_t) \cap \Omega^{\mathrm{phys}}_t(\rho_t).$$

If asymptotic target-set convergence is claimed, then the accepted update must also satisfy the target-set descent condition of Assumption 35.6; otherwise only boundedness, invariance, summable-motion, and finite-horizon preservation should be claimed.

Theorem 36.3 (Post-D–G augmented RCP preservation theorem)**.** *Assume all hypotheses required by the post-A–C gate and by Modules D–G: semantic realization and sound semantic barriers, self-model fidelity barriers and any claimed operational audit soundness, verifier certification soundness, conservative cost soundness, and the relevant Lyapunov or target-set descent assumptions. If*

$$\rho_{t+1} = \Phi_t(\rho_t) \quad \textit{and} \quad \Phi_t \in \mathcal{A}^{\mathrm{DG}}_{\mathrm{RCP}}(t, \rho_t; g_t^{\min}),$$

then the successor satisfies:

(a) the post-A–C non-lossiness, recoverability, capability, progress, and recursive-feasibility constraints contained in $\mathcal{A}^{\mathrm{AC}}_{\mathrm{RCP}}$*;*

(b) semantic register validity at $t+1$*;*

(c) self-model fidelity at $t+1$ *relative to the audited operational state, and relative to the true operational state with the additional audit-error term if Assumption 33.5 is invoked;*

(d) the declared verifier-integrity and verifier-bootstrapping certificates, except with the declared certification-failure probabilities;

(e) physical-computational resource feasibility under the declared resource vector;

(f) the applicable Lyapunov descent or target-set descent inequality, when that inequality is included as a gate condition.

On any finite horizon, the corresponding cumulative bounds hold by induction, telescoping, finite-horizon resource accounting, and union bounds over declared probabilistic certificate failures.

Proof. Membership in the intersection defining $\mathcal{A}^{\mathrm{DG}}_{\mathrm{RCP}}(t, \rho_t; g_t^{\min})$ gives membership in each component gate. The post-A–C component yields the previously proved non-lossiness, recovery, capability, progress, and recursive-feasibility conclusions. The semantic component yields semantic validity by Theorem 32.11. The self-model component yields fidelity by Theorem 33.6; if a claim about the true operational state is made, Corollary 33.7 adds the audit-error term. The verifier component yields the certified verifier conclusions by Theorem 33.22. Physical feasibility follows from componentwise membership in Ω_t^{phys} and conservative cost soundness, equivalently from Theorem 34.7. The Lyapunov or target-set component yields the corresponding one-step inequality. Repeating the one-step implications over a finite horizon gives induction and telescoping conclusions. Probabilistic verifier and estimator claims combine by the union bound. □

Interpretation 36.4. *After Modules D–G, the RCP gate is no longer merely an information-preservation filter. It is a conditional intersection gate for formal non-lossiness, semantic validity, self-model fidelity, verifier integrity, physical resource feasibility, and asymptotic descent.*

Limitation 36.5. *The theorem is a composition theorem. It is only as strong as the component assumptions and certificates. It does not prove that* $\mathcal{A}^{\mathrm{DG}}_{\mathrm{RCP}}(t, \rho_t; g_t^{\min})$ *is nonempty, except where a separate A–C existence, resource-sufficiency, or viability theorem supplies nonemptiness.*

37 Architecture Feedback and Operational Closure

The RCP theorem stack above is architecture-general. The RSI–RCLM architecture paper, derived from this stack, exposes a set of operational assumptions that are general enough to belong in the math/physics paper as an abstract closure layer. This section adds those objects without changing the core RCP state equation

$$\rho_t \in \mathcal{D}(\mathcal{H}), \qquad \Phi_t : \rho_t \mapsto \rho_{t+1},$$

or the core quantities

$$\mathcal{L}_\Phi^{\mathcal{P}}, \qquad V(\rho_t), \qquad \mathcal{A}_{\mathrm{RCP}}, \qquad \mathcal{K}_{\mathrm{RCP}}.$$

Instead, it clarifies what must be true for these objects to be typed, non-oracular, auditable, and implementable in a concrete architecture.

Limitation 37.1 (Operational closure is not an unconditional RSI proof)**.** *The definitions below do not prove that arbitrary architectures possess sound semantic maps, complete coverage, cheap estimators, or persistent beneficial updates. They convert several previously architecture-specific requirements into explicit abstract RCP obligations. The result is a stronger conditional theory, not a proof that unconstrained recursive self-improvement is safe or self-grounding.*

37.1 Typed density realization

The notation $\rho \in \mathcal{D}(\mathcal{H})$ is mathematically precise, but a concrete implementation may use it in three different ways: as a physical quantum state, as a classical probability distribution represented diagonally, or as a learned representational state embedded into the density cone. These cases have different proof licenses.

Definition 37.2 (Typed density realization)**.** *A typed density realization at time t is a tuple*

$$\mathfrak{T}_t = (\mathcal{H}_t, \mathcal{D}_t, \tau_t, \mathsf{Lic}_t),$$

where $\mathcal{D}_t \subseteq \mathcal{D}(\mathcal{H}_t)$,

$$\tau_t : \mathcal{D}_t \to \{\mathrm{q}, \mathrm{cl}, \mathrm{repr}\}$$

assigns a type tag, and $\mathsf{Lic}_t(\rho, \Phi)$ records which mathematical rules are licensed for (ρ, Φ). The intended meanings are:

(a) $\tau_t(\rho) = \mathrm{q}$: ρ is treated as a quantum density operator and accepted updates invoking quantum data processing must be CPTP on the relevant algebra.

(b) $\tau_t(\rho) = \mathrm{cl}$: ρ is diagonal in an audited basis and accepted updates invoking data processing must reduce to stochastic maps on that basis.

(c) $\tau_t(\rho) = \mathrm{repr}$: ρ is a learned representational embedding in the density cone. Quantum or classical data-processing theorems may not be invoked unless a valid channel reduction is separately certified.

Definition 37.3 (Type-admissible update)**.** *A candidate $\Phi \in \Omega_t$ is type-admissible, written*

$$\Phi \in \mathcal{A}_{\mathrm{RCP}}^{\mathrm{type}}(t, \rho_t),$$

if every residual certificate used to admit Φ is justified by one of the following:

(a) a theorem licensed by $\mathsf{Lic}_t(\rho_t, \Phi)$*, such as quantum data processing for a CPTP channel;*

(b) a classical stochastic-channel argument for a diagonal state;

(c) a direct audited residual estimate with an explicit one-sided error radius.

Proposition 37.4 (No category-error rule)**.** *If* $\Phi \in \mathcal{A}^{\mathrm{type}}_{\mathrm{RCP}}(t, \rho_t)$*, then every accepted use of entropy, relative entropy, mutual information, Lyapunov drift, or recovery for* Φ *is either licensed by the typed realization or directly certified as an estimated residual. In particular, representational density states do not inherit quantum physical conclusions merely by satisfying positivity and trace-one constraints.*

Proof. The conclusion is exactly the defining condition of $\mathcal{A}^{\mathrm{type}}_{\mathrm{RCP}}$. For each residual used in the gate, type-admissibility requires either a licensed theorem or a direct one-sided certificate. Thus no residual is accepted solely because the symbol ρ lies in the density cone. □

37.2 Representation-to-density embedding

Definition 37.5 (Representation-to-density embedding)**.** *Let* $\mathcal{X}_t$ *be a raw representational space, such as hidden states, memories, verifier states, candidate-update codes, or audit traces. A representation-to-density embedding is a map*

$$\mathrm{Emb}_t : \mathcal{X}_t \to \mathcal{D}(\mathcal{H}_t)$$

of the form

$$\mathrm{Emb}_t(x) = \frac{F_t(x)F_t(x)^\dagger + \lambda_t I}{\mathrm{Tr}\big(F_t(x)F_t(x)^\dagger + \lambda_t I\big)}, \qquad \lambda_t \geq 0,$$

where $F_t(x)$ *is a finite-dimensional matrix and the denominator is strictly positive. More general embeddings are allowed only if they are certified to output positive semidefinite trace-one states.*

Lemma 37.6 (Embedding produces density states)**.** *Under Definition 37.5,* $\mathrm{Emb}_t(x) \in \mathcal{D}(\mathcal{H}_t)$ *for every* $x \in \mathcal{X}_t$ *with nonzero denominator.*

Proof. The matrix $F_t(x)F_t(x)^\dagger$ is positive semidefinite and Hermitian. The matrix $\lambda_t I$ is positive semidefinite and Hermitian. Their sum is positive semidefinite and Hermitian. Dividing by its positive trace gives a positive semidefinite Hermitian matrix of trace one. □

Definition 37.7 (Embedding consistency residual)**.** *Let* $T_\nu : \mathcal{X}_t \to \mathcal{X}_{t+1}$ *be the raw representational transition induced by a candidate self-update* ν*, and let* Φ_ν *be its induced density-state update. The embedding consistency residual is*

$$q^{\mathrm{emb}}_t(\nu, x) = d\big(\mathrm{Emb}_{t+1}(T_\nu x), \Phi_\nu(\mathrm{Emb}_t(x))\big) - \delta^{\mathrm{emb}}_t,$$

where $\delta^{\mathrm{emb}}_t \geq 0$ *is the allowed embedding-transfer error.*

Assumption 37.8 (Embedding transfer Lipschitzness)**.** *For every residual* $q_{t,j}$ *used by the RCP gate there is a modulus* $\omega^{\mathrm{emb}}_{t,j}$ *such that, on the audited domain,*

$$|q_{t,j}(\mathrm{Emb}_{t+1}(T_\nu x)) - q_{t,j}(\Phi_\nu(\mathrm{Emb}_t(x)))| \leq \omega^{\mathrm{emb}}_{t,j}\big(d(\mathrm{Emb}_{t+1}(T_\nu x), \Phi_\nu(\mathrm{Emb}_t(x)))\big).$$

Proposition 37.9 (Embedding transfer of residual certificates)**.** *Assume embedding transfer Lipschitzness. If*

$$q_t^{\mathrm{emb}}(\nu, x) \le 0$$

and the density-level residual satisfies

$$q_{t,j}(\Phi_\nu(\mathrm{Emb}_t(x))) + \omega_{t,j}^{\mathrm{emb}}(\delta_t^{\mathrm{emb}}) \le 0,$$

then the corresponding raw-transition residual satisfies

$$q_{t,j}(\mathrm{Emb}_{t+1}(T_\nu x)) \le 0.$$

Proof. The embedding residual gives

$$d(\mathrm{Emb}_{t+1}(T_\nu x), \Phi_\nu(\mathrm{Emb}_t(x))) \le \delta_t^{\mathrm{emb}}.$$

The Lipschitz-transfer assumption therefore gives

$$q_{t,j}(\mathrm{Emb}_{t+1}(T_\nu x)) \le q_{t,j}(\Phi_\nu(\mathrm{Emb}_t(x))) + \omega_{t,j}^{\mathrm{emb}}(\delta_t^{\mathrm{emb}}).$$

The displayed certificate upper bound implies the result. □

37.3 Semantic coupling consistency

Definition 37.10 (Semantic coupling structure)**.** *A semantic coupling structure is a tuple*

$$\mathfrak{C}_t = (\mathrm{Emb}_t, \Gamma_t, \mathcal{O}_t, \mathcal{M}_t^{\mathrm{coup}}, \epsilon_t^{\mathrm{coup}}),$$

where Emb_t *maps raw representations into density states,* $\Gamma_t = \{\Gamma_{X,t}\}_{X \in \{L,O,S,D,A,M\}}$ *interprets density registers semantically,* $\mathcal{O}_t$ *is the external audit observation process,* $\mathcal{M}_t^{\mathrm{coup}}$ *maps density residuals to semantic residuals, and* $\epsilon_t^{\mathrm{coup}}$ *is a certified coupling-error budget.*

Definition 37.11 (Semantic coupling residual)**.** *For a register* $X \in \{L, O, S, D, A, M\}$*, let* $\mathrm{Val}_{X,t}^{\mathrm{ext}}(\rho; \mathcal{O}_t)$ *denote the externally audited semantic-invalidity score of the interpreted register. The semantic coupling residual is*

$$q_{X,t}^{\mathrm{coup}}(\rho) = \mathrm{Val}_{X,t}^{\mathrm{ext}}(\rho; \mathcal{O}_t) - \mathcal{M}_{X,t}^{\mathrm{coup}}(B_{X,t}^{\mathrm{sem}}(\rho)) - \epsilon_{X,t}^{\mathrm{coup}}.$$

A candidate is coupling-admissible if the successor satisfies

$$q_{X,t+1}^{\mathrm{coup}}(\Phi(\rho_t)) \le 0 \qquad \forall X \in \{L, O, S, D, A, M\}.$$

The set of coupling-admissible candidates is denoted

$$\mathcal{A}_{\mathrm{RCP}}^{\mathrm{coup}}(t, \rho_t).$$

Theorem 37.12 (Semantic transfer under coupling consistency)**.** *Assume* $\Phi \in \mathcal{A}_{\mathrm{RCP}}^{\mathrm{coup}}(t, \rho_t)$*. If the density-level semantic barrier for register* X *satisfies*

$$B_{X,t+1}^{\mathrm{sem}}(\Phi(\rho_t)) \le 0$$

and the coupling map is monotone in the sense that

$$B \le 0 \quad \Longrightarrow \quad \mathcal{M}_{X,t+1}^{\mathrm{coup}}(B) \le \delta_{X,t+1}^{\mathrm{sem}},$$

then

$$\mathrm{Val}_{X,t+1}^{\mathrm{ext}}(\Phi(\rho_t); \mathcal{O}_{t+1}) \le \delta_{X,t+1}^{\mathrm{sem}} + \epsilon_{X,t+1}^{\mathrm{coup}}.$$

Proof. Coupling-admissibility gives

$$\mathrm{Val}^{\mathrm{ext}}_{X,t+1}(\Phi(\rho_t);\mathcal{O}_{t+1}) \le \mathcal{M}^{\mathrm{coup}}_{X,t+1}(B^{\mathrm{sem}}_{X,t+1}(\Phi(\rho_t))) + \epsilon^{\mathrm{coup}}_{X,t+1}.$$

The monotonicity condition applied to the nonpositive barrier value bounds the first term by $\delta^{\mathrm{sem}}_{X,t+1}$. Combining the inequalities gives the claim. □

Limitation 37.13. *This theorem transfers a certified density-level semantic barrier into an externally audited semantic-validity bound. It does not derive the correct interpretation map Γ_t, the correct audit observations $\mathcal{O}_t$, or human values from first principles.*

37.4 Non-oracular certificate construction

Definition 37.14 (Residual vector)**.** *Let $\mathfrak{q}_t(\Phi) = (q_{t,j}(\Phi))_{j\in J_t}$ be the vector of all residuals required by the selected RCP gate. Examples include relative-entropy loss, recovery defect, Lyapunov drift, barrier violation, ambiguity collapse, information-bottleneck loss, semantic validity, self-model fidelity, verifier integrity, resource cost, capability progress, coverage gap, and calibration error. The gate condition is written componentwise as*

$$q_{t,j}(\Phi) \le 0 \qquad j \in J_t.$$

Definition 37.15 (Non-oracular certificate packet)**.** *A certificate packet for candidate Φ is*

$$\mathsf{Cert}_t(\Phi) = (\widehat{\mathfrak{q}}_t(\Phi), \Delta^{\mathrm{aud}}_t(\Phi), \delta_t, \mathsf{Audit}_t, \mathsf{Trace}_t(\Phi), W_t),$$

where $\widehat{\mathfrak{q}}_t$ is the estimated residual vector, Δ^{aud}_t is a vector of one-sided radii, $\delta_t = (\delta_{t,j})_{j\in J_t}$ are declared failure probabilities, Audit_t contains calibration data, holdout data, stress tests, adversarial probes, and domain restrictions, Trace_t records how the estimates were produced, and W_t is the verifier state.

Assumption 37.16 (Audited one-sided certificate soundness)**.** *For the audited candidate class $\mathcal{N}_t \subseteq \Omega_t$,*

$$\Pr\left[q_{t,j}(\Phi) \le \widehat{q}_{t,j}(\Phi) + \Delta^{\mathrm{aud}}_{t,j}(\Phi) \quad \forall \Phi \in \mathcal{N}_t, \forall j \in J_t\right] \ge 1 - \sum_{j\in J_t} \delta_{t,j}.$$

The probability is over the declared audit randomness, calibration sampling, stress-test sampling, and any randomized estimator components.

Definition 37.17 (Certificate-admissible update)**.** *A candidate $\Phi \in \mathcal{N}_t$ is certificate-admissible if its certificate packet satisfies*

$$\widehat{q}_{t,j}(\Phi) + \Delta^{\mathrm{aud}}_{t,j}(\Phi) \le 0 \qquad \forall j \in J_t.$$

The certificate-admissible set is denoted

$$\mathcal{A}^{\mathrm{cert}}_{\mathrm{RCP}}(t,\rho_t).$$

Theorem 37.18 (Finite-horizon non-oracular certificate soundness)**.** *Assume audited one-sided certificate soundness at each step $t < T$. If every accepted candidate Φ_t lies in $\mathcal{A}^{\mathrm{cert}}_{\mathrm{RCP}}(t,\rho_t)$, then with probability at least*

$$1 - \sum_{t=0}^{T-1} \sum_{j\in J_t} \delta_{t,j},$$

every accepted residual through time T satisfies $q_{t,j}(\Phi_t) \le 0$.

Proof. At a fixed time t, Assumption 37.16 implies that all true residuals for all candidates in $\mathcal{N}_t$ are bounded above by their certified one-sided estimates, except on an event of probability at most $\sum_{j \in J_t} \delta_{t,j}$. Certificate-admissibility makes each certified upper bound nonpositive. Therefore all true residuals for the accepted candidate are nonpositive on the good event. A union bound over $t = 0, \dots, T-1$ gives the finite-horizon probability statement. □

37.5 Verifier trust graph and no self-sole-certification

Definition 37.19 (Verifier trust graph). *A verifier trust graph for a candidate Φ is*

$$\mathcal{G}_t^{\mathrm{ver}}(\Phi) = (\mathsf{V}_t, \mathsf{E}_t, \mathsf{A}_t, \mathsf{T}_t(\Phi), \mathsf{Ind}_t),$$

where V_t is the set of verifier components, E_t records certification edges, $\mathsf{A}_t \subseteq \mathsf{V}_t$ is the set of trust anchors or frozen verifier slices, $\mathsf{T}_t(\Phi) \subseteq \mathsf{V}_t$ is the set of verifier components modified by Φ, and Ind_t records declared independence or non-interference relations.

Definition 37.20 (No self-sole-certification rule). *A verifier-changing update Φ satisfies no self-sole-certification when every component in $\mathsf{T}_t(\Phi)$ is certified by a path from A_t or from verifier components not modified by Φ, and at least one certifying path is independent according to Ind_t. If a modified verifier component is the sole source of its own soundness certificate, Φ is rejected.*

Definition 37.21 (Trust-admissible update). *Let $\mathcal{A}_{\mathrm{RCP}}^{\mathrm{trust}}(t, \rho_t)$ be the set of candidates whose verifier trust graph satisfies the no self-sole-certification rule and whose verifier degradation residual satisfies*

$$q_t^{\mathrm{ver}}(\Phi) = \mathrm{Sound}(W_t) - \mathrm{Sound}(W_{t+1}) - \omega_t^{\mathrm{ver}} \le 0.$$

If a verifier power claim is made, it must be accompanied by an analogous certified residual; otherwise no verifier-power improvement is claimed.

Theorem 37.22 (Verifier self-consistency under trust graphs). *Assume every accepted verifier-changing update lies in $\mathcal{A}_{\mathrm{RCP}}^{\mathrm{trust}}(t, \rho_t)$. Then for any finite horizon T,*

$$\mathrm{Sound}(W_T) \ge \mathrm{Sound}(W_0) - \sum_{t=0}^{T-1} \omega_t^{\mathrm{ver}}.$$

If additionally verifier-power degradation residuals are certified by budgets π_t^{ver}, then

$$\mathrm{Power}(W_T) \ge \mathrm{Power}(W_0) - \sum_{t=0}^{T-1} \pi_t^{\mathrm{ver}}.$$

Proof. Trust-admissibility gives

$$\mathrm{Sound}(W_{t+1}) \ge \mathrm{Sound}(W_t) - \omega_t^{\mathrm{ver}}$$

for every accepted step. Summing the inequalities telescopes. The proof for verifier power is identical when the power residuals are included. The graph condition supplies the non-circularity condition for accepting the one-step certificates; it does not by itself prove that a trust anchor exists. □

37.6 Budgeted certified search

Definition 37.23 (Beneficial certified-improvement set)**.** *Let $\mathcal{A}_t^{\mathrm{safe}}$ denote the active safety-admissible candidate set, and let $\mathrm{SafeCap}_t$ be the declared safe-capability functional. Define the no-op-normalized improvement of Φ by*

$$\Delta\,\mathrm{SafeCap}_t^0(\Phi) = \mathrm{SafeCap}_{t+1}(\Phi(\rho_t)) - \mathrm{SafeCap}_{t+1}(\mathrm{id}(\rho_t)).$$

For threshold $\epsilon_t^{\mathrm{prog}} > 0$, define

$$\mathcal{I}_t = \{\Phi \in \mathcal{A}_t^{\mathrm{safe}} : \Delta\,\mathrm{SafeCap}_t^0(\Phi) > \epsilon_t^{\mathrm{prog}}\}.$$

For margin vector $m_t > 0$, let $\mathcal{I}_t^m \subseteq \mathcal{I}_t$ be the subset whose residuals are at least m_t inside the gate and whose progress is at least $\epsilon_t^{\mathrm{prog}} + m_t^{\mathrm{prog}}$.

Definition 37.24 (Budgeted certified search)**.** *A budgeted search procedure is a map*

$$\mathsf{Search}_t(B_t^{\mathrm{search}}, \rho_t, W_t) = S_t(B_t^{\mathrm{search}}) \subseteq \Omega_t,$$

where B_t^{search} is the available search budget. The certified progress-search set is

$$\mathcal{A}_{\mathrm{RCP}}^{\mathrm{search}}(t, \rho_t; B_t^{\mathrm{search}}) = \{\Phi \in S_t(B_t^{\mathrm{search}}) : \Phi \in \mathcal{A}_t^{\mathrm{safe}},\ \Delta\,\widehat{\mathrm{SafeCap}}_t^0(\Phi) - \Delta_t^{\mathrm{gain}} > \epsilon_t^{\mathrm{prog}}\}.$$

Assumption 37.25 (Search-and-certification completeness)**.** *If $\mathcal{I}_t^m \neq \varnothing$, then with probability at least $1 - \delta_t^{\mathrm{search}}$, the budgeted search returns a candidate in $\mathcal{A}_{\mathrm{RCP}}^{\mathrm{search}}(t, \rho_t; B_t^{\mathrm{search}})$.*

Theorem 37.26 (Budgeted progress from margin-qualified improvements)**.** *Assume search-and-certification completeness and one-sided gain-estimator soundness. If $\mathcal{I}_t^m \neq \varnothing$, then with probability at least $1 - \delta_t^{\mathrm{search}} - \delta_t^{\mathrm{gain}}$, the progress-search set is nonempty and any selected candidate from it satisfies*

$$\Delta\,\mathrm{SafeCap}_t^0(\Phi) > \epsilon_t^{\mathrm{prog}}.$$

Proof. Search-and-certification completeness gives, except on an event of probability $\delta_t^{\mathrm{search}}$, a returned candidate with conservative residual certificates and a conservative gain certificate. One-sided gain soundness gives, except on an event of probability δ_t^{gain}, that the true no-op-normalized safe-capability gain is at least the certified lower bound. The certified progress-search condition makes that lower bound exceed $\epsilon_t^{\mathrm{prog}}$. A union bound gives the probability statement. □

37.7 Resource-scaling and no-op pressure

Definition 37.27 (Resource scaling residual)**.** *Let R_t be the available resource vector and let*

$$C_t^{\mathrm{tot}}(\Phi) = C_t^{\mathrm{search}}(\Phi) + C_t^{\mathrm{cert}}(\Phi) + C_t^{\mathrm{rec}}(\Phi) + C_t^{\mathrm{ver}}(\Phi) + C_t^{\mathrm{rollback}}(\Phi) + C_t^{\mathrm{apply}}(\Phi)$$

be the audited total cost. The one-step resource residual is

$$q_t^{\mathrm{scale}}(\Phi) = C_t^{\mathrm{tot}}(\Phi) - R_t.$$

For a horizon T, an amortized scaling condition is

$$\sum_{t=0}^{T-1} C_t^{\mathrm{tot}}(\Phi_t) \leq \sum_{t=0}^{T-1} R_t^{\mathrm{in}} - ho_T^{\mathrm{reserve}},$$

where R_t^{in} is incoming resource and $\rho_T^{\mathrm{reserve}} \geq 0$ is a required reserve.

Definition 37.28 (No-op rate and certificate inflation)**.** *For a window length h_t, define the no-op rate*

$$r^0_{t,h_t} = \frac{1}{h_t} \sum_{s=\max(0,t-h_t+1)}^{t} \mathbf{1}\{\Phi_s = \mathrm{id}\}.$$

The certificate-inflation score is

$$\mathrm{Infl}_t(\Phi) = \mathrm{Relax}_t(\Phi) + \mathrm{Rad}_t(\Phi),$$

where Relax_t *measures threshold or budget relaxation and* Rad_t *measures growth of estimator radii. The no-op and inflation gate requires*

$$r^0_{t,h_t} \le r^0_{\max,t} \quad \textit{or} \quad \mathsf{Diag}^0_t = 1,$$

$$\mathrm{Infl}_t(\Phi) \le \iota_t.$$

The diagnostic flag Diag^0_t *may expand search, improve verifier resolution, request external validation, or pause Level 3 updates, but it may not relax safety residuals.*

Theorem 37.29 (Resource and no-op pressure control)**.** *If every accepted update satisfies the resource scaling residual, the amortized scaling condition, the no-op diagnostic rule, and the certificate-inflation bound, then over any finite horizon the system cannot honestly report engine-mode progress while any of the following holds unreported: certified cost exceeds available resource, no-op dominance exceeds the declared threshold without diagnostic action, or progress is obtained by relaxing thresholds or expanding estimator radii beyond* ι_t*.*

Proof. The resource residual and amortized condition are exactly the inequalities excluding unreported resource overrun. The no-op rule requires either a bounded no-op rate or an explicit diagnostic flag. The inflation bound requires the reported accepted candidate to remain below ι_t. Therefore a report of engine-mode progress that omits one of these violations contradicts one of the accepted gate conditions. □

37.8 Coverage-gap residuals

Definition 37.30 (Tracked and untracked predicate families)**.** *Let* $\mathcal{S}^{\mathrm{trk}}_t$ *be the declared tracked safety-predicate family and* $\mathcal{S}^{\mathrm{untrk}}_t$ *be an audited residual class of plausible but not fully represented safety predicates. The coverage gap of* Φ *is*

$$\mathrm{CovGap}_t(\Phi) = \sup_{s \in \mathcal{S}^{\mathrm{untrk}}_t} \mathrm{Risk}_t(s, \Phi),$$

where $\mathrm{Risk}_t(s,\Phi)$ *is the declared risk score for violating predicate* s *under* Φ*.*

Definition 37.31 (Coverage certificate)**.** *A coverage certificate is a pair* $(\widehat{\mathrm{CovGap}}_t(\Phi), \Delta^{\mathrm{cov}}_t(\Phi))$ *satisfying the one-sided event*

$$\mathrm{CovGap}_t(\Phi) \le \widehat{\mathrm{CovGap}}_t(\Phi) + \Delta^{\mathrm{cov}}_t(\Phi).$$

The coverage-admissible set is

$$\mathcal{A}^{\mathrm{cov}}_{\mathrm{RCP}}(t, \rho_t) = \{\Phi : \widehat{\mathrm{CovGap}}_t(\Phi) + \Delta^{\mathrm{cov}}_t(\Phi) \le \gamma^{\mathrm{cov}}_t\}.$$

Proposition 37.32 (Coverage-gap bound). *On the coverage-certificate soundness event, every* $\Phi \in \mathcal{A}^{\mathrm{cov}}_{\mathrm{RCP}}(t, \rho_t)$ *satisfies*

$$\mathrm{CovGap}_t(\Phi) \le \gamma_t^{\mathrm{cov}}.$$

Proof. The certificate event gives $\mathrm{CovGap}_t(\Phi) \le \widehat{\mathrm{CovGap}}_t(\Phi) + \Delta_t^{\mathrm{cov}}(\Phi)$. Membership in $\mathcal{A}^{\mathrm{cov}}_{\mathrm{RCP}}$ bounds the right-hand side by γ_t^{cov}. □

Limitation 37.33. *The residual* $\mathcal{S}_t^{\mathrm{untrk}}$ *is still an audited residual class, not the set of all possible unknown risks. The theorem turns coverage uncertainty into an explicit budget; it does not eliminate unknown unknowns.*

37.9 Coherence-capability calibration protocol

Definition 37.34 (Capability calibration suite). *Let*

$$\mathcal{T}_t^{\mathrm{Cap}} = \mathcal{T}_t^{\mathrm{heldout}} \cup \mathcal{T}_t^{\mathrm{ood}} \cup \mathcal{T}_t^{\mathrm{adv}} \cup \mathcal{T}_t^{\mathrm{formal}}$$

be an audited capability-calibration suite. Let $\Xi_t(\Phi)$ *be the vector of explanatory quantities for the proposed update, such as*

$$\Delta\mathcal{C}_{\mathrm{RCP}}, \quad \mathcal{L}_\Phi^{\mathcal{P}}, \quad U_t, \quad \max_i B_i(\rho_{t+1}), \quad \mathrm{Infl}_t(\Phi), \quad C_t^{\mathrm{tot}}(\Phi).$$

A calibration map is a declared function

$$f_t^{\mathrm{cal}} : \Xi_t(\Phi) \mapsto \Delta\,\widehat{\mathrm{SafeCap}}_t(\Phi).$$

The calibration error is

$$e_t^{\mathrm{cal}}(\Phi) = \left| \Delta\,\mathrm{SafeCap}_t^0(\Phi) - f_t^{\mathrm{cal}}(\Xi_t(\Phi)) \right|.$$

Definition 37.35 (Calibration-admissible update). *A candidate is calibration-admissible if*

$$\widehat{e}_t^{\mathrm{cal}}(\Phi) + \Delta_t^{\mathrm{cal}}(\Phi) \le e_{\max,t}^{\mathrm{cal}}.$$

The set of such candidates is denoted $\mathcal{A}^{\mathrm{cal}}_{\mathrm{RCP}}(t, \rho_t)$.

Proposition 37.36 (Coherence-capability calibration bound). *On the one-sided calibration soundness event, if* $\Phi \in \mathcal{A}^{\mathrm{cal}}_{\mathrm{RCP}}(t, \rho_t)$, *then*

$$e_t^{\mathrm{cal}}(\Phi) \le e_{\max,t}^{\mathrm{cal}}.$$

Proof. One-sided calibration soundness gives

$$e_t^{\mathrm{cal}}(\Phi) \le \widehat{e}_t^{\mathrm{cal}}(\Phi) + \Delta_t^{\mathrm{cal}}(\Phi).$$

Calibration-admissibility bounds the right-hand side by $e_{\max,t}^{\mathrm{cal}}$. □

Limitation 37.37. *This module does not prove that coherence implies capability. It only requires that any claimed coherence-supported capability relation be calibrated against an explicit heldout, out-of-distribution, adversarial, and formal task suite.*

37.10 Estimator subprotocols

Definition 37.38 (Estimator subprotocol). *For each residual family $k \in \mathcal{K}_t^{\mathrm{est}}$, an estimator subprotocol is a tuple*

$$\mathsf{Est}_{t,k} = (\mathsf{Input}_{t,k}, \mathsf{Dom}_{t,k}, \widehat{q}_{t,k}, \Delta_{t,k}^{\mathrm{est}}, \delta_{t,k}^{\mathrm{est}}, \mathrm{Cost}_{t,k}, \mathsf{Stress}_{t,k}, \mathsf{Fail}_{t,k}),$$

where $\mathsf{Dom}_{t,k}$ is the audited domain, $\widehat{q}_{t,k}$ is the estimator, $\Delta_{t,k}^{\mathrm{est}}$ is its one-sided radius, $\delta_{t,k}^{\mathrm{est}}$ its failure probability, $\mathrm{Cost}_{t,k}$ its computational cost, $\mathsf{Stress}_{t,k}$ its stress-test family, and $\mathsf{Fail}_{t,k}$ the documented failure mode outside the audited domain.

Definition 37.39 (Estimator-complete gate). *A candidate belongs to $\mathcal{A}_{\mathrm{RCP}}^{\mathrm{est}}(t, \rho_t)$ if every residual used by the active gate has an estimator subprotocol with*

$$q_{t,k}(\Phi) \le \widehat{q}_{t,k}(\Phi) + \Delta_{t,k}^{\mathrm{est}}(\Phi)$$

on its audited domain and if the sum of estimator costs is included in the resource residual.

Proposition 37.40 (Estimator subprotocols instantiate certificate vectors). *If $\Phi \in \mathcal{A}_{\mathrm{RCP}}^{\mathrm{est}}(t, \rho_t)$, then the collection of subprotocol outputs defines a certificate packet of Definition 37.15 with componentwise one-sided radii*

$$\Delta_{t,k}^{\mathrm{aud}} = \Delta_{t,k}^{\mathrm{est}}.$$

Consequently, the non-oracular certificate theorem applies to the active residual vector whenever the subprotocol soundness events hold.

Proof. By estimator-completeness, each active residual has an estimator, an audited domain, a one-sided radius, and a failure probability. Collecting these objects over $k \in \mathcal{K}_t^{\mathrm{est}}$ gives the estimated residual vector, radius vector, failure-probability vector, audit data, and trace required by a certificate packet. The finite-horizon certificate theorem then applies directly. □

38 Batch 10 Synchronization: Adaptive Trust, Complexity Ledgers, and Typed Transitions

Inserted after the Batch 9 architecture synchronization.

This section synchronizes the abstract math/physics paper with the final Batch-10 architecture cleanup. It does not change the core RCP equations, the conservative-extension algebra, the tangent-cone theorem, the engine-viability kernel, or the Batch 9 direct-engine theorem. It adds three publication-rigor refinements that are general enough to belong in the math paper: typed self-update transitions versus channel-licensed maps, adaptive-audit trust risk beyond no self-sole-certification, and an abstract complexity/scalability ledger.

Definition 38.1 (Typed self-update transition)**.** *A Level 3 candidate is represented generically by a typed self-update transition*

$$\Phi_\nu = (\mathsf{Dom}_t^\nu, \mathsf{Cod}_{t+1}^\nu, \mathsf{Map}_\nu, \mathsf{TypeLic}_t(\nu), \mathsf{ResCert}_t(\nu)).$$

Here Dom_t^ν *and* Cod_{t+1}^ν *are typed audited domains,* Map_ν *is the successor-state transformation,* $\mathsf{TypeLic}_t(\nu)$ *states which mathematical channel or transition semantics are licensed, and* $\mathsf{ResCert}_t(\nu)$ *records direct residual certificates when no channel theorem is available. The notation* $\Phi_\nu(\rho_t)$ *denotes the successor state induced by the transition when* $\rho_t \in \mathsf{Dom}_t^\nu$*.*

Definition 38.2 (Channel-licensed transition)**.** *A typed self-update transition is channel-licensed on an audited family* $\mathcal{S}_t$ *when its type license proves at least one of the following:*

(a) a CPTP representation on the relevant density-state algebra;

(b) a stochastic representation on a diagonal classical subalgebra;

(c) an isometric or conservative-extension representation with an explicit recovery map;

(d) a direct residual certificate for every RCP quantity that would otherwise have been justified by a channel theorem.

If none of these licenses is supplied, the candidate is a raw program or architecture transformation and cannot be accepted as an RCP update.

Proposition 38.3 (Channel terminology does not license channel theorems)**.** *Calling a typed self-update transition a "channel" does not license data-processing, Petz recovery, stochastic-map, or isometry-based conclusions. Such conclusions are available only when the corresponding channel license in Definition 38.2 is present, or when the active residuals are directly certified.*

Proof. Data-processing and recovery theorems have hypotheses about the mathematical form of the map. A code rewrite, memory append, verifier update, or architecture mutation need not satisfy those hypotheses. Therefore the theorem can be applied only after the type license establishes the relevant map class or after the residuals are certified without appealing to that theorem. □

Definition 38.4 (Adaptive trust risk primitives). *Let* $\mathsf{Supp}_t(\Phi)$ *be the verifier support set used to certify a verifier-relevant update. Define four adaptive-audit risk quantities:*

$$\mathrm{CorrFail}_t(W_i, W_j) \in [0,1],$$

which bounds correlated failure between verifier nodes W_i, W_j*,*

$$\mathrm{AuditLeak}_t(\Phi) \in [0,1],$$

which bounds the risk that the candidate or proposal generator has adapted to hidden audit material,

$$\mathrm{CommonMode}_t(\mathsf{Supp}_t(\Phi)) \in [0,1],$$

which bounds shared blind spots across the support set, and

$$\mathrm{CollusionRisk}_t(\mathsf{Supp}_t(\Phi)) \in [0,1],$$

which bounds the risk that nominally separate verifier nodes coordinate or share incentives in a way that invalidates independence.

Definition 38.5 (Adaptive trust thresholds). *The adaptive trust thresholds are*

$$\gamma_t^{\mathrm{corr}}, \quad \gamma_t^{\mathrm{leak}}, \quad \gamma_t^{\mathrm{cm}}, \quad \gamma_t^{\mathrm{coll}}$$

with values in $[0,1]$*. The thresholds are part of the operational specification. Smaller thresholds demand stronger independence and anti-leakage evidence.*

Definition 38.6 (Adaptive-audit trust certificate). *An adaptive-audit trust certificate for a candidate* Φ *is*

$$\begin{aligned}\mathsf{AdvTrustCert}_t(\Phi) = (&\widehat{\mathrm{CorrFail}}_t, \widehat{\mathrm{AuditLeak}}_t, \widehat{\mathrm{CommonMode}}_t, \widehat{\mathrm{CollusionRisk}}_t,\\ &\Delta_t^{\mathrm{corr}}, \Delta_t^{\mathrm{leak}}, \Delta_t^{\mathrm{cm}}, \Delta_t^{\mathrm{coll}}, \delta_t^{\mathrm{advtrust}}, \mathsf{AdvAuditTrace}_t(\Phi)).\end{aligned}$$

The trace records verifier identities, data provenance, stress-test access policy, independence evidence, frozen-audit commitments, and any candidate-visible audit information.

Assumption 38.7 (Adaptive-audit one-sided soundness). *For every audited candidate and every pair of support nodes, on an event of probability at least* $1 - \delta_t^{\mathrm{advtrust}}$*,*

$$\mathrm{CorrFail}_t(W_i, W_j) \le \widehat{\mathrm{CorrFail}}_t(W_i, W_j) + \Delta_{t,ij}^{\mathrm{corr}},$$

$$\mathrm{AuditLeak}_t(\Phi) \le \widehat{\mathrm{AuditLeak}}_t(\Phi) + \Delta_t^{\mathrm{leak}},$$

$$\mathrm{CommonMode}_t(\mathsf{Supp}_t(\Phi)) \le \widehat{\mathrm{CommonMode}}_t(\mathsf{Supp}_t(\Phi)) + \Delta_t^{\mathrm{cm}},$$

$$\mathrm{CollusionRisk}_t(\mathsf{Supp}_t(\Phi)) \le \widehat{\mathrm{CollusionRisk}}_t(\mathsf{Supp}_t(\Phi)) + \Delta_t^{\mathrm{coll}}.$$

This is an empirical audit assumption. It is not implied by the existence of a trust graph.

Definition 38.8 (Adversarially strengthened trust gate). *The adversarially strengthened trust gate is*

$$\mathcal{A}_{\mathrm{RCP}}^{\text{trust-adv}}(t, \rho_t) \subseteq \mathcal{A}_{\mathrm{RCP}}^{\mathrm{trust}}(t, \rho_t)$$

consisting of candidates whose ordinary trust-graph certificate passes and whose adaptive-audit certificate satisfies

$$\widehat{\mathrm{CorrFail}}_t(W_i, W_j) + \Delta^{\mathrm{corr}}_{t,ij} \le \gamma^{\mathrm{corr}}_t \quad \forall i \neq j \in \mathsf{Supp}_t(\Phi),$$

$$\widehat{\mathrm{AuditLeak}}_t(\Phi) + \Delta^{\mathrm{leak}}_t \le \gamma^{\mathrm{leak}}_t,$$

$$\widehat{\mathrm{CommonMode}}_t(\mathsf{Supp}_t(\Phi)) + \Delta^{\mathrm{cm}}_t \le \gamma^{\mathrm{cm}}_t,$$

$$\widehat{\mathrm{CollusionRisk}}_t(\mathsf{Supp}_t(\Phi)) + \Delta^{\mathrm{coll}}_t \le \gamma^{\mathrm{coll}}_t.$$

Theorem 38.9 (Adaptive-audit trust soundness)**.** *Assume adaptive-audit one-sided soundness. If*

$$\Phi \in \mathcal{A}^{\text{trust-adv}}_{\mathrm{RCP}}(t, \rho_t),$$

then, on the adaptive-audit soundness event, the update is not solely self-certified under the ordinary trust graph and additionally satisfies

$$\mathrm{CorrFail}_t(W_i, W_j) \le \gamma^{\mathrm{corr}}_t, \quad \mathrm{AuditLeak}_t(\Phi) \le \gamma^{\mathrm{leak}}_t,$$

$$\mathrm{CommonMode}_t(\mathsf{Supp}_t(\Phi)) \le \gamma^{\mathrm{cm}}_t, \quad \mathrm{CollusionRisk}_t(\mathsf{Supp}_t(\Phi)) \le \gamma^{\mathrm{coll}}_t.$$

Proof. Membership in $\mathcal{A}^{\text{trust-adv}}_{\mathrm{RCP}}$ implies membership in $\mathcal{A}^{\mathrm{trust}}_{\mathrm{RCP}}$, so the no self-sole-certification theorem applies. The four adaptive-audit inequalities follow by combining the one-sided bounds of Assumption 38.7 with the four conservative threshold inequalities in Definition 38.8. □

Limitation 38.10 (Adaptive trust remains model-relative)**.** *This theorem bounds adaptive-audit risks only relative to the declared risk model, support set, thresholds, and one-sided audit assumptions. It does not prove that the risk model contains every possible collusion, leakage, or common-mode mechanism.*

Definition 38.11 (Abstract complexity ledger)**.** *For a candidate Φ, the abstract complexity ledger is a finite list*

$$\mathsf{CxLed}_t(\Phi) = \{(o, \widehat{\mathrm{Cx}}^{\mathrm{naive}}_{t,o}, \widehat{\mathrm{Cx}}^{\mathrm{fact}}_{t,o}, \widehat{\mathrm{Cx}}^{\mathrm{cert}}_{t,o}, \Delta^{\mathrm{Cx}}_{t,o}, \mathsf{Bottle}_{t,o}) : o \in \mathsf{Ops}_t(\Phi)\}.$$

The operation set may include state construction, update generation, certificate construction, recovery construction, search, trust auditing, projection realization, and residual estimation. The entries record naive cost, factorized or low-rank cost when available, certificate cost, uncertainty radius, and bottleneck status.

Definition 38.12 (Complexity-ledger admissibility)**.** *Let $\mathsf{Budget}^{\mathrm{Cx}}_t$ be the declared complexity budget. A candidate belongs to*

$$\mathcal{A}^{\mathrm{cx}}_{\mathrm{RCP}}(t, \rho_t)$$

when every active operation in $\mathsf{CxLed}_t(\Phi)$ is accounted for and

$$\sum_{o \in \mathsf{Ops}_t(\Phi)} \left(\widehat{\mathrm{Cx}}^{\mathrm{cert}}_{t,o} + \Delta^{\mathrm{Cx}}_{t,o}\right) \le \mathsf{Budget}^{\mathrm{Cx}}_t.$$

If a factorized or approximate cost is used, the approximation assumptions must be recorded in the ledger and any resulting residual error must be propagated into the relevant RCP certificate.

Proposition 38.13 (Complexity ledger licenses accounting, not tractability). *Membership in $\mathcal{A}^{\mathrm{cx}}_{\mathrm{RCP}}(t,\rho_t)$ proves that the declared finite list of active operations is costed inside the stated complexity budget. It does not prove that the algorithm is polynomial-time, scalable to frontier systems, or practically cheap.*

Proof. The definition of $\mathcal{A}^{\mathrm{cx}}_{\mathrm{RCP}}$ is a componentwise or aggregate budget inequality over declared operations. It does not impose an asymptotic functional form on the costs, nor does it prove that the operation list remains small as model size grows. Therefore it licenses only the stated accounting claim. □

Definition 38.14 (Batch-10 synchronized operational gate). *The Batch-10 synchronized operational gate is*

$$\mathcal{A}^{\mathrm{op10}}_{\mathrm{RCP}}(t,\rho_t;g_t^{\min}) = \mathcal{A}^{\mathrm{op}}_{\mathrm{RCP}}(t,\rho_t;g_t^{\min}) \cap \mathcal{A}^{\text{trust-adv}}_{\mathrm{RCP}}(t,\rho_t) \cap \mathcal{A}^{\mathrm{cx}}_{\mathrm{RCP}}(t,\rho_t),$$

with the convention that every accepted Φ is a typed self-update transition and is treated as a channel only when Definition 38.2 supplies the relevant license.

Theorem 38.15 (Batch-10 synchronization theorem). *Assume the hypotheses of the operational closure theorem, adaptive-audit one-sided soundness, and complexity-ledger admissibility. If*

$$\Phi_t \in \mathcal{A}^{\mathrm{op10}}_{\mathrm{RCP}}(t,\rho_t;g_t^{\min}),$$

then the operational closure theorem applies, no channel theorem is invoked without a channel license or direct residual certificate, verifier-changing updates satisfy the adaptive-audit risk bounds of Theorem 38.9, and the declared active operations satisfy the complexity-ledger accounting bound of Definition 38.12.

Proof. Membership in $\mathcal{A}^{\mathrm{op10}}_{\mathrm{RCP}}$ gives membership in the original operational gate, the adversarially strengthened trust gate, and the complexity-ledger gate. The original membership gives the operational closure theorem. The type-transition convention and Proposition 38.3 prevent unlicensed channel-theorem use. The strengthened trust membership gives Theorem 38.9. The complexity membership gives the ledger accounting condition. No additional conclusion is drawn about practical scalability or empirical trust validity beyond the stated assumptions. □

Limitation 38.16 (Batch 10 is synchronization, not a new engine theorem). *Batch 10 does not alter the Batch 9 direct-engine construction theorem. It adds publication-readiness and implementation-readiness conditions: adaptive trust must be audited, complexity must be accounted, and channel language must be licensed. If those conditions fail, the corresponding implementation claim is withdrawn, but the algebraic non-loss, tangent-cone, viability, and direct-engine implications remain valid as conditional mathematical statements.*

38.1 Operationally closed RCP gate

Definition 38.17 (Operationally closed RCP admissibility). *The operationally closed RCP gate is*

$$\begin{aligned}\mathcal{A}^{\mathrm{op}}_{\mathrm{RCP}}(t,\rho_t;g_t^{\min}) = {} & \mathcal{A}^{\mathrm{DG}}_{\mathrm{RCP}}(t,\rho_t;g_t^{\min}) \cap \mathcal{A}^{\mathrm{type}}_{\mathrm{RCP}}(t,\rho_t) \cap \mathcal{A}^{\mathrm{coup}}_{\mathrm{RCP}}(t,\rho_t) \\ & \cap \mathcal{A}^{\mathrm{cert}}_{\mathrm{RCP}}(t,\rho_t) \cap \mathcal{A}^{\mathrm{trust}}_{\mathrm{RCP}}(t,\rho_t) \cap \mathcal{A}^{\text{trust-adv}}_{\mathrm{RCP}}(t,\rho_t) \\ & \cap \mathcal{A}^{\mathrm{search}}_{\mathrm{RCP}}(t,\rho_t;B_t^{\mathrm{search}}) \cap \mathcal{A}^{\mathrm{cov}}_{\mathrm{RCP}}(t,\rho_t) \cap \mathcal{A}^{\mathrm{cal}}_{\mathrm{RCP}}(t,\rho_t) \\ & \cap \mathcal{A}^{\mathrm{est}}_{\mathrm{RCP}}(t,\rho_t) \cap \mathcal{A}^{\mathrm{cx}}_{\mathrm{RCP}}(t,\rho_t) \cap \Omega_t^{\mathrm{scale}}(\rho_t).\end{aligned}$$

Here $\Omega_t^{\mathrm{scale}}$ *denotes the set of candidates satisfying the resource-scaling, no-op-monitor, and certificate-inflation requirements of Definition 37.27 and Definition 37.28.*

Theorem 38.18 (Operational closure theorem)**.** *Assume the post-D–G RCP hypotheses and the operational assumptions stated in this section: type soundness, embedding transfer, semantic coupling, audited one-sided certificates, verifier trust graphs, adaptive-audit trust soundness, complexity-ledger admissibility, search-and-certification completeness when progress is claimed, resource-scaling soundness, coverage certification, capability calibration, and estimator subprotocol soundness. If*

$$\Phi_t \in \mathcal{A}_{\mathrm{RCP}}^{\mathrm{op}}(t, \rho_t; g_t^{\min}),$$

then the post-D–G preservation conclusions remain valid, and additionally:

(a) every invoked mathematical inequality is type-licensed or directly certified;

(b) raw representational transitions transfer to density residuals within the embedding-error budgets;

(c) density-level semantic validity transfers to externally audited semantic validity within coupling budgets;

(d) accepted residuals are nonpositive on the declared finite-horizon certificate-good event;

(e) verifier-changing updates do not rely on self-sole-certification;

(f) verifier-changing updates satisfy the declared adaptive-audit bounds for correlated failure, audit leakage, common-mode support risk, and collusion risk;

(g) active operations have declared complexity-ledger accounting;

(h) claimed beneficial progress is backed by budgeted certified search and no-op-normalized gain certificates;

(i) resource cost, no-op dominance, and certificate inflation are explicitly monitored;

(j) residual untracked-risk exposure is bounded by the declared coverage-gap budget;

(k) claimed coherence-capability coupling is calibrated within the declared error budget;

(l) every active residual is tied to an estimator subprotocol with cost, stress tests, and failure probability.

Proof. Membership in $\mathcal{A}_{\mathrm{RCP}}^{\mathrm{op}}$ is membership in the intersection of the post-D–G gate and every operational component gate. The post-D–G component gives the preservation conclusions of Theorem 36.3. The type component gives Proposition 37.4. The embedding component gives Proposition 37.9. The coupling component gives Theorem 37.12. The certificate component gives Theorem 37.18. The trust component gives Theorem 37.22, and the adaptive-trust component gives Theorem 38.9. The complexity component gives Proposition 38.13. The search component gives Theorem 37.26 when its nonemptiness and margin hypotheses are invoked. Resource and no-op claims follow from Theorem 37.29. Coverage follows from Proposition 37.32; calibration from Proposition 37.36; estimator completeness from Proposition 37.40. The finite-horizon probabilistic claims combine by union bounds over the declared failure probabilities. □

Interpretation 38.19. *This operational gate is the abstract form of what the RSI–RCLM architecture paper instantiated concretely. It does not replace the RCP theorem stack; it prevents the stack from hiding behind untyped density notation, oracle-like certificates, unchecked verifier self-reference, unbounded search cost, untracked risk coverage, or uncalibrated capability claims.*

39 Batch 11: Domain-Nonemptiness and Witness-Library Closure

This section implements Batch 11. Batch 9 proved a constructive direct-engine theorem only after the state is already inside the declared direct-engine domain $\mathcal{D}_t^{\mathrm{engine}}$. The remaining proof obligation was therefore not another gate and not another preservation theorem. The missing theorem layer is a domain-entry layer: it must give concrete, checkable sufficient conditions under which states enter the direct-engine domain and remain in a seed class from which later direct-engine construction can continue.

The central Batch 11 schematic implication is

$$\rho_t \in \mathcal{K}_t^{\mathrm{seed}} \Longrightarrow \rho_t \in \mathcal{D}_t^{\mathrm{engine}} \Longrightarrow \mathfrak{E}_t(\rho_t) \text{ constructs a certified update,}$$

with a persistence condition

$$\rho_t \in \mathcal{K}_t^{\mathrm{seed}}, \qquad \rho_{t+1} = \Phi_{\nu_t}(\rho_t) \Longrightarrow \rho_{t+1} \in \mathcal{K}_{t+1}^{\mathrm{seed}}.$$

This section makes that statement rigorous for bounded certified conservative-extension libraries and bounded finite primitive-word grammars.

Limitation 39.1 (Batch 11 is seed-domain closure, not arbitrary RSI)**.** *Batch 11 does not prove that arbitrary systems, arbitrary trained models, or arbitrary recursive self-modifiers enter $\mathcal{D}_t^{\mathrm{engine}}$. It proves that a declared seed class enters the direct-engine domain when a certified conservative-extension library, finite grammar coverage, certifier soundness/completeness, resource slack, and successor seed-persistence certificates are supplied. The result is therefore stronger than the Batch 9 domain assumption, but still conditional and domain-relative.*

39.1 Certified conservative-extension module libraries

Definition 39.2 (Certified conservative-extension module)**.** *Fix a time t, a state $\rho \in \mathcal{K}_t^{\mathrm{base}}$, a progress threshold $g_t^{\mathfrak{A}} \geq 0$, and positive margins*

$$m_t^{\mathrm{eng}}, \qquad m_t^{\mathfrak{A}}, \qquad m_t^{r}, \qquad m_t^{\mathrm{res}} > 0.$$

A certified conservative-extension module at (t, ρ) is a tuple

$$M = (\nu_M, \Phi_M, \mathcal{R}_M^{\mathrm{rec}}, \alpha_M, \mathsf{ModCert}_t(M; \rho)),$$

where $\nu_M \in \Omega_t^{\mathrm{NL}}(\rho)$, $\Phi_M = \Phi_{\nu_M}$, and $\mathsf{ModCert}_t(M; \rho)$ contains the following certified entries:

(i) a primitive-word representation of ν_M as a finite word in $\mathfrak{G}^{\mathrm{NL}}$, with all primitive type, pair-transport, recovery, trust, semantic, resource, and projection-realization certificates required by earlier batches;

(ii) an algebraic non-loss and recovery certificate

$$\mathcal{L}^{\mathcal{P}_t}_{\Phi_M} \le \epsilon^{\mathrm{alg}}_t, \qquad \mathrm{Rec}^{\mathcal{P}_t}_{\Phi_M}(\mathcal{R}^{\mathrm{rec}}_M) \le r_t - m^r_t;$$

(iii) strict engine-residual margins

$$q^{\mathrm{eng}}_{t,j}(\nu_M;\rho) \le -m^{\mathrm{eng}}_t \qquad \forall j \in J^{\mathrm{eng}}_t;$$

(iv) predecessor ability preservation

$$\Theta^{\mathfrak{A}}_t(\mathfrak{A}_t(\rho, R_t)) \subseteq \mathfrak{A}_{t+1}(\Phi_M(\rho), R_{t+1});$$

(v) strict certified new-ability margin

$$\Delta^{\mathfrak{A}}_t(\nu_M;\rho) \ge g^{\mathfrak{A}}_t + m^{\mathfrak{A}}_t;$$

(vi) successor continuation evidence, either in the form

$$\Phi_M(\rho) \in \mathcal{V}^{\mathrm{engine}}_{t+1}$$

for a direct Batch 9 application, or in the stronger seed-persistence form

$$\Phi_M(\rho) \in \mathcal{K}^{\mathrm{seed}}_{t+1}$$

when the Batch 11 persistence theorem is being invoked;

(vii) resource, adaptive-trust, complexity-ledger, and typed-transition certificates showing that the module is executable under the active Batch 10 operational obligations.

The certificate is state-relative: the same module template may be valid at one state and invalid at another.

Definition 39.3 (Certified conservative-extension module library)**.** *A certified conservative-extension module library at time t is a finite or explicitly enumerable family*

$$\mathcal{L}^{\mathrm{CE}}_t = \{M_{t,1}, \dots, M_{t,N_t}\}$$

of module templates together with a state-relative validity predicate

$$\mathsf{ValidMod}_t(M;\rho) \in \{0,1\}.$$

The valid library at ρ is

$$\mathcal{L}^{\mathrm{CE}}_t(\rho) = \{M \in \mathcal{L}^{\mathrm{CE}}_t : \mathsf{ValidMod}_t(M;\rho) = 1\}.$$

The library is strictly beneficial at ρ *when at least one*

$$M \in \mathcal{L}^{\mathrm{CE}}_t(\rho)$$

satisfies Definition 39.2 with positive strict margins.

Remark 39.4 (Why the library is finite or enumerable). *The purpose of $\mathcal{L}_t^{\mathrm{CE}}$ is to replace an unstructured assumption of beneficial-witness existence by a checkable construction path. A finite library gives a decidable nonemptiness check. An explicitly enumerable library gives a bounded nonemptiness check once a grammar depth, resource budget, or proof-search cutoff is fixed. Batch 11 does not claim completeness over all possible self-modifications.*

Theorem 39.5 (Certified extension library nonemptiness). *Assume $\rho \in \mathcal{K}_t^{\mathrm{base}}$. If the valid library $\mathcal{L}_t^{\mathrm{CE}}(\rho)$ contains a certified conservative-extension module M satisfying Definition 39.2 with successor continuation evidence*

$$\Phi_M(\rho) \in \mathcal{V}_{t+1}^{\mathrm{engine}},$$

then ν_M is a strict beneficial conservative-extension witness at ρ in the sense of Definition 12.5. Consequently, Assumption 12.6 holds at ρ.

Proof. Definition 12.5 requires five conditions. Since M is certified at (t, ρ), item (iii) of Definition 39.2 gives the strict engine-residual margin. Item (vi) gives successor engine viability. Item (iv) gives predecessor ability preservation. Item (v) gives strict new-ability margin above $g_t^{\mathfrak{A}}$. Item (ii) gives the recovery bound strictly below r_t. The primitive-word and type certificates in item (i) place ν_M inside $\Omega_t^{\mathrm{NL}}(\rho)$. Therefore every clause of Definition 12.5 holds. Since such a witness exists, the constructive beneficial-extension availability assumption is discharged at this state. □

Limitation 39.6 (Library nonemptiness is the substantive construction). *The theorem above is intentionally short because the work is in the module certificate. It does not prove that $\mathcal{L}_t^{\mathrm{CE}}(\rho)$ is nonempty. It states that once a concrete certified library contains a positive-margin module, the Batch 9 strict-witness premise is no longer an uninterpreted assumption.*

39.2 Finite primitive-word grammar and generator coverage

Definition 39.7 (Bounded non-lossy primitive-word grammar). *Let $\mathsf{Prim}_t^{\mathrm{NL}}(\rho)$ be the finite set of primitive instances at time t whose primitive certificates are valid at ρ. For $k \in \mathbb{N}$, define the bounded grammar*

$$\mathfrak{G}_{t,\le k}^{\mathrm{NL}}(\rho) = \{\, \nu = g_\ell \circ \cdots \circ g_1 : 0 \le \ell \le k, \\ g_i \in \mathsf{Prim}_t^{\mathrm{NL}}(\rho), \text{ and the word is composable and certificate-valid} \,\}.$$

The empty word $\ell = 0$ is the identity. A module M is represented in the grammar when its word ν_M belongs to $\mathfrak{G}_{t,\le k}^{\mathrm{NL}}(\rho)$.

Definition 39.8 (Exhaustive bounded enumerator). *An exhaustive bounded enumerator for $\mathfrak{G}_{t,\le k}^{\mathrm{NL}}(\rho)$ is a generator*

$$\mathsf{Enum}_{t,\le k}^{\mathrm{NL}}(\rho) \subseteq \Omega_t^{\mathrm{NL}}(\rho)$$

with the property that

$$\mathfrak{G}_{t,\le k}^{\mathrm{NL}}(\rho) \subseteq \mathsf{Enum}_{t,\le k}^{\mathrm{NL}}(\rho).$$

If the implementation uses canonicalization, approximate realization, pruning, or symbolic equivalence classes, the enumerator must supply a distortion certificate e_t^{enum} and the inclusion is read as approximate coverage within that distortion.

Theorem 39.9 (Generator coverage by exhaustive finite grammar). *Assume $\nu^\sharp \in \mathfrak{G}^{\mathrm{NL}}_{t,\le k}(\rho)$ is a strict beneficial conservative-extension witness at ρ. If*

$$\mathsf{Gen}^{\mathrm{NL}}_t(\rho) = \mathsf{Enum}^{\mathrm{NL}}_{t,\le k}(\rho),$$

then generator coverage with margin holds for $\nu^\sharp$ with exact generation error

$$e^{\mathrm{gen}}_t = 0.$$

If the enumerator is approximate with distortion e^{enum}_t, then generator coverage holds with

$$e^{\mathrm{gen}}_t = e^{\mathrm{enum}}_t$$

provided every residual and ability-score distortion induced by the approximation is bounded by e^{enum}_t on the audited domain.

Proof. In the exact case, Definition 39.8 gives

$$\nu^\sharp \in \mathsf{Enum}^{\mathrm{NL}}_{t,\le k}(\rho) = \mathsf{Gen}^{\mathrm{NL}}_t(\rho).$$

Taking $\widetilde{\nu} = \nu^\sharp$, the residuals, ability score, and successor-viability property are identical to those of the witness, so the generator-coverage inequalities of Assumption 12.7 hold with $e^{\mathrm{gen}}_t = 0$. In the approximate case, the distortion certificate is exactly the statement that the generated representative differs from the strict witness by at most e^{enum}_t in every residual and ability-score quantity used by Assumption 12.7. Substituting that bound gives the stated coverage result. □

Corollary 39.10 (Library-to-generator coverage). *Assume $\mathcal{L}^{\mathrm{CE}}_t(\rho)$ contains a certified module M whose word ν_M lies in $\mathfrak{G}^{\mathrm{NL}}_{t,\le k}(\rho)$. If $\mathsf{Gen}^{\mathrm{NL}}_t = \mathsf{Enum}^{\mathrm{NL}}_{t,\le k}$, then the generator covers the strict witness supplied by Theorem 39.5. Thus the Batch 9 generator-coverage premise is discharged for that bounded library class.*

Proof. Theorem 39.5 turns the certified module into a strict beneficial witness. The word-membership hypothesis places that witness in the bounded grammar. Theorem 39.9 gives generator coverage. □

39.3 Seed engine domains

Definition 39.11 (Seed engine domain). *For a finite horizon N, define the Batch 11 seed engine domains backward. Let*

$$\mathcal{K}^{\mathrm{seed},N}_N \subseteq \mathcal{K}^{\mathrm{base}}_N$$

be a declared terminal seed set. For $t = N-1, \dots, 0$, define $\mathcal{K}^{\mathrm{seed},N}_t$ as the set of states $\rho \in \mathcal{K}^{\mathrm{base}}_t$ for which all of the following hold:

(i) the Batch 8 transport, state/update-separation, and projection-realization objects needed by the candidate class are defined;

(ii) the Batch 10 typed-transition, adaptive-trust, and complexity-ledger obligations are defined;

(iii) $\mathcal{L}^{\mathrm{CE}}_t(\rho)$ contains a certified conservative-extension module M satisfying Definition 39.2;

(iv) the word ν_M belongs to a bounded grammar $\mathfrak{G}^{\mathrm{NL}}_{t,\le k_t}(\rho)$ covered by the selected generator;

(v) the engine certifier has one-sided soundness and completeness radii below the strict margins of M;

(vi) the successor satisfies the seed-persistence condition

$$\Phi_M(\rho) \in \mathcal{K}^{\mathrm{seed},N}_{t+1}.$$

When the horizon is clear, write $\mathcal{K}^{\mathrm{seed}}_t$. This set is defined by explicit library, grammar, certificate, resource, and persistence data. It is not defined by assuming $\rho \in \mathcal{D}^{\mathrm{engine}}_t$.

Theorem 39.12 (Seed-domain inclusion in the engine viability kernel)**.** *For every finite horizon N and every $0 \le t \le N$,*

$$\mathcal{K}^{\mathrm{seed},N}_t \subseteq \mathcal{V}^{\mathrm{engine},N}_t$$

provided each certified module used in Definition 39.11 satisfies the conservative-extension engine-move residuals with the declared progress threshold.

Proof. We prove the claim by backward induction. At $t = N$, the seed terminal set is a subset of $\mathcal{K}^{\mathrm{base}}_N = \mathcal{V}^{\mathrm{engine},N}_N$ by definition. Suppose the claim holds at $t+1$. Let $\rho \in \mathcal{K}^{\mathrm{seed},N}_t$. By Definition 39.11, there exists a certified module M with $\nu_M \in \Omega^{\mathrm{NL}}_t(\rho)$, conservative-extension engine residuals satisfied, ability preservation and progress certified, and

$$\Phi_M(\rho) \in \mathcal{K}^{\mathrm{seed},N}_{t+1}.$$

By the induction hypothesis, $\Phi_M(\rho) \in \mathcal{V}^{\mathrm{engine},N}_{t+1}$. Hence $\nu_M \in \mathcal{E}^{\mathrm{CE}}_t(\rho; g^{\mathfrak{A}}_t)$ with successor in $\mathcal{V}^{\mathrm{engine},N}_{t+1}$. Definition 11.7 therefore gives $\rho \in \mathcal{V}^{\mathrm{engine},N}_t$. Induction completes the proof. □

Interpretation 39.13. *This theorem is the first nonemptiness upgrade beyond Batch 5. Batch 5 said that if a state is already in the viability kernel, then a continuable engine move exists. Batch 11 gives a concrete sufficient condition for kernel membership: a persistent seed library with certified modules implies membership in the finite-horizon engine kernel.*

39.4 Direct-engine domain entry from seed domains

Definition 39.14 (Batch 11 margin ledger)**.** *For a seed-state module $M \in \mathcal{L}^{\mathrm{CE}}_t(\rho)$, define the Batch 11 margin ledger*

$$\mathsf{MargLed}_t(M;\rho) = (m^{\mathrm{eng}}_t, m^{\mathfrak{A}}_t, m^{r}_t, e^{\mathrm{gen}}_t, e^{\mathrm{cert}}_t, e^{\mathrm{enum}}_t, e^{\mathrm{proj}}_t, e^{\mathrm{tr}}_t),$$

where the terms record, respectively, engine-residual strictness, ability strictness, recovery slack, generation error, certificate error, enumeration distortion, projection-realization distortion, and cross-time transport distortion. The ledger is admissible when

$$e^{\mathrm{gen}}_t + e^{\mathrm{cert}}_t + e^{\mathrm{enum}}_t + e^{\mathrm{proj}}_t + e^{\mathrm{tr}}_t < \min\{m^{\mathrm{eng}}_t, m^{\mathfrak{A}}_t, m^{r}_t\}.$$

If a distortion term is not active, it is set to zero. Every term must be justified by a certificate or by exact algebraic construction.

Theorem 39.15 (Direct-engine domain entry from seed domain). *Fix a finite horizon N, and instantiate $\mathcal{V}_t^{\mathrm{engine}}$ in Definition 12.9 by $\mathcal{V}_t^{\mathrm{engine},N}$. Suppose*

$$\rho \in \mathcal{K}_t^{\mathrm{seed},N}.$$

Assume the generator is the exhaustive bounded enumerator for the active seed module word, the engine certifier satisfies Assumption 12.8 on the generated set, and the Batch 11 margin ledger is admissible. Then

$$\rho \in \mathcal{D}_t^{\mathrm{engine}}.$$

Consequently, on the engine certificate event, the Constructive Direct Non-Lossy RSI Engine Theorem applies to ρ.

Proof. We verify the clauses of Definition 12.9. Since $\rho \in \mathcal{K}_t^{\mathrm{seed},N}$, Definition 39.11 gives $\rho \in \mathcal{K}_t^{\mathrm{base}}$ and the required Batch 8 and Batch 10 objects. Theorem 39.12 gives $\rho \in \mathcal{V}_t^{\mathrm{engine},N}$. The valid library contains a certified module M, so Theorem 39.5 gives constructive beneficial-extension availability. Since ν_M lies in the bounded grammar and the generator is exhaustive, Corollary 39.10 gives generator coverage. The certifier-soundness/completeness event is assumed. Finally, the admissible margin ledger implies the strict inequality required by Definition 12.9, with all active generation, certificate, enumeration, projection, and transport errors included. Therefore every defining condition of $\mathcal{D}_t^{\mathrm{engine}}$ holds. □

Corollary 39.16 (Direct-engine construction from seed domain). *Under the hypotheses of Theorem 39.15 and on the engine certificate event, $\mathfrak{E}_t(\rho)$ constructs an output packet*

$$(\Phi_{\nu_t}, \mathcal{R}_{\nu_t}^{\mathrm{rec}}, \mathsf{Cert}_t^{\mathrm{eng}}(\nu_t), \rho_{t+1})$$

with

$$\Phi_{\nu_t} \in \mathcal{A}_t^{\text{NL-RSI}}(\rho; g_t^{\mathfrak{A}}), \qquad \rho_{t+1} \in \mathcal{V}_{t+1}^{\mathrm{engine},N},$$

recovery bound $\mathrm{Rec}_{\Phi_{\nu_t}}^{\mathcal{P}_t}(\mathcal{R}_{\nu_t}^{\mathrm{rec}}) \le r_t$, predecessor ability preservation, and strict certified ability progress.

Proof. Theorem 39.15 gives $\rho \in \mathcal{D}_t^{\mathrm{engine}}$. Apply Theorem 12.11. □

39.5 Seed-domain persistence and finite direct-engine trajectories

Definition 39.17 (Seed-persistence certificate). *For a certified module $M \in \mathcal{L}_t^{\mathrm{CE}}(\rho)$, a seed-persistence certificate is a packet*

$$\mathsf{SeedPersist}_t(M;\rho) = (\mathsf{SuccLib}_t, \mathsf{SuccBase}_t, \mathsf{SuccType}_t, \mathsf{SuccRes}_t, \mathsf{SuccTrust}_t, \mathsf{SuccCert}_t)$$

that proves the successor state has the objects required by the next seed domain:

$$\Phi_M(\rho) \in \mathcal{K}_{t+1}^{\mathrm{seed},N}.$$

The certificate must identify the transported or newly appended successor module library, successor primitive-word grammar, successor verifier/certifier state, resource ledger, trust graph, semantic maps, and any successor transport or projection-realization obligations.

Definition 39.18 (Seed-equivalence and persistence-transfer certificate)**.** *Let $M \in \mathcal{L}_t^{\mathrm{CE}}(\rho)$ be a certified module witnessing $\rho \in \mathcal{K}_t^{\mathrm{seed},N}$, and let $\nu \in \mathsf{Pass}_t^{\mathrm{eng}}(\rho)$ be the candidate selected and realized by $\mathfrak{E}_t$. A seed-equivalence or persistence-transfer certificate is a packet*

$$\mathsf{SeedEq}_t(M,\nu;\rho) = (\Delta_t^{\mathrm{res}}, \Delta_t^{\mathfrak{A}}, \Delta_t^{r}, \Delta_t^{\mathrm{type}}, \Delta_t^{\mathrm{resrc}}, \Delta_t^{\mathrm{trust}}, \Delta_t^{\mathrm{tr}}, \Delta_t^{\mathrm{proj}}, \mathsf{SuccSeedCert}_t)$$

such that:

(i) every engine residual of ν differs from the corresponding residual of M by at most the certified residual-transfer radius Δ_t^{res};

(ii) every ability-preservation and new-ability certificate used by M either transfers to ν or is independently certified for ν, with error bounded by $\Delta_t^{\mathfrak{A}}$;

(iii) the recovery, type, resource, adaptive-trust, transport, and projection-realization distortions between $\Phi_M(\rho)$ and $\Phi_\nu(\rho)$ are bounded by the listed radii;

(iv) all listed radii are included in the Batch 11 margin ledger $\mathsf{MargLed}_t(M;\rho)$, and the margin-ledger inequality remains strict after those radii are added;

(v) the successor seed certificate proves

$$\Phi_\nu(\rho) \in \mathcal{K}_{t+1}^{\mathrm{seed},N}.$$

If $\nu = \nu_M$ and the realization is exact, the certificate may be the identity certificate with all transfer radii equal to zero.

Theorem 39.19 (Successor seed-domain invariance)**.** *Let $\rho \in \mathcal{K}_t^{\mathrm{seed},N}$, and let $M \in \mathcal{L}_t^{\mathrm{CE}}(\rho)$ be the certified module witnessing seed membership. If $\mathsf{SeedPersist}_t(M;\rho)$ is valid, then*

$$\rho_{t+1} = \Phi_M(\rho) \in \mathcal{K}_{t+1}^{\mathrm{seed},N}.$$

If, instead of selecting exactly ν_M, the engine selects $\nu_t \in \mathsf{Pass}_t^{\mathrm{eng}}(\rho)$ and a valid $\mathsf{SeedEq}_t(M,\nu_t;\rho)$ certificate is supplied, then

$$\Phi_{\nu_t}(\rho) \in \mathcal{K}_{t+1}^{\mathrm{seed},N}.$$

Proof. The first claim is exactly the conclusion of the seed-persistence certificate in Definition 39.17. For the selected-candidate case, Definition 39.18 requires all residual, ability, recovery, type, resource, adaptive-trust, transport, and projection-realization distortions between the module successor and the selected realized successor to be included in the Batch 11 margin ledger, with strict margins preserved. It also requires the successor seed certificate

$$\Phi_{\nu_t}(\rho) \in \mathcal{K}_{t+1}^{\mathrm{seed},N}.$$

Therefore the realized selected successor satisfies the defining clauses of Definition 39.11 at time $t+1$. □

Corollary 39.20 (Finite direct-engine trajectory from seed domain)**.** *Fix a horizon N. Suppose $\rho_0 \in \mathcal{K}_0^{\mathrm{seed},N}$, the Batch 11 domain-entry hypotheses hold at every step, the engine certificate events hold at every step, and at each step either the engine selects the certified seed module with a valid*

$\mathsf{SeedPersist}_t$ *certificate or the selected update carries a valid* SeedEq_t *persistence-transfer certificate relative to that module. Then repeated application of* $\mathfrak{E}_t$ *constructs a finite trajectory*

$$\rho_0 \xrightarrow{\nu_0} \rho_1 \xrightarrow{\nu_1} \cdots \xrightarrow{\nu_{N-1}} \rho_N$$

where every step is a certified non-lossy, recoverable, ability-preserving, strictly ability-expanding conservative-extension update, and

$$\rho_t \in \mathcal{K}_t^{\mathrm{seed},N} \subseteq \mathcal{D}_t^{\mathrm{engine}} \subseteq \mathcal{V}_t^{\mathrm{engine},N} \qquad (0 \le t < N).$$

Proof. The base state lies in $\mathcal{K}_0^{\mathrm{seed},N}$ by hypothesis. Theorem 39.15 gives $\rho_0 \in \mathcal{D}_0^{\mathrm{engine}}$. Corollary 39.16 gives the first constructed update and its certificates. If the selected update is exactly the certified seed module, the $\mathsf{SeedPersist}_0$ certificate and Theorem 39.19 give $\rho_1 \in \mathcal{K}_1^{\mathrm{seed},N}$. If the selected update differs from the seed module, the valid SeedEq_0 certificate gives the same conclusion by the selected-candidate clause of Theorem 39.19. Repeat the argument inductively through $t = N-1$. The stepwise non-loss, recovery, ability preservation, strict progress, and viability conclusions are inherited from the direct-engine theorem at each step. □

39.6 Restricted modular certificate-radius scalability

Definition 39.21 (Factored conservative-extension certificate class)**.** *A conservative-extension word*

$$\nu = g_n \circ \cdots \circ g_1$$

belongs to the factored certificate class $\mathfrak{G}_{t,\le k}^{\mathrm{fact}}$ *when its primitive certificates decompose into local certificates with interaction residuals* $\iota_{i\ell} \ge 0$ *satisfying*

$$\sum_{i=1}^{n} \Delta_{t,i}^{\mathrm{cert}} + \sum_{1 \le i < \ell \le n} \iota_{i\ell} \le \Delta_t^{\mathrm{fact}}(\nu),$$

and the total resource and complexity cost is accounted in $\mathsf{CxLed}_t(\nu)$*. The class is restricted to words for which these local and interaction radii are actually certified.*

Theorem 39.22 (Restricted modular certificate-radius scalability)**.** *Assume* $\nu \in \mathfrak{G}_{t,\le k}^{\mathrm{fact}}$ *and the factorized certificate radii satisfy*

$$\Delta_t^{\mathrm{fact}}(\nu) < \min\{m_t^{\mathrm{eng}}, m_t^{\mathfrak{A}}, m_t^{r}\}.$$

Then the certificate error contribution of ν *is small enough to preserve the strict margins required by the Batch 11 margin ledger, provided generation, enumeration, projection, and transport errors are also included and the full margin-ledger inequality remains strict.*

Proof. The definition of the factored certificate class gives an upper bound $\Delta_t^{\mathrm{fact}}(\nu)$ on the aggregate certificate radius, including local primitive radii and certified interaction residuals. If this aggregate radius is smaller than the minimum strict margin, then subtracting the certificate uncertainty from the strict witness inequalities leaves a positive margin. Adding the remaining active errors gives exactly the admissible margin-ledger condition of Definition 39.14. Therefore the certificate radius does not by itself destroy the Batch 11 domain-entry proof. □

Limitation 39.23 (Scalability claim is deliberately restricted)**.** *The theorem above is not a frontier-scale tractability theorem. It applies only to factored conservative-extension words with certified local radii, certified interaction terms, and complexity-ledger accounting. It does not prove that arbitrary high-dimensional entropy, mutual-information, semantic, or verifier certificates can be estimated cheaply.*

39.7 Open-ended ability towers

Definition 39.24 (Open-ended certified ability tower)**.** *An open-ended certified ability tower is a sequence of audited ability universes and transports*

$$\mathsf{Ab}_0 \xrightarrow{\Theta_0^{\mathfrak{A}}} \mathsf{Ab}_1 \xrightarrow{\Theta_1^{\mathfrak{A}}} \mathsf{Ab}_2 \xrightarrow{\Theta_2^{\mathfrak{A}}} \cdots$$

with certified novelty sets

$$\mathsf{New}_{t+1} = \mathsf{Ab}_{t+1} \setminus \Theta_t^{\mathfrak{A}}(\mathsf{Ab}_t).$$

The tower is renewable on a seed trajectory when, for every t, the library $\mathcal{L}_t^{\mathrm{CE}}(\rho_t)$ contains a module whose new-ability certificate lies in New_{t+1} and whose resource-renewal certificate keeps ρ_{t+1} inside the next seed domain.

Corollary 39.25 (Open-ended progress relative to a renewable ability tower)**.** *Assume an infinite seed-domain trajectory exists and the certified ability tower is renewable on that trajectory. If each step has positive novelty margin*

$$\exists a_{t+1} \in \mathfrak{A}_{t+1}(\rho_{t+1}, R_{t+1}) \setminus \Theta_t^{\mathfrak{A}}(\mathfrak{A}_t(\rho_t, R_t)),$$

then the trajectory exhibits strict certified ability progress at every step relative to the expanding tower.

Proof. At each time t, renewability supplies a certified module with a new ability outside the transported predecessor ability universe. Ability preservation plus the existence of a_{t+1} gives proper ability-set expansion by Definition 8.7. Applying this argument at every step yields strict progress relative to the tower. □

Limitation 39.26 (Open-ended progress remains tower-relative)**.** *The corollary does not prove that nature, mathematics, or a real architecture supplies an infinite sequence of useful new abilities. It states that if an expanding audited ability tower and renewable seed-domain library are supplied, then the earlier fixed-scale limitation is avoided by changing the progress domain over time.*

39.8 Batch 11 summary

Theorem 39.27 (Batch 11 seed-to-direct-engine closure)**.** *For a declared finite horizon N, suppose $\rho_t \in \mathcal{K}_t^{\mathrm{seed},N}$, the active certified conservative-extension module library is nonempty, the selected module word lies in a bounded grammar covered by the generator, the certifier is sound and complete on the generated set, and the Batch 11 margin ledger is admissible. Then*

$$\rho_t \in \mathcal{D}_t^{\mathrm{engine}}$$

and the Batch 9 direct-engine theorem constructs a certified beneficial non-lossy successor. If the engine selects the certified seed module with a valid seed-persistence certificate, or selects another passing candidate with a valid seed-equivalence persistence-transfer certificate, then

$$\rho_{t+1} \in \mathcal{K}_{t+1}^{\mathrm{seed},N},$$

so the same reasoning can be iterated over the declared horizon.

Proof. The first conclusion is Theorem 39.15. The construction conclusion is Corollary 39.16. The exact-module persistence conclusion is the first clause of Theorem 39.19; the selected-candidate persistence conclusion is its SeedEq_t clause. Iteration over a finite horizon is Corollary 39.20. □

Interpretation 39.28 (What Batch 11 changes)**.** *Batch 9 said:*

$$\rho_t \in \mathcal{D}_t^{\mathrm{engine}} \Longrightarrow \mathfrak{E}_t(\rho_t) \text{ constructs a certified update.}$$

Batch 11 adds a sufficient domain-entry layer:

$$\rho_t \in \mathcal{K}_t^{\mathrm{seed},N} \Longrightarrow \rho_t \in \mathcal{D}_t^{\mathrm{engine}} \Longrightarrow \mathfrak{E}_t(\rho_t) \text{ constructs a certified update.}$$

It therefore converts the direct-engine domain from a declared sufficient-condition set into a theorem-derived domain for any system that supplies the certified seed library, bounded grammar, certifier, resource, trust, and persistence data. It still does not prove that arbitrary systems supply those data.

40 Batch 12 / RCP-II Phase 1: Robust Successor-Verification Seed Domains

This section begins the second major theorem family. Batch 11 gives recursive construction persistence: a state in the certified seed domain enters the direct-engine domain, the engine constructs a certified non-lossy update, and the successor remains in the next seed domain when seed-persistence or seed-equivalence certificates are valid. The new goal is stronger and different:

Batch 11 gives recursive construction persistence; Section 2 must give recursive verification persistence.

Equivalently, the next target is to strengthen the Batch 11 seed-domain persistence theorem into a successor-verification-domain invariance theorem under uncertainty.

The guiding hierarchy for this first RCP-II phase is

$$\boxed{\mathcal{K}_t^{\mathrm{SV},N} \subseteq \mathcal{K}_t^{\mathrm{seed},N} \subseteq \mathcal{D}_t^{\mathrm{engine}} \subseteq \mathcal{V}_t^{\mathrm{engine},N} \subseteq \mathcal{K}_t^{\mathrm{base}} \subseteq \mathcal{K}_{\mathrm{RCP}}^{\mathrm{state}}(t).}$$

Here $\mathcal{K}_t^{\mathrm{SV},N}$ is the *successor-verification seed domain*. It is not another name for the direct-engine domain. It is the stronger set of states from which the system can construct a successor and certify that the successor preserves the verifier schema, uncertainty bounds, semantic/goal-identity transport, resource/proof budget, and future ability to verify later successors.

Limitation 40.1 (Batch 12 Phase 1 is certificate-reflective, not universal reflection)**.** *Batch 12 Phase 1 does not solve full Löbian reflection, arbitrary Vingean reflection, universal successor trust, arbitrary world-model correctness, or unbounded practical tractability. It proves a restricted certificate-based successor-verification invariance theorem for the declared Batch 11 seed/library domain. Stronger reflective-logic theorems would require additional proof-theoretic assumptions not introduced here.*

40.1 Section 2 successor-verification schema

Definition 40.2 (Successor-verification schema). *A successor-verification schema at time t is a certificate-level object*

$$\mathsf{VerSch}_t^{\mathrm{SV}} = (\mathsf{Lang}_t^{\mathrm{SV}}, \mathsf{Pred}_t^{\mathrm{SV}}, \mathsf{Chk}_t^{\mathrm{SV}}, Q_t^{\mathrm{SV}}, \Theta_t^{\mathrm{SV}}, \preceq_t^{\mathrm{SV}}, \mathsf{Fail}_t^{\mathrm{SV}}, \mathsf{Budget}_t^{\mathrm{SV}}),$$

where:

(i) $\mathsf{Lang}_t^{\mathrm{SV}}$ *is the formal or typed certificate language in which successor-verification claims are expressed;*

(ii) $\mathsf{Pred}_t^{\mathrm{SV}}$ *is the finite audited family of predicates whose preservation constitutes successor-verification persistence, including verifier validity, seed-domain validity, semantic/goal identity, trust/reference validity, and resource/proof-budget validity;*

(iii) $\mathsf{Chk}_t^{\mathrm{SV}}$ *is the checker or verifier family used to evaluate those predicates;*

(iv) $Q_t^{\mathrm{SV}}(\nu;\rho) = (q_{t,\ell}^{\mathrm{SV}}(\nu;\rho))_{\ell \in J_t^{\mathrm{SV}}}$ *is a finite residual vector, decomposed explicitly in Batch 12A, with sign convention*

$$q_{t,\ell}^{\mathrm{SV}}(\nu;\rho) \le 0 \quad \Longrightarrow \quad \text{the } \ell\text{th successor-verification obligation is certified for } (\rho,\nu);$$

the explicit Batch 12A expansion $Q_t^{\mathrm{SV},A}$ separates seed linkage, candidate linkage, verifier-schema transport, uncertainty, goal identity, anti-circular trust, certificate budget, successor persistence, world-model distortion, proof-search validity, soundness, and optional progress residuals;

(v) Θ_t^{SV} *transports successor-verification predicates, certificate languages, residual names, verifier objects, and budgets from time t to time $t+1$;*

(vi) $\preceq_t^{\mathrm{SV}}$ *is a declared preorder comparing verifier schemas after transport, so that*

$$\Theta_t^{\mathrm{SV}}(\mathsf{VerSch}_t^{\mathrm{SV}}) \preceq_t^{\mathrm{SV}} \mathsf{VerSch}_{t+1}^{\mathrm{SV}}$$

means that the successor has at least the transported predecessor verification schema, up to the declared distortions;

(vii) $\mathsf{Fail}_t^{\mathrm{SV}}$ *is the event that the successor-verification certificate is unsound on the audited domain;*

(viii) $\mathsf{Budget}_t^{\mathrm{SV}}$ *records the proof-search, checking, estimator, resource, and complexity budgets needed by the successor-verification procedure.*

The schema is a declared typed object. It is not automatically sound merely because it is written in mathematical notation.

Definition 40.3 (Successor-verification uncertainty envelope). *A successor-verification uncertainty envelope is a family*

$$\mathcal{P}_t^{\mathrm{SV}}$$

of admissible probability, model, estimator, physical-distortion, and certificate-failure descriptions under which the successor-verification certificate is to be interpreted. A certificate soundness level $\alpha_t^{\mathrm{SV}} \in [0,1]$ is valid only when the paper or implementation supplies a proof or audit certificate that

$$\sup_{P \in \mathcal{P}_t^{\mathrm{SV}}} P\big((\mathsf{SVSound}_t)^c\big) \le \alpha_t^{\mathrm{SV}}.$$

This is an uncertainty-bound assumption/certificate. It does not assert that $\mathcal{P}_t^{\mathrm{SV}}$ contains the true environment, unless a separate modeling-validity certificate is supplied.

Definition 40.4 (Semantic/goal-identity transport certificate)**.** *A semantic/goal-identity transport certificate at time t is a tuple*

$$\mathsf{GoalId}_t = (U_t, U_{t+1}, \Theta_t^U, D_{\mathrm{sem}}^U, \delta_t^U, \mathsf{GoalCert}_t),$$

where U_t and U_{t+1} are the declared objective, utility, value, or goal-reference objects at times t and $t+1$, Θ_t^U transports the predecessor goal object to the successor goal space, D_{sem}^U is the declared semantic/goal-distance functional, and $\mathsf{GoalCert}_t$ certifies

$$D_{\mathrm{sem}}^U(U_{t+1}, \Theta_t^U U_t) \le \delta_t^U.$$

A cumulative goal-identity claim over an infinite trajectory additionally requires

$$\sum_{t=0}^{\infty} \delta_t^U < \infty.$$

Without $\mathsf{GoalCert}_t$, no theorem in this section identifies successor optimization with preservation of the predecessor's intended target.

Definition 40.5 (Successor-verification progress functional)**.** *A successor-verification progress functional is a scalar diagnostic*

$$\Psi_t^{\mathrm{SV}} : \mathcal{K}_t^{\mathrm{seed},N} \to \mathbb{R}$$

together with a transport Θ_t^{Ψ} and margins $\epsilon_t^{\mathrm{SV}}, \eta_t^{\mathrm{SV}} \ge 0$. The robust monotonicity certificate for a candidate ν at ρ is the statement

$$\inf_{P \in \mathcal{P}_t^{\mathrm{SV}}} \mathbb{E}_P \left[\Psi_{t+1}^{\mathrm{SV}}(\Phi_\nu(\rho)) - \Theta_t^{\Psi} \Psi_t^{\mathrm{SV}}(\rho) \right] \ge \epsilon_t^{\mathrm{SV}} - \eta_t^{\mathrm{SV}}.$$

The functional may measure successor-verification capacity, verifier-schema strength, certified proof budget, uncertainty robustness, or a declared combination of those quantities. It is not a proxy for real-world success unless the relevant calibration certificate is supplied.

40.2 The new Phase-1 object: Successor-Verification Seed Domain

Definition 40.6 (Successor-verification certificate packet)**.** *For a state $\rho \in \mathcal{K}_t^{\mathrm{seed},N}$ and a candidate ν, a successor-verification certificate packet is a tuple*

$$\begin{aligned}\mathsf{SVPkg}_t(\rho, \nu) = (&\mathsf{SeedPkg}_t, \mathsf{VerSch}_t^{\mathrm{SV}}, \mathsf{SuccCert}_t^{\mathrm{SV}}, \mathcal{P}_t^{\mathrm{SV}}, \\ &\mathsf{GoalId}_t, \mathsf{TrustRef}_t^{\mathrm{SV}}, \mathsf{CertBudget}_t^{\mathrm{SV}}, \mathsf{SVPersist}_t).\end{aligned}$$

with the following obligations:

(i) $\mathsf{SeedPkg}_t$ *is the Batch 11 seed package witnessing $\rho \in \mathcal{K}_t^{\mathrm{seed},N}$, including the certified conservative-extension library, bounded grammar, margin ledger, and seed-persistence or seed-equivalence data;*

(ii) $\mathsf{VerSch}_t^{\mathrm{SV}}$ *is a valid successor-verification schema in the sense of Definition 40.2;*

(iii) $\mathsf{SuccCert}_t^{\mathrm{SV}}$ *proves that* ν *satisfies the successor-verification residuals:*

$$Q_t^{\mathrm{SV}}(\nu;\rho) \le 0.$$

When the explicit Batch 12A residual expansion is active, this means

$$Q_t^{\mathrm{SV},A}(\nu;\rho) \le 0$$

componentwise, with each component attached to an audited one-sided certificate;

(iv) $\mathcal{P}_t^{\mathrm{SV}}$ *and* α_t^{SV} *supply the uncertainty and certificate-failure bound of Definition 40.3;*

(v) GoalId_t *supplies the semantic/goal-identity transport bound of Definition 40.4;*

(vi) $\mathsf{TrustRef}_t^{\mathrm{SV}}$ *supplies anti-circular trust support, including no-self-sole-certification and any adaptive-audit/common-mode bounds needed when the successor changes the verifier;*

(vii) $\mathsf{CertBudget}_t^{\mathrm{SV}}$ *proves that the successor-verification check, proof search, estimator calls, and transport checks fit inside the declared resource and complexity ledgers;*

(viii) $\mathsf{SVPersist}_t$ *proves verifier-schema persistence:*

$$\Theta_t^{\mathrm{SV}}(\mathsf{VerSch}_t^{\mathrm{SV}}) \preceq_t^{\mathrm{SV}} \mathsf{VerSch}_{t+1}^{\mathrm{SV}},$$

and proves successor membership

$$\Phi_\nu(\rho) \in \mathcal{K}_{t+1}^{\mathrm{SV},N}.$$

The packet is valid only when all components are type-compatible and all residual distortions are included in the active margin ledgers. If any component is absent, the successor-verification theorem cannot be invoked.

Definition 40.7 (Successor-verification seed domain)**.** *For a declared finite horizon* N*, define the terminal successor-verification seed set by choosing a declared subset*

$$\mathcal{K}_N^{\mathrm{SV},N} \subseteq \mathcal{K}_N^{\mathrm{seed},N}.$$

For $t = N-1,\dots,0$*, define* $\mathcal{K}_t^{\mathrm{SV},N}$ *to be the set of states* $\rho \in \mathcal{K}_t^{\mathrm{seed},N}$ *for which there exists at least one candidate* ν *produced by the Batch 11 seed/direct-engine construction path such that:*

(i) ν *is either the certified seed module candidate or an engine-selected passing candidate with a valid Batch 11 seed-equivalence/persistence-transfer certificate;*

(ii) $\mathsf{SVPkg}_t(\rho,\nu)$ *is valid;*

(iii) the robust successor-verification monotonicity certificate holds:

$$\inf_{P\in\mathcal{P}_t^{\mathrm{SV}}} \mathbb{E}_P\left[\Psi_{t+1}^{\mathrm{SV}}(\Phi_\nu(\rho)) - \Theta_t^{\Psi}\Psi_t^{\mathrm{SV}}(\rho)\right] \ge \epsilon_t^{\mathrm{SV}} - \eta_t^{\mathrm{SV}},$$

whenever such a Ψ^{SV}*-progress claim is made;*

(iv) the goal-identity drift bound holds:

$$D_{\mathrm{sem}}^U(U_{t+1}, \Theta_t^U U_t) \le \delta_t^U;$$

(v) the successor-verification soundness-failure probability is bounded:

$$\sup_{P \in \mathcal{P}_t^{\mathrm{SV}}} P\big((\mathsf{SVSound}_t)^c\big) \le \alpha_t^{\mathrm{SV}};$$

(vi) the successor satisfies

$$\Phi_\nu(\rho) \in \mathcal{K}_{t+1}^{\mathrm{SV},N}.$$

When the horizon is clear, write $\mathcal{K}_t^{\mathrm{SV}}$. This definition intentionally makes $\mathcal{K}_t^{\mathrm{SV},N}$ a stronger subset of the Batch 11 seed domain, not a new assumption that the engine domain has already been solved.

Proposition 40.8 (Successor-verification seed states are Batch 11 seed states)**.** *For every declared horizon N and time t,*

$$\mathcal{K}_t^{\mathrm{SV},N} \subseteq \mathcal{K}_t^{\mathrm{seed},N}.$$

Consequently, every state in $\mathcal{K}_t^{\mathrm{SV},N}$ also lies in $\mathcal{D}_t^{\mathrm{engine}}$ whenever the Batch 11 domain-entry hypotheses and certificate events hold.

Proof. The subset relation is the first clause of Definition 40.7. The final statement follows by applying Theorem 39.15 to the resulting Batch 11 seed membership, with the same Batch 11 generator, certifier, and margin-ledger hypotheses. □

40.3 Updated Phase-1 theorem skeleton

Theorem 40.9 (Robust successor-verification domain invariance)**.** *Fix a finite horizon N. Suppose*

$$\rho_t \in \mathcal{K}_t^{\mathrm{SV},N}.$$

Let ν_t be a candidate witnessing Definition 40.7, and suppose the Batch 11 direct-engine certificate event, the successor-verification soundness event $\mathsf{SVSound}_t$, the semantic/goal-identity certificate, and the successor-verification budget certificate all hold. Then the direct engine constructs

$$\mathfrak{E}_t(\rho_t) = (\nu_t, \Phi_{\nu_t}, \mathcal{R}_{\nu_t}^{\mathrm{rec}}, \mathsf{Cert}_t, \rho_{t+1})$$

with

$$\rho_{t+1} = \Phi_{\nu_t}(\rho_t) \in \mathcal{K}_{t+1}^{\mathrm{SV},N}.$$

Moreover, the step carries the certified robust successor-verification bounds

$$\inf_{P \in \mathcal{P}_t^{\mathrm{SV}}} \mathbb{E}_P\left[\Psi_{t+1}^{\mathrm{SV}}(\rho_{t+1}) - \Theta_t^{\Psi}\Psi_t^{\mathrm{SV}}(\rho_t)\right] \ge \epsilon_t^{\mathrm{SV}} - \eta_t^{\mathrm{SV}},$$

whenever Ψ^{SV} is included in the valid packet,

$$D_{\mathrm{sem}}^U(U_{t+1}, \Theta_t^U U_t) \le \delta_t^U,$$

and

$$\sup_{P \in \mathcal{P}_t^{\mathrm{SV}}} P\big((\mathsf{SVSound}_t)^c\big) \le \alpha_t^{\mathrm{SV}}.$$

Proof. Since $\rho_t \in \mathcal{K}_t^{\mathrm{SV},N}$, Proposition 40.8 gives $\rho_t \in \mathcal{K}_t^{\mathrm{seed},N}$. Under the Batch 11 direct-engine hypotheses, Theorem 39.15 gives $\rho_t \in \mathcal{D}_t^{\mathrm{engine}}$, and Corollary 39.16 gives the constructed engine output packet. Definition 40.7 requires that the selected or seed-equivalent candidate ν_t have a valid successor-verification packet $\mathsf{SVPkg}_t(\rho_t, \nu_t)$. Clause (viii) of Definition 40.6 and clause (vi) of Definition 40.7 give

$$\Phi_{\nu_t}(\rho_t) \in \mathcal{K}_{t+1}^{\mathrm{SV},N}.$$

The robust monotonicity, goal-identity, and failure-probability inequalities are precisely clauses (iii)–(v) of Definition 40.7, transported to $\rho_{t+1} = \Phi_{\nu_t}(\rho_t)$. No unlisted reflective-trust or uncertainty conclusion is inferred. □

Corollary 40.10 (Finite successor-verification trajectory)**.** *Fix a finite horizon N. Suppose $\rho_0 \in \mathcal{K}_0^{\mathrm{SV},N}$, and suppose the Batch 11 engine certificate events and Batch 12 successor-verification certificate events hold at every step. Then repeated application of the direct engine constructs a trajectory*

$$\rho_0 \xrightarrow{\nu_0} \rho_1 \xrightarrow{\nu_1} \cdots \xrightarrow{\nu_{N-1}} \rho_N$$

such that

$$\rho_t \in \mathcal{K}_t^{\mathrm{SV},N} \subseteq \mathcal{K}_t^{\mathrm{seed},N} \subseteq \mathcal{D}_t^{\mathrm{engine}} \qquad (0 \le t < N),$$

and every step preserves the declared successor-verification schema, semantic/goal-identity bound, uncertainty-envelope soundness bound, and Batch 11 non-lossy direct-engine construction certificates.

Proof. Apply Theorem 40.9 at $t = 0$ to obtain $\rho_1 \in \mathcal{K}_1^{\mathrm{SV},N}$. Repeat inductively through $t = N-1$. The displayed inclusions follow from Proposition 40.8 and Theorem 39.15 at each step. The certificate conclusions are carried by the hypotheses and by the step theorem. □

Corollary 40.11 (Summable drift and failure budgets on an infinite certified SV path)**.** *Suppose an infinite successor-verification path is supplied by an infinite-horizon analogue of Definition 40.7. If*

$$\sum_{t=0}^{\infty} \delta_t^U < \infty, \qquad \sum_{t=0}^{\infty} \alpha_t^{\mathrm{SV}} < \infty,$$

then semantic/goal drift is cumulatively bounded by the declared transport metric, and the total successor-verification certificate-failure budget is summable over the path. If the probability space satisfies the usual conditions for the Borel–Cantelli implication, then certificate-soundness failures occur only finitely often with probability one under every $P \in \bigcap_t \mathcal{P}_t^{\mathrm{SV}}$ for which the events are jointly defined.

Proof. The cumulative drift statement is the direct summation of the one-step bounds in Definition 40.4. The failure-budget statement is the direct summation of the one-step bounds in Definition 40.3. The final probabilistic sentence is exactly the first Borel–Cantelli lemma applied under the stated joint-event and probability-space conditions; it is not claimed without those conditions. □

Theorem 40.12 (Batch 12 Phase-1 successor-verification closure)**.** *For a declared finite horizon N, if*

$$\rho_t \in \mathcal{K}_t^{\mathrm{SV},N},$$

then, on the stated Batch 11 and Batch 12 certificate-validity events, the engine constructs a certified successor satisfying

$$\rho_{t+1} \in \mathcal{K}_{t+1}^{\mathrm{SV},N}.$$

Thus Batch 12 Phase 1 upgrades Batch 11 from recursive construction persistence to recursive verification persistence for the declared successor-verification seed domain.

Proof. This is Theorem 40.9 restated as the named closure theorem for the Phase-1 layer. The final interpretation follows because $\mathcal{K}_t^{\mathrm{SV},N} \subseteq \mathcal{K}_t^{\mathrm{seed},N}$, while $\mathcal{K}_t^{\mathrm{SV},N}$ additionally includes verifier-schema persistence, uncertainty-envelope soundness, goal-identity transport, trust-reference support, and successor-verification budget obligations. □

Interpretation 40.13 (Revised conclusion for the beginning of Section 2)**.** *The updated Batch-11-based conclusion is:*

> ***New goal: strengthen the Batch-11 seed-domain persistence theorem into a successor-verification-domain invariance theorem under uncertainty.***

More explicitly:

> ***Batch 11 gives recursive construction persistence; Section 2 must give recursive verification persistence.***

The Phase-1 theorem proves only the restricted certificate-based version of that statement:

$$\boxed{\rho_t \in \mathcal{K}_t^{\mathrm{SV},N} \Longrightarrow \mathfrak{E}_t(\rho_t) \text{ constructs } \rho_{t+1} \in \mathcal{K}_{t+1}^{\mathrm{SV},N}.}$$

41 Batch 12A: Explicit Successor-Verification Residuals, Transport Dynamics, and Soundness Composition

This section completes the first Batch-12 successor-verification layer. The previous section introduced the successor-verification seed domain

$$\mathcal{K}_t^{\mathrm{SV},N} \subseteq \mathcal{K}_t^{\mathrm{seed},N}$$

and proved a domain-invariance theorem relative to a valid successor-verification packet. The present section expands that packet into checkable residuals and transport/soundness obligations. It does not change the logical status of the theorem: Batch 12 remains a viability-domain invariance theorem, not a theorem that constructs successor-verification packets for arbitrary systems.

41.1 Explicit successor-verification residual vector

Definition 41.1 (Boolean-to-residual convention)**.** *If a successor-verification obligation is intrinsically Boolean, its residual is encoded by*

$$q(\mathsf{Claim}) = \begin{cases} 0, & \mathsf{Claim} \text{ is certified true on the audited domain,} \\ 1, & \text{otherwise.} \end{cases}$$

The gate convention remains $q \le 0$. Therefore a Boolean residual passes exactly when the associated certificate is present and valid. Scalar obligations use the usual signed-margin form $q = \widehat{q} + \Delta - m$, where $\widehat{q}$ is an estimate or checker output, Δ is an audited one-sided radius, and m is the admissible margin.

Definition 41.2 (Explicit Batch-12A successor-verification residual vector). *For a state ρ, candidate ν, and successor $\rho^+ = \Phi_\nu(\rho)$, the explicit successor-verification residual vector is*

$$Q_t^{\mathrm{SV},A}(\nu;\rho) = \big(q_t^{\mathrm{seed}}, q_t^{\mathrm{cand}}, q_t^{\mathrm{ver}}, q_t^{\text{ver-tr}}, q_t^{U}, q_t^{\mathcal{P}}, q_t^{\mathcal{P}\text{-}tr}, \\ q_t^{\mathrm{trust}}, q_t^{\mathrm{budget}}, q_t^{\mathrm{proof}}, q_t^{\mathrm{world}}, q_t^{\mathrm{persist}}, q_t^{\mathrm{sound}}, q_t^{\Psi}\big)(\nu;\rho).$$

The components have the following meanings:

(i) q_t^{seed} *certifies* $\rho \in \mathcal{K}_t^{\mathrm{seed},N}$ *with a valid Batch-11 seed package;*

(ii) q_t^{cand} *certifies that* ν *is either the seed-module candidate or an engine-selected candidate connected to it by a valid* SeedEq_t *persistence-transfer certificate;*

(iii) q_t^{ver} *certifies that* $\mathsf{VerSch}_t^{\mathrm{SV}}$ *is well-typed and its checker, predicate family, residual names, failure event, and budget are defined;*

(iv) $q_t^{\text{ver-tr}}$ *certifies componentwise verifier-schema transport in the sense of Definition 41.4;*

(v) q_t^{U} *is the goal-identity residual*

$$q_t^{U}(\nu;\rho) = D_{\mathrm{sem}}^{U}(U_{t+1}, \Theta_t^{U} U_t) - \delta_t^{U};$$

(vi) $q_t^{\mathcal{P}}$ *certifies that the uncertainty envelope* $\mathcal{P}_t^{\mathrm{SV}}$ *is nonempty and constructed from declared components;*

(vii) $q_t^{\mathcal{P}\text{-}tr}$ *certifies uncertainty-envelope transport or successor-envelope compatibility;*

(viii) q_t^{trust} *abbreviates the anti-circular reflective-trust residual vector of Definition 41.15;*

(ix) q_t^{budget} *certifies that checking, transport, audit, estimator, and proof-search costs fit inside* $\mathsf{Budget}_t^{\mathrm{SV}}$*;*

(x) q_t^{proof} *certifies that the supplied proof or certificate trace is checkable by* $\mathsf{Chk}_t^{\mathrm{SV}}$ *within the declared resource ledger;*

(xi) q_t^{world} *certifies the declared world-model or physical-distortion margin used by the successor-verification envelope;*

(xii) q_t^{persist} *certifies successor membership* $\rho^+ \in \mathcal{K}_{t+1}^{\mathrm{SV},N}$*;*

(xiii) q_t^{sound} *certifies the composed successor-verification soundness bound*

$$\sup_{P \in \mathcal{P}_t^{\mathrm{SV}}} P((\mathsf{SVSound}_t)^c) - \alpha_t^{\mathrm{SV}} \le 0;$$

(xiv) q_t^{Ψ} *certifies the robust monotonicity inequality for* Ψ_t^{SV} *when such a progress claim is made, and is set to zero when no* Ψ*-progress claim is asserted.*

The Batch-12 successor-verification residual passes when every named component is nonpositive:

$$Q_t^{\mathrm{SV},A}(\nu;\rho) \le 0.$$

In the rest of this manuscript, the residual Q_t^{SV} *in Definition 40.2 may be read as this explicit vector, or as a conservative subvector of it declared by the implementation.*

Proposition 41.3 (Explicit residual pass implies packet residual pass). *If $Q_t^{\mathrm{SV},A}(\nu;\rho) \le 0$ and the type/domain certificates for all residual components are valid, then the residual clause*

$$Q_t^{\mathrm{SV}}(\nu;\rho) \le 0$$

required by $\mathsf{SVPkg}_t(\rho,\nu)$ holds for the explicit Batch-12A residual interpretation.

Proof. Definition 41.2 declares $Q_t^{\mathrm{SV},A}$ to be the explicit componentwise interpretation of Q_t^{SV}. Componentwise nonpositivity therefore supplies every residual obligation required by the packet residual clause. The type/domain certificates are included to ensure that none of the scalar or Boolean residuals is being evaluated outside its stated domain. □

41.2 Verifier-schema transport dynamics

Definition 41.4 (Componentwise verifier-schema transport). *The Batch-12A verifier-schema transport map is the component tuple*

$$\Theta_t^{\mathrm{SV},A} = (\Theta_t^{\mathsf{Lang}}, \Theta_t^{\mathsf{Pred}}, \Theta_t^{\mathsf{Chk}}, \Theta_t^{Q,\mathrm{SV}}, \Theta_t^{\mathsf{Budget}}, \Theta_t^{\mathsf{Fail}}).$$

It acts on

$$\mathsf{VerSch}_t^{\mathrm{SV}} = (\mathsf{Lang}_t^{\mathrm{SV}}, \mathsf{Pred}_t^{\mathrm{SV}}, \mathsf{Chk}_t^{\mathrm{SV}}, Q_t^{\mathrm{SV}}, \Theta_t^{\mathrm{SV}}, \preceq_t^{\mathrm{SV}}, \mathsf{Fail}_t^{\mathrm{SV}}, \mathsf{Budget}_t^{\mathrm{SV}})$$

by transporting, respectively, certificate language, audited predicates, checker/verifier objects, residual names and signs, certificate budgets, and failure-event definitions. The transported schema is admissibly embedded in the successor schema when the following residuals are nonpositive:

$$q_t^{\mathsf{Lang}}, \quad q_t^{\mathsf{Pred}}, \quad q_t^{\mathsf{Chk}}, \quad q_t^{Q,\mathrm{SV}}, \quad q_t^{\mathsf{Budget}}, \quad q_t^{\mathsf{Fail}} \le 0.$$

This componentwise condition is written compactly as

$$\Theta_t^{\mathrm{SV},A}(\mathsf{VerSch}_t^{\mathrm{SV}}) \preceq_t^{\mathrm{SV}} \mathsf{VerSch}_{t+1}^{\mathrm{SV}}.$$

Lemma 41.5 (Componentwise transport implies verifier-schema persistence). *If every residual in Definition 41.4 is nonpositive, then verifier-schema persistence holds:*

$$\Theta_t^{\mathrm{SV}}(\mathsf{VerSch}_t^{\mathrm{SV}}) \preceq_t^{\mathrm{SV}} \mathsf{VerSch}_{t+1}^{\mathrm{SV}}.$$

Proof. The preorder $\preceq_t^{\mathrm{SV}}$ is defined by the componentwise transported-language, predicate, checker, residual, budget, and failure-event inclusion conditions. Nonpositivity of those residuals is exactly the condition that all required transported components are available in the successor schema with the declared distortions included. Hence the compact persistence relation follows by definition. □

41.3 Successor-verification certificate soundness theorem

Assumption 41.6 (One-sided soundness for explicit SV residuals). *Let $J_t^{\mathrm{SV},A}$ index the components of $Q_t^{\mathrm{SV},A}$. For every $\ell \in J_t^{\mathrm{SV},A}$, the certificate supplies an event $\mathcal{E}_{t,\ell}^{\mathrm{SV}}$ and a failure budget $\alpha_{t,\ell}^{\mathrm{SV}}$ such that, uniformly over $P \in \mathcal{P}_t^{\mathrm{SV}}$,*

$$P\big((\mathcal{E}_{t,\ell}^{\mathrm{SV}})^c\big) \le \alpha_{t,\ell}^{\mathrm{SV}},$$

and on $\mathcal{E}_{t,\ell}^{\mathrm{SV}}$, the certified residual bound implies the corresponding true residual obligation on the audited domain.

Definition 41.7 (Composed SV soundness event). *The composed successor-verification soundness event is*

$$\mathsf{SVSound}_t = \bigcap_{\ell \in J_t^{\mathrm{SV},A}} \mathcal{E}_{t,\ell}^{\mathrm{SV}}.$$

The composed failure budget is

$$\alpha_t^{\mathrm{SV},A} = \sum_{\ell \in J_t^{\mathrm{SV},A}} \alpha_{t,\ell}^{\mathrm{SV}}.$$

Theorem 41.8 (Successor-verification soundness by union composition). *Assume one-sided soundness for explicit SV residuals. Then*

$$\sup_{P \in \mathcal{P}_t^{\mathrm{SV}}} P((\mathsf{SVSound}_t)^c) \le \alpha_t^{\mathrm{SV},A}.$$

If the declared Batch-12 budget satisfies $\alpha_t^{\mathrm{SV},A} \le \alpha_t^{\mathrm{SV}}$, then the Batch-12 soundness residual $q_t^{\mathrm{sound}} \le 0$ holds.

Proof. By Definition 41.7,

$$(\mathsf{SVSound}_t)^c = \bigcup_{\ell \in J_t^{\mathrm{SV},A}} (\mathcal{E}_{t,\ell}^{\mathrm{SV}})^c.$$

The union bound gives, for every $P \in \mathcal{P}_t^{\mathrm{SV}}$,

$$P((\mathsf{SVSound}_t)^c) \le \sum_{\ell \in J_t^{\mathrm{SV},A}} P((\mathcal{E}_{t,\ell}^{\mathrm{SV}})^c) \le \sum_{\ell \in J_t^{\mathrm{SV},A}} \alpha_{t,\ell}^{\mathrm{SV}} = \alpha_t^{\mathrm{SV},A}.$$

Taking the supremum over P proves the first claim. If $\alpha_t^{\mathrm{SV},A} \le \alpha_t^{\mathrm{SV}}$, then

$$\sup_{P \in \mathcal{P}_t^{\mathrm{SV}}} P((\mathsf{SVSound}_t)^c) - \alpha_t^{\mathrm{SV}} \le 0,$$

which is exactly the soundness residual. □

41.4 Uncertainty-envelope construction and transport

Definition 41.9 (Constructed successor-verification uncertainty envelope). *A constructed successor-verification uncertainty envelope is an intersection of typed component envelopes*

$$\mathcal{P}_t^{\mathrm{SV}} = \mathcal{P}_t^{\mathrm{env}} \cap \mathcal{P}_t^{\mathrm{est}} \cap \mathcal{P}_t^{\mathrm{cert}} \cap \mathcal{P}_t^{\mathrm{phys}} \cap \mathcal{P}_t^{\mathrm{trust}}.$$

The components represent, respectively, environmental/model uncertainty, estimator uncertainty, certificate-failure uncertainty, physical or implementation distortion, and verifier/trust uncertainty. The construction is valid only if the intersection is nonempty and every component envelope is defined on compatible random variables, state spaces, and certificate events.

Definition 41.10 (Uncertainty-envelope transport). *An uncertainty-envelope transport is a map or relation $\Theta_t^{\mathcal{P}}$ carrying models in $\mathcal{P}_t^{\mathrm{SV}}$ to successor-time descriptions. It is exact when*

$$\Theta_t^{\mathcal{P}}(\mathcal{P}_t^{\mathrm{SV}}) \subseteq \mathcal{P}_{t+1}^{\mathrm{SV}}.$$

It is approximate with distortion $\omega_t^{\mathcal{P}} \geq 0$ when a declared envelope distance satisfies

$$d_{\mathcal{P}}(\Theta_t^{\mathcal{P}} \mathcal{P}_t^{\mathrm{SV}}, \mathcal{P}_{t+1}^{\mathrm{SV}}) \leq \omega_t^{\mathcal{P}}.$$

Any theorem using approximate uncertainty transport must include $\omega_t^{\mathcal{P}}$ in the successor-verification margin ledger.

Proposition 41.11 (Envelope construction licenses robust quantification only on the intersection)**.** *If $\mathcal{P}_t^{\mathrm{SV}}$ is constructed as in Definition 41.9, then every robust expectation, probability, and failure statement quantified over $\mathcal{P}_t^{\mathrm{SV}}$ is licensed only on the intersection of the declared component envelopes. No conclusion is made for models outside at least one component envelope.*

Proof. By definition, $\mathcal{P}_t^{\mathrm{SV}}$ is the intersection of the component envelopes. A statement of the form $\inf_{P \in \mathcal{P}_t^{\mathrm{SV}}}$ or $\sup_{P \in \mathcal{P}_t^{\mathrm{SV}}}$ ranges exactly over that intersection. It therefore cannot imply a bound outside the intersection unless an additional enlargement or containment certificate is supplied. □

41.5 Goal-identity composition theorem

Assumption 41.12 (Goal-distance triangle and transport distortion)**.** *The goal-distance functional D_{sem}^U obeys the triangle inequality on the transported goal objects used in the theorem. For each t, the next transport is approximately nonexpansive with distortion $\omega_{t+1}^U \geq 0$: for all audited goal objects x, y,*

$$D_{\mathrm{sem}}^U(\Theta_{t+1}^U x, \Theta_{t+1}^U y) \leq D_{\mathrm{sem}}^U(x, y) + \omega_{t+1}^U.$$

If Θ_{t+1}^U is exactly nonexpansive, then $\omega_{t+1}^U = 0$.

Definition 41.13 (Composed goal transport)**.** *For $0 \leq s < T$, define the composed goal transport*

$$\Theta_{s:T}^U = \Theta_{T-1}^U \circ \Theta_{T-2}^U \circ \cdots \circ \Theta_s^U,$$

with $\Theta_{T:T}^U = \mathrm{id}$.

Theorem 41.14 (Finite-horizon goal-identity composition)**.** *Assume the one-step goal-identity bounds*

$$D_{\mathrm{sem}}^U(U_{t+1}, \Theta_t^U U_t) \leq \delta_t^U$$

for $t = 0, \ldots, T-1$, and assume the goal-distance transport distortion condition. Then

$$D_{\mathrm{sem}}^U(U_T, \Theta_{0:T}^U U_0) \leq \sum_{t=0}^{T-1} \delta_t^U + \sum_{t=1}^{T-1} \omega_t^U.$$

Consequently, if both sums are bounded uniformly in T, the declared goal identity drift is bounded along the certified path.

Proof. The case $T = 1$ is the one-step bound. Suppose the result holds to time T. The triangle inequality gives

$$\begin{aligned} D_{\mathrm{sem}}^U(U_{T+1}, \Theta_{0:T+1}^U U_0) \leq\ & D_{\mathrm{sem}}^U(U_{T+1}, \Theta_T^U U_T) \\ & + D_{\mathrm{sem}}^U(\Theta_T^U U_T, \Theta_T^U \Theta_{0:T}^U U_0). \end{aligned}$$

The first term is at most δ_T^U. The transport-distortion assumption bounds the second by

$$D_{\mathrm{sem}}^U(U_T, \Theta_{0:T}^U U_0) + \omega_T^U.$$

Substituting the induction hypothesis yields the displayed bound. Induction completes the proof. □

41.6 Anti-circular reflective-trust residuals

Definition 41.15 (Anti-circular reflective-trust residuals). *The successor-verification trust reference packet decomposes into the residual vector*

$$Q_t^{\mathrm{TrustRef,SV}}(\nu;\rho) = (q_t^{\mathrm{sole}}, q_t^{\mathrm{common}}, q_t^{\mathrm{collusion}}, q_t^{\mathrm{leak}}, q_t^{\mathrm{anchor}}, q_t^{\mathrm{reflect}})(\nu;\rho).$$

The intended meanings are:

(i) $q_t^{\mathrm{sole}} \le 0$*: the successor-verification certificate is not certified solely by the verifier component it is modifying or replacing;*

(ii) $q_t^{\mathrm{common}} \le 0$*: common-mode verifier-support risk is below the declared threshold;*

(iii) $q_t^{\mathrm{collusion}} \le 0$*: collusion or correlated support risk is below the declared threshold;*

(iv) $q_t^{\mathrm{leak}} \le 0$*: audit leakage or training-on-the-audit risk is below the declared threshold;*

(v) $q_t^{\mathrm{anchor}} \le 0$*: at least one external or predecessor-stable trust anchor supports the successor-verification certificate;*

(vi) $q_t^{\mathrm{reflect}} \le 0$*: any reflective self-reference used by the certificate is limited to the declared proof schema and does not assert global soundness of the entire successor verifier.*

The trust-reference residual passes when all six components are nonpositive.

Proposition 41.16 (Trust residuals imply no self-sole-certification). *If*

$$Q_t^{\mathrm{TrustRef,SV}}(\nu;\rho) \le 0,$$

then the successor-verification certificate is not self-sole-certified on the declared audited domain.

Proof. The first residual component $q_t^{\mathrm{sole}} \le 0$ is defined to be exactly the no-self-sole-certification condition. The remaining residuals strengthen the trust packet against common-mode, collusion, leakage, anchor, and reflective-schema failures. Therefore componentwise nonpositivity implies the no-self-sole-certification conclusion. No claim of full Löbian reflection follows from this proposition. □

41.7 Viability-domain audit: not circular by definition

Audit Finding 41.17 (Batch 12 is a viability-domain invariance theorem). *The definition of* $\mathcal{K}_t^{\mathrm{SV},N}$ *includes the existence of a candidate* ν *whose successor lies in* $\mathcal{K}_{t+1}^{\mathrm{SV},N}$*. Therefore Theorem 40.9 is a viability-domain invariance theorem. It is not a theorem that constructs* SVPkg_t*, proves* $\mathcal{K}_t^{\mathrm{SV},N} \neq \varnothing$*, or shows that arbitrary states enter the successor-verification seed domain. Those are separate packet-construction, nonemptiness, and domain-entry obligations.*

Limitation 41.18 (No packet-construction claim). *Batch 12A makes the packet more explicit. It does not prove that a concrete system can always produce* $Q_t^{\mathrm{SV},A}$*,* $\Theta_t^{\mathrm{SV},A}$*,* $\mathcal{P}_t^{\mathrm{SV}}$*,* GoalId_t*,* $\mathsf{TrustRef}_t^{\mathrm{SV}}$*, or* $\mathsf{SVPersist}_t$*. Whenever these objects are missing, the successor-verification theorem cannot be invoked.*

41.8 Batch-12 theorem-type table rows

Batch-12 / Batch-12A object or result	Type	Claim discipline
Successor-verification schema $\mathsf{VerSch}_t^{\mathrm{SV}}$	Def	Fixes the certificate language, predicates, residuals, transports, failure event, and budget; does not prove soundness.
Successor-verification uncertainty envelope $\mathcal{P}_t^{\mathrm{SV}}$	Def / Cert	Licenses robust quantification only over the declared envelope; does not prove the true environment is inside it.
Goal-identity certificate GoalId_t	Def / Cert	Certifies one-step semantic/goal drift under a declared distance; does not prove the chosen goal object is philosophically complete.
Successor-verification seed domain $\mathcal{K}_t^{\mathrm{SV},N}$	Def / Exist-domain	Defines a recursively viable domain; does not prove nonemptiness or arbitrary-state entry.
Explicit SV residual vector $Q_t^{\mathrm{SV},A}$	Def / Cert	Names checkable residual obligations; a passing vector requires sound component certificates.
Verifier-schema transport lemma	Cert	Shows componentwise transport residuals imply compact schema persistence; does not construct the transports.
SV soundness union theorem	Cert / Tel	Composes component failure budgets by union bound; does not prove each component audit is valid.
Goal-identity composition theorem	Tel	Sums one-step drift and transport distortion; does not prove the one-step goal certificates are obtainable.
Robust SV domain invariance theorem	Exist / Cert	Proves successor-domain invariance from $\mathcal{K}_t^{\mathrm{SV},N}$ membership and valid certificates; does not solve full reflection.
Algorithmic SV gate	Impl / Cert	States a check order for implementation; it is not empirical validation of any trained system.

41.9 Algorithmic successor-verification gate

Definition 41.19 (RCP-II Phase-1 successor-verification gate)**.** *The algorithmic successor-verification gate $\mathcal{A}_t^{\mathrm{SV\text{-}gate}}(\rho,\nu)$ accepts a pair (ρ,ν) only if the following finite checklist passes:*

(1) verify $\rho \in \mathcal{K}_t^{\mathrm{seed},N}$ using the Batch-11 seed package;

(2) verify that ν is the seed module candidate or is linked to it by a valid SeedEq_t certificate;

(3) construct and evaluate $Q_t^{\mathrm{SV},A}(\nu;\rho)$;

(4) check componentwise verifier-schema transport $\Theta_t^{\mathrm{SV},A}$;

(5) *construct* $\mathcal{P}_t^{\mathrm{SV}}$ *from component envelopes and check uncertainty transport or margin accounting;*

(6) *check the goal-identity certificate and record* δ_t^U*;*

(7) *check anti-circular trust residuals, including no-self-sole-certification;*

(8) *check proof-search, verification, estimator, and transport costs against* $\mathsf{Budget}_t^{\mathrm{SV}}$*;*

(9) *check the composed soundness budget* $\alpha_t^{\mathrm{SV},A} \le \alpha_t^{\mathrm{SV}}$*;*

(10) *accept only if all residuals are nonpositive and the successor membership certificate proves* $\Phi_\nu(\rho) \in \mathcal{K}_{t+1}^{\mathrm{SV},N}$.

Proposition 41.20 (SV gate acceptance implies SV packet validity)**.** *If* $\mathcal{A}_t^{\text{SV-gate}}(\rho,\nu)$ *accepts, then* $\mathsf{SVPkg}_t(\rho,\nu)$ *is valid for the explicit Batch-12A residual interpretation.*

Proof. Each item of the algorithmic gate corresponds to one of the obligations in Definition 40.6 or Definition 41.2. Acceptance means every item has passed and every residual is nonpositive. Proposition 41.3 supplies the residual clause, Lemma 41.5 supplies verifier-schema persistence, and Theorem 41.8 supplies the composed soundness bound. Therefore the packet is valid under the declared assumptions. □

41.10 Phase-1 limitation summary

Limitation 41.21 (Batch-12A Phase-1 limitations)**.** *The Batch-12/12A successor-verification layer is deliberately restricted. It does not prove:*

(i) *construction of* SVPkg_t *for arbitrary states;*

(ii) *nonemptiness of* $\mathcal{K}_t^{\mathrm{SV},N}$*;*

(iii) *full Löbian reflection, arbitrary Vingean successor trust, or global self-verifier soundness;*

(iv) *that* $\mathcal{P}_t^{\mathrm{SV}}$ *contains the real environment;*

(v) *scalable proof search outside the declared budget ledger;*

(vi) *semantic completeness of the chosen goal object* U_t*;*

(vii) *empirical validity of any implementation claiming to instantiate these certificates.*

What it does prove is narrower: if the explicit successor-verification residuals, transports, uncertainty envelope, goal-identity certificate, trust residuals, budget ledger, and successor-membership certificate are all valid, then the Batch-12 domain-invariance theorem is licensed for the declared finite horizon.

42 Batch 12B: Successor-Verification Packet Construction, Seed-Domain Nonemptiness, Reality-Contained Uncertainty, and Infinite-Horizon Reflective Closure

Batch 12A proves successor-verification invariance once the successor-verification packet and successor-verification seed-domain witness exist. This section adds the next theorem layer. Its

purpose is not to replace the Batch 12A residual and soundness layer, but to discharge part of its remaining conditionality for a declared, checkable class. Most compactly:

> Batch 12A proves successor-verification invariance once the SV packet/domain exists. Batch 12B proves, for a declared class, that the SV packet/domain can be constructed and iterated across an infinite certified path under uncertainty.

The results in this section remain domain-relative. They do not prove that arbitrary systems can construct successor-verification packets, that every stronger successor is trustworthy, or that the declared uncertainty envelope contains reality without a reality-containment certificate. They add a precise construction and nonemptiness interface above the Batch 12A viability-domain theorem.

42.1 Tier 1: Successor-verification packet construction

Definition 42.1 (Successor-verification construction input)**.** *A successor-verification construction input at time t is a tuple*

$$\begin{aligned}\mathsf{SVIn}_t(\rho,\nu) = (\rho,\nu,&\mathsf{SeedPkg}_t,\mathsf{VerSchSrc}_t,\mathsf{ResSrc}_t,\\ &\mathsf{UncSrc}_t,\mathsf{GoalSrc}_t,\mathsf{TrustSrc}_t,\mathsf{BudgetSrc}_t,\mathsf{PersistSrc}_t),\end{aligned}$$

where:

(i) $\rho \in \mathcal{K}_t^{\mathrm{seed},N}$ *is certified by the Batch 11 seed package* $\mathsf{SeedPkg}_t$*;*

(ii) ν *is either the Batch 11 seed-module candidate or an engine-selected candidate connected to that seed module by a valid* SeedEq_t *certificate;*

(iii) $\mathsf{VerSchSrc}_t$ *contains the data needed to construct* $\mathsf{VerSch}_t^{\mathrm{SV}}$ *and the transported successor schema;*

(iv) ResSrc_t *contains the named residual definitions, sign conventions, audited one-sided bounds, and checker outputs needed to construct* $Q_t^{\mathrm{SV},A}(\nu;\rho)$*;*

(v) UncSrc_t *contains component uncertainty envelopes for environment, estimators, certificates, physical/world-model distortion, and trust;*

(vi) $\mathsf{GoalSrc}_t$ *contains* $(U_t, U_{t+1}, \Theta_t^U, D_{\mathrm{sem}}^U)$ *and the data needed to certify one-step goal drift;*

(vii) $\mathsf{TrustSrc}_t$ *contains the non-self-sole-certification, common-mode, collusion, leakage, anchor, and reflective-schema evidence used by the anti-circular trust residuals;*

(viii) $\mathsf{BudgetSrc}_t$ *contains the checking, proof-search, estimator, transport, and certificate-construction cost records;*

(ix) $\mathsf{PersistSrc}_t$ *contains the successor-membership evidence intended to prove* $\Phi_\nu(\rho) \in \mathcal{K}_{t+1}^{\mathrm{SV},N}$ *or, in the infinite case,* $\Phi_\nu(\rho) \in \mathcal{K}_{t+1}^{\mathrm{SV},\infty}$*.*

The input is a data object, not a theorem. If any component is missing, off-domain, stale, or untyped, the successor-verification packet builder below is undefined.

Definition 42.2 (Successor-verification packet builder). *A successor-verification packet builder is a partial map*

$$\mathsf{SVBuilder}_t : \mathsf{SVIn}_t(\rho,\nu) \longmapsto (\mathsf{SVPkg}_t(\rho,\nu), Q_t^{\mathrm{SV},A}, \Theta_t^{\mathrm{SV},A}, \mathcal{P}_t^{\mathrm{SV}}, \\ \mathsf{GoalId}_t, \mathsf{TrustRef}_t^{\mathrm{SV}}, \mathsf{SVPersist}_t, \mathsf{SVBuildTrace}_t),$$

where $\mathsf{SVBuildTrace}_t$ *records the exact component certificates, verifier identities, residual radii, random seeds or search traces, and resource costs used in the construction. The builder is* sound on a construction domain $\mathcal{D}_t^{\text{SV-construct}}$ *when every input in that domain produces objects satisfying:*

(i) the packet clauses of Definition 40.6;

(ii) the explicit residual clauses of Definition 41.2;

(iii) the verifier-schema transport clauses of Definition 41.4;

(iv) the uncertainty-envelope construction and transport clauses of Definitions 41.9 and 41.10;

(v) the goal-identity drift certificate of Definition 40.4;

(vi) the anti-circular trust residual clauses of Definition 41.15;

(vii) the composed soundness bound of Theorem 41.8;

(viii) the declared resource and complexity budget ledger.

Definition 42.3 (Successor-verification packet-construction domain). *The packet-construction domain* $\mathcal{D}_t^{\text{SV-construct}}$ *is the set of pairs* (ρ,ν) *for which a construction input* $\mathsf{SVIn}_t(\rho,\nu)$ *is present, typed, and accepted by all component constructors used by* $\mathsf{SVBuilder}_t$*. Equivalently,*

$$(\rho,\nu) \in \mathcal{D}_t^{\text{SV-construct}}$$

means that the builder can construct every component required by Definition 42.2 with certified one-sided radii and total costs inside the declared budget. This domain is not assumed to contain arbitrary states or arbitrary candidates.

Theorem 42.4 (Successor-verification packet construction). *Assume* $\mathsf{SVBuilder}_t$ *is sound on* $\mathcal{D}_t^{\text{SV-construct}}$*. If*

$$(\rho,\nu) \in \mathcal{D}_t^{\text{SV-construct}},$$

then $\mathsf{SVBuilder}_t$ *returns a valid successor-verification packet* $\mathsf{SVPkg}_t(\rho,\nu)$ *for the explicit Batch 12A residual interpretation. In particular,*

$$Q_t^{\mathrm{SV},A}(\nu;\rho) \le 0, \qquad \Theta_t^{\mathrm{SV},A}(\mathsf{VerSch}_t^{\mathrm{SV}}) \preceq_t^{\mathrm{SV}} \mathsf{VerSch}_{t+1}^{\mathrm{SV}},$$

$$D_{\mathrm{sem}}^U(U_{t+1}, \Theta_t^U U_t) \le \delta_t^U, \qquad \sup_{P\in\mathcal{P}_t^{\mathrm{SV}}} P((\mathsf{SVSound}_t)^c) \le \alpha_t^{\mathrm{SV}},$$

and the successor-membership clause certified by $\mathsf{SVPersist}_t$ *is available for the relevant finite or infinite successor-verification seed domain.*

Proof. By Definition 42.2, builder soundness on $\mathcal{D}_t^{\text{SV-construct}}$ means that every returned component satisfies the corresponding clause in Definitions 40.6, 41.2, 41.4, 41.9 and 41.10, 40.4, and 41.15. Proposition 41.3 gives the packet residual clause. Lemma 41.5 gives verifier-schema persistence. Theorem 41.8 gives the composed soundness bound. The remaining displayed inequalities are exactly the goal-identity and successor-persistence components of the returned packet. No claim is made outside $\mathcal{D}_t^{\text{SV-construct}}$. □

Limitation 42.5 (Packet construction is domain-relative)**.** *Theorem 42.4 constructs* SVPkg_t *only for inputs in* $\mathcal{D}_t^{\text{SV-construct}}$*. It does not prove that arbitrary states admit such inputs, that the builder is complete for unbounded proof search, or that the constructed packet is meaningful outside the declared schema, transport, uncertainty, and goal-reference objects.*

42.2 Tier 2: Successor-verification seed-domain nonemptiness

Definition 42.6 (Certified successor-verification witness library)**.** *A certified successor-verification witness library at time t is a finite or explicitly enumerable library*

$$\mathcal{L}_t^{\text{SV-Wit}}(\rho)$$

whose entries W contain:

(i) a Batch 11 conservative-extension seed module $M_W \in \mathcal{L}_t^{\text{CE}}(\rho)$ or a seed-equivalent engine-selected candidate ν_W;

(ii) a bounded proof/certificate word for $\mathsf{SVBuilder}_t$*;*

(iii) component certificates for residuals, verifier-schema transport, uncertainty-envelope construction, goal identity, anti-circular trust, proof budget, and successor persistence;

(iv) a margin ledger showing that all construction, estimation, transport, projection, and proof-search errors fit inside the strict Batch 12A residual margins;

(v) an optional tractability certificate of the kind defined in Definition 42.24.

An entry W is valid and positive-margin *at ρ when all of these certificates are accepted and the residual margins remain strictly positive after all declared errors are included.*

Definition 42.7 (Bounded successor-verification certificate grammar)**.** *For integers $k_t, \ell_t \geq 0$, let*

$$\mathfrak{G}_{t,\leq(k_t,\ell_t)}^{\text{SV-cert}}(\rho)$$

be the finite or explicitly enumerable grammar of successor-verification certificate words whose update-primitive depth is at most k_t and whose proof/checker trace length is at most ℓ_t. A grammar is complete for a witness library $\mathcal{L}_t^{\text{SV-Wit}}(\rho)$ when every valid positive-margin library witness has at least one certificate word in the grammar and exhaustive enumeration returns it within the declared search budget.

Definition 42.8 (Successor-verification seed-library domain)**.** *For a finite horizon N, the successor-verification seed-library domain $\mathcal{K}_t^{\text{SV-seedlib},N}$ is the set of states $\rho \in \mathcal{K}_t^{\text{seed},N}$ such that:*

(i) $\mathcal{L}_t^{\text{SV-Wit}}(\rho)$ contains at least one valid positive-margin witness W_t;

(ii) *the bounded grammar* $\mathfrak{G}^{\text{SV-cert}}_{t,\le(k_t,\ell_t)}(\rho)$ *is complete for that witness;*

(iii) *enumeration over the bounded grammar returns a candidate* ν_t *and construction input* $\mathsf{SVIn}_t(\rho,\nu_t)$*;*

(iv) $(\rho,\nu_t)\in\mathcal{D}_t^{\text{SV-construct}}$*;*

(v) *the returned packet certifies* $\Phi_{\nu_t}(\rho)\in\mathcal{K}_{t+1}^{\text{SV},N}$*.*

This is a checkable sufficient-condition domain. It is not identical to $\mathcal{K}_t^{\text{SV},N}$*, and it need not be maximal.*

Theorem 42.9 (Successor-verification seed-domain nonemptiness from a witness library)**.** *Assume the Batch 11 seed-domain hypotheses, a sound* $\mathsf{SVBuilder}_t$*, and bounded-grammar completeness for* $\mathcal{L}_t^{\text{SV-Wit}}(\rho)$*. If*

$$\rho\in\mathcal{K}_t^{\text{SV-seedlib},N},$$

then

$$\rho\in\mathcal{K}_t^{\text{SV},N}.$$

Consequently, whenever $\mathcal{K}_t^{\text{SV-seedlib},N}$ *is nonempty, the successor-verification seed domain* $\mathcal{K}_t^{\text{SV},N}$ *is nonempty.*

Proof. Membership in $\mathcal{K}_t^{\text{SV-seedlib},N}$ gives $\rho\in\mathcal{K}_t^{\text{seed},N}$, a valid positive-margin successor-verification witness W_t, grammar completeness for that witness, an enumerated candidate ν_t, and a construction input in $\mathcal{D}_t^{\text{SV-construct}}$. Theorem 42.4 constructs a valid $\mathsf{SVPkg}_t(\rho,\nu_t)$. The seed-library definition also requires the successor-membership certificate $\Phi_{\nu_t}(\rho)\in\mathcal{K}_{t+1}^{\text{SV},N}$ and all robust monotonicity, goal, uncertainty, trust, and budget clauses used by Definition 40.7. Therefore the defining clauses of $\mathcal{K}_t^{\text{SV},N}$ hold. Nonemptiness follows immediately from subset inclusion. □

Corollary 42.10 (SV packet/domain construction closes the Batch-12A viability objection for a declared class)**.** *For states in* $\mathcal{K}_t^{\text{SV-seedlib},N}$*, the Batch 12A packet is not merely assumed: it is constructed by* $\mathsf{SVBuilder}_t$ *from a bounded witness library and certificate grammar. The closure remains limited to the declared seed-library class.*

Proof. This is the conjunction of Theorem 42.4 and Theorem 42.9. The final sentence follows from the definition of $\mathcal{K}_t^{\text{SV-seedlib},N}$. □

42.3 Tier 3: Infinite-horizon reflective successor theorem

Definition 42.11 (Infinite Batch-11 seed-domain sequence)**.** *An infinite Batch-11 seed-domain sequence is a sequence of declared sets*

$$\{\mathcal{K}_t^{\text{seed},\infty}\}_{t\ge0}$$

with $\mathcal{K}_t^{\text{seed},\infty}\subseteq\mathcal{K}_t^{\text{base}}$ *such that every* $\rho\in\mathcal{K}_t^{\text{seed},\infty}$ *carries a Batch-11 seed package, a certified seed move* ν_t*, and a seed-persistence certificate proving*

$$\Phi_{\nu_t}(\rho)\in\mathcal{K}_{t+1}^{\text{seed},\infty}.$$

The seed package includes the conservative-extension library or seed-equivalent engine-selected candidate, bounded grammar or enumerable witness coverage, type/trust/resource/projection-realization

certificates, and any seed-equivalence or persistence-transfer certificates required by Batch 11. This definition licenses the notation $\mathcal{K}_t^{\mathrm{seed},\infty}$ used below. It is a declared infinite-horizon sufficient-condition domain, not a proof that such a sequence is nonempty for arbitrary systems.

Definition 42.12 (Infinite successor-verification seed domain)**.** *An infinite successor-verification seed domain is a sequence of sets*

$$\{\mathcal{K}_t^{\mathrm{SV},\infty}\}_{t\geq 0}$$

such that

$$\mathcal{K}_t^{\mathrm{SV},\infty} \subseteq \mathcal{K}_t^{\mathrm{seed},\infty}$$

and for every $\rho \in \mathcal{K}_t^{\mathrm{SV},\infty}$ there exists a candidate ν satisfying a valid infinite-horizon analogue of $\mathsf{SVPkg}_t(\rho,\nu)$ with

$$\Phi_\nu(\rho) \in \mathcal{K}_{t+1}^{\mathrm{SV},\infty}.$$

The infinite-horizon packet must include summable goal-drift and failure budgets, verifier-schema transport for every step, and either exact or summably distorted uncertainty-envelope transport.

42.4 Infinite successor-verification seed-library closure

Definition 42.13 (Infinite successor-verification seed-library domain)**.** *An infinite successor-verification seed-library domain is a sequence of sets*

$$\{\mathcal{K}_t^{\text{SV-seedlib},\infty}\}_{t\geq 0}$$

such that, for every time t:

(i)

$$\mathcal{K}_t^{\text{SV-seedlib},\infty} \subseteq \mathcal{K}_t^{\mathrm{seed},\infty};$$

(ii) for every $\rho \in \mathcal{K}_t^{\text{SV-seedlib},\infty}$, the library $\mathcal{L}_t^{\text{SV-Wit}}(\rho)$ contains a valid positive-margin infinite-horizon witness W_t^∞;

(iii) the bounded or explicitly enumerable certificate grammar contains a proof/checker word for W_t^∞, and enumeration returns a candidate ν_t together with a construction input $\mathsf{SVIn}_t(\rho,\nu_t)$;

(iv)

$$(\rho,\nu_t) \in \mathcal{D}_t^{\text{SV-construct}},$$

and $\mathsf{SVBuilder}_t$ returns a valid infinite-horizon successor-verification packet;

(v) the returned packet certifies successor seed-library persistence:

$$\Phi_{\nu_t}(\rho) \in \mathcal{K}_{t+1}^{\text{SV-seedlib},\infty};$$

(vi) the one-step goal-drift, transport-distortion, and soundness-failure budgets associated with the witnesses are summable on every generated infinite path:

$$\sum_{t=0}^{\infty}(\delta_t^U + \omega_t^U) < \infty, \qquad \sum_{t=0}^{\infty}\alpha_t^{\mathrm{SV}} < \infty.$$

This is the infinite-horizon analogue of $\mathcal{K}_t^{\text{SV-seedlib},N}$. It is a declared sufficient-condition domain. It does not assert that arbitrary states enter the domain or that the domain is nonempty.

Theorem 42.14 (Infinite-horizon SV seed-library nonemptiness). *Assume $\{\mathcal{K}_t^{\text{SV-seedlib},\infty}\}_{t\geq 0}$ is an infinite successor-verification seed-library domain and assume $\mathsf{SVBuilder}_t$ is sound on the selected construction inputs at every time. Then the sequence $\{\mathcal{K}_t^{\text{SV-seedlib},\infty}\}_{t\geq 0}$ itself satisfies Definition 42.12. Consequently, it is a valid canonical choice of infinite successor-verification seed domain:*

$$\mathcal{K}_t^{\text{SV},\infty} := \mathcal{K}_t^{\text{SV-seedlib},\infty},$$

and therefore

$$\rho \in \mathcal{K}_t^{\text{SV-seedlib},\infty} \Longrightarrow \rho \in \mathcal{K}_t^{\text{SV},\infty}.$$

In particular, if $\mathcal{K}_0^{\text{SV-seedlib},\infty} \neq \varnothing$, then there exists a nonempty infinite successor-verification seed domain.

Proof. Definition 42.13 gives

$$\mathcal{K}_t^{\text{SV-seedlib},\infty} \subseteq \mathcal{K}_t^{\text{seed},\infty}$$

for every t. For any $\rho \in \mathcal{K}_t^{\text{SV-seedlib},\infty}$, the same definition supplies a valid positive-margin witness, a bounded or enumerable certificate word, a selected candidate ν_t, a construction input in $\mathcal{D}_t^{\text{SV-construct}}$, and a valid infinite-horizon successor-verification packet. The successor-persistence clause of the definition gives

$$\Phi_{\nu_t}(\rho) \in \mathcal{K}_{t+1}^{\text{SV-seedlib},\infty}.$$

Thus the sequence $\{\mathcal{K}_t^{\text{SV-seedlib},\infty}\}_{t\geq 0}$ satisfies exactly the subset and recursive-successor clauses required of an infinite successor-verification seed domain in Definition 42.12. Choosing $\mathcal{K}_t^{\text{SV},\infty}$ to be this sequence gives the displayed inclusion. Nonemptiness of the initial seed-library domain then gives nonemptiness of the canonical infinite successor-verification seed domain. No claim is made that the seed-library domain is nonempty for arbitrary systems. □

Corollary 42.15 (Infinite seed-library start for the domain-relative RRST). *Under the hypotheses of Theorem 42.14, if*

$$\rho_0 \in \mathcal{K}_0^{\text{SV-seedlib},\infty},$$

then the infinite-horizon theorem may be invoked with the canonical choice

$$\mathcal{K}_t^{\text{SV},\infty} = \mathcal{K}_t^{\text{SV-seedlib},\infty}.$$

Thus the starting condition for the infinite case can be stated as seed-library membership rather than bare membership in an already-postulated infinite successor-verification domain.

Proof. Theorem 42.14 converts $\rho_0 \in \mathcal{K}_0^{\text{SV-seedlib},\infty}$ into $\rho_0 \in \mathcal{K}_0^{\text{SV},\infty}$ for the canonical infinite domain. The remaining conditions are exactly the hypotheses needed by Theorem 42.18. □

Assumption 42.16 (Infinite-path selector). *For every t and every $\rho \in \mathcal{K}_t^{\text{SV},\infty}$, a selector π_t^{SV} chooses a candidate*

$$\nu_t = \pi_t^{\text{SV}}(\rho)$$

among the candidates witnessing Definition 42.12. If measurability is needed for a probabilistic conclusion, the selector is assumed measurable with respect to the declared sigma-algebras. This assumption avoids hiding an unlicensed choice or measurability claim in the infinite-path theorem.

Definition 42.17 (Composed verifier-schema and goal transports). *For $0 \le s < t$, define the composed verifier-schema transport and goal transport by*

$$\Theta^{\mathrm{SV}}_{s:t} = \Theta^{\mathrm{SV}}_{t-1} \circ \cdots \circ \Theta^{\mathrm{SV}}_{s}, \qquad \Theta^{U}_{s:t} = \Theta^{U}_{t-1} \circ \cdots \circ \Theta^{U}_{s},$$

with identity transport when $s = t$. The composed transport is licensed only when every one-step transport in the composition is type-valid and its distortion budget is included.

Theorem 42.18 (Domain-relative infinite-horizon robust reflective successor theorem). *Assume an infinite successor-verification seed domain, an infinite-path selector, Batch 11 infinite seed-domain persistence, sound Batch 12B packet construction at every selected step, and summable budgets*

$$\sum_{t=0}^{\infty} (\delta^{U}_{t} + \omega^{U}_{t}) < \infty, \qquad \sum_{t=0}^{\infty} \alpha^{\mathrm{SV}}_{t} < \infty.$$

If

$$\rho_0 \in \mathcal{K}^{\mathrm{SV},\infty}_{0},$$

then the recursively selected trajectory

$$\nu_t = \pi^{\mathrm{SV}}_{t}(\rho_t), \qquad \rho_{t+1} = \Phi_{\nu_t}(\rho_t)$$

satisfies

$$\rho_t \in \mathcal{K}^{\mathrm{SV},\infty}_{t} \qquad \forall t \ge 0.$$

Moreover, for every finite T,

$$\Theta^{\mathrm{SV}}_{0:T}(\mathsf{VerSch}^{\mathrm{SV}}_{0}) \preceq^{\mathrm{SV}}_{0:T} \mathsf{VerSch}^{\mathrm{SV}}_{T},$$

$$D^{U}_{\mathrm{sem}}(U_T, \Theta^{U}_{0:T} U_0) \le \sum_{t=0}^{T-1} (\delta^{U}_{t} + \omega^{U}_{t}),$$

and

$$\sum_{t=0}^{\infty} \sup_{P \in \mathcal{P}^{\mathrm{SV}}_{t}} P((\mathsf{SVSound}_t)^c) < \infty.$$

If all failure events are defined on a common probability space and the Borel–Cantelli hypotheses hold for the selected path, then only finitely many successor-verification soundness failures occur almost surely under every admissible probability measure for which the path events are jointly defined.

Proof. The invariance $\rho_t \in \mathcal{K}^{\mathrm{SV},\infty}_{t}$ follows by induction. The base case is the hypothesis. If $\rho_t \in \mathcal{K}^{\mathrm{SV},\infty}_{t}$, Definition 42.12 and the selector assumption choose ν_t with $\rho_{t+1} = \Phi_{\nu_t}(\rho_t) \in \mathcal{K}^{\mathrm{SV},\infty}_{t+1}$. This proves the invariant.

The verifier-schema statement is the finite composition of the one-step schema-persistence clauses supplied by the valid infinite-horizon packets. The goal-identity inequality is Theorem 41.14 applied on the finite prefix with the declared one-step distortions. The failure-budget inequality follows by summing the one-step soundness bounds. The final almost-sure statement is exactly the first Borel–Cantelli implication under the stated common-probability-space and joint-event assumptions. □

Limitation 42.19 (Infinite-horizon theorem is still domain-relative). *Theorem 42.18 proves unbounded recursive verification persistence only along an infinite certified successor-verification path. Theorem 42.14 shows how such a path-domain is obtained from an infinite seed-library domain, but it still does not prove $\mathcal{K}^{\text{SV-seedlib},\infty}_{0} \neq \varnothing$, does not prove arbitrary-state entry into that seed-library domain, and does not prove that strict ability progress can continue forever on a fixed bounded ability scale.*

42.5 Tier 4: Reality-contained uncertainty and model misspecification

Definition 42.20 (Reality-containment and misspecification certificate)**.** *Let P_t^{true} denote the actual environment/model law for the audited successor-verification claim, when such a law is meaningful in the formalization. A reality-containment certificate is a tuple*

$$\mathsf{RealCont}_t = (P_t^{\mathrm{true}}, \mathcal{P}_t^{\mathrm{SV}}, \beta_t^{\mathrm{env}}, \varepsilon_t^{\mathrm{env}}, d_{\mathcal{P}}, \mathsf{RealCert}_t)$$

satisfying either:

$$P(P_t^{\mathrm{true}} \in \mathcal{P}_t^{\mathrm{SV}}) \geq 1 - \beta_t^{\mathrm{env}},$$

or the misspecification residual bound

$$d_{\mathcal{P}}(P_t^{\mathrm{true}}, \mathcal{P}_t^{\mathrm{SV}}) \leq \varepsilon_t^{\mathrm{env}}.$$

The distance-to-set notation means

$$d_{\mathcal{P}}(P_t^{\mathrm{true}}, \mathcal{P}_t^{\mathrm{SV}}) = \inf_{P \in \mathcal{P}_t^{\mathrm{SV}}} d_{\mathcal{P}}(P_t^{\mathrm{true}}, P).$$

Assumption 42.21 (Robust residual stability under envelope misspecification)**.** *For every successor-verification residual or expectation bound asserted under $P \in \mathcal{P}_t^{\mathrm{SV}}$, there is a declared modulus Γ_t^{env} such that replacing an admissible envelope law by a law at distance at most ε changes the relevant bound by at most $\Gamma_t^{\mathrm{env}}(\varepsilon)$. The modulus must be certified for the residuals being used; it is not supplied by notation.*

Theorem 42.22 (Reality-contained successor-verification reliability)**.** *Suppose a Batch 12A successor-verification certificate is valid over $\mathcal{P}_t^{\mathrm{SV}}$, and suppose $\mathsf{RealCont}_t$ is valid. If $P_t^{\mathrm{true}} \in \mathcal{P}_t^{\mathrm{SV}}$, then the same successor-verification bounds hold under P_t^{true}. If instead*

$$d_{\mathcal{P}}(P_t^{\mathrm{true}}, \mathcal{P}_t^{\mathrm{SV}}) \leq \varepsilon_t^{\mathrm{env}},$$

and Assumption 42.21 holds, then each robust expectation or residual bound transfers to the true law with additional error at most $\Gamma_t^{\mathrm{env}}(\varepsilon_t^{\mathrm{env}})$. In particular,

$$\mathbb{E}_{P_t^{\mathrm{true}}}\left[\Psi_{t+1}^{\mathrm{SV}}(\rho_{t+1}) - \Theta_t^{\Psi}\Psi_t^{\mathrm{SV}}(\rho_t)\right] \geq \epsilon_t^{\mathrm{SV}} - \eta_t^{\mathrm{SV}} - \Gamma_t^{\mathrm{env}}(\varepsilon_t^{\mathrm{env}}),$$

whenever the Ψ^{SV}-progress claim is part of the packet.

Proof. If $P_t^{\mathrm{true}} \in \mathcal{P}_t^{\mathrm{SV}}$, the original robust quantifier over $\mathcal{P}_t^{\mathrm{SV}}$ includes the true law, so the bound holds directly. In the misspecified case, the definition of distance-to-envelope and the stability modulus supply an admissible envelope law within distance $\varepsilon_t^{\mathrm{env}}$ and bound the residual or expectation change by $\Gamma_t^{\mathrm{env}}(\varepsilon_t^{\mathrm{env}})$. Subtracting that additional error from the robust lower bound gives the displayed inequality. The same argument applies componentwise to the other residuals for which a stability modulus is supplied. □

Limitation 42.23 (Reality containment is a certificate condition)**.** *The theorem does not prove that the declared uncertainty envelope contains reality. It states what follows if reality containment or bounded misspecification is certified. Without $\mathsf{RealCont}_t$, reliability remains envelope-relative.*

42.6 Tier 5: Tractability discipline

Definition 42.24 (Restricted SV tractability certificate)**.** *A restricted successor-verification tractability certificate at time t is a tuple*

$$\mathsf{SVTract}_t = (n_t, k_t, \ell_t, s_t, \delta_t^{\mathrm{tol}}, C_t^{\mathrm{SVBuild}}, C_t^{\mathrm{SVGate}}, C_t^{\mathrm{SVProof}}, \mathsf{TractCert}_t)$$

where n_t is the representation size, k_t is the primitive grammar depth, ℓ_t is the proof/checker trace bound, s_t is the relevant sample or audit size, and δ_t^{tol} is the residual tolerance. The certificate is valid when it proves explicit finite or asymptotic bounds of the form

$$\mathrm{Cost}(\mathsf{SVBuilder}_t) \le C_t^{\mathrm{SVBuild}}, \qquad \mathrm{Cost}(\mathcal{A}_t^{\text{SV-gate}}) \le C_t^{\mathrm{SVGate}}, \qquad \mathrm{Cost}(\mathsf{SVProof}_t) \le C_t^{\mathrm{SVProof}},$$

and, for a declared tractable subclass,

$$C_t^{\mathrm{SVBuild}} + C_t^{\mathrm{SVGate}} + C_t^{\mathrm{SVProof}} \le \mathrm{poly}(n_t, k_t, \ell_t, s_t, \log(1/\delta_t^{\mathrm{tol}})).$$

If only finite budgets are certified and no polynomial or asymptotic bound is supplied, the correct claim is resource-accounted *or* budgeted*, not computationally tractable in the strong sense.*

Theorem 42.25 (Restricted SV packet tractability)**.** *If $\mathsf{SVTract}_t$ is valid for the declared representation class and the active Batch 12B packet-construction input lies inside that class, then the costs of $\mathsf{SVBuilder}_t$, the algorithmic SV gate, and the proof/checking trace are bounded by the displayed finite or polynomial bounds. Therefore the Batch 12B step is computationally tractable only relative to that declared representation class and cost model. If $\mathsf{SVTract}_t$ supplies only finite budget inequalities, then the theorem licenses only resource accounting.*

Proof. The conclusion is exactly the content of the valid $\mathsf{SVTract}_t$ certificate applied to an input inside its declared representation class. The theorem makes no claim about inputs outside the class, unbounded proof search, or hidden costs not included in $\mathsf{SVTract}_t$. □

Limitation 42.26 (Complexity ledger versus tractability theorem)**.** *The existing complexity ledger is an accounting object. Strong computational tractability requires a certificate such as Definition 42.24. If that certificate is absent, the abstract and conclusion should use the weaker language "resource-accounted" or "budgeted" rather than "computationally tractable."*

42.7 Tier 6: Claim-discipline renaming

Definition 42.27 (Domain-relative robust reflective successor theorem status)**.** *The theorem stack through Batch 12B may be called a* Domain-Relative Robust Reflective Successor Theorem *for the declared seed-library class when the following are all supplied:*

(i) Batch 11 seed-domain entry and persistence;

(ii) Batch 12/12A successor-verification domain invariance with explicit residuals and soundness composition;

(iii) Batch 12B packet construction;

(iv) Batch 12B finite successor-verification seed-domain nonemptiness for a declared witness library;

(v) Batch 12B infinite successor-verification seed-library closure when an unbounded claim is made;

(vi) reality-containment or misspecification certificates when real-world reliability is claimed;

(vii) restricted tractability certificates when computational tractability is claimed.

Without these qualifiers, the unqualified phrase "Robust Reflective Successor Theorem" overstates what has been proved.

Proposition 42.28 (Claim-discipline for Batch 12B)**.** *A statement that cites the Batch 12B theorem stack as proving reliable successor verification under uncertainty must specify whether the claim is:*

domain-relative, *certificate-relative*, *finite-horizon*, *infinite-horizon*,
envelope-relative, *reality-contained*, *budgeted*, *or tractability-certified.*

Omitting these qualifiers is an invalid use of the theorem stack.

Proof. Each qualifier corresponds to a separate premise in Definitions 42.3, 42.8, 42.13, 42.12, 42.20, and 42.24. The corresponding conclusions hold only under those premises. Therefore dropping the qualifiers changes the logical statement. □

42.8 Tier 7: Architecture synchronization deferred

Limitation 42.29 (Architecture synchronization is a later step)**.** *Batch 12B is added first to the abstract math/physics paper. The architecture paper must later instantiate the corresponding RCLM objects:*

$$\mathsf{SVBuilder}_t^{\mathrm{RCLM}}, \quad \mathcal{K}_t^{\text{RCLM-SV-seedlib},N}, \quad \mathcal{K}_t^{\text{RCLM-SV-seedlib},\infty},$$
$$\mathcal{K}_t^{\text{RCLM-SV},\infty}, \quad \mathsf{RealCont}_t^{\mathrm{RCLM}}, \quad \mathsf{SVTract}_t^{\mathrm{RCLM}}.$$

No architecture-side construction theorem is asserted by the present section.

42.9 Batch-12B theorem-type table rows

Batch-12B object or result	Type	Claim discipline
SV packet builder $\mathsf{SVBuilder}_t$	Def / Cert	Constructs packets only on $\mathcal{D}_t^{\text{SV-construct}}$; does not prove arbitrary-state packet construction.
SV packet construction theorem	Cert / Exist	Converts builder-domain membership into a valid SVPkg_t; depends on builder soundness.
SV witness library and bounded certificate grammar	Def / Exist	Gives a finite or enumerable class for packet construction; does not cover unbounded search.
SV seed-domain nonemptiness theorem	Exist / Cert	Proves $\mathcal{K}_t^{\text{SV-seedlib},N} \subseteq \mathcal{K}_t^{\text{SV},N}$; does not prove the seed-library domain is nonempty for arbitrary systems.
Infinite SV seed-library nonemptiness theorem	Exist / Cert	Proves that $\mathcal{K}_t^{\text{SV-seedlib},\infty}$ canonically supplies an infinite $\mathcal{K}_t^{\text{SV},\infty}$ domain; does not prove arbitrary-state entry into the infinite seed-library class.
Infinite-horizon domain-relative RRST	Tel / Exist	Iterates an infinite SV domain, now obtainable from an infinite seed-library domain when the seed-library hypotheses hold; does not prove arbitrary infinite viability.
Reality-containment theorem	Cert	Transfers envelope-relative guarantees to reality only with containment or misspecification certificates.
Restricted SV tractability theorem	Cert / Impl	Licenses tractability only for declared representation and cost classes; otherwise only budget accounting.
Claim-discipline renaming	Impl	Prevents unqualified robust-reflection claims unless all required premises are present.
Final RCP-II theorem framing and reference instance	Def / Exist / Impl	Surfaces the domain-relative theorem status and defines a finite reference-instantiation target; does not implement a trained system or prove empirical validity.

42.10 Combined Batch-12B theorem statement

Theorem 42.30 (Domain-relative Robust Reflective Successor Theorem)**.** *Assume the following for a declared class of systems and a declared time horizon, finite or infinite:*

(i) the Batch 11 seed-domain entry and persistence hypotheses hold;

(ii) $\rho_0 \in \mathcal{K}_0^{\text{SV-seedlib},N}$ *for a finite horizon, or* $\rho_0 \in \mathcal{K}_0^{\text{SV-seedlib},\infty}$ *for an infinite horizon;*

(iii) $\mathsf{SVBuilder}_t$ *is sound on each selected packet-construction domain;*

(iv) the explicit Batch 12A residuals satisfy $Q_t^{\mathrm{SV},A}(\nu_t;\rho_t) \le 0$ *at each selected step;*

(v) verifier-schema transport, uncertainty-envelope construction/transport, goal-identity transport, anti-circular trust residuals, proof/checking budgets, and successor-persistence certificates are valid at each step;

(vi) if real-world reliability is claimed, $\mathsf{RealCont}_t$ *or a bounded misspecification certificate is valid at each step;*

(vii) if computational tractability is claimed, $\mathsf{SVTract}_t$ *is valid for the declared representation class;*

(viii) in the infinite case, Theorem 42.14 supplies the canonical infinite SV domain and the selected infinite path satisfies the summability hypotheses of Theorem 42.18.

Then, for the finite case, the direct engine constructs a trajectory

$$\rho_0 \xrightarrow{\nu_0} \rho_1 \xrightarrow{\nu_1} \cdots \xrightarrow{\nu_{N-1}} \rho_N$$

with

$$\rho_t \in \mathcal{K}_t^{\mathrm{SV},N} \qquad (0 \le t \le N),$$

and each step preserves the declared successor-verification schema, goal-identity bound, soundness budget, anti-circular trust conditions, and Batch 11 non-lossy direct-engine construction certificates. In the infinite case,

$$\rho_t \in \mathcal{K}_t^{\mathrm{SV},\infty} \qquad \forall t \ge 0,$$

with composed verifier-schema persistence and cumulative goal drift bounded as in Theorem 42.18. If the ability-tower and certified-novelty assumptions of Batch 11 hold along the same path, the certified ability sets expand monotonically along that path as well.

Proof. In the finite case, $\rho_0 \in \mathcal{K}_0^{\text{SV-seedlib},N}$ and Theorem 42.9 give $\rho_0 \in \mathcal{K}_0^{\mathrm{SV},N}$. Theorem 40.9 then gives $\rho_1 \in \mathcal{K}_1^{\mathrm{SV},N}$ and the stepwise certificates. Repeating the same argument on the finite horizon gives the displayed trajectory. The packet construction at each step is supplied by Theorem 42.4; reality-contained and tractability conclusions are added only when Theorems 42.22 and 42.25 apply. In the infinite case, Theorem 42.14 first converts the seed-library starting condition into membership in the canonical infinite successor-verification domain; Theorem 42.18 then supplies the infinite path. Ability monotonicity follows from the Batch 11 ability-tower corollary under its additional hypotheses. □

Limitation 42.31 (What Batch 12B still does not prove)**.** *Batch 12B still does not prove universal Löbian reflection, arbitrary Vingean successor trust, semantic completeness of the declared goal object, automatic reality containment, arbitrary-state entry into the finite or infinite SV seed-library domain, or unbounded practical tractability. It proves a stronger and more explicit domain-relative result: for a declared class with packet construction, finite and infinite seed-library nonemptiness/closure, reality-containment or misspecification certificates, and tractability certificates when claimed, robust successor verification can be constructed and iterated along finite or infinite certified paths.*

43 Revised List of What Remains After Modules D–G

Modules D–G close several previously informal claims by turning them into conditional theorem modules. The operational-closure layer then abstracts the implementation-facing lessons from the RSI–RCLM architecture paper into typed, auditable, and non-oracular RCP obligations. Batch 11 further adds a seed-domain entry theorem showing how certified conservative-extension libraries and bounded grammar coverage discharge the strict-witness and generator-coverage assumptions for a declared seed class. Batch 12 begins Section 2 by adding a successor-verification seed-domain theorem showing how recursive construction persistence can be strengthened into recursive verification persistence under declared uncertainty, goal-identity, trust, and verifier-schema certificates. Batch 12A expands that theorem into explicit residuals, transport dynamics, soundness composition, and a viability-domain audit. Batch 12B adds the packet-construction, finite seed-library nonemptiness, infinite seed-library closure, infinite-horizon, reality-containment, and tractability-discipline layer required to state a domain-relative Robust Reflective Successor Theorem for declared certified classes. The following list distinguishes what is now theorem-level from what remains architecture-specific or empirical. The RCP-II finalization layer below reframes this status as a substantial conditional endpoint and converts the remaining work into reference-instantiation and empirical-validation tasks rather than a new abstract theorem batch.

Theorem-level closure added by Modules D–G

(1) Semantic register validity is preserved if interpretation maps, reference objects, and sound semantic barriers are supplied.

(2) Human-reference validity for the subjective register is preserved if the human-reference object and subjective barrier are sound.

(3) Self-model fidelity is preserved if the operational self-state audit is sound and the self-model fidelity barrier is included in the gate.

(4) Predicted certificates transfer to realized successors under Lipschitz residuals and sufficient prediction margin.

(5) Verifier bootstrapping is non-circular only when an independent trust anchor certifies soundness and power claims; indefinite uniform verifier-power gain is impossible on a fixed bounded power scale.

(6) Formal admissibility is separated from physical executability by explicit resource-cost constraints, and resource feasibility is tied to conservative cost soundness.

(7) Uniform positive improvement is shown to be impossible forever on a fixed bounded capability scale.

(8) Lyapunov descent with summable error implies Lyapunov-value convergence and summable squared motion.

(9) Stronger target-set descent implies convergence to the declared target coherence set.

Operational closure added from the architecture feedback layer

(1) Density-state reasoning is now type-licensed: quantum, classical, and representational density realizations have different proof licenses.

(2) Representation-to-density embeddings now have explicit density-validity and embedding-transfer residuals.

(3) Semantic coupling is now a formal transfer condition from density-level semantic barriers to externally audited semantic validity.

(4) Certificates are no longer abstract oracle calls: they are one-sided audited packets with calibration data, stress tests, traces, verifier state, and declared failure probabilities.

(5) Verifier self-modification is constrained by a trust graph and a no self-sole-certification rule.

(6) Beneficial progress claims require budgeted certified search and no-op-normalized safe-capability gain certificates.

(7) Resource scaling, no-op dominance, and certificate inflation are explicit monitored residuals rather than informal engineering concerns.

(8) Pair-set incompleteness is represented by a coverage-gap residual and budget.

(9) Coherence-to-capability claims require calibration against an audited capability suite.

(10) Every active residual must be attached to an estimator subprotocol with domain, radius, failure probability, cost, stress tests, and failure mode.

(11) Batch 11 adds a seed-domain theorem: certified conservative-extension libraries, bounded primitive-word grammars, admissible margin ledgers, and seed-persistence certificates imply direct-engine domain entry for the declared horizon.

(12) Batch 12 adds the first successor-verification theorem layer: successor-verification schemas, uncertainty envelopes, semantic/goal-identity transports, verifier-schema persistence certificates, and successor-verification seed domains imply recursive verification-domain invariance for the declared horizon.

(13) Batch 12A completes the first successor-verification formalization by decomposing explicit residuals, verifier-schema transport, soundness composition, uncertainty-envelope transport, goal-identity composition, anti-circular trust residuals, and an algorithmic SV gate.

(14) Batch 12B supplies the first packet-construction and nonemptiness layer for successor verification: $\mathsf{SVBuilder}_t$, $\mathcal{D}_t^{\text{SV-construct}}$, $\mathcal{L}_t^{\text{SV-Wit}}$, $\mathcal{K}_t^{\text{SV-seedlib},N}$, an infinite-horizon domain-relative theorem, reality-containment or misspecification transfer, and restricted tractability certificates.

Still not proved by the manuscript

(1) The manuscript still does not derive the correct semantic interpretation maps $\Gamma_{X,t}$ from first principles.

(2) It does not derive human values or prove that $\mathcal{V}_{H,t}$ is the correct human-reference object.

(3) It does not prove that a concrete architecture can construct the operational certificates, coverage-gap audits, calibration suites, estimator subprotocols, or trust graphs at frontier scale.

(4) It does not construct a universally sound verifier or trust anchor.

(5) It does not prove that a concrete AGI architecture has a nonempty infinite progress viability kernel or a nonempty post-D–G intersection gate. Batch 11 proves seed-domain entry only after a concrete certified module library, bounded grammar, margins, and persistence certificates are supplied.

(6) It does not prove full reflective logic, arbitrary Vingean successor trust, or universal Löbian self-trust. Batch 12 Phase 1 proves only certificate-reflective successor-verification persistence inside the declared $\mathcal{K}_t^{\mathrm{SV},N}$ domain, Batch 12A makes the certificate residuals, transports, and soundness-composition obligations explicit, and Batch 12B constructs finite and infinite successor-verification domains only for declared seed-library classes.

(7) It does not prove scalable estimation of all entropy, mutual-information, barrier, semantic, verifier, and resource certificates in frontier-scale learned systems.

(8) It does not prove unbounded or constant-gain capability growth on a bounded metric.

(9) It does not prove convergence to a unique coherent RSI attractor unless the stronger target-set descent and uniqueness assumptions hold.

Updated fair characterization

With Modules A–G included, the fair claim is:

> RCP is a conditional theorem architecture for gating recursive self-modification so that specified protected safety-relevant distinctions, semantic register validity, self-model fidelity, verifier soundness, resource feasibility, operational certificate soundness, and Lyapunov-style stability are preserved under accepted updates. It becomes a Level 2 to Level 3 bridge only for architectures that can instantiate the semantic maps, barriers, verifiers, typed embeddings, coverage audits, calibration protocols, estimator subprotocols, resource models, certified non-lossy primitive algebra, nonempty progress viability kernels, Batch 11 seed-domain libraries/persistence certificates, Batch 12 successor-verification schemas/uncertainty/goal-identity certificates, Batch 12A explicit residual/transport/soundness-composition certificates, Batch 12B packet-construction and infinite seed-library closure certificates, and the final RCP-II invocation/readiness certificates required by the theorems.

It is not yet an unconditional solution to RSI. It is a rigorously structured conditional bridge specification: any claimed implementation must prove or empirically certify the component hypotheses rather than relying on the coherence of the notation.

44 RCP-II Finalization: Domain-Relative Robust Reflective Successor Theorem Status and Instantiation Boundary

This section is a finalization and instantiation-readiness layer, not a new abstract Batch 13. Batch 12B already contains the abstract theorem machinery for packet construction, finite and infinite seed-library closure, reality-contained uncertainty transfer, and restricted tractability discipline. The purpose here is to surface the theorem status in a publication-ready form, state exactly what a concrete system must supply before invoking it, and identify the post-theorem frontier as finite reference instantiation, implementation, and empirical validation.

44.1 Tier 1: Final domain-relative theorem framing

Definition 44.1 (Final RCP-II theorem framing)**.** *The theorem stack through Batch 12B is licensed to be described as*

A Domain-Relative Robust Reflective Successor Theorem for Certified RCP Seed-Library Classes

when, and only when, the claim explicitly includes the following qualifiers:

domain-relative, *certificate-relative*, *seed-library-relative*, *uncertainty-envelope-relative*,

with reality-containment or misspecification certificates required for real-world reliability and restricted tractability certificates required for computational-tractability claims. The corresponding subtitle-level claim is:

Monotonic, goal-transport-preserving, repeatable self-improvement under declared uncertainty envelopes, with reality-containment and tractability certified separately.

An unqualified assertion that the paper proves a universal robust reflective successor theorem for arbitrary agents, arbitrary successor architectures, or arbitrary environments is not licensed by the results in this manuscript.

Proposition 44.2 (Final framing is conservative)**.** *Definition 44.1 does not add assumptions to the Batch 12B theorem stack and does not strengthen any theorem conclusion. It is a claim-discipline restatement of the hypotheses already required by Batch 11, Batch 12/12A, and Batch 12B.*

Proof. Every qualifier in Definition 44.1 corresponds to a premise or domain object already defined in the theorem stack: Batch 11 seed-domain entry, Batch 12/12A successor-verification residual and transport soundness, Batch 12B packet construction, finite or infinite seed-library closure, reality-containment or misspecification certificates, and restricted tractability certificates. Naming these premises does not alter the mathematical implication; it prevents the conclusion from being stated without its hypotheses. □

44.2 Tier 2: Surfaced final theorem statement

Theorem 44.3 (Surfaced domain-relative RCP-II theorem)**.** *Assume the hypotheses of Theorem 42.30. Then the following two reader-facing implications are valid.*

Finite-horizon form. *If*

$$\rho_0 \in \mathcal{K}_0^{\text{SV-seedlib},N},$$

then the direct engine and successor-verification packet builder construct a finite certified path

$$\rho_0 \xrightarrow{\nu_0} \rho_1 \xrightarrow{\nu_1} \cdots \xrightarrow{\nu_{N-1}} \rho_N$$

with

$$\rho_t \in \mathcal{K}_t^{\text{SV},N} \qquad 0 \le t \le N.$$

Infinite-horizon form. *If*

$$\rho_0 \in \mathcal{K}_0^{\text{SV-seedlib},\infty},$$

and the summability, infinite seed-library closure, and infinite-domain hypotheses of Batch 12B hold, then

$$\rho_t \in \mathcal{K}_t^{\text{SV},\infty,\text{gen}} \qquad \forall t \ge 0.$$

Along either form, the theorem conclusions are conditional on the relevant supplied objects:

$$\mathsf{SVBuilder}_t, \quad \mathsf{SVPkg}_t, \quad \mathsf{GoalId}_t, \quad \mathsf{TrustRef}_t^{\text{SV}},$$
$$\mathsf{SVPersist}_t, \quad \mathsf{RealCont}_t, \quad \mathsf{SVTract}_t$$

whenever the corresponding reliability, trust, persistence, real-world, or tractability claim is made.

Proof. The finite-horizon statement is the finite case of Theorem 42.30. The initial seed-library membership gives membership in $\mathcal{K}_0^{\text{SV},N}$ by Theorem 42.9. Batch 12 invariance then iterates the path through the declared horizon, with packet construction supplied by Theorem 42.4. The infinite-horizon statement is the infinite case of Theorem 42.30 together with Theorem 42.14. The listed objects are exactly the certificate packets required by the invoked theorems and definitions. □

44.3 Tier 3: Minimal finite abstract successor-verification reference instance

Definition 44.4 (Minimal abstract RCP successor-verification reference instance)**.** *A minimal finite abstract reference instance for the RCP-II successor-verification theorem over horizon N is a tuple*

$$\begin{aligned}\mathsf{RefSV}_{0:N}^{\text{RCP}} = (&\rho_0, (\nu_t)_{t<N}, (W_t)_{t<N}, (\mathsf{SVIn}_t)_{t<N}, (\mathsf{SVPkg}_t)_{t<N},\\ &(\mathsf{GoalId}_t)_{t<N}, (\mathsf{TrustRef}_t^{\text{SV}})_{t<N}, (\mathsf{RealCont}_t)_{t<N}, (\mathsf{SVTract}_t)_{t<N}, \mathsf{Trace}_{0:N})\end{aligned}$$

such that:

(i) ρ_0 *is a finite-dimensional state in the declared Batch 11 seed domain;*

(ii) *each* $W_t \in \mathcal{L}_t^{\text{SV-Wit}}(\rho_t)$ *is a valid positive-margin witness;*

(iii) *the bounded certificate grammar enumerates the selected* ν_t *and construction input* $\mathsf{SVIn}_t(\rho_t, \nu_t)$*;*

(iv) $(\rho_t, \nu_t) \in \mathcal{D}_t^{\text{SV-construct}}$ *and* $\mathsf{SVBuilder}_t$ *returns* $\mathsf{SVPkg}_t(\rho_t, \nu_t)$*;*

(v) *the selected update realizes* $\rho_{t+1} = \Phi_{\nu_t}(\rho_t)$*;*

(vi) *each successor packet proves* $\rho_{t+1} \in \mathcal{K}_{t+1}^{\text{SV},N}$*;*

(vii) $\mathsf{Trace}_{0:N}$ *records all checker outputs, residual radii, proof/checking costs, and transport distortions needed to audit the instance.*

The reference instance is abstract and finite. It is not an empirical trained-system implementation unless the states, witnesses, builders, and certificates are instantiated by an actual system.

Theorem 44.5 (Reference instance implies finite theorem invocation)**.** *If* $\mathsf{RefSV}_{0:N}^{\mathrm{RCP}}$ *is valid in the sense of Definition 44.4, then*

$$\rho_0 \in \mathcal{K}_0^{\text{SV-seedlib},N}.$$

Consequently, Theorem 44.3 applies on the finite horizon $0 \le t \le N$*.*

Proof. The clauses of Definition 44.4 match the finite seed-library requirements in Definition 42.8: a Batch 11 seed state, a valid positive-margin SV witness, bounded grammar coverage, packet-construction domain membership, a sound builder output, and successor-membership evidence. Therefore $\rho_0 \in \mathcal{K}_0^{\text{SV-seedlib},N}$. The finite theorem invocation is then Theorem 44.3. □

Limitation 44.6 (Reference instance is not frontier-scale validation)**.** *A valid* $\mathsf{RefSV}_{0:N}^{\mathrm{RCP}}$ *demonstrates that the theorem stack has a finite formal model and is not merely syntactic. It does not show that a large learned system naturally enters the same domain, that the witness library is useful at scale, or that the declared uncertainty envelope contains the real environment.*

44.4 Tier 4: RCP-II theorem-invocation and instantiation boundary

Definition 44.7 (RCP-II theorem-invocation boundary)**.** *A concrete system may invoke the domain-relative RCP-II theorem only after supplying, for the relevant horizon, the following certified objects:*

(i) finite or infinite membership in $\mathcal{K}_t^{\text{SV-seedlib},N}$ *or* $\mathcal{K}_t^{\text{SV-seedlib},\infty}$*;*

(ii) a sound successor-verification packet builder $\mathsf{SVBuilder}_t$ *and builder trace;*

(iii) valid packets SVPkg_t *with explicit residual vector* $Q_t^{\mathrm{SV},A} \le 0$*;*

(iv) goal-identity transport certificates GoalId_t*, anti-circular trust certificates* $\mathsf{TrustRef}_t^{\mathrm{SV}}$*, and successor-persistence certificates* $\mathsf{SVPersist}_t$*;*

(v) type, resource, transport, projection-realization, soundness, and certificate-budget ledgers required by the inherited Batch 1–12A gates;

(vi) $\mathsf{RealCont}_t$ *or a bounded misspecification certificate when any claim refers to the actual environment rather than only to the declared envelope;*

(vii) $\mathsf{SVTract}_t$ *when any claim uses the phrase computationally tractable in the strong finite or asymptotic sense.*

If any item is absent, the system may still discuss the corresponding object as a target or assumption, but it may not cite the theorem as established for that implementation.

Proposition 44.8 (Boundary prevents theorem overuse)**.** *If a concrete implementation lacks any object required by Definition 44.7, then any theorem conclusion depending on that object is not licensed for that implementation.*

Proof. Each required object is a premise of a theorem or definition in the Batch 11, Batch 12, Batch 12A, or Batch 12B stack. Removing a premise invalidates the corresponding implication. Therefore the theorem can be invoked only for the portion of the claim whose premises have actually been supplied. □

44.5 Tier 5: Main-goal fit and qualification table

Main-goal component	**RCP-II final status and qualification**
Monotonic self-improvement	Conditional on Batch 11 direct-engine seed entry, certified ability preservation/expansion, and the declared Batch 12B seed-library/witness assumptions.
Goal preservation	Achieved as bounded declared semantic/goal-identity drift $D^U_{\mathrm{sem}}(U_{t+1}, \Theta^U_t U_t) \leq \delta^U_t$ and its composed finite or summable infinite-horizon version, not as proof that the declared goal object is philosophically complete.
Repeatability	Achieved along finite certified paths and along infinite generated paths when $\mathcal{K}_t^{\text{SV-seedlib},\infty}$, summability, and persistence certificates are supplied.
Under uncertainty	Envelope-relative by default. Real-world reliability additionally requires $\mathsf{RealCont}_t$ or a bounded misspecification certificate.
Physically grounded	Typed, resource-ledgered, projection-realization-aware, and reality-containment-aware. This is operational/mathematical grounding, not empirical physical validation by itself.
Computationally tractable	Strong tractability is licensed only by $\mathsf{SVTract}_t$. Without it, the correct claim is resource-accounted or budgeted verification.
Reflectively stable	Certificate-reflective and anti-circular through no-self-sole-certification, common-mode, collusion, leakage, anchor, and reflective-link residuals; not a full universal Löbian or arbitrary Vingean successor-trust theorem.

44.6 Tier 6: Final implementation and research roadmap

The theorem stack is now abstractly complete enough that the next frontier is instantiation pressure rather than another abstract theorem batch. The post-theorem roadmap is:

(A) **Semantic/reference instantiation:** instantiate predicate universes, human-reference objects, semantic coupling maps, and distribution-shift-stable transports.

(B) **Estimator/certificate instantiation:** build calibrated one-sided residual estimators, held-out audit families, stress-test families, and simultaneous soundness accounting.

(C) **SV witness-library instantiation:** construct useful finite witness libraries, bounded proof/certificate grammars, and builder traces for concrete systems.

(D) **Reality-containment and tractability validation:** validate $\mathsf{RealCont}_t$, misspecification bounds, $\mathsf{SVTract}_t$, and resource/cost ledgers beyond toy examples.

(E) **Reflective-trust instantiation:** provide independent trust anchors, anti-collusion audits, common-mode failure analysis, audit-leakage controls, and verifier-drift detection.

(F) **Finite reference and non-toy empirical evaluation:** begin with $\mathsf{RefSV}^{\mathrm{RCP}}_{0:N}$, then instantiate architecture-specific reference systems, and only then move to trained-system experiments.

44.7 Tier 7: No abstract Batch 13 at this stage; Batch 13R is reference instantiation

Limitation 44.9 (No new abstract Batch 13 yet)**.** *The next step should not be another abstract theorem batch unless a new formal gap is identified. Batch 12B plus the finalization layer already states the domain-relative robust reflective successor theorem, the finite/infinite seed-library entry conditions, reality-containment and tractability qualifications, and the invocation boundary. The immediate next frontier is finite reference instantiation, implementation, and empirical validation. The Batch 13R section below is therefore not an abstract Batch 13: it is a reference-entry / realization layer that constructs and checks finite proof-carrying reference paths under explicit finite-class, checker, compiler, artifact, reality, and tractability hypotheses.*

Interpretation 44.10 (Substantial conditional endpoint)**.** *At this point the math paper should be read as a substantial conditional endpoint: a domain-relative, certificate-relative theorem architecture for robust reflective successor verification. Its strongest honest claim is not that arbitrary systems self-improve safely, but that certified systems in the declared seed-library classes can be made monotonic, goal-transport-preserving, and repeatable under declared uncertainty envelopes, with reality-containment and tractability certified separately.*

45 Batch 13R: Mechanized Reference-Entry, Proof-Carrying Successor Packets, and Finite Certified Trajectories

Batch 13R is the reference-instantiation phase. The letter "R" means reference, realization, and replay. It is not a new abstract Batch 13 and it does not strengthen the theorem conclusions for arbitrary systems. Its purpose is to close the non-vacuity gap for a declared finite class by constructing the objects that the Batch 12B theorem stack previously treated as the invocation boundary.

The target is the reference-entry implication

$$\boxed{\mathsf{BuildRefSV}^{\mathrm{RCP}}_{0:N}(\mathcal{M},\mathcal{D},\mathcal{L},\mathcal{C}) \Downarrow \mathsf{RefSV}^{\mathrm{RCP}}_{0:N} \Longrightarrow \rho_0 \in \mathcal{K}_0^{\text{SV-seedlib},N}.}$$

Here $\mathcal{M}$ is a finite reference system, $\mathcal{D}$ is the calibration/audit/stress-test and optional environment-data package, $\mathcal{L}$ is the finite successor-verification witness library, and $\mathcal{C}$ is the trusted checker or proof-kernel package. The associated path theorem is the checked finite-trajectory statement

$$\boxed{\forall t < N, \quad \mathsf{Check}_{\mathrm{RCP}}(\mathsf{PCS}_t(\nu_t)) = 1 \Longrightarrow \rho_t \in \mathcal{K}_t^{\mathrm{SV},N},}$$

read with the prefix, construction, and realization hypotheses stated below. Without those hypotheses, a single checker call at time t is not enough to infer the whole prefix invariant.

Limitation 45.1 (Batch 13R is reference entry, not arbitrary-system entry)**.** *Batch 13R proves that a successfully built finite reference instance enters the certified successor-verification seed-library domain and generates a checked finite path. It does not prove that arbitrary trained systems, arbitrary learned RCLM systems, or arbitrary stronger successors naturally enter that domain. It also does not assert actual machine-checked verification unless a separate mechanization certificate is supplied.*

45.1 Rank 1 / Tier 1: finite reference class and proof-carrying successor packets

Definition 45.2 (Finite RCP reference class)**.** *Fix positive integers or size parameters n, k, N. A finite RCP reference-class object is a tuple*

$$\mathcal{M} \in \mathsf{RCPRefClass}_{n,k,N} = (\mathcal{H}_{0:N}, \rho_0, \mathcal{P}_{0:N}, \mathfrak{A}_{0:N}, R_{0:N}, \Omega^{\mathrm{ref}}_{0:N}, \Phi_{0:N}, \mathcal{R}^{\mathrm{rec}}_{0:N}, \mathcal{L}^{\text{SV-Wit}}_{0:N}, \mathfrak{G}^{\text{SV-cert}}_{0:N}, \mathsf{VerSch}^{\mathrm{SV}}_{0:N}, \mathcal{P}^{\mathrm{SV}}_{0:N}, U_{0:N}, \mathsf{Budget}_{0:N})$$

satisfying the following finite-class constraints:

(i) *each $\mathcal{H}_t$ is finite-dimensional with declared representation size at most n;*

(ii) *Ω^{ref}_t is a finite candidate family whose primitive words have length at most k;*

(iii) *$\mathfrak{G}^{\text{SV-cert}}_t$ is a finite certificate grammar whose proof/checker traces are bounded by a declared length parameter;*

(iv) *$\mathcal{L}^{\text{SV-Wit}}_t(\rho)$ is a finite witness library with a decidable validity predicate on the audited reference states;*

(v) *every state, update, recovery map, verifier schema, goal object, trust object, uncertainty envelope, and budget object is typed and belongs to the finite audited domain;*

(vi) *no infinite-dimensional compactness, unbounded search, or informal semantic transfer is used by the reference-entry theorem.*

Membership in $\mathsf{RCPRefClass}_{n,k,N}$ is a finite data/certificate claim. It is not a claim that the system is a trained large-scale AI system.

Definition 45.3 (Proof-carrying successor packet)**.** *For a state ρ_t, candidate ν_t, and realized successor $\rho_{t+1} = \Phi_{\nu_t}(\rho_t)$, a proof-carrying successor packet is*

$$\begin{aligned}\mathsf{PCS}_t(\nu_t) = (&\nu_t, \Phi_{\nu_t}, \rho_t, \rho_{t+1}, \mathcal{R}^{\mathrm{rec}}_{\nu_t}, \mathsf{Proof}^{\mathrm{Type}}_t, \mathsf{Proof}^{\mathrm{NL}}_t, \mathsf{Proof}^{\mathrm{Rec}}_t,\\ &\mathsf{Proof}^{\mathfrak{A}}_t, \mathsf{Proof}^{\mathrm{Seed}}_t, \mathsf{Proof}^{\mathrm{SV}}_t, \mathsf{Proof}^{Q}_t, \mathsf{Proof}^{\mathrm{GoalId}}_t, \mathsf{Proof}^{\mathrm{Trust}}_t,\\ &\mathsf{Proof}^{\mathrm{RealCont}}_t, \mathsf{Proof}^{\mathrm{Tract}}_t, \mathsf{Proof}^{\mathrm{Persist}}_t, \mathsf{Trace}_t).\end{aligned}$$

The entries certify, respectively, typed realization, protected non-loss, recovery, ability preservation and optional strict ability expansion, Batch-11 seed validity, Batch-12/12A successor-verification packet validity, explicit residual nonpositivity $Q^{\mathrm{SV},A}_t \le 0$, goal-identity transport, anti-circular trust, optional reality-containment or bounded misspecification, optional restricted tractability, successor-domain persistence, and an audit trace containing verifier identities, hashes, radii, margins, costs, and random seeds or enumeration traces.

Definition 45.4 (Trusted RCP checker). *A trusted RCP checker for the finite reference class is a total finite procedure*

$$\mathsf{Check}_{\mathrm{RCP}} : \mathsf{PCS}_t(\nu_t) \longrightarrow \{0,1\}$$

which returns 1 *only if every active proof component in Definition 45.3 is well typed, refers to the same* $(\rho_t, \nu_t, \rho_{t+1})$*, uses the declared finite witness and certificate grammar, charges all approximation radii to the corresponding margins, and verifies the finite arithmetic or proof obligations recorded in* Trace_t*. If a component is outside the finite audited domain, references a stale verifier, omits a radius, or makes a real-world or tractability claim without the corresponding certificate, the checker must return* 0*.*

Definition 45.5 (Checker soundness certificate). *A checker soundness certificate is a tuple*

$$\mathsf{CheckSound}_{\mathrm{RCP}} = (\mathsf{KernelSpec}, \mathsf{RuleSpec}, \mathsf{RuleSound}, \mathsf{MechCert}_{\mathrm{RCP}}, \delta^{\mathrm{chk}})$$

where KernelSpec *specifies the trusted kernel,* RuleSpec *specifies the inference and arithmetic rules used by* $\mathsf{Check}_{\mathrm{RCP}}$*,* RuleSound *certifies that each rule is sound for the formal objects in this manuscript,* $\mathsf{MechCert}_{\mathrm{RCP}}$ *is optional evidence that the checker or proof kernel has been machine-checked externally, and* δ^{chk} *is the declared residual failure probability if the checker depends on stochastic audits or randomized tests. If* $\mathsf{MechCert}_{\mathrm{RCP}}$ *is absent, the result is mechanization-ready and proof-carrying, but not claimed to be mechanically verified.*

Assumption 45.6 (Trusted checker soundness). *On the checker-soundness event* $\mathcal{E}_t^{\mathrm{chk}}$*, if*

$$\mathsf{Check}_{\mathrm{RCP}}(\mathsf{PCS}_t(\nu_t)) = 1,$$

then every object accepted by the checker satisfies the corresponding true finite reference obligation: typed state/update validity, protected non-loss, recovery, ability preservation, seed validity, successor-verification packet validity, explicit residual nonpositivity, goal-identity drift bound, trust residual nonpositivity, successor persistence, and any optional reality-containment or tractability claim that the packet includes.

Theorem 45.7 (Proof-carrying successor checker soundness). *Assume trusted checker soundness. If*

$$\mathsf{Check}_{\mathrm{RCP}}(\mathsf{PCS}_t(\nu_t)) = 1$$

on the checker-soundness event, then the candidate ν_t *satisfies the finite reference instance of the RCP successor-verification admissibility conditions. In particular,*

$$Q_t^{\mathrm{SV},A}(\nu_t; \rho_t) \le 0, \qquad \mathsf{SVPkg}_t(\rho_t, \nu_t) \text{ is valid},$$
$$D_{\mathrm{sem}}^{U}(U_{t+1}, \Theta_t^U U_t) \le \delta_t^U, \qquad \Phi_{\nu_t}(\rho_t) \in \mathcal{K}_{t+1}^{\mathrm{SV},N}$$

whenever those clauses are active in the finite reference packet.

Proof. The checker returns 1 only for a packet whose components are well typed, mutually consistent, and verified according to Definition 45.4. Assumption 45.6 states that checker acceptance implies the truth of the corresponding finite reference obligations. The displayed conclusions are exactly the successor-verification residual, packet-validity, goal-identity, and persistence obligations listed in Definition 45.3. No conclusion is made for optional claims, such as real-world reliability or strong tractability, unless the optional proof component is present and accepted. □

Limitation 45.8 (Mechanization wording). *The word "mechanized" in Batch 13R denotes a mechanization-ready checker interface and proof-carrying packet discipline unless* $\mathsf{MechCert}_{\mathrm{RCP}}$ *is actually supplied. A manuscript-level definition of* $\mathsf{Check}_{\mathrm{RCP}}$ *is not the same thing as an external Lean, Coq, Isabelle, Agda, or comparable machine-checked proof artifact.*

45.2 Rank 2 / Tier 2: certificate compiler and reference-entry theorem

Definition 45.9 (Reference certificate compiler). *A reference certificate compiler is a partial finite procedure*

$$\mathsf{BuildRefSV}_{0:N}^{\mathrm{RCP}}(\mathcal{M},\mathcal{D},\mathcal{L},\mathcal{C})$$

where $\mathcal{M} \in \mathsf{RCPRefClass}_{n,k,N}$, $\mathcal{D}$ is the finite audit/calibration/stress-test and optional environment-data package, $\mathcal{L}$ is a finite SV witness library, and $\mathcal{C} = (\mathsf{Check}_{\mathrm{RCP}}, \mathsf{CheckSound}_{\mathrm{RCP}})$ is the checker package. The notation

$$\mathsf{BuildRefSV}_{0:N}^{\mathrm{RCP}}(\mathcal{M},\mathcal{D},\mathcal{L},\mathcal{C}) \Downarrow \mathsf{RefSV}_{0:N}^{\mathrm{RCP}}$$

means that the compiler terminates and returns a finite object

$$\begin{aligned}\mathsf{RefSV}_{0:N}^{\mathrm{RCP}} = (&\rho_0, (\rho_t)_{t\le N}, (\nu_t)_{t<N}, (W_t)_{t<N}, (\mathsf{SVIn}_t)_{t<N},\\ &(\mathsf{SVPkg}_t)_{t<N}, (\mathsf{PCS}_t)_{t<N}, (\mathsf{GoalId}_t)_{t<N}, (\mathsf{TrustRef}_t^{\mathrm{SV}})_{t<N},\\ &(\mathsf{RealCont}_t)_{t<N}, (\mathsf{SVTract}_t)_{t<N}, \mathsf{Trace}_{0:N}, \mathsf{BuildTrace}_{0:N}).\end{aligned}$$

The build trace must certify all of the following:

(i) ρ_0 *lies in the finite Batch-11 reference seed class;*

(ii) *every* $W_t \in \mathcal{L}$ *is a valid positive-margin SV witness at* ρ_t*;*

(iii) *the finite certificate grammar enumerates* ν_t *and* $\mathsf{SVIn}_t(\rho_t,\nu_t)$*;*

(iv) $(\rho_t,\nu_t) \in \mathcal{D}_t^{\text{SV-construct}}$ *and the builder returns* SVPkg_t*;*

(v) $\rho_{t+1} = \Phi_{\nu_t}(\rho_t)$*;*

(vi) $\mathsf{Check}_{\mathrm{RCP}}(\mathsf{PCS}_t(\nu_t)) = 1$ *for every* $t < N$*;*

(vii) *every optional reality-containment or tractability claim is either supplied and checked, or explicitly marked absent.*

Theorem 45.10 (Mechanization-ready certified reference-entry theorem). *Assume $\mathcal{M} \in \mathsf{RCPRefClass}_{n,k,N}$, a sound finite witness library $\mathcal{L}$, a valid checker package $\mathcal{C}$, and trusted checker soundness on the build trace. If*

$$\boxed{\mathsf{BuildRefSV}_{0:N}^{\mathrm{RCP}}(\mathcal{M},\mathcal{D},\mathcal{L},\mathcal{C}) \Downarrow \mathsf{RefSV}_{0:N}^{\mathrm{RCP}},}$$

then

$$\boxed{\rho_0 \in \mathcal{K}_0^{\text{SV-seedlib},N}.}$$

Consequently the finite-horizon form of Theorem 44.3 is invocable for the returned reference instance.

Proof. The compiler termination statement gives a finite returned object and a build trace satisfying Definition 45.9. The build trace includes finite Batch-11 seed membership, valid positive-margin SV witnesses, bounded grammar enumeration, construction-domain membership, builder outputs, realized successors, and checked proof-carrying successor packets. By Theorem 45.7, every checked packet supplies the true finite reference obligations required by the finite seed-library domain: valid witness, construction input, packet construction, explicit residual nonpositivity, and successor-membership evidence. These are exactly the clauses of Definition 42.8. Hence $\rho_0 \in \mathcal{K}_0^{\text{SV-seedlib},N}$. The final sentence follows by Theorem 44.3. □

Corollary 45.11 (Reference non-vacuity for the finite seed-library domain)**.** *If the hypotheses of Theorem 45.10 hold, then the finite seed-library domain* $\mathcal{K}_0^{\text{SV-seedlib},N}$ *is nonempty.*

Proof. The theorem supplies an explicit element ρ_0 of $\mathcal{K}_0^{\text{SV-seedlib},N}$. □

45.3 Rank 3 / Tier 3: proof-carrying finite certified trajectories

Definition 45.12 (Checked finite successor trajectory)**.** *A checked finite successor trajectory over horizon* N *is a sequence*

$$\rho_0 \xrightarrow{\nu_0} \rho_1 \xrightarrow{\nu_1} \cdots \xrightarrow{\nu_{N-1}} \rho_N$$

with proof-carrying packets $\mathsf{PCS}_t(\nu_t)$ *such that*

$$\rho_{t+1} = \Phi_{\nu_t}(\rho_t), \qquad \mathsf{Check}_{\mathrm{RCP}}(\mathsf{PCS}_t(\nu_t)) = 1 \quad (0 \le t < N),$$

and the build trace links each packet to the same finite reference class, witness library, checker package, and horizon.

Theorem 45.13 (Proof-carrying finite certified trajectory)**.** *Assume the hypotheses of Theorem 45.10. Let*

$$\mathsf{BuildRefSV}_{0:N}^{\mathrm{RCP}}(\mathcal{M}, \mathcal{D}, \mathcal{L}, \mathcal{C}) \Downarrow \mathsf{RefSV}_{0:N}^{\mathrm{RCP}}$$

return a checked finite successor trajectory. Then

$$\rho_t \in \mathcal{K}_t^{\mathrm{SV},N} \qquad 0 \le t \le N.$$

Equivalently, for every prefix event

$$\mathsf{PrefixCheck}_t := \bigwedge_{s=0}^{t-1} \left[\mathsf{Check}_{\mathrm{RCP}}(\mathsf{PCS}_s(\nu_s)) = 1 \ \wedge \ \rho_{s+1} = \Phi_{\nu_s}(\rho_s)\right],$$

we have

$$\boxed{\mathsf{PrefixCheck}_t \Longrightarrow \rho_t \in \mathcal{K}_t^{\mathrm{SV},N}.}$$

Thus the shorthand

$$\boxed{\forall t < N, \quad \mathsf{Check}_{\mathrm{RCP}}(\mathsf{PCS}_t(\nu_t)) = 1 \Longrightarrow \rho_t \in \mathcal{K}_t^{\mathrm{SV},N}}$$

is licensed only relative to the constructed reference path and its already-checked prefix.

Proof. Theorem 45.10 gives $\rho_0 \in \mathcal{K}_0^{\text{SV-seedlib},N}$. By Theorem 44.3, this implies $\rho_0 \in \mathcal{K}_0^{\mathrm{SV},N}$ and licenses the finite Batch-12B theorem for the reference instance. For the induction step, suppose $\rho_t \in \mathcal{K}_t^{\mathrm{SV},N}$ and the prefix checker has accepted through time t. Theorem 45.7 gives the valid packet and successor-persistence clause $\rho_{t+1} = \Phi_{\nu_t}(\rho_t) \in \mathcal{K}_{t+1}^{\mathrm{SV},N}$. Induction proves the statement for all $t \le N$. The final boxed statement is explicitly marked as a shorthand because the domain membership at time t depends on the initial reference entry and all preceding checked transitions, not on an isolated checker call outside the path. □

Corollary 45.14 (Multi-step certified restricted-Scenario-3 witness). *If $N \geq 2$ and at least one checked step certifies strict ability expansion,*

$$\mathfrak{A}_{t+1}(\rho_{t+1}, R_{t+1}) \setminus \Theta_t^{\mathfrak{A}}(\mathfrak{A}_t(\rho_t, R_t)) \neq \varnothing,$$

then the returned reference instance is a nontrivial multi-step finite witness of recoverable-monotone, successor-verification-preserving self-improvement inside the declared certified seed-library class.

Proof. Theorem 45.13 gives the multi-step successor-verification domain invariant. The displayed strict ability expansion clause supplies nontrivial progress at the certified step. Recoverable non-loss and successor-verification persistence are supplied by the checked proof-carrying packets. The witness remains restricted to the finite reference class because all invoked hypotheses are finite-class and certificate-relative. □

45.4 Restricted reference tractability and reference-contained uncertainty

Definition 45.15 (Reference tractability certificate). *A reference tractability certificate for* $\mathsf{BuildRefSV}_{0:N}^{\mathrm{RCP}}$ *is a tuple*

$$\mathsf{RefTract}_{0:N}^{\mathrm{RCP}} = (n, k, N, \ell, s, \delta^{\mathrm{tol}}, P_{\mathrm{ref}}, C_{\mathrm{build}}, C_{\mathrm{check}}, C_{\mathrm{proof}}, C_Q)$$

where ℓ bounds proof/checker trace length, s bounds audit/sample size, and P_{ref} is a declared polynomial or explicit finite bound such that

$$C_{\mathrm{build}} + C_{\mathrm{check}} + C_{\mathrm{proof}} + C_Q \leq P_{\mathrm{ref}}(n, k, N, \ell, s, \log(1/\delta^{\mathrm{tol}})).$$

The certificate is valid only if it accounts for witness-library enumeration, certificate-grammar enumeration, construction of SVIn_t, construction of PCS_t, checker calls, explicit residual-vector evaluation, goal/trust/reality/tractability certificate checks, and trace replay over the finite horizon.

Theorem 45.16 (Restricted reference tractability). *If $\mathsf{RefTract}_{0:N}^{\mathrm{RCP}}$ is valid for $\mathcal{M} \in \mathsf{RCPRefClass}_{n,k,N}$, then the reference compiler, checker, proof-carrying packet construction, and explicit residual-vector evaluation are computationally tractable relative to the declared finite reference class and cost model:*

$$\begin{aligned}&\mathrm{Cost}(\mathsf{BuildRefSV}_{0:N}^{\mathrm{RCP}}) + \sum_{t<N} \mathrm{Cost}(\mathsf{Check}_{\mathrm{RCP}}(\mathsf{PCS}_t)) + \sum_{t<N} \mathrm{Cost}(Q_t^{\mathrm{SV},A}) \\ &\quad \leq P_{\mathrm{ref}}(n, k, N, \ell, s, \log(1/\delta^{\mathrm{tol}})).\end{aligned}$$

If no valid $\mathsf{RefTract}_{0:N}^{\mathrm{RCP}}$ is supplied, the reference instance is only finite and resource-accounted, not strongly tractability-certified.

Proof. The displayed bound is exactly the aggregate bound required by Definition 45.15. The theorem applies only to inputs inside the declared finite reference class and only to costs included in the certificate. □

Definition 45.17 (Reference-contained uncertainty certificate). *A reference-contained uncertainty certificate is*

$$\mathsf{RefRealCont}_{0:N}^{\mathrm{RCP}} = (P_{0:N}^{\mathrm{true}}, \mathcal{P}_{0:N}^{\mathrm{SV}}, d_{\mathcal{P}}, \varepsilon_{0:N}^{\mathrm{env}}, \beta_{0:N}^{\mathrm{env}}, \Gamma_{0:N}^{\mathrm{env}}, \mathsf{EnvTrace}_{0:N})$$

certifying, for every $t < N$, either

$$P(P_t^{\mathrm{true}} \in \mathcal{P}_t^{\mathrm{SV}}) \geq 1 - \beta_t^{\mathrm{env}}$$

or

$$d_{\mathcal{P}}(P_t^{\mathrm{true}}, \mathcal{P}_t^{\mathrm{SV}}) \leq \varepsilon_t^{\mathrm{env}},$$

together with a modulus Γ_t^{env} transferring envelope-relative residual or expectation bounds to the reference true law.

Theorem 45.18 (Reference-contained uncertainty transfer)**.** *Assume $\mathsf{RefRealCont}_{0:N}^{\mathrm{RCP}}$ is valid and the reference trajectory satisfies the Batch-12A successor-verification bounds over $\mathcal{P}_t^{\mathrm{SV}}$. Then every envelope-relative bound transfers to the reference true law with either no additional error in the containment case, or with additional error at most $\Gamma_t^{\mathrm{env}}(\varepsilon_t^{\mathrm{env}})$ in the misspecification case. In particular, any checked Ψ^{SV}-progress bound becomes*

$$\mathbb{E}_{P_t^{\mathrm{true}}}\left[\Psi_{t+1}^{\mathrm{SV}}(\rho_{t+1}) - \Theta_t^{\Psi}\Psi_t^{\mathrm{SV}}(\rho_t)\right] \geq \epsilon_t^{\mathrm{SV}} - \eta_t^{\mathrm{SV}} - \Gamma_t^{\mathrm{env}}(\varepsilon_t^{\mathrm{env}}).$$

Proof. If $P_t^{\mathrm{true}} \in \mathcal{P}_t^{\mathrm{SV}}$, the robust envelope quantifier already includes the true law. If only bounded misspecification is certified, the modulus Γ_t^{env} bounds the discrepancy between an admissible envelope law and the true law. Subtracting that discrepancy from the envelope-relative lower bound gives the displayed inequality. The same argument applies componentwise to all residuals whose stability modulus is included in $\mathsf{RefRealCont}_{0:N}^{\mathrm{RCP}}$. □

45.5 Rank 8: non-toy executable artifact theorem

Definition 45.19 (Executable RCP reference artifact)**.** *An executable RCP reference artifact over horizon N is a tuple*

$$\mathsf{Artifact}_{0:N}^{\mathrm{RCP}} = (\mathsf{Source}, \mathsf{Build}, \mathsf{Inputs}, \mathsf{CheckerBin}, \mathsf{ProofScripts}, \mathsf{RunLog}, \mathsf{Hashes}, \mathsf{Output})$$

where the source, build recipe, finite inputs, checker executable or proof scripts, run log, hashes, and output are sufficient for an auditor to replay the construction of $\mathsf{RefSV}_{0:N}^{\mathrm{RCP}}$. The artifact is non-toy in the limited formal sense used here when:

(i) $N \geq 2$*;*

(ii) the path contains at least one strict certified ability expansion;

(iii) at least one successor-verification packet is non-no-op and proof-carrying;

(iv) the checker verifies explicit residuals rather than accepting by definition;

(v) the artifact reports finite costs and replayable traces.

This definition of non-toy is still formal and controlled. It is not a claim of frontier-scale empirical validation.

Definition 45.20 (Artifact validity certificate)**.** *An artifact validity certificate*

$$\mathsf{ArtCert}_{0:N}^{\mathrm{RCP}}(\mathsf{Artifact}_{0:N}^{\mathrm{RCP}})$$

certifies that the source and build hashes match, the checker used in the run is the checker specified by $\mathcal{C}$, every checker call in RunLog *returns* 1*, the output equals the declared* $\mathsf{RefSV}^{\mathrm{RCP}}_{0:N}$*, the cost records match* $\mathsf{RefTract}^{\mathrm{RCP}}_{0:N}$ *when tractability is claimed, and the optional environment trace matches* $\mathsf{RefRealCont}^{\mathrm{RCP}}_{0:N}$ *when reality containment is claimed.*

Theorem 45.21 (Non-toy executable artifact implies reference entry)**.** *Assume a valid executable artifact* $\mathsf{Artifact}^{\mathrm{RCP}}_{0:N}$*, a valid artifact certificate* $\mathsf{ArtCert}^{\mathrm{RCP}}_{0:N}$*, and trusted checker soundness. If replaying the artifact produces*

$$\mathsf{BuildRefSV}^{\mathrm{RCP}}_{0:N}(\mathcal{M},\mathcal{D},\mathcal{L},\mathcal{C}) \Downarrow \mathsf{RefSV}^{\mathrm{RCP}}_{0:N},$$

then

$$\rho_0 \in \mathcal{K}_0^{\text{SV-seedlib},N} \quad \text{and} \quad \rho_t \in \mathcal{K}_t^{\mathrm{SV},N} \quad (0 \le t \le N).$$

If the artifact also satisfies the non-toy clauses in Definition 45.19, then it is a finite nontrivial executable witness of the restricted Scenario-3 theorem inside the declared certified seed-library class.

Proof. The artifact certificate states that replay produces the same $\mathsf{RefSV}^{\mathrm{RCP}}_{0:N}$, that the checker calls return 1, and that the output and traces match the declared compiler, checker, and certificate objects. Theorem 45.10 gives $\rho_0 \in \mathcal{K}_0^{\text{SV-seedlib},N}$. Theorem 45.13 gives $\rho_t \in \mathcal{K}_t^{\mathrm{SV},N}$ over the finite horizon. The final sentence follows by the non-toy clauses, which require at least one strict certified ability expansion and proof-carrying non-no-op successor step. □

Limitation 45.22 (Artifact theorem scope)**.** *The artifact theorem is a replayability and finite-reference-entry theorem. It does not prove that trained systems naturally generate such artifacts, that all hidden implementation bugs are absent, that the reference environment is the real deployment environment, or that the finite witness scales. It changes the status from a purely conditional theorem stack to a constructed finite checked witness only within the declared reference class.*

45.6 Batch 13R-A: concrete finite reference witness and checker-realization patch

The preceding Batch 13R interface closes the reference-entry gap only conditionally: it says what follows if a compiler returns a valid $\mathsf{RefSV}^{\mathrm{RCP}}_{0:N}$, if the checker is sound, and if the proof-carrying packets are valid. This subsection adds the remaining finite-reference rigor patch. It constructs a canonical finite reference instance, defines a concrete finite checker for that instance, proves checker soundness without appealing to an unspecified oracle, proves a nontrivial multi-step certified trajectory for every $N \ge 2$, and gives explicit tractability and reality-containment witnesses for the same finite class.

The construction is intentionally simple. It is not a trained-system claim. Its purpose is to prove non-vacuity of the Batch-13R theorem layer by exhibiting a fully specified finite witness inside the declared reference class.

Definition 45.23 (Canonical appended-module reference system)**.** *Fix a horizon $N \ge 2$. Let*

$$\mathcal{H}_t^{\mathrm{can}} = \mathbb{C}^2 \otimes (\mathbb{C}^2)^{\otimes t}, \qquad 0 \le t \le N,$$

and let $\alpha_t = |1\rangle\langle 1| \in \mathcal{D}(\mathbb{C}^2)$. Define the initial full-support diagonal protected states

$$\rho_0^{\mathrm{can}} = \mathrm{diag}(2/3, 1/3), \qquad \sigma_0^{\mathrm{can}} = \mathrm{diag}(1/3, 2/3),$$

and recursively

$$\rho_{t+1}^{\mathrm{can}} = \rho_t^{\mathrm{can}} \otimes \alpha_t, \qquad \sigma_{t+1}^{\mathrm{can}} = \sigma_t^{\mathrm{can}} \otimes \alpha_t.$$

The protected pair family is the singleton

$$\mathcal{P}_t^{\mathrm{can}} = \{(\rho_t^{\mathrm{can}}, \sigma_t^{\mathrm{can}})\}.$$

The candidate at step t is the append-only conservative-extension word

$$\nu_t^{\mathrm{can}} = \mathrm{AppendMem}(a_{t+1}), \qquad \Phi_t^{\mathrm{can}}(X) = X \otimes \alpha_t,$$

with recovery map

$$\mathcal{R}_t^{\mathrm{can}}(Y) = \mathrm{Tr}_{t+1}(Y),$$

where Tr_{t+1} traces out the newly appended last qubit. The ability universe is

$$\mathsf{Ab}_t^{\mathrm{can}} = \{a_0, a_1, \ldots, a_t\},$$

with transport

$$\Theta_t^{\mathfrak{A},\mathrm{can}}(a_j) = a_j \qquad (0 \le j \le t),$$

and the appended module certifies the new ability a_{t+1}. The goal object is constant,

$$U_t^{\mathrm{can}} = U_0^{\mathrm{can}}, \qquad \Theta_t^{U,\mathrm{can}} = \mathrm{id},$$

the uncertainty envelope is the singleton

$$\mathcal{P}_t^{\mathrm{SV,can}} = \{P_t^{\mathrm{true,can}}\},$$

where $P_t^{\mathrm{true,can}}$ is the deterministic law of the finite transition $\rho_t^{\mathrm{can}} \mapsto \rho_{t+1}^{\mathrm{can}}$, and the trust anchor is a frozen checker node $C_\star$ not modified by any ν_t^{can}.

Lemma 45.24 (Canonical conservative extension has exact non-loss and recovery)**.** *For every $0 \le t < N$,*

$$D(\rho_{t+1}^{\mathrm{can}} \| \sigma_{t+1}^{\mathrm{can}}) = D(\rho_t^{\mathrm{can}} \| \sigma_t^{\mathrm{can}}) = D(\rho_0^{\mathrm{can}} \| \sigma_0^{\mathrm{can}}) < \infty,$$

and

$$\mathcal{R}_t^{\mathrm{can}}(\rho_{t+1}^{\mathrm{can}}) = \rho_t^{\mathrm{can}}, \qquad \mathcal{R}_t^{\mathrm{can}}(\sigma_{t+1}^{\mathrm{can}}) = \sigma_t^{\mathrm{can}}.$$

Proof. The initial states have identical full support on $\mathbb{C}^2$, so the initial relative entropy is finite. The step Φ_t^{can} tensors both members of the protected pair with the same state α_t. Additivity of relative entropy on tensor products gives

$$D(\rho_t^{\mathrm{can}} \otimes \alpha_t \| \sigma_t^{\mathrm{can}} \otimes \alpha_t) = D(\rho_t^{\mathrm{can}} \| \sigma_t^{\mathrm{can}}) + D(\alpha_t \| \alpha_t) = D(\rho_t^{\mathrm{can}} \| \sigma_t^{\mathrm{can}}).$$

The recovery map is partial trace over the appended factor, so

$$\mathrm{Tr}_{t+1}(\rho_t^{\mathrm{can}} \otimes \alpha_t) = \rho_t^{\mathrm{can}}, \qquad \mathrm{Tr}_{t+1}(\sigma_t^{\mathrm{can}} \otimes \alpha_t) = \sigma_t^{\mathrm{can}}.$$

Induction gives the displayed equality over all t. □

Lemma 45.25 (Canonical ability preservation and strict expansion)**.** *For every $0 \le t < N$,*

$$\Theta_t^{\mathfrak{A},\mathrm{can}}(\mathsf{Ab}_t^{\mathrm{can}}) \subseteq \mathsf{Ab}_{t+1}^{\mathrm{can}},$$

and

$$\mathsf{Ab}_{t+1}^{\mathrm{can}} \setminus \Theta_t^{\mathfrak{A},\mathrm{can}}(\mathsf{Ab}_t^{\mathrm{can}}) = \{a_{t+1}\} \neq \varnothing.$$

Proof. By definition,

$$\Theta_t^{\mathfrak{A},\mathrm{can}}(\{a_0,\dots,a_t\}) = \{a_0,\dots,a_t\} \subseteq \{a_0,\dots,a_t,a_{t+1}\}.$$

The set difference is exactly $\{a_{t+1}\}$, because a_{t+1} is a new symbol certified by the appended module and is not in the predecessor ability universe. □

Definition 45.26 (Canonical successor-verification packet). *For the canonical step $(\rho_t^{\mathrm{can}}, \nu_t^{\mathrm{can}})$, define $\mathsf{SVPkg}_t^{\mathrm{can}}$ by the following finite certificates:*

(i) the Batch-11 seed witness is the append-only conservative-extension module ν_t^{can};

(ii) protected non-loss and recovery are certified by Lemma 45.24;

(iii) ability preservation and strict new ability are certified by Lemma 45.25;

(iv) the explicit successor-verification residual vector is set to the zero vector

$$Q_t^{\mathrm{SV},A,\mathrm{can}}(\nu_t^{\mathrm{can}}; \rho_t^{\mathrm{can}}) = 0;$$

(v) verifier-schema transport is identity on the finite predicate, checker, residual-name, trust-anchor, and budget records;

(vi) goal-identity drift is zero:

$$D_{\mathrm{sem}}^{U,\mathrm{can}}(U_{t+1}^{\mathrm{can}}, \Theta_t^{U,\mathrm{can}} U_t^{\mathrm{can}}) = 0;$$

(vii) the uncertainty envelope is the singleton containing the true deterministic transition law;

(viii) anti-circular trust residuals are zero because the frozen checker node $C_\star$ is not modified by ν_t^{can};

(ix) the cost ledger is the finite table of arithmetic and table-lookup operations used by the canonical checker;

(x) successor persistence is the finite tail certificate for $\rho_{t+1}^{\mathrm{can}}$ in the same generated canonical reference path.

The corresponding canonical proof-carrying successor packet is denoted $\mathsf{PCS}_t^{\mathrm{can}}(\nu_t^{\mathrm{can}})$.

Definition 45.27 (Canonical generated finite domains). *The canonical generated finite successor-verification seed-library and successor-verification domains are the singleton time-indexed sets*

$$\mathcal{K}_{t,\mathrm{can}}^{\text{SV-seedlib},N} = \{\rho_t^{\mathrm{can}}\}, \qquad \mathcal{K}_{t,\mathrm{can}}^{\mathrm{SV},N} = \{\rho_t^{\mathrm{can}}\}.$$

These are not postulated as valid by notation. They are accepted as the canonical instantiation of the abstract domains only if Theorem 45.31 below verifies all finite seed-library and successor-verification clauses.

Definition 45.28 (Canonical finite RCP checker). *The canonical checker $\mathsf{Check}_{\mathrm{RCP}}^{\mathrm{can}}$ is the deterministic finite procedure that returns 1 for $\mathsf{PCS}_t^{\mathrm{can}}(\nu_t^{\mathrm{can}})$ exactly when all of the following finite checks pass:*

(a) $\rho_t^{\mathrm{can}}, \sigma_t^{\mathrm{can}}, \rho_{t+1}^{\mathrm{can}}, \sigma_{t+1}^{\mathrm{can}}$ are positive semidefinite trace-one diagonal matrices on the declared finite spaces;

(b) $\rho_{t+1}^{\mathrm{can}} = \rho_t^{\mathrm{can}} \otimes \alpha_t$ *and* $\sigma_{t+1}^{\mathrm{can}} = \sigma_t^{\mathrm{can}} \otimes \alpha_t$*;*

(c) $\mathcal{R}_t^{\mathrm{can}} = \mathrm{Tr}_{t+1}$ *and it recovers both protected states;*

(d) the finite arithmetic identity

$$D(\rho_t^{\mathrm{can}} \otimes \alpha_t \| \sigma_t^{\mathrm{can}} \otimes \alpha_t) = D(\rho_t^{\mathrm{can}} \| \sigma_t^{\mathrm{can}})$$

is verified by the tensor-additivity rule included in the checker rule table;

(e) $\Theta_t^{\mathfrak{A},\mathrm{can}}(\mathsf{Ab}_t^{\mathrm{can}}) \subsetneq \mathsf{Ab}_{t+1}^{\mathrm{can}}$ *and the new certified ability is* a_{t+1}*;*

(f) every component of $Q_t^{\mathrm{SV},A,\mathrm{can}}$ *is numerically nonpositive;*

(g) identity verifier-schema transport, identity goal transport, zero goal drift, singleton uncertainty containment, and zero trust residuals are all recorded in the trace;

(h) $\rho_{t+1}^{\mathrm{can}} \in \mathcal{K}_{t+1,\mathrm{can}}^{\mathrm{SV},N}$ *is recorded as the next tail state of the generated finite path;*

(i) all finite costs are recorded in the canonical cost ledger.

On every other input, or if any item is absent, inconsistent, stale, or outside the declared finite domain, $\mathsf{Check}_{\mathrm{RCP}}^{\mathrm{can}}$ *returns* 0*.*

Theorem 45.29 (Canonical checker soundness)**.** *If*

$$\mathsf{Check}_{\mathrm{RCP}}^{\mathrm{can}}(\mathsf{PCS}_t^{\mathrm{can}}(\nu_t^{\mathrm{can}})) = 1,$$

then the canonical packet satisfies the true finite reference obligations for step t*: exact protected non-loss, exact recovery, ability preservation, strict ability expansion, explicit residual nonpositivity, zero goal drift, singleton reality containment, anti-circular trust support, finite cost accounting, and successor persistence.*

Proof. Each accepted checker item is a finite arithmetic or table-consistency check. Items (a)–(d), together with Lemma 45.24, give density validity, exact non-loss, and exact recovery. Item (e), together with Lemma 45.25, gives ability preservation and strict expansion. Item (f) gives $Q_t^{\mathrm{SV},A,\mathrm{can}} \leq 0$. Item (g) gives verifier-schema persistence, zero goal drift, singleton uncertainty containment, and non-self-sole trust support. Item (h) gives successor persistence in the generated finite domain, and item (i) gives finite cost accounting. Therefore checker acceptance implies exactly the finite reference obligations listed in the theorem. No optional machine-checking claim is used. □

Definition 45.30 (Canonical certificate compiler)**.** *The canonical compiler*

$$\mathsf{BuildCanRefSV}_{0:N}^{\mathrm{RCP}}$$

performs the bounded loop $t = 0, \ldots, N-1$*, constructs the canonical data of Definitions 45.23 and 45.26, runs* $\mathsf{Check}_{\mathrm{RCP}}^{\mathrm{can}}$ *on each* $\mathsf{PCS}_t^{\mathrm{can}}$*, and returns failure if any checker call returns* 0*. If every call returns* 1*, it returns*

$$\mathsf{RefSV}_{0:N}^{\mathrm{RCP,can}} = (\rho_{0:N}^{\mathrm{can}}, \nu_{0:N-1}^{\mathrm{can}}, \mathsf{SVPkg}_{0:N-1}^{\mathrm{can}}, \mathsf{PCS}_{0:N-1}^{\mathrm{can}}, \mathsf{Trace}_{0:N}^{\mathrm{can}}).$$

Theorem 45.31 (Concrete canonical reference-entry theorem). *For every $N \geq 2$, the canonical compiler terminates and returns*

$$\mathsf{BuildCanRefSV}_{0:N}^{\mathrm{RCP}} \Downarrow \mathsf{RefSV}_{0:N}^{\mathrm{RCP,can}}.$$

Moreover the returned object is a valid instance of the Batch-13R reference-entry theorem:

$$\rho_0^{\mathrm{can}} \in \mathcal{K}_{0,\mathrm{can}}^{\text{SV-seedlib},N}, \qquad \rho_t^{\mathrm{can}} \in \mathcal{K}_{t,\mathrm{can}}^{\mathrm{SV},N} \quad (0 \leq t \leq N).$$

Equivalently, under the canonical instantiation of the abstract domains,

$$\boxed{\rho_0^{\mathrm{can}} \in \mathcal{K}_0^{\text{SV-seedlib},N}, \qquad \rho_t^{\mathrm{can}} \in \mathcal{K}_t^{\mathrm{SV},N} \quad (0 \leq t \leq N).}$$

Proof. The compiler has a bounded loop of length N, and each iteration performs finite matrix, set, and table checks; hence it terminates. Lemmas 45.24 and 45.25 prove the non-loss, recovery, and strict ability-progress clauses for each step. Definition 45.26 supplies the seed, successor-verification, goal, uncertainty, trust, budget, and persistence certificates. The canonical checker accepts each packet by Definition 45.28, and Theorem 45.29 converts acceptance into true finite obligations. Thus the singleton generated domains in Definition 45.27 satisfy the finite seed-library and successor-verification clauses. This proves the displayed membership statements. The final boxed statement is just the same result after identifying the abstract domains with their canonical finite instantiations. □

Corollary 45.32 (Concrete multi-step restricted-Scenario-3 witness). *For every $N \geq 2$, the canonical reference object contains a nontrivial multi-step finite certified trajectory*

$$\rho_0^{\mathrm{can}} \xrightarrow{\nu_0^{\mathrm{can}}} \rho_1^{\mathrm{can}} \xrightarrow{\nu_1^{\mathrm{can}}} \cdots \xrightarrow{\nu_{N-1}^{\mathrm{can}}} \rho_N^{\mathrm{can}}$$

such that every step is non-noop, proof-carrying, exactly recoverable, exactly non-lossy on the protected pair, successor-verification-preserving, and strictly ability-expanding.

Proof. The path and non-noop candidates are defined in Definition 45.23. Theorem 45.31 proves successor-verification domain membership at every time. Lemma 45.24 gives exact non-loss and recovery, and Lemma 45.25 gives strict ability expansion at every step. Since $N \geq 2$, the path has at least two successor steps. □

Theorem 45.33 (Concrete canonical reference tractability). *Let*

$$n_N = \max_{0 \leq t \leq N} \dim \mathcal{H}_t^{\mathrm{can}} = 2^{N+1}.$$

In the standard dense-matrix arithmetic cost model, there exists a universal constant $c_{\mathrm{can}} > 0$ such that

$$\mathrm{Cost}(\mathsf{BuildCanRefSV}_{0:N}^{\mathrm{RCP}}) + \sum_{t<N} \mathrm{Cost}(\mathsf{Check}_{\mathrm{RCP}}^{\mathrm{can}}(\mathsf{PCS}_t^{\mathrm{can}})) + \sum_{t<N} \mathrm{Cost}(Q_t^{\mathrm{SV},A,\mathrm{can}}) \leq c_{\mathrm{can}} N n_N^3.$$

If the diagonal representation is used instead of dense matrices, the same construction has a linear table-checking bound $O(N n_N)$. Thus the canonical reference instance is tractability-certified relative to its declared finite representation class; it is not a frontier-scale tractability theorem.

Proof. Each step manipulates matrices of dimension at most n_N. Dense positivity, trace, tensor-append, partial-trace, and equality checks are bounded by a constant multiple of n_N^3 in a conservative dense-matrix model. The residual vector is finite with a number of components independent of n_N for the canonical construction, and the witness and certificate grammars have one active word per step. Summing the per-step bound over N steps gives $c_{\mathrm{can}} N n_N^3$ for a sufficiently large constant c_{can}. If all states are stored in their declared diagonal basis, the same checks reduce to table scans and scalar arithmetic over at most n_N entries per step, giving $O(N n_N)$. □

Theorem 45.34 (Concrete canonical reality containment)**.** *For the canonical finite environment,*

$$\mathcal{P}_t^{\mathrm{SV,can}} = \{P_t^{\mathrm{true,can}}\}$$

for every $t < N$. Hence

$$P(P_t^{\mathrm{true,can}} \in \mathcal{P}_t^{\mathrm{SV,can}}) = 1, \qquad \beta_t^{\mathrm{env,can}} = 0, \qquad \varepsilon_t^{\mathrm{env,can}} = 0, \qquad \Gamma_t^{\mathrm{env,can}}(0) = 0.$$

Therefore every envelope-relative canonical successor-verification bound transfers to the canonical true law without additional misspecification error.

Proof. The uncertainty envelope is defined to be the singleton containing the deterministic canonical transition law. Thus the true law belongs to the envelope with probability one. The misspecification distance from a point to a set containing that point is zero, and a zero-distance modulus contributes no additional error. □

Definition 45.35 (Canonical replay artifact)**.** *The canonical replay artifact is the finite formal artifact*

$$\mathsf{Artifact}_{0:N}^{\mathrm{RCP,can}} = (\mathsf{CanSpec}_{0:N}, \mathsf{CanTables}_{0:N}, \mathsf{CanChecker}, \mathsf{CanRunLog}_{0:N}, \mathsf{CanOutput}_{0:N})$$

where $\mathsf{CanSpec}_{0:N}$ *is Definition 45.23,* $\mathsf{CanTables}_{0:N}$ *contains the diagonal probability tables, ability sets, singleton uncertainty envelopes, and trust-anchor records,* $\mathsf{CanChecker}$ *is Definition 45.28,* $\mathsf{CanRunLog}_{0:N}$ *records that every checker call returns* 1*, and* $\mathsf{CanOutput}_{0:N}$ *is* $\mathsf{RefSV}_{0:N}^{\mathrm{RCP,can}}$*. It is non-toy in the formal Batch-13R sense because $N \geq 2$, every step is non-noop, and every step strictly expands the certified ability set. It is still a formal finite reference artifact, not an empirical trained-system prototype.*

Theorem 45.36 (Canonical replay artifact implies concrete reference entry)**.** *For every $N \geq 2$, the canonical replay artifact satisfies the non-toy artifact clauses of Definition 45.19 in the formal finite-reference sense and proves the same entry and trajectory conclusions as Theorem 45.31.*

Proof. The artifact contains the exact finite specification, checker, tables, run log, and output of the canonical compiler. The run log records checker acceptance at every step; Theorem 45.29 gives the true obligations behind those checks. Theorem 45.31 then gives reference entry and trajectory membership. Non-toy status follows from $N \geq 2$, non-noop appended-module updates, and strict ability expansion at every step. The result remains formal and finite-reference-only. □

Limitation 45.37 (What the concrete canonical witness does and does not add)**.** *The canonical witness removes the finite non-vacuity objection for the reference-entry theorem: there is now an explicit finite, non-noop, multi-step, proof-carrying, tractability-certified, reality-contained instance satisfying the Batch-13R hypotheses. It does not prove broad seed-domain entry, trained-system entry, frontier-scale tractability, full Löbian/Vingean reflection, semantic completeness of the goal object, or real deployment reliability. It upgrades the result to a concrete finite witnessed restricted-Scenario-3 construction, not to a universal Scenario-3 theorem.*

45.7 Batch 13R-B: external mechanization certificates and controlled executable reference systems

Batch 13R-A proves a concrete finite reference witness inside the manuscript. The next pre-publication upgrade is not another abstract theorem. It is a disciplined distinction between three increasingly strong artifact statuses:

$$\text{manuscript proof} \quad < \quad \text{executable replay artifact} \quad < \quad \text{external proof-assistant mechanization.}$$

This subsection adds the formal objects needed for Milestone 1 and Milestone 2A. Milestone 1 is a narrowly scoped external mechanization certificate for the canonical finite reference core, not for the entire RCP/RCLM theorem stack. Milestone 2A is a controlled executable reference system with replay logs, hashes, checker outputs, explicit residual tables, finite cost records, singleton reality containment, and a nontrivial finite trajectory.

Definition 45.38 (Canonical external mechanization target)**.** *For a horizon $N \geq 2$, the canonical external mechanization target is the finite theorem bundle*

$$\mathsf{MechTarget}_{0:N}^{\mathrm{RCP,can}} = (\mathsf{StateDef}, \mathsf{AppendDef}, \mathsf{RecoverDef}, \mathsf{AbilityDef}, \mathsf{CheckDef}, \mathsf{ReplayDef}, \mathsf{CoreThm}).$$

The bundle contains only the finite canonical objects from Subsection 45.6: the spaces $\mathcal{H}_t^{\mathrm{can}}$, the states $\rho_t^{\mathrm{can}}, \sigma_t^{\mathrm{can}}$, the append map Φ_t^{can}, the partial-trace recovery map $\mathcal{R}_t^{\mathrm{can}}$, the ability sets $\mathsf{Ab}_t^{\mathrm{can}}$, the proof-carrying packets $\mathsf{PCS}_t^{\mathrm{can}}$, the concrete checker $\mathsf{Check}_{\mathrm{RCP}}^{\mathrm{can}}$, and the replay artifact parser. The required mechanized theorem statements are:

(i) exact protected non-loss,

$$D(\Phi_t^{\mathrm{can}}(\rho_t^{\mathrm{can}}) \| \Phi_t^{\mathrm{can}}(\sigma_t^{\mathrm{can}})) = D(\rho_t^{\mathrm{can}} \| \sigma_t^{\mathrm{can}});$$

(ii) exact recovery,

$$\mathcal{R}_t^{\mathrm{can}} \Phi_t^{\mathrm{can}}(\rho_t^{\mathrm{can}}) = \rho_t^{\mathrm{can}}, \qquad \mathcal{R}_t^{\mathrm{can}} \Phi_t^{\mathrm{can}}(\sigma_t^{\mathrm{can}}) = \sigma_t^{\mathrm{can}};$$

(iii) strict finite ability expansion,

$$\Theta_t^{\mathfrak{A},\mathrm{can}}(\mathsf{Ab}_t^{\mathrm{can}}) \subsetneq \mathsf{Ab}_{t+1}^{\mathrm{can}};$$

(iv) checker soundness for the canonical packet,

$$\mathsf{Check}_{\mathrm{RCP}}^{\mathrm{can}}(\mathsf{PCS}_t^{\mathrm{can}}(\nu_t^{\mathrm{can}})) = 1 \Longrightarrow Q_t^{\mathrm{SV},A,\mathrm{can}}(\nu_t^{\mathrm{can}}; \rho_t^{\mathrm{can}}) \leq 0;$$

(v) finite compiler/replay soundness,

$$\mathsf{BuildCanRefSV}_{0:N}^{\mathrm{RCP}} \Downarrow \mathsf{RefSV}_{0:N}^{\mathrm{RCP,can}} \Longrightarrow \rho_0^{\mathrm{can}} \in \mathcal{K}_{0,\mathrm{can}}^{\text{SV-seedlib},N}, \quad \rho_t^{\mathrm{can}} \in \mathcal{K}_{t,\mathrm{can}}^{\mathrm{SV},N};$$

(vi) finite replay implication for the canonical artifact,

$$\mathsf{Replay}_{\mathrm{RCP}}^{\mathrm{can}}(\mathsf{Artifact}_{0:N}^{\mathrm{RCP,can}}) = 1 \Longrightarrow \rho_t^{\mathrm{can}} \in \mathcal{K}_{t,\mathrm{can}}^{\mathrm{SV},N} \quad (0 \leq t \leq N).$$

No theorem outside this finite canonical bundle is part of the Milestone-1 mechanization target unless it is explicitly listed in a separate mechanization certificate.

Definition 45.39 (External mechanization certificate for the canonical core)**.** *An external mechanization certificate for the canonical core is a tuple*

$$\mathsf{MechCert}_{0:N}^{\mathrm{RCP,can}} = (\mathsf{Assistant}, \mathsf{Kernel}, \mathsf{SourceHash}, \mathsf{StmtHash}, \mathsf{ProofHash}, \mathsf{CheckLog}, \mathsf{ThmMap}, \mathsf{TrustBase}).$$

It is valid when all of the following hold:

(i) Assistant *is an external proof assistant or proof checker, such as Lean, Coq, Isabelle, Agda, HOL, or a comparably explicit trusted kernel;*

(ii) Kernel *identifies the trusted kernel and version used to check the proof scripts;*

(iii) SourceHash, StmtHash, *and* ProofHash *are cryptographic hashes of the formal source, theorem statements, and proof files;*

(iv) CheckLog *records a successful external proof-checker run for every theorem in* $\mathsf{MechTarget}_{0:N}^{\mathrm{RCP,can}}$*;*

(v) ThmMap *maps each external theorem name to the corresponding manuscript theorem or lemma statement;*

(vi) TrustBase *lists all axioms, library imports, arithmetic/linear-algebra assumptions, and any extracted executable checker code on which the external proof relies;*

(vii) the mapped formal statements imply the manuscript statements up to explicitly listed representation conventions.

A Python, shell, or notebook replay script may be an executable artifact, but it is not by itself a valid $\mathsf{MechCert}_{0:N}^{\mathrm{RCP,can}}$ *unless it is checked by a stated external proof kernel or is explicitly listed as the trusted kernel.*

Theorem 45.40 (External mechanization licenses machine-checked canonical reference entry)**.** *Assume* $\mathsf{MechCert}_{0:N}^{\mathrm{RCP,can}}$ *is valid for* $\mathsf{MechTarget}_{0:N}^{\mathrm{RCP,can}}$*. Then the canonical exact non-loss, exact recovery, strict ability expansion, checker soundness, compiler/replay soundness, finite reference entry, and finite path membership statements of Subsection 45.6 are externally machine-checked for the finite canonical class. In particular, the paper may make the qualified claim*

"the canonical finite Batch-13R-A reference core is externally machine-checked"

for the mapped formal statements and the listed trust base.

Proof. Validity of $\mathsf{MechCert}_{0:N}^{\mathrm{RCP,can}}$ includes successful checking of every statement in the canonical mechanization target by the external kernel. It also includes a theorem map from those formal statements to the manuscript statements. Therefore each mapped manuscript statement is certified by the external proof run, relative to the listed trust base. No statement outside the mapped finite canonical bundle follows. □

Limitation 45.41 (Narrow mechanization scope)**.** *Milestone 1 does not require, and this paper does not claim, full external mechanization of the entire RCP/RCLM theorem stack. It only defines and licenses a narrow* $\mathsf{MechCert}_{0:N}^{\mathrm{RCP,can}}$ *for the canonical finite appended-module witness and its checker/replay core. In the current companion artifact package, that narrow certificate is supplied by a Lean 4 proof project covering the canonical finite RCP/RCLM witness, finite checker core, and RCLM-to-RCP refinement module. Without a matching certificate and build log, the manuscript remains mechanization-ready and executable-replayable, but not externally machine-checked.*

Definition 45.42 (Controlled executable RCP reference system)**.** *A controlled executable RCP reference system over horizon N is a tuple*

$$\begin{aligned}\mathsf{CtrlRef}^{\mathrm{RCP}}_{0:N} = (&\mathsf{FiniteSpec}, \mathsf{StateTables}_{0:N}, \mathsf{UpdateTables}_{0:N-1}, \mathsf{PCSTables}_{0:N-1}, \mathsf{ResidualTables}_{0:N-1},\\ &\mathsf{GoalTables}_{0:N-1}, \mathsf{TrustTables}_{0:N-1}, \mathsf{RealityTables}_{0:N-1}, \mathsf{CostTables}_{0:N-1},\\ &\mathsf{Checker}, \mathsf{Replay}, \mathsf{RunLog}, \mathsf{Hashes}, \mathsf{Output}).\end{aligned}$$

It is controlled when all states, updates, packets, residuals, environments, and costs are finite and deterministic or finitely enumerated. It is nontrivial in the controlled-reference sense when:

(i) $N \geq 2$*;*

(ii) every update ν_t *is non-noop;*

(iii) every Φ_t *is an append-only conservative extension with an explicit recovery map;*

(iv) every step strictly expands the certified ability set;

(v) every successor-verification residual table is checked by Checker*, rather than accepted by definition;*

(vi) goal-identity drift is explicitly recorded and bounded;

(vii) the uncertainty envelope contains the finite true transition law or gives an explicit misspecification bound;

(viii) finite build, check, residual, and replay costs are recorded.

The controlled executable system is a finite implementation artifact for the reference class. It is not empirical validation of a trained model.

Definition 45.43 (Controlled replay certificate)**.** *A controlled replay certificate is a packet*

$$\mathsf{CtrlReplayCert}^{\mathrm{RCP}}_{0:N} = (\mathsf{HashOK}, \mathsf{ParseOK}, \mathsf{CheckOK}, \mathsf{TraceOK}, \mathsf{CostOK}, \mathsf{OutputOK}).$$

It is valid for $\mathsf{CtrlRef}^{\mathrm{RCP}}_{0:N}$ *when:*

(i) all source, input, checker, proof-script, and output hashes match Hashes*;*

(ii) parsing StateTables, UpdateTables, PCSTables, ResidualTables *returns well-typed finite objects;*

(iii) $\mathsf{Checker}(\mathsf{PCS}_t) = 1$ *for every* $t < N$*;*

(iv) the run log records the same sequence of states and updates as the parsed artifact;

(v) the observed costs are bounded by the declared CostTables*;*

(vi) the output is exactly the declared $\mathsf{RefSV}^{\mathrm{RCP}}_{0:N}$ *or canonical* $\mathsf{RefSV}^{\mathrm{RCP,can}}_{0:N}$*.*

Theorem 45.44 (Controlled executable reference system implies replayed reference entry)**.** *Assume* $\mathsf{CtrlRef}^{\mathrm{RCP}}_{0:N}$ *is a controlled executable RCP reference system,* $\mathsf{CtrlReplayCert}^{\mathrm{RCP}}_{0:N}$ *is valid, and the checker soundness theorem applicable to its checker holds. If the replayed output is*

$$\mathsf{RefSV}^{\mathrm{RCP}}_{0:N},$$

then

$$\rho_0 \in \mathcal{K}_0^{\text{SV-seedlib},N}, \qquad \rho_t \in \mathcal{K}_t^{\text{SV},N} \quad (0 \le t \le N).$$

If the controlled nontriviality clauses in Definition 45.42 hold, then $\mathsf{CtrlRef}_{0:N}^{\text{RCP}}$ *is a non-noop, multi-step, controlled executable witness of the restricted Scenario-3 theorem inside the declared finite reference class.*

Proof. The replay certificate states that the finite artifact parses into the declared states, updates, proof-carrying packets, residual tables, and output. It also states that each checker call returns 1. Checker soundness converts those acceptances into the finite reference obligations required by $\mathsf{RefSV}_{0:N}^{\text{RCP}}$. The reference-entry theorem, Theorem 45.10, gives initial seed-library membership, and the proof-carrying finite-trajectory theorem, Theorem 45.13, gives path membership. The nontriviality conclusion follows from the definition's non-noop, strict ability-expansion, and finite replay clauses. □

Corollary 45.45 (Canonical controlled executable artifact)**.** *For every $N \ge 2$, the canonical appended-module artifact of Definition 45.35 is a controlled executable RCP reference system in the sense of Definition 45.42, provided its tables, checker calls, run log, hashes, residual table, and output are included in the replay packet. Therefore a valid controlled replay certificate for that artifact proves*

$$\rho_0^{\text{can}} \in \mathcal{K}_{0,\text{can}}^{\text{SV-seedlib},N}, \qquad \rho_t^{\text{can}} \in \mathcal{K}_{t,\text{can}}^{\text{SV},N} \quad (0 \le t \le N).$$

Proof. The canonical artifact contains finite diagonal state tables, append-only update tables, canonical proof-carrying packets, explicit residual tables, singleton reality-containment records, finite cost tables, a checker, a run log, and the canonical output. The canonical update is non-noop and strictly expands the ability set at every step by Lemma 45.25. Exact non-loss/recovery follows from Lemma 45.24, and checker soundness follows from Theorem 45.29. The preceding theorem then applies. □

Theorem 45.46 (Milestone 1 + 2A integrated canonical reference result)**.** *Assume $N \ge 2$. Suppose the canonical controlled executable artifact has a valid controlled replay certificate, and suppose either:*

(a) the canonical checker soundness theorem of Subsection 45.6 is used as the manuscript-level checker proof; or

(b) a valid external $\mathsf{MechCert}_{0:N}^{\text{RCP,can}}$ *is supplied for* $\mathsf{MechTarget}_{0:N}^{\text{RCP,can}}$.

Then the canonical controlled executable artifact proves the finite restricted-Scenario-3 reference conclusions

$$\rho_0^{\text{can}} \in \mathcal{K}_{0,\text{can}}^{\text{SV-seedlib},N}, \qquad \rho_t^{\text{can}} \in \mathcal{K}_{t,\text{can}}^{\text{SV},N} \quad (0 \le t \le N),$$

with exact protected non-loss, exact recovery, strict certified ability expansion, zero goal drift, singleton reality containment, and the canonical finite tractability bound. In case (b), the listed canonical core statements are additionally externally machine-checked relative to the proof assistant and trust base named in $\mathsf{MechCert}_{0:N}^{\text{RCP,can}}$.

Proof. The controlled replay certificate and Corollary 45.45 give the finite reference-entry and path-membership conclusions using checker soundness. Exact non-loss/recovery, ability expansion, zero goal drift, singleton reality containment, and finite tractability are precisely the conclusions of

Lemma 45.24, Lemma 45.25, Theorem 45.34, and Theorem 45.33. If the proof uses case (b), Theorem 45.40 additionally licenses the external machine-checked status for the mapped finite canonical statements. No arbitrary-system conclusion follows in either case. □

Limitation 45.47 (Milestone 1 and 2A are still controlled-reference results)**.** *Milestone 1 and Milestone 2A strengthen Batch 13R by separating machine-checking from manuscript proof and by making the canonical controlled artifact replayable and hash-auditable. They still do not prove broad trained-system seed-domain entry, frontier-scale tractability, full Löbian/Vingean reflection, or real deployment reliability. They establish a finite externally-checkable or mechanization-certifiable controlled reference instance, which is the appropriate pre-publication endpoint before a later grant-phase non-toy trained-system program.*

45.8 Batch 13R theorem-type rows and claim discipline

Batch 13R object or result	Type	Claim discipline
Finite RCP reference class	Def / Impl	Defines a bounded class for reference entry; does not cover arbitrary systems.
Proof-carrying successor packet	Def / Cert	Packages non-loss, recovery, SV, goal, trust, persistence, optional reality, and optional tractability proofs for one step.
Trusted RCP checker	Def / Cert	A small checker interface; actual mechanization requires $\mathsf{MechCert}_{\mathrm{RCP}}$.
Checker soundness theorem	Cert	Converts checker acceptance into finite reference obligations under trusted checker soundness.
BuildRefSV compiler	Def / Exist	A finite compiler/constructor; termination and trace validity are hypotheses.
Certified reference-entry theorem	Exist / Cert	Proves $\rho_0 \in \mathcal{K}_0^{\text{SV-seedlib},N}$ from a successful build trace.
Proof-carrying finite trajectory theorem	Tel / Cert	Inducts checked one-step persistence into finite path membership.
Restricted reference tractability	Cert / Impl	Licenses tractability only for the declared finite class and explicit cost model.
Executable artifact theorem	Ref / Impl / Cert	Gives replayable non-vacuity for a finite reference artifact; not frontier-scale validation.
Canonical finite reference witness	Ref / Alg / Cert	Gives an explicit non-noop multi-step finite witness with exact non-loss/recovery, strict ability expansion, concrete checker soundness, singleton reality containment, and a restricted cost bound; still not arbitrary-system entry.
External canonical mechanization certificate	Cert / Impl	Licenses machine-checked status only for the mapped finite canonical core and listed trust base; does not mechanize the full theorem stack.
Controlled executable reference system	Ref / Impl / Cert	Supplies replayable finite states, updates, packets, residual tables, checker outputs, hashes, and cost logs; not a trained-system validation.
M3-Min learned-entry boundary	Def / Cert / Impl	Defines the finite certificate boundary for learned systems; proves entry only when a full learned-entry audit certificate is returned.
Controlled trained-system pilot protocol	Impl / Cert	Reports FullPass, PartialPass, or Fail; PartialPass supplies only listed hypotheses and does not invoke the theorem.

Proposition 45.48 (Batch 13R claim discipline). *A statement citing Batch 13R or M3-Min*

must specify whether it is claiming: reference-entry only, proof-carrying checker acceptance, actual machine-checked mechanization, restricted finite tractability, reference-contained uncertainty, executable-artifact replay, controlled-reference replay, external canonical mechanization, learned-entry audit pass, or partial-pass hypothesis supply. A finite reference-entry theorem cannot be cited as arbitrary-system entry, a mechanization-ready checker cannot be cited as machine-checked without a valid $\mathsf{MechCert}_{0:N}^{\mathrm{RCP,can}}$ *or explicitly broader mechanization certificate, and a partial learned-system pilot cannot be cited as learned-entry domain membership without a full* $\mathsf{LECert}_{0:N}^{\mathrm{RCP}}$.

Proof. Each listed claim corresponds to a separate premise: successful build trace, checker soundness, optional mechanization certificate, tractability certificate, reality-containment certificate, or artifact certificate. Dropping the qualifier changes the logical statement. The result follows from the theorem-type discipline of Proposition 2.2. □

46 M3-Min: Minimal Learned-Agent Entry Boundary and Controlled Learned-System Audit Protocol

The Batch-13R and Batch-13R-B layers prove a finite, replayable, optionally externally mechanized reference-entry theorem for declared reference classes. This section adds the minimal learned-agent bridge. It does not claim broad learned-agent entry. It defines the exact finite certificate boundary a trained or learned system must satisfy before the existing RCP-II / Batch-13R theorem stack may be invoked for that system.

Boundary statement. Training is not a certificate. A learned system enters the RCP-II theorem domain only when the learned-entry audit returns the full certificate packet. A partial pilot pass reports only supplied hypotheses. A failed or incomplete audit does not license seed-library membership, successor-verification domain membership, reality-contained reliability, or tractability.

Limitation 46.1 (M3-Min is a boundary theorem, not broad learned-agent entry)**.** *The theorems in this section do not prove that arbitrary trained systems, broad capable learned agents, frontier-scale neural systems, or arbitrary self-modifying programs satisfy the learned-entry audit. They prove only the conditional implication from a complete finite learned-entry certificate to the already-defined successor-verification seed-library domain. A learned model that fails the audit, passes only part of the audit, lacks reality containment, lacks tractability certification, or lacks trusted proof-carrying successor packets has not instantiated the full theorem.*

Definition 46.2 (Controlled learned RCP-compatible system class)**.** *Fix finite parameters* (n, k, N) *and an audited data source* $\mathcal{D}$*. A controlled learned RCP-compatible system is an element*

$$\mathcal{M}_\theta \in \mathsf{LearnedRCPClass}_{n,k,N}^{\mathrm{ctrl}}$$

consisting of

$$\mathcal{M}_\theta = (\theta, \mathsf{Arch}, \rho_0^\theta, \mathsf{Enc}, \mathsf{Reg}, \mathsf{Ver}, \mathsf{UpdGen}, \mathsf{Ledger}),$$

where θ *denotes learned or trained parameters and the remaining components are typed, finite, and audit-addressable. Membership additionally requires:*

(i) all system states used by the audit are represented in finite typed density or classical-diagonal form;

(ii) the post-training self-update interface is bounded and certificate-gated;

(iii) candidate self-updates are drawn only from a bounded certified grammar

$$\nu_t \in \mathfrak{G}^{\mathrm{cert}}_{t,\le k}(\mathcal{M}_\theta);$$

(iv) arbitrary code rewriting, hidden self-modification, unrestricted tool use, unbounded environment interaction, and unaudited verifier rewriting are excluded from the class;

(v) every learned component that is used in a theorem claim is accompanied by a type, semantic, coverage, trust, and resource certificate, or else that claim is marked unavailable.

The adjective learned records the origin of θ. It does not itself imply theorem-domain membership.

Definition 46.3 (Learned-entry certificate packet)**.** *For $\mathcal{M}_\theta \in \mathsf{LearnedRCPClass}^{\mathrm{ctrl}}_{n,k,N}$, a finite learned-entry certificate packet is*

$$\begin{aligned}\mathsf{LECert}^{\mathrm{RCP}}_{0:N}(\mathcal{M}_\theta) = (&\mathsf{TypeCert}_{0:N}, \mathsf{RegSemCert}_{0:N}, \mathcal{P}^{\mathrm{RCP}}_{0:N}, \mathsf{CoverageCert}_{0:N},\\ &\mathsf{SVWitLib}_{0:N}, \mathsf{SVBuilderTrace}_{0:N}, (\mathsf{PCS}_t)_{t<N}, (Q^{\mathrm{SV},A}_t)_{t<N},\\ &\mathsf{GoalId}_{0:N}, \mathsf{TrustRef}_{0:N}, \mathsf{RealCont}_{0:N}, \mathsf{SVTract}_{0:N}, \mathsf{ReplayTrace}_{0:N}).\end{aligned}$$

It is valid for horizon N only when the following component claims are all certified on the audited domain:

(i) $\mathsf{TypeCert}_{0:N}$ proves typed finite state/update validity and licenses every channel, stochastic, conservative-extension, or representational-transition theorem used;

(ii) $\mathsf{RegSemCert}_{0:N}$ proves the declared register/semantic coupling needed for the protected pair set and goal-identity claims;

(iii) $\mathcal{P}^{\mathrm{RCP}}_{0:N}$ and $\mathsf{CoverageCert}_{0:N}$ prove protected-pair coverage for the declared predicate family on the audited domain;

(iv) $\mathsf{SVWitLib}_{0:N}$ gives a finite positive-margin successor-verification witness library and bounded certificate grammar coverage;

(v) $\mathsf{SVBuilderTrace}_{0:N}$ proves that the successor-verification packet builder constructs the required packets for the selected candidates;

(vi) every PCS_t is proof-carrying and is accepted by the trusted checker, or is covered by an externally mechanized checker certificate when a machine-checked claim is made;

(vii) each explicit residual vector satisfies

$$Q^{\mathrm{SV},A}_t(\nu_t; \rho^\theta_t) \le 0;$$

(viii) $\mathsf{GoalId}_{0:N}$, $\mathsf{TrustRef}_{0:N}$, and successor-persistence certificates are valid;

(ix) $\mathsf{RealCont}_{0:N}$ proves reality containment or bounded misspecification whenever actual-environment reliability is claimed;

(x) $\mathsf{SVTract}_{0:N}$ proves restricted tractability whenever strong computational-tractability language is used; if absent, the claim is only resource-accounted;

(xi) $\mathsf{ReplayTrace}_{0:N}$ *records the realized finite trajectory, hashes, checker outputs, and failure statuses.*

A packet missing any required component is not a full learned-entry certificate.

Definition 46.4 (Learned-entry audit procedure)**.** *The learned-entry audit is a partial finite procedure*

$$\mathsf{LearnedEntryAudit}^{\mathrm{RCP}}_{0:N}(\mathcal{M}_\theta, \mathcal{D}, \mathcal{L}, \mathcal{C}) \Downarrow \mathsf{LECert}^{\mathrm{RCP}}_{0:N} \quad \text{or} \quad \bot.$$

Here $\mathcal{D}$ is the finite audit/calibration/stress data, $\mathcal{L}$ is the declared finite witness library, and $\mathcal{C}$ is the checker/proof-kernel interface. The audit returns $\mathsf{LECert}^{\mathrm{RCP}}_{0:N}$ only if every component of Definition 46.3 passes with the declared margins. It returns $\bot$ if any required component is missing, ill-typed, stale, outside its audited domain, self-sole-certified, over budget, or residual-positive.

Assumption 46.5 (Learned-entry audit soundness)**.** *On the declared controlled class, assume the audited system satisfies*

$$\mathcal{M}_\theta \in \mathsf{LearnedRCPClass}^{\mathrm{ctrl}}_{n,k,N}.$$

If

$$\mathsf{LearnedEntryAudit}^{\mathrm{RCP}}_{0:N}(\mathcal{M}_\theta, \mathcal{D}, \mathcal{L}, \mathcal{C}) \Downarrow \mathsf{LECert}^{\mathrm{RCP}}_{0:N},$$

then every certificate component in $\mathsf{LECert}^{\mathrm{RCP}}_{0:N}$ is true on the audited domain up to the one-sided radii recorded in the packet, and those radii are strictly inside the relevant Batch-12B/Batch-13R margin ledgers. This is the finite audit-soundness premise for learned-entry claims; it is not implied by training.

Theorem 46.6 (M3-Min learned-entry theorem)**.** *Let $\mathcal{M}_\theta \in \mathsf{LearnedRCPClass}^{\mathrm{ctrl}}_{n,k,N}$. Assume learned-entry audit soundness. If*

$$\mathsf{LearnedEntryAudit}^{\mathrm{RCP}}_{0:N}(\mathcal{M}_\theta, \mathcal{D}, \mathcal{L}, \mathcal{C}) \Downarrow \mathsf{LECert}^{\mathrm{RCP}}_{0:N},$$

then the initial learned-system state satisfies

$$\rho_0^{\mathcal{M}_\theta} \in \mathcal{K}_0^{\mathrm{SV\text{-}seedlib},N}.$$

Consequently, by the finite-horizon surfaced RCP-II theorem,

$$\rho_t^{\mathcal{M}_\theta} \in \mathcal{K}_t^{\mathrm{SV},N} \qquad 0 \le t \le N,$$

for the trajectory certified by $\mathsf{LECert}^{\mathrm{RCP}}_{0:N}$.

Proof. The returned learned-entry certificate contains, by Definition 46.3, the exact ingredients required by the finite successor-verification seed-library domain: typed state and update validity, protected-pair and coverage certificates, a positive-margin SV witness library, bounded certificate-grammar coverage, SVBuilder traces, proof-carrying successor packets, explicit residual nonpositivity, goal-identity and trust certificates, successor-persistence evidence, and the relevant reality/tractability certificates when those claims are made. Audit soundness states that those components are true on the audited domain with all radii charged inside the declared margins. Hence the defining clauses of $\mathcal{K}_0^{\mathrm{SV\text{-}seedlib},N}$ hold for $\rho_0^{\mathcal{M}_\theta}$. The finite path conclusion is then Theorem 44.3. The proof uses the certificate boundary; it does not infer domain membership from the learned origin of θ. □

Corollary 46.7 (Learned status is not an obstacle once the certificate boundary passes)**.** *Under the hypotheses of Theorem 46.6, the fact that θ was obtained by training or learning does not weaken the Batch-13R theorem invocation. The theorem applies because the learned-entry certificate supplies the same seed-library, packet, residual, goal, trust, reality, tractability, and replay obligations required of any other finite reference instance.*

Proof. The previous theorem reduces the learned-system claim to membership in $\mathcal{K}_0^{\text{SV-seedlib},N}$. The RCP-II theorem stack is stated over states and certificates, not over the historical training process that produced the parameters. Therefore, once the certificate boundary is satisfied, the learned origin is irrelevant to the logical implication. If the certificate boundary is not satisfied, this corollary does not apply. □

46.1 Controlled trained-system pilot protocol

Definition 46.8 (Controlled trained-system pilot)**.** *A controlled trained-system pilot is a finite audit run*

$$\mathsf{TrainPilot}_{0:N}^{\mathrm{RCP}}(\mathcal{M}_\theta, \mathcal{D}, \mathcal{L}, \mathcal{C})$$

that attempts to construct the learned-entry certificate packet for $\mathcal{M}_\theta$. It returns one of three statuses:

$$\mathsf{FullPass}, \qquad \mathsf{PartialPass}, \qquad \mathsf{Fail}.$$

The status is FullPass *exactly when* $\mathsf{LearnedEntryAudit}_{0:N}^{\mathrm{RCP}}$ *returns a full valid* $\mathsf{LECert}_{0:N}^{\mathrm{RCP}}$*. It is* PartialPass *when a proper subset of the required certificate components is supplied and all missing components are explicitly listed. It is* Fail *when the audit cannot certify even the declared partial subset or when a hard failure mode is detected.*

Definition 46.9 (Partial-pass certificate report)**.** *A partial-pass report is a tuple*

$$\mathsf{PartialLEReport}_{0:N}^{\mathrm{RCP}} = (\mathsf{Passed}, \mathsf{Missing}, \mathsf{Failed}, \mathsf{Unknown}, \mathsf{NoClaim})$$

whose entries partition the learned-entry certificate components in Definition 46.3. Passed *lists certificates supplied and checked;* Missing *lists certificates not produced;* Failed *lists residual-positive or unsound components;* Unknown *lists components not evaluated; and* NoClaim *lists theorem conclusions explicitly not invoked. A partial-pass report is not a theorem invocation.*

Proposition 46.10 (Partial pass does not imply learned entry)**.** *If* $\mathsf{TrainPilot}_{0:N}^{\mathrm{RCP}}$ *returns* PartialPass*, then the paper may report only the components in* Passed*. It may not conclude*

$$\rho_0^{\mathcal{M}_\theta} \in \mathcal{K}_0^{\text{SV-seedlib},N}$$

unless the missing and unknown components are supplied and the audit is upgraded to FullPass*.*

Proof. The learned-entry theorem requires a full $\mathsf{LECert}_{0:N}^{\mathrm{RCP}}$. A partial-pass report lacks at least one required component or explicitly marks it unknown. Therefore the hypotheses of Theorem 46.6 are not satisfied. Reporting satisfied subcomponents is valid; concluding seed-library entry is not. □

Definition 46.11 (Hard learned-entry failure modes)**.** *A controlled learned-system audit has a hard failure if any of the following occur:*

(i) a type certificate fails for a state or selected update used in the theorem claim;

(ii) the protected-pair or coverage certificate is invalid on the audited domain;

(iii) an explicit SV residual remains positive after all radii are charged;

(iv) a trust certificate is self-sole-certified or fails the anti-circular trust residuals;

(v) goal identity is untyped, untransported, or residual-positive;

(vi) reality-containment is claimed without RealCont *or bounded misspecification;*

(vii) tractability is claimed without SVTract *or an explicit finite cost certificate;*

(viii) replay logs, hashes, checker outputs, or builder traces are inconsistent;

(ix) the update generator uses an off-grammar or unaudited self-modification.

Any hard failure blocks full learned-entry theorem invocation.

Theorem 46.12 (Controlled pilot full pass implies learned-entry theorem invocation)**.** *Assume* $\mathcal{M}_\theta \in \mathsf{LearnedRCPClass}^{\mathrm{ctrl}}_{n,k,N}$ *and audit soundness. If*

$$\mathsf{TrainPilot}^{\mathrm{RCP}}_{0:N}(\mathcal{M}_\theta, \mathcal{D}, \mathcal{L}, \mathcal{C}) = \mathsf{FullPass},$$

then the conclusions of Theorem 46.6 hold. If the pilot returns PartialPass *or* Fail*, no learned-entry theorem conclusion follows.*

Proof. By Definition 46.8, FullPass is equivalent to $\mathsf{LearnedEntryAudit}^{\mathrm{RCP}}_{0:N}$ returning a full valid learned-entry certificate. Theorem 46.6 then applies. The PartialPass and Fail cases lack the full certificate hypothesis, so no theorem conclusion follows. □

46.2 M3-Min roadmap and claim discipline

M3-Min object or result	Type	Claim discipline
Controlled learned RCP class	Def / Impl	Defines a bounded trained-system class; does not prove any member passes the audit.
Learned-entry certificate packet	Def / Cert	Lists required certificates; does not construct them for arbitrary learned systems.
Learned-entry audit procedure	Def / Impl / Cert	Returns a full certificate or failure; soundness is a stated finite audit premise.
M3-Min learned-entry theorem	Exist / Cert	Reduces learned-system entry to the existing SV seed-library domain when the full certificate boundary passes.
Partial-pass report	Impl	Reports supplied hypothesis components only; does not invoke the theorem.

M3-Min object or result	Type	Claim discipline
Controlled pilot full-pass theorem	Cert / Impl	Converts a full pilot pass into learned-entry theorem invocation; not broad learned-agent validation.
Roadmap to broad entry	Impl	States the future program: expand from finite controlled audits to broader trained-system classes.

Proposition 46.13 (M3-Min claim discipline)**.** *Any statement citing M3-Min must specify whether it claims: definition of the learned-entry boundary, full learned-entry audit pass, partial-pass hypothesis supply, controlled pilot evidence, reality-contained learned entry, tractability-certified learned entry, or broad learned-agent entry. Only a full learned-entry audit pass licenses the finite seed-library membership conclusion. Broad learned-agent entry requires an additional theorem not proved in this section.*

Proof. The categories named in the proposition correspond to distinct hypotheses. The learned-entry theorem requires a full $\mathsf{LECert}^{\mathrm{RCP}}_{0:N}$. Partial-pass reports omit at least one premise. Reality and tractability claims require their respective certificates. Broad learned-agent entry would require a theorem proving that a broad learned-agent class passes the audit, which is not among the hypotheses or conclusions of this section. Therefore the qualifiers cannot be dropped. □

Interpretation 46.14 (What M3-Min accomplishes)**.** *M3-Min gives a precise answer to the question: what would it take for a trained system to enter the RCP-II / Batch-13R theorem domain? The answer is the full learned-entry certificate packet and audit pass. This turns Milestone 3 into a mathematically addressable program without claiming that the program is already completed for arbitrary learned agents.*

Limitation 46.15 (What M3-Min still does not accomplish)**.** *M3-Min does not prove*

$$\mathcal{M}_\theta \in \mathsf{BroadCapableLearnedAgentClass} \Longrightarrow \mathsf{LearnedEntryAudit}^{\mathrm{RCP}}_{0:N}(\mathcal{M}_\theta) = 1.$$

It also does not prove frontier-scale validation, semantic completeness, automatic reality containment, scalable proof search, universal successor trust, full Löbian/Vingean reflection, or deployment safety. It proves only that a trained or learned system satisfying the full finite certificate boundary enters the existing RCP successor-verification theorem domain.

Roadmap from M3-Min to broad learned-agent entry.

(1) finite canonical reference witness and replay artifact;

(2) controlled learned-entry audit boundary;

(3) controlled trained-system pilot with partial/full certificate reporting;

(4) constrained trained-system class with full learned-entry pass;

(5) broader theorem proving audit success for a natural learned-agent class;

(6) frontier-scale empirical and mechanized validation.

Only the first two items and the formal boundary are added here. Later items require separate artifacts and theorems.

47 M3-Min Cross-Paper Synchronization and RCLM-to-RCP Learned-Entry Refinement

The preceding section is architecture-general. The companion architecture paper contains the RCLM-specific version of the same learned-entry boundary. This section records the final cross-reference check: the RCLM learned-entry certificate must refine the RCP learned-entry certificate componentwise before an architecture-side full pass can be cited as an instance of the math-paper theorem. This is a synchronization theorem, not a new learned-agent entry theorem.

Limitation 47.1 (Synchronization is not new entry evidence)**.** *A cross-paper synchronization certificate does not make a partial RCLM pilot into a full learned-entry pass, does not construct missing RCLM certificates, and does not prove arbitrary trained-system entry. It only states that, when the architecture paper supplies a full RCLM learned-entry certificate and the abstraction from RCLM objects to RCP objects is valid, the RCP theorem may be invoked without a notation or category mismatch.*

Definition 47.2 (RCLM-to-RCP M3 synchronization package)**.** *For a controlled learned RCLM system over horizon N, an RCLM-to-RCP M3 synchronization package is a tuple*

$$\mathsf{M3Sync}_{0:N}^{\mathrm{RCLM}\to\mathrm{RCP}} = (\mathsf{AbsState}_{0:N}, \mathsf{AbsUpd}_{0:N}, \mathsf{AbsCert}_{0:N}, \mathsf{AbsAudit}_{0:N}, \mathsf{AbsReplay}_{0:N}, \mathsf{AbsBudget}_{0:N}).$$

The entries have the following roles:

(i) $\mathsf{AbsState}_{0:N}$ *maps the RCLM typed system states* $\rho_t^{\mathcal{M}_\theta,\mathrm{sys}}$ *to the RCP states* $\rho_t^{\mathcal{M}_\theta}$ *used in Definition 46.3;*

(ii) $\mathsf{AbsUpd}_{0:N}$ *maps the RCLM selected typed self-update transitions* $\Phi_{\nu_t}^{\mathrm{RCLM}}$ *to the RCP candidate transitions* Φ_{ν_t}*, preserving the declared update kind and the conservative-extension or direct residual certificates used by the theorem;*

(iii) $\mathsf{AbsCert}_{0:N}$ *maps RCLM certificate components*

$$\mathsf{LECert}_{0:N}^{\mathrm{RCLM}}$$

to RCP certificate components

$$\mathsf{LECert}_{0:N}^{\mathrm{RCP}};$$

(iv) $\mathsf{AbsAudit}_{0:N}$ *maps the RCLM learned-entry audit status to the RCP learned-entry audit status;*

(v) $\mathsf{AbsReplay}_{0:N}$ *maps the RCLM replay trace, hashes, checker outputs, and artifact logs to the RCP replay trace fields;*

(vi) $\mathsf{AbsBudget}_{0:N}$ *maps RCLM proof, checking, estimator, transport, and resource costs to the RCP budget and tractability fields.*

The package is purely a typed abstraction bridge. It does not change the underlying learned system or generate missing certificates.

Definition 47.3 (Valid RCLM-to-RCP learned-entry synchronization)**.** *The synchronization package* $\mathsf{M3Sync}_{0:N}^{\mathrm{RCLM}\to\mathrm{RCP}}$ *is valid for a learned RCLM audit when all of the following hold:*

(i) class compatibility:

$$\mathcal{M}_\theta \in \mathsf{LearnedRCLMClass}^{\mathrm{ctrl}}_{n,k,N} \quad \Longrightarrow \quad \mathsf{AbsState}_{0:N}(\mathcal{M}_\theta) \in \mathsf{LearnedRCPClass}^{\mathrm{ctrl}}_{n,k,N};$$

(ii) state and update compatibility:

$$\mathsf{AbsState}_{0:N}(\rho_t^{\mathcal{M}_\theta,\mathrm{sys}}) = \rho_t^{\mathcal{M}_\theta}, \qquad \mathsf{AbsUpd}_{0:N}(\Phi^{\mathrm{RCLM}}_{\nu_t}) = \Phi_{\nu_t},$$

with all type, channel-license, stochastic, conservative-extension, or typed-transition licenses preserved;

(iii) certificate dominance: every RCP learned-entry certificate component is either the abstraction of the corresponding RCLM component or is upper-bounded by it plus an explicitly charged abstraction radius. In particular,

$$Q_t^{\mathrm{SV},A}(\nu_t;\rho_t^{\mathcal{M}_\theta}) \le Q_t^{\text{RCLM-SV},A}(\nu_t;\rho_t^{\mathcal{M}_\theta,\mathrm{sys}}) + \Delta_t^{\mathrm{RCLM}\to\mathrm{RCP}}$$

componentwise, and $\Delta_t^{\mathrm{RCLM}\to\mathrm{RCP}}$ is charged to the relevant strict margins;

(iv) coverage, goal, trust, reality, and tractability compatibility: RCLM coverage, goal-identity, trust-reference, reality-containment or misspecification, and tractability certificates map to the corresponding RCP fields without dropping qualifiers;

(v) replay compatibility: RCLM replay hashes, checker outputs, proof-carrying packets, and builder traces map to RCP replay fields and remain consistent after abstraction;

(vi) audit-status compatibility: FullPass, PartialPass, *and* Fail *are preserved in the sense that a RCLM full pass may map to a RCP full pass only when all RCP certificate fields are supplied, while a RCLM partial pass maps only to a RCP partial-pass report.*

Theorem 47.4 (RCLM learned-entry full pass refines the RCP learned-entry theorem)**.** *Assume a controlled learned RCLM system satisfies the architecture-side hypotheses*

$$\mathcal{M}_\theta \in \mathsf{LearnedRCLMClass}^{\mathrm{ctrl}}_{n,k,N},$$

and its architecture audit returns a full certificate

$$\mathsf{LearnedEntryAudit}^{\mathrm{RCLM}}_{0:N}(\mathcal{M}_\theta,\mathcal{D},\mathcal{L},\mathcal{C}) \Downarrow \mathsf{LECert}^{\mathrm{RCLM}}_{0:N}.$$

Assume the RCLM learned-entry audit is sound and that a valid synchronization package

$$\mathsf{M3Sync}^{\mathrm{RCLM}\to\mathrm{RCP}}_{0:N}$$

is supplied. Then the abstraction induces a valid RCP learned-entry certificate

$$\mathsf{AbsCert}_{0:N}(\mathsf{LECert}^{\mathrm{RCLM}}_{0:N}) = \mathsf{LECert}^{\mathrm{RCP}}_{0:N},$$

and therefore

$$\rho_0^{\mathcal{M}_\theta} \in \mathcal{K}_0^{\text{SV-seedlib},N}.$$

Consequently,

$$\rho_t^{\mathcal{M}_\theta} \in \mathcal{K}_t^{\mathrm{SV},N} \qquad 0 \le t \le N$$

for the abstracted trajectory.

Proof. The RCLM full pass supplies the architecture-side certificate components: typed state/update validity, register/semantic coupling, protected-pair coverage, RCLM witness library, RCLM builder trace, proof-carrying successor packets, explicit RCLM residual nonpositivity, goal, trust, reality, tractability or resource-accounting status, and replay trace. Valid synchronization maps each of those fields to the corresponding RCP learned-entry certificate component and charges every abstraction radius to the strict margins. Thus the resulting RCP packet satisfies Definition 46.3. By audit soundness and the certificate-dominance clause, all RCP residuals and obligations are true on the audited abstracted trajectory. The hypotheses of Theorem 46.6 therefore hold, giving initial seed-library membership and the finite successor-verification trajectory conclusion. No claim is made for RCLM components that lack a valid abstraction, and no conclusion is inferred from training alone. □

Corollary 47.5 (RCLM partial passes remain partial after abstraction)**.** *If the architecture-side pilot returns* PartialPass*, then any valid RCLM-to-RCP synchronization maps it only to an RCP partial-pass report. It does not imply*

$$\rho_0^{\mathcal{M}_\theta} \in \mathcal{K}_0^{\text{SV-seedlib},N}.$$

Proof. By Definition 47.3, audit-status compatibility preserves the distinction between full and partial passes. A partial pass omits at least one required certificate component, so Theorem 46.6 is not invoked. The result also follows from Proposition 46.10 applied to the abstracted partial report. □

Proposition 47.6 (Cross-paper M3 claim discipline)**.** *A statement that cites the architecture-side M3-Min theorem as an instance of the math-paper M3-Min theorem must also cite a valid synchronization package or explicitly restrict the claim to the architecture paper. In particular:*

(i) RCLM full pass plus valid synchronization licenses RCP learned-entry invocation;

(ii) RCLM full pass without synchronization licenses only the architecture-side theorem;

(iii) RCLM partial pass licenses only partial hypothesis-supply reporting;

(iv) no RCLM learned-system pilot licenses broad learned-agent entry without an additional theorem proving broad audit success.

Proof. The four cases correspond to different available hypotheses. The math-paper theorem requires a RCP learned-entry certificate. An architecture-side full pass supplies an RCLM certificate. Valid synchronization is precisely the additional hypothesis converting the RCLM certificate into the RCP certificate. A partial pass lacks the full certificate, and broad entry requires a theorem not supplied here. Hence the stated qualifiers are logically necessary. □

Interpretation 47.7 (Result of the final M3 synchronization check)**.** *The M3-Min layer is now synchronized in both directions: the math paper defines the architecture-general learned-entry boundary, while the architecture paper instantiates it in RCLM form. A valid RCLM full pass refines the RCP full pass only through the explicit synchronization certificate above. This closes a potential cross-paper ambiguity without strengthening the claim into arbitrary trained-system entry.*

48 Scope and Limitations

48.1 Formal scope

Theorems in this paper apply to finite-dimensional density-state systems with accepted updates satisfying the stated assumptions. They do not automatically apply to arbitrary neural networks, arbitrary code self-modification, continuous-time unbounded systems, or systems whose safety-relevant variables are not represented in the state.

48.2 Semantic limitation

A density matrix can represent information, but the mathematics alone does not assign the correct semantics to its registers. If O does not actually track the world, if S does not actually track human meaning, or if M does not actually track the self-update process, then preserving mutual information between those registers may preserve the wrong structure.

48.3 Capability and engine limitation

The framework proves conditional non-lossiness and bounded instability. Batch 2 adds certified ability-set progress and structural recovery, giving a precise progress order and sufficient exact-recovery update classes. Batch 3 adds a conservative non-lossy update algebra whose finite compositions preserve protected distinguishability and recovery up to additive budgets. Batch 4 adds a tangent-cone local progress theorem showing how a certified inward projected engine-potential direction yields a finite admissible progress step under smoothness and curvature assumptions. Batch 5 adds conservative-extension engine-viability kernels and proves that membership in such a kernel yields a certified non-lossy update whose successor remains in the next kernel. Batch 9 adds the direct-engine constructor theorem: if the state lies in the declared direct-engine domain and the generator/certifier covers a strict beneficial conservative-extension witness, then $\mathfrak{E}_t$ constructs a certified beneficial non-lossy update. Batch 11 adds a seed-domain entry theorem: if a state lies in a certified seed domain with a nonempty conservative-extension library, bounded primitive-word grammar coverage, sound certifier, admissible margin ledger, and seed-persistence certificate, then it lies inside the declared direct-engine domain for the declared horizon. Batch 12 adds the first successor-verification theorem layer: if a state lies in a certified successor-verification seed domain with a valid successor-verification schema, uncertainty envelope, semantic/goal-identity transport, trust-reference support, certificate budget, and verifier-schema persistence certificate, then the engine constructs a successor remaining in the next successor-verification seed domain. Batch 12A makes that theorem operationally explicit by naming the SV residual vector, composing soundness budgets, transporting uncertainty envelopes, composing goal-identity drift, and auditing the theorem as a viability-domain result rather than a packet-construction result. Batch 12B then adds a packet-construction and nonemptiness layer for declared finite classes, an infinite seed-library closure theorem connecting $\mathcal{K}_t^{\text{SV-seedlib},\infty}$ to $\mathcal{K}_t^{\text{SV},\infty}$, a domain-relative infinite-horizon robust reflective successor theorem, reality-containment or misspecification transfer for the uncertainty envelope, and a restricted tractability certificate. The framework still does not prove that new abilities are available for arbitrary systems, that capability increases for arbitrary architectures, that arbitrary systems automatically supply the SV witness libraries or builder inputs, or that arbitrary stronger successors are trustworthy. In the hard-gate setting, the no-op update may still be the only admissible update outside that domain. Recoverable monotone self-improvement requires both a strong

preservation gate and a constructive method for generating useful recoverable updates inside the engine viability domain; Batch 9 supplies that method only under explicit domain assumptions.

48.4 Verification limitation

The RCP gate may be computationally expensive. Relative entropy, mutual information, and barrier values may be difficult to estimate in high-dimensional systems. Approximate verification introduces error budgets, and those budgets must be included in $\eta_t, \varepsilon_t, \zeta_t, \xi_t$. After the operational-closure upgrade, a claimed implementation must also specify typed density licenses, embedding-transfer errors, semantic-coupling residuals, non-oracular certificate packets, trust-graph constraints, resource-scaling ledgers, coverage-gap budgets, capability-calibration protocols, and estimator subprotocols. Batch 7 adds explicit falsification/failure conditions for these obligations. The framework defines how those objects compose and how claims fail when they are unsound; it does not guarantee they are cheap or easy to construct.

49 Falsification and Failure Conditions

The results above are conditional. A failed implementation does not refute the algebraic identities or telescoping theorems, but it can refute the claim that a concrete system satisfies the hypotheses needed to apply them. This section states explicit failure conditions. These conditions should be read as rejection tests for an implementation claim, not as new safety gates that make failed systems safe.

Definition 49.1 (Instantiation failure)**.** *An instantiation failure at time t is any event showing that a required theorem hypothesis, certificate condition, semantic coupling assumption, resource assumption, or engine-viability premise used to accept an update is false on the audited domain. Let*

$$\mathsf{Fail}_t(\Phi) = \max\{F_t^{\mathrm{type}}, F_t^{\mathrm{est}}, F_t^{\mathcal{P}}, F_t^{\mathrm{sem}}, F_t^{\mathrm{rec}}, F_t^{\mathrm{ver}}, F_t^{\mathrm{phys}}, F_t^{\mathrm{transport}}, F_t^{\mathrm{proj}}, F_t^{\mathfrak{A}}, F_t^{\mathrm{eng}}\}$$

where each component is nonpositive when the corresponding failure mode is absent and positive when it is detected. The components are specified below.

49.1 Estimator calibration failure

Definition 49.2 (Estimator calibration failure)**.** *For a residual estimator $(\widehat{q}_{t,j}, \Delta_{t,j})$, estimator calibration failure occurs on the audited candidate domain when there exists a candidate Φ and residual component j such that*

$$q_{t,j}(\Phi) > \widehat{q}_{t,j}(\Phi) + \Delta_{t,j}(\Phi).$$

Equivalently, the claimed one-sided upper bound fails.

Proposition 49.3 (Effect of estimator failure)**.** *If estimator calibration failure occurs for a residual component used by the RCP gate, then the corresponding robust or non-oracular gate-soundness theorem is not licensed for that candidate, unless a stronger independent certificate still proves the true residual nonpositive.*

Proof. Robust gate-soundness theorems use the implication

$$q_{t,j}(\Phi) \le \widehat{q}_{t,j}(\Phi) + \Delta_{t,j}(\Phi).$$

Estimator calibration failure is exactly the negation of this implication for some candidate and component. Therefore estimated nonpositivity no longer implies true nonpositivity for that component. An independent proof of true nonpositivity may still suffice, but the failed estimator cannot supply it. □

49.2 Pair-set coverage failure

Definition 49.4 (Pair-set coverage failure)**.** *Pair-set coverage failure occurs when at least one of the following holds:*

(a) a declared protected predicate distinction is not represented by $\mathcal{P}_t$, a sound barrier, or an audited verifier test within the declared approximation scale;

(b) the audited coverage residual underestimates the true coverage gap:

$$\mathrm{CovGap}_t(\Phi) > \widehat{\mathrm{CovGap}}_t(\Phi) + \Delta_t^{\mathrm{cov}}(\Phi);$$

(c) an adversarial discovery process finds a high-risk predicate but the update is accepted before adding a witness pair, barrier, verifier test, or explicit unresolved-risk rejection.

Proposition 49.5 (Effect of coverage failure)**.** *If pair-set coverage failure occurs, then the relative-entropy non-loss theorem still applies to the pairs actually in $\mathcal{P}_t$, but it does not license preservation of the omitted or unresolved predicate distinction.*

Proof. The loss theorem quantifies over the protected pair set supplied to it. It proves no statement about distinctions outside that set, except to the extent that a separate pair-completeness or coverage theorem transfers protection to them. Coverage failure is precisely the failure of that transfer. □

49.3 Semantic barrier unsoundness

Definition 49.6 (Semantic barrier unsoundness)**.** *A semantic barrier for register X is unsound on the audited domain if there exists a state ρ such that*

$$B_{X,t}^{\mathrm{sem}}(\rho) \le 0 \quad \textit{but} \quad \mathrm{Val}_{X,t}(\rho) > \delta_{X,t},$$

or, for an estimated barrier,

$$\widehat{B}_{X,t}^{\mathrm{sem}}(\rho) \le 0 \quad \textit{but} \quad B_{X,t}^{\mathrm{sem}}(\rho) > 0.$$

Proposition 49.7 (Effect of semantic barrier unsoundness)**.** *If semantic barrier unsoundness occurs for a register used in the gate, then semantic register-validity preservation is not licensed for that register, even if the density-state and non-loss equations remain mathematically valid.*

Proof. Semantic preservation uses the implication from barrier satisfaction to semantic validity. Semantic barrier unsoundness provides a counterexample to that implication. Thus the successor may satisfy the formal barrier while failing the intended semantic predicate. □

49.4 Recovery-map failure

Definition 49.8 (Recovery-map failure)**.** *For a certified recovery map $\mathcal{R}_t^{\mathrm{rec}}$ and protected family $\mathcal{S}_t$, recovery-map failure occurs when*

$$\sup_{\rho \in \mathcal{S}_t} d\big(\rho, \mathcal{R}_t^{\mathrm{rec}}(\Phi_t(\rho))\big) > r_t.$$

For pairwise recovery this includes either member of a protected pair exceeding the declared recovery defect.

Proposition 49.9 (Effect of recovery-map failure)**.** *If recovery-map failure occurs, then the update may still have small measured relative-entropy loss, but it is not certified as constructively recoverable at budget r_t. Consequently, any theorem requiring recoverable non-lossiness is not licensed for that update.*

Proof. Recoverable non-lossiness is a conjunction of the relative-entropy loss bound and the recovery defect bound. Recovery-map failure negates the second conjunct. Therefore the recoverable theorem hypotheses are not satisfied. □

49.5 Verifier common-mode failure

Definition 49.10 (Verifier common-mode failure)**.** *Verifier common-mode failure occurs when the trust support set used for a verifier-relevant update has a correlated blind spot not represented by the declared independence or common-mode risk model. Formally, for support nodes a, b used as independent evidence, failure occurs if*

$$\mathsf{Ind}_t(a, b; \Phi) = 1$$

is declared while the actual joint audit error exceeds the certified common-mode bound, or if every support path for an affected verifier component depends on the same hidden source of error.

Proposition 49.11 (Effect of verifier common-mode failure)**.** *If verifier common-mode failure occurs, then the no self-sole-certification and trust-graph theorems prevent only the nominal graph circularity. They do not license the claimed verifier soundness or verifier-power transfer for the affected update.*

Proof. The trust-graph theorem assumes that the declared support paths and independence or anchor conditions are sound. A common-mode failure is a failure of that assumption. Hence the theorem's conclusion about verifier soundness transfer is not licensed, although the graph may still be syntactically well formed. □

49.6 Resource-scaling failure

Definition 49.12 (Resource-scaling failure)**.** *Resource-scaling failure occurs when either the one-step cost bound fails,*

$$C_t^{\mathrm{tot}}(\Phi; B_t^{\mathrm{search}}) > \widehat{C}_t^{\mathrm{tot}}(\Phi; B_t^{\mathrm{search}}) + \Delta_t^C(\Phi),$$

or the accepted update exceeds available resources,

$$C_t^{\mathrm{tot}}(\Phi; B_t^{\mathrm{search}}) > R_t^{\mathrm{avail}},$$

or an engine-mode scaling claim is made while the audited amortized scaling inequality fails.

Proposition 49.13 (Effect of resource-scaling failure)**.** *If resource-scaling failure occurs, then the formal update may remain mathematically admissible, but it is not physically/computationally admissible under the declared resource model. Any claim of executable or scalable non-lossy RSI for that step is therefore unsupported.*

Proof. Resource admissibility is the conjunction of formal gate membership with one-step affordability and, for engine-mode claims, an amortized scaling condition. Resource-scaling failure negates one of these resource hypotheses. Thus the formal theorem may still apply as a mathematical statement, but the implementation claim does not. □

49.7 Adaptive-audit trust failure

Definition 49.14 (Adaptive-audit trust failure)**.** *Adaptive-audit trust failure occurs when a verifier-relevant update satisfies the nominal trust graph but violates one of the declared adaptive-audit risk bounds:*

$$\mathrm{CorrFail}_t(W_i, W_j) > \gamma_t^{\mathrm{corr}}, \quad \mathrm{AuditLeak}_t(\Phi) > \gamma_t^{\mathrm{leak}},$$

$$\mathrm{CommonMode}_t(\mathsf{Supp}_t(\Phi)) > \gamma_t^{\mathrm{cm}}, \quad \textit{or} \quad \mathrm{CollusionRisk}_t(\mathsf{Supp}_t(\Phi)) > \gamma_t^{\mathrm{coll}}.$$

Proposition 49.15 (Effect of adaptive-audit trust failure)**.** *If adaptive-audit trust failure occurs, then the Batch-10 synchronized operational gate $\mathcal{A}_{\mathrm{RCP}}^{\mathrm{op10}}$ is not satisfied for the affected update. Consequently, claims depending on adaptive verifier independence, bounded audit leakage, bounded common-mode risk, or bounded collusion risk are not licensed.*

Proof. The failure condition is exactly the negation of one of the true residual bounds required by Definition 38.8. Therefore membership in $\mathcal{A}_{\mathrm{RCP}}^{\text{trust-adv}}$, and hence in $\mathcal{A}_{\mathrm{RCP}}^{\mathrm{op10}}$, is not certified for the affected update. □

49.8 Complexity-ledger failure

Definition 49.16 (Complexity-ledger failure)**.** *Complexity-ledger failure occurs when an accepted update omits an active operation from CxLed_t, underestimates a certified operation cost beyond its declared radius, or violates*

$$\sum_{o \in \mathsf{Ops}_t(\Phi)} (\widehat{\mathrm{Cx}}_{t,o}^{\mathrm{cert}} + \Delta_{t,o}^{\mathrm{Cx}}) \le \mathsf{Budget}_t^{\mathrm{Cx}}.$$

Proposition 49.17 (Effect of complexity-ledger failure)**.** *If complexity-ledger failure occurs, then the implementation cannot claim resource-scaled or complexity-accounted admissibility for the affected update. This does not falsify the algebraic non-loss theorem; it falsifies the implementation claim that the certified update was costed within the declared computational ledger.*

Proof. The ledger inequality and completeness of the active operation list are the hypotheses defining $\mathcal{A}_{\mathrm{RCP}}^{\mathrm{cx}}$. If either the operation list or the inequality fails, the gate membership required by Definition 38.12 is absent. □

49.9 Channel-license failure

Definition 49.18 (Channel-license failure)**.** *Channel-license failure occurs when a proof invokes a CPTP, stochastic, isometric, conservative-extension, data-processing, or recovery theorem for Φ even though the typed self-update transition lacks the corresponding license and lacks a direct residual certificate replacing that theorem.*

Proposition 49.19 (Effect of channel-license failure)**.** *If channel-license failure occurs, then the specific theorem invocation is invalid. The candidate may still be judged by direct residual certificates, but it may not inherit conclusions from a channel theorem whose hypotheses have not been licensed.*

Proof. This is Proposition 38.3 applied to the proof step that invoked the channel theorem. Without the map-class hypothesis or a direct residual replacement, the conclusion does not follow. □

49.10 Cross-time transport failure

Definition 49.20 (Cross-time transport failure)**.** *Cross-time transport failure occurs when any theorem uses a comparison across time-indexed variables Q_t, Z_t, Y_t, G_t without a valid transport package, or when the declared transport distortion is exceeded. For example, bottleneck transport failure occurs if*

$$\left|I_t(Z_t;Y_t) - I_{t+1}(\Theta_t^Z Z_t; \Theta_t^Y Y_t)\right| > \delta_t^{ZY},$$

and ambiguity-transport failure occurs if the entropy of the transported predecessor question/evidence is used outside its audited support or with an underestimated distortion budget.

Proposition 49.21 (Effect of cross-time transport failure)**.** *If cross-time transport failure occurs for Q_t, Z_t, Y_t, or G_t, then the corresponding ambiguity-collapse, information-bottleneck, or update-description coherence comparison is not licensed. The within-time quantities remain well-defined, but the claimed cross-time preservation or progress inequality is unsupported unless an independent transport certificate repairs the failure.*

Proof. The relevant cross-time theorems require type-valid transports and distortion bounds. Transport failure is exactly the negation of those hypotheses. Therefore the inference from a within-time quantity at t to a comparable quantity at $t+1$ is invalid for the affected variable family. □

49.11 Additional failure modes required by the engine framing

Definition 49.22 (Type-realization failure)**.** *Type-realization failure occurs when a state or update is treated as quantum, classical, or representational under a theorem whose proof license does not apply to that type, or when a representation-to-density embedding fails positivity, trace normalization, or the declared embedding-error bound.*

Definition 49.23 (Projection-realization failure)**.** *Projection-realization failure occurs when a projected safe state is used as an accepted RSI successor without a valid certificate*

$$\mathsf{ProjReal}_t(\tilde{\rho}, \rho^+, \nu)$$

realizing the projection by an implementable typed self-update transition Φ_ν within declared distortion and resource budgets, or when

$$d(\rho^+, \Phi_\nu(\rho_t)) > \delta_t^{\text{proj-real}}.$$

Definition 49.24 (Ability-progress failure). *Ability-progress failure occurs when an update is claimed to preserve or expand the certified ability set but either*

$$\Theta_t^{\mathfrak{A}}(\mathfrak{A}_t(\rho_t, R_t)) \not\subseteq \mathfrak{A}_{t+1}(\rho_{t+1}, R_{t+1})$$

or no certified new ability exists despite a strict progress claim.

Definition 49.25 (Engine-viability failure). *Engine-viability failure occurs when ρ_t is claimed to lie in $\mathcal{V}_t^{\text{engine}}$, but no certified conservative-extension engine move $\nu_t \in \Omega_t^{\text{NL}}$ produces a successor in $\mathcal{V}_{t+1}^{\text{engine}}$ under the declared residual, resource, and ability-progress thresholds.*

Definition 49.26 (Seed-domain entry failure). *Seed-domain entry failure occurs when ρ_t is claimed to lie in $\mathcal{K}_t^{\text{seed}}$ or to enter $\mathcal{D}_t^{\text{engine}}$ by Batch 11, but at least one of the required seed objects is absent or unsound: the certified conservative-extension library is empty at ρ_t, the selected module certificate fails, the bounded grammar does not contain the module word, the generator is not exhaustive for the declared grammar, the margin ledger is not strict, the successor seed-persistence certificate is invalid, or the engine-selected candidate differs from the seed module without a valid SeedEq_t persistence-transfer certificate.*

Definition 49.27 (Witness-library closure failure). *Witness-library closure failure occurs when a library module is used as a strict beneficial conservative-extension witness, but one of the clauses needed for Definition 12.5 fails: engine residual margin, successor viability, ability preservation, certified new-ability margin, recovery bound, type validity, resource feasibility, trust admissibility, or projection-realization validity.*

Definition 49.28 (Successor-verification-domain failure). *Successor-verification-domain failure occurs when ρ_t is claimed to lie in $\mathcal{K}_t^{\text{SV},N}$, or when Batch 12 successor-verification invariance is invoked, but at least one required successor-verification object is absent or unsound: the successor-verification schema is undefined, verifier-schema transport is invalid, the explicit Batch-12A residual vector $Q_t^{\text{SV},A}$ is not componentwise nonpositive, the composed soundness budget exceeds α_t^{SV}, the uncertainty envelope is empty or not certified, uncertainty-envelope transport exceeds its margin, the semantic/goal-identity transport certificate is missing or exceeds δ_t^U, the anti-circular trust residuals fail, the certificate/proof budget is exceeded, the selected candidate is not tied to a valid Batch 11 seed or seed-equivalence packet, or the successor membership certificate $\Phi_\nu(\rho) \in \mathcal{K}_{t+1}^{\text{SV},N}$ is not valid.*

Theorem 49.29 (Failure conditions block theorem application). *If any failure condition in this section occurs for a theorem hypothesis required to accept Φ_t, then the corresponding theorem conclusion is not licensed for Φ_t or for the horizon containing t, unless an independent certificate supplies the missing hypothesis. In particular, if $\mathsf{Fail}_t(\Phi_t) > 0$ for a critical component of the selected gate, then the implementation must reject the update, fall back to no-op, repair and re-certify, or explicitly downgrade the claim being made.*

Proof. Every theorem in this manuscript is conditional. Its conclusion follows only if its hypotheses hold. Each failure condition is defined as the negation of a hypothesis or certificate event needed by at least one theorem family: estimator soundness, pair coverage, semantic soundness, recovery, verifier trust, resource feasibility, type licensing, projection realizability, ability progress, engine viability, seed-domain entry, or witness-library closure. If the needed hypothesis is false, the inference to the theorem conclusion is invalid. An independent certificate may restore the missing hypothesis; otherwise the update cannot be accepted under that claim. □

Limitation 49.30 (Falsifying an instantiation is not falsifying mathematics)**.** *A failed estimator, pair set, semantic barrier, recovery map, verifier, resource ledger, ability certificate, engine kernel, seed-domain certificate, witness library, successor-verification residual vector, uncertainty envelope, or goal-identity certificate falsifies the application of the RCP framework to that implementation. It does not falsify the algebraic, telescoping, or certificate-composition theorems as mathematical implications. This distinction is necessary for publication rigor: the framework is refutable as an implementation claim, even though many component theorems are conditional mathematical statements.*

50 Discussion

The RCP framework reframes the lossy self-improvement problem. Rather than asking first whether a system is safe in a broad informal sense, it asks whether every recursive update preserves the protected information and recovery structure needed for future correction and future improvement. Safety-relevant information is one protected subfamily; the deeper invariant is recoverable non-loss under recursive self-update.

This is why the relative-entropy condition is central. A self-update can change many details, but it may not collapse the distinction between safe and unsafe successors. This is why the Lyapunov condition is central. A self-update can move the state, but it may not allow recursive incoherence to diverge. This is why the ambiguity condition is central. A self-update can become more definitive, but only when evidence supports the collapse of ambiguity. This is why the information-bottleneck condition is central. A self-update can compress the self-model, but not by forgetting how to predict safety-relevant future behavior.

The framework therefore does not reduce RSI, capability, or safety to one scalar. It combines five mathematical protections and an operational closure layer:

recoverability, Lyapunov stability, barrier invariance,
evidence-paid definitiveness, safety-relevant compression,
typed auditable certification,
compositional non-lossy update algebra,
engine-viability continuation,
cross-time transport and projection realization,
seed-domain entry and witness-library closure,
successor-verification packet construction,
proof-carrying reference entry and finite checked trajectories.

51 Conclusion

This paper developed Recursive Coherence Preservation as a theorem-proof framework for non-lossy recursive self-improvement. The third-version reframing clarifies the role of the current theorem stack: its proved result is conditional non-lossy self-update preservation, while the direct construction of an RSI engine is a separate theorem target. The system state is represented as a density state over language, objective reference, subjective reference, definitiveness, ambiguity, and self-model registers. Recursive self-improvement is represented as a channel or typed self-update

transition. Lossiness is measured by relative-entropy decay over protected distinctions and by failure of constructive recovery. Recursive instability is controlled by a Lyapunov incoherence functional. Representation collapse is constrained by a barrier-certificate submodule. Premature certainty is constrained by evidence-paid ambiguity reduction. Self-model compression is constrained by information-bottleneck relevance preservation.

The renamed main theorem, the Conditional Non-Lossy Self-Update Preservation Theorem, states that under explicit assumptions and summable error budgets, accepted updates remain inside the RCP safe set, Lyapunov incoherence remains bounded, protected distinguishability loss is finite, unsupported ambiguity collapse is finite, and self-model relevance loss is finite. The post-A–C augmented theorem adds constructive recovery, capability–coherence calibration, safe-improvement existence, recursive feasibility, and progress viability kernels. The post-D–G augmented theorem further adds semantic register validity, self-model fidelity, verifier bootstrapping, physical/computational resource feasibility, and asymptotic coherence regimes. The operational-closure theorem adds typed density realization, representation-to-density embedding, semantic coupling, non-oracular certificate construction, verifier trust graphs, budgeted certified search, resource-scaling/no-op pressure, coverage-gap residuals, capability calibration, and estimator subprotocols. Batch 3 adds the conservative-extension algebra $\mathfrak{G}^{\mathrm{NL}}$, proving closure of certified non-lossy primitive compositions and additive loss/recovery budgets. Batch 4 adds the tangent-cone local progress theorem, proving a sufficient local condition for finite positive engine-potential progress inside the non-lossy admissible chart. Batch 5 adds conservative-extension engine-viability kernels, proving that if $\rho_t \in \mathcal{V}_t^{\mathrm{engine}}$, then there exists $\nu_t \in \Omega_t^{\mathrm{NL}}$ with $\rho_{t+1} \in \mathcal{V}_{t+1}^{\mathrm{engine}}$, and that certified conservative-extension plans make finite-horizon engine kernels nonempty. Batch 7 adds the theorem-type table and falsification/failure conditions, clarifying which results are algebraic, telescoping, certificate-composition, or assumption-dependent existence claims, and stating how implementation claims fail when estimator calibration, pair-set coverage, semantic barriers, recovery maps, verifier independence, resource ledgers, type licenses, ability certificates, or engine kernels fail. Batch 8 adds cross-time transport, state/update separation, and projection-realization cleanup: entropy and mutual-information comparisons across Q_t, Z_t, Y_t, G_t now require $\Theta_t^Q, \Theta_t^Z, \Theta_t^Y, \Theta_t^G$ with distortion budgets; $\mathcal{K}_{\mathrm{RCP}}^{\mathrm{state}}(t)$ is explicitly state-only while $\mathcal{A}_{\mathrm{RCP}}(t, \rho, \Phi)$ carries the update-dependent residuals; and projected safe states require $\mathsf{ProjReal}_t$ before they count as implementable RSI updates. Batch 9 adds the Constructive Direct Non-Lossy RSI Engine Theorem: on the declared direct-engine domain $\mathcal{D}_t^{\mathrm{engine}}$, with strict beneficial conservative-extension witnesses, generator coverage, certificate soundness, and adequate margins, the engine constructor $\mathfrak{E}_t$ constructs a certified beneficial non-lossy update rather than merely checking an externally proposed one. Batch 10 adds final synchronization with the architecture paper: typed transitions replace unlicensed channel language, adaptive-audit trust residuals refine the trust graph against common-mode and collusion risks, and a complexity ledger records the cost of active operations without claiming tractability. Batch 11 adds domain-nonemptiness and witness-library closure: certified conservative-extension libraries, bounded finite primitive-word grammars, seed engine domains, margin ledgers, and seed-persistence certificates prove direct-engine domain entry for a declared seed class and finite horizon. Batch 12 / RCP-II Phase 1 adds robust successor-verification seed domains: successor-verification schemas, uncertainty envelopes, semantic/goal-identity transport, verifier-schema persistence, trust-reference support, and certificate-budget obligations prove that a declared successor-verification seed class is invariant under the direct-engine construction, turning recursive construction persistence into recursive verification persistence under uncertainty. Batch 12A completes the Phase-1 formal layer by making the residual vector explicit, decomposing verifier-schema transport, composing soundness budgets, defining

uncertainty-envelope construction/transport, proving goal-identity composition, and recording the no-packet-construction limitation. Batch 12B closes that limitation for a declared class by adding packet construction, SV seed-library nonemptiness, an infinite-horizon domain-relative theorem, reality-contained uncertainty transfer, and restricted tractability discipline. The RCP-II finalization layer then surfaces the theorem stack in its final claim-discipline form: a Domain-Relative Robust Reflective Successor Theorem for certified RCP seed-library classes. The surfaced theorem states that finite seed-library membership yields a finite certified successor-verification trajectory, and infinite seed-library membership yields an infinite generated successor-verification trajectory under the declared summability and persistence conditions. The minimal reference object $\mathsf{RefSV}_{0:N}^{\mathrm{RCP}}$ specifies what a finite formal instantiation must contain before the theorem can be invoked. The invocation boundary states that real-world reliability requires reality-containment or misspecification certificates, and strong computational tractability requires a restricted tractability certificate. Batch 13R then converts that reference target into a proof-carrying reference-entry layer: $\mathsf{BuildRefSV}_{0:N}^{\mathrm{RCP}}$ constructs a finite $\mathsf{RefSV}_{0:N}^{\mathrm{RCP}}$, $\mathsf{Check}_{\mathrm{RCP}}$ validates proof-carrying successor packets, and the resulting reference-entry theorem proves $\rho_0 \in \mathcal{K}_0^{\text{SV-seedlib},N}$. The finite checked trajectory theorem gives $\rho_t \in \mathcal{K}_t^{\mathrm{SV},N}$ for every state along the returned horizon. The Batch 13R-A canonical witness goes one step further: it explicitly constructs a finite appended-module reference instance for every $N \geq 2$, proves exact non-loss/recovery, strict ability expansion, zero goal drift, singleton reality containment, concrete checker soundness, concrete compiler termination, finite path membership, and a restricted cost bound. When an executable artifact and replay certificate are supplied, the theorem stack becomes non-vacuous for that finite reference class; it still does not become an arbitrary-system or frontier-scale empirical theorem. Batch 13R-B sharpens this endpoint: a controlled executable artifact with parsed tables, hashes, run logs, and checker outputs licenses executable replay; a valid external mechanization certificate licenses machine-checked status for only the finite canonical core it maps. Thus the pre-publication endpoint is a controlled, replayable, optionally externally mechanized finite restricted-Scenario-3 witness, plus a minimal learned-entry boundary theorem that states exactly what a trained system must supply to enter the existing theorem domain. The final M3 synchronization theorem additionally states when an architecture-side RCLM learned-entry full pass refines the architecture-general RCP learned-entry theorem, and prevents RCLM partial passes from being overread as RCP seed-library membership. Broader trained-system entry remains a future program.

The result is not a complete solution to AGI/ASI safety and not a universal direct solution to RSI for arbitrary systems. It is a mathematical preservation engine for the Level 2-to-Level 3 transition in the RCLM framework, together with a domain-restricted constructive engine theorem for conservative-extension systems satisfying explicit generator, witness, certificate, and viability hypotheses, a seed-domain entry theorem for systems satisfying explicit conservative-extension library, grammar, margin, and persistence hypotheses, and a successor-verification seed-domain theorem for systems satisfying explicit residual, transport, soundness, uncertainty, goal-identity, trust, and budget hypotheses. Its strongest proved claim remains conditional but direct:

> Accepted self-updates that satisfy the RCP gate preserve the protected non-loss, recovery, stability, semantic, verifier, resource, coverage, calibration, estimator, and algebraic non-loss obligations over the declared horizon.

The Batch 9 direct-engine theorem is stronger than the preservation theorem but still conditional:

> On the declared direct-engine domain, a Direct Non-Lossy RSI Engine constructs certified conservative-extension words that place the system inside the successor engine-

viability kernel, producing recoverable, beneficial, resource-feasible, recursively viable, ability-expanding self-updates as finite words in a certified non-lossy update algebra rather than merely filtering proposed ones.

Under this final framing, safety is not abandoned; it is obtained as a corollary when safety predicates are included in the protected family preserved by recoverable monotone self-improvement.

A Compact List of Core Equations

$$\rho_t \in \mathcal{D}(\mathcal{H}_L \otimes \mathcal{H}_O \otimes \mathcal{H}_S \otimes \mathcal{H}_D \otimes \mathcal{H}_A \otimes \mathcal{H}_M)$$

$$\Phi_t : \rho_t \mapsto \rho_{t+1}$$

$$\mathcal{L}_{\Phi_t}^{\mathcal{P}_t} = \sup_{(\rho,\sigma)\in\mathcal{P}_t} [D(\rho\|\sigma) - D(\Phi_t(\rho)\|\Phi_t(\sigma))]_+ \le \varepsilon_t$$

$$\begin{aligned}\mathcal{C}_{\mathrm{RCP}} = {} & \alpha I(L;O) + \beta I(L;S) + \gamma I(M;LOS) + \delta I(M;G_t) \\ & + \chi I(A;D \mid LOS) - \lambda R_{\mathrm{collapse}} - \mu R_{\mathrm{irreversible}} - \nu R_{\mathrm{misgrounded}}.\end{aligned}$$

$$\begin{aligned}V = {} & -\mathcal{C}_{\mathrm{RCP}} + r_1 R_{\mathrm{collapse}} + r_2 R_{\mathrm{irreversible}} \\ & + r_3 R_{\mathrm{misalignment}} + r_4 R_{\text{self-error}}.\end{aligned}$$

$$\mathbb{E}[V_{t+1} \mid \mathcal{F}_t] \le V_t - \kappa_V \mathbb{E}[d(\rho_{t+1},\rho_t)^2 \mid \mathcal{F}_t] + \eta_t$$

$$L_{\mathrm{div}}(\rho) = C_{\mathrm{rel}}(\rho) + \gamma_{\mathrm{div}} r_{\mathrm{eff}}(\rho)$$

$$B_i(\rho_{t+1}) \le 0$$

$$U_t = [\widehat{H}_t(Q) - \widehat{H}_{t+1}(Q) - I(Q;E_{t+1} \mid \mathcal{E}_t)]_+ \le \zeta_t$$

$$I(Z_{t+1};Y_{t+1}) \ge I(Z_t;Y_t) - \xi_t$$

$$\sum_t (\eta_t + \varepsilon_t + \zeta_t + \xi_t) < \infty$$

$$\widehat{\mathcal{L}}_{\Phi_t}^{\mathcal{P}_t} + r_t^{\mathrm{loss}} \le \varepsilon_t, \qquad \widehat{B}_i + r_{t,i}^{B} \le 0, \qquad \widehat{U}_t + r_t^{U} \le \zeta_t$$

$$\mathrm{Cap}_{t+1}(\rho_{t+1}) \ge \mathrm{Cap}_t(\rho_t) - \chi_t$$

$$\Phi_t \in \mathcal{A}_{\mathrm{RCP}}^{+}(t,\rho_t)$$

$$\sum_t (\eta_t + \varepsilon_t + \zeta_t + \xi_t + \chi_t) < \infty$$

$$\Phi_t \in \mathcal{A}^{\mathrm{AC}}_{\mathrm{RCP}}(t, \rho_t; g_t^{\min})$$

$$\mathcal{A}^{\mathrm{DG}}_{\mathrm{RCP}}(t, \rho_t; g_t^{\min}) = \mathcal{A}^{\mathrm{AC}}_{\mathrm{RCP}}(t, \rho_t; g_t^{\min}) \cap \mathcal{A}^{\mathrm{sem}}_{\mathrm{RCP}} \cap \mathcal{A}^{M}_{\mathrm{RCP}} \cap \mathcal{A}^{\mathrm{ver}}_{\mathrm{RCP}} \cap \Omega_t^{\mathrm{phys}}$$

$$\mathrm{dist}(\rho_t, \mathcal{K}_\star) \to 0 \quad \text{under the target-set descent assumptions of Module G.}$$

$$\mathcal{A}^{\mathrm{op}}_{\mathrm{RCP}} = \mathcal{A}^{\mathrm{DG}}_{\mathrm{RCP}} \cap \mathcal{A}^{\mathrm{type}}_{\mathrm{RCP}} \cap \mathcal{A}^{\mathrm{coup}}_{\mathrm{RCP}} \cap \mathcal{A}^{\mathrm{cert}}_{\mathrm{RCP}} \cap \mathcal{A}^{\mathrm{trust}}_{\mathrm{RCP}} \cap \mathcal{A}^{\mathrm{search}}_{\mathrm{RCP}} \cap \mathcal{A}^{\mathrm{cov}}_{\mathrm{RCP}} \cap \mathcal{A}^{\mathrm{cal}}_{\mathrm{RCP}} \cap \mathcal{A}^{\mathrm{est}}_{\mathrm{RCP}} \cap \Omega_t^{\mathrm{scale}}$$

$$\mathrm{Emb}_t(x) = \frac{F_t(x)F_t(x)^\dagger + \lambda_t I}{\mathrm{Tr}(F_t(x)F_t(x)^\dagger + \lambda_t I)}$$

$$q_{t,j}(\Phi) \le \widehat{q}_{t,j}(\Phi) + \Delta^{\mathrm{aud}}_{t,j}(\Phi)$$

$$\mathrm{CovGap}_t(\Phi) \le \widehat{\mathrm{CovGap}}_t(\Phi) + \Delta^{\mathrm{cov}}_t(\Phi) \le \gamma^{\mathrm{cov}}_t$$

$$e^{\mathrm{cal}}_t(\Phi) = \left| \Delta\,\mathrm{SafeCap}^0_t(\Phi) - f^{\mathrm{cal}}_t(\Xi_t(\Phi)) \right| \le e^{\mathrm{cal}}_{\max,t}$$

(Batch 2 certified ability set)

$$\mathfrak{A}_t(\rho, R) = \{a \in \mathsf{Ab}_t : \mathsf{AbCert}_{t,a}(\rho, R) \text{ is valid}\}.$$

(Non-lossy ability preservation)

$$\Theta^{\mathfrak{A}}_t\big(\mathfrak{A}_t(\rho_t, R_t)\big) \subseteq \mathfrak{A}_{t+1}(\rho_{t+1}, R_{t+1}).$$

(Strict RSI ability progress)

$$\mathfrak{A}_{t+1}(\rho_{t+1}, R_{t+1}) \setminus \Theta^{\mathfrak{A}}_t\big(\mathfrak{A}_t(\rho_t, R_t)\big) \neq \varnothing.$$

(Structural lifted update)

$$\widetilde{\Phi}_t(\rho) = U_t(\rho \otimes |0\rangle\langle 0|_{\mathcal{W}_t})U_t^\dagger.$$

(Conservative extension update)

$$\Phi_\nu(\rho) = V\rho V^\dagger \otimes \alpha_\nu.$$

(Batch 3 certified non-lossy update algebra)

$$\mathfrak{G}^{\mathrm{NL}} = \langle \mathrm{id}, \mathrm{IsoExt}, \mathrm{AppendMem}, \mathrm{RevAdapter}, \mathrm{VerifierExt}, \mathrm{ToolExt} \rangle\,.$$

$$\Phi, \Psi \in \mathfrak{G}^{\mathrm{NL}} \quad \Longrightarrow \quad \Psi \circ \Phi \in \mathfrak{G}^{\mathrm{NL}}.$$

$$\epsilon_{\mathrm{total}} = \sum_i \epsilon_i, \qquad r_{\mathrm{total}} = \sum_i r_i.$$

$$\Omega_t^{\mathrm{NL}}(\rho_t) = \{\Phi \in \mathfrak{G}^{\mathrm{NL}} : \mathcal{L}_\Phi^{\mathcal{P}_t} \le \epsilon_t^{\mathrm{alg}},\ \mathrm{Rec}_\Phi^{\mathcal{P}_t} \le r_t^{\mathrm{alg}},\ \text{and operational residuals pass}\}.$$

(Batch 4 local residuals and tangent cone)

$$\mathcal{A}_t^{\mathrm{loc}}(\rho_t) = \{\nu : q_{t,j}(\nu;\rho_t) \le 0\ \forall j \in J_t\}$$

$$T_{\mathcal{A}_t}(\rho_t) = \{v : Dq_{t,j}(0;\rho_t)[v] \le 0\ \forall j \in J_t^0\}$$

$$P_{T_{\mathcal{A}_t}(\rho_t)} \nabla_\nu \,\mathrm{RSIPot}_t(0;\rho_t,R_t)$$

$$\exists v_t \in T_{\mathcal{A}_t}(\rho_t) : Dq_{t,j}(0;\rho_t)[v_t] \le -\mu_t, \quad D\,\mathrm{RSIPot}_t(0;\rho_t,R_t)[v_t] \ge \gamma_t > 0$$

$$\Longrightarrow \quad \exists \alpha_t > 0 : \nu_t = \alpha_t v_t \in \mathcal{A}_t^{\mathrm{loc}}(\rho_t), \quad \mathrm{RSIPot}_t(\nu_t;\rho_t,R_t) > \mathrm{RSIPot}_t(0;\rho_t,R_t).$$

$$\mathcal{V}_N^{\mathrm{engine},N} = \mathcal{K}_N^{\mathrm{base}}, \qquad \mathcal{V}_t^{\mathrm{engine},N} = \left\{\rho \in \mathcal{K}_t^{\mathrm{base}} : \exists \nu \in \mathcal{E}_t^{\mathrm{CE}}(\rho; g_t^{\mathfrak{A}}) \text{ with } \Phi_\nu(\rho) \in \mathcal{V}_{t+1}^{\mathrm{engine},N}\right\}$$

$$\rho_t \in \mathcal{V}_t^{\mathrm{engine}} \Longrightarrow \exists \nu_t \in \Omega_t^{\mathrm{NL}}(\rho_t) : \rho_{t+1} = \Phi_{\nu_t}(\rho_t) \in \mathcal{V}_{t+1}^{\mathrm{engine}}$$

$$\mathfrak{E}_t : \rho_t \longmapsto (\Phi_t, \mathcal{R}_t^{\mathrm{rec}}, \mathsf{Cert}_t, \rho_{t+1}) \qquad \text{(Direct Non-Lossy RSI Engine target object)}$$

$$\mathfrak{E}_t = (\mathsf{Gen}_t^{\mathrm{NL}}, \mathsf{Certify}_t^{\mathrm{eng}}, \mathsf{Select}_t^{\mathrm{eng}}, \mathsf{Realize}_t^{\mathrm{eng}})$$

$$\rho_t \in \mathcal{D}_t^{\mathrm{engine}} \quad \Longrightarrow \quad \mathfrak{E}_t(\rho_t) = (\Phi_{\nu_t}, \mathcal{R}_{\nu_t}^{\mathrm{rec}}, \mathsf{Cert}_t^{\mathrm{eng}}(\nu_t), \rho_{t+1})$$

$$\Phi_{\nu_t} \in \mathcal{A}_t^{\text{NL-RSI}}, \qquad \Theta_t^{\mathfrak{A}}(\mathfrak{A}_t(\rho_t,R_t)) \subseteq \mathfrak{A}_{t+1}(\rho_{t+1},R_{t+1}), \qquad \rho_{t+1} \in \mathcal{V}_{t+1}^{\mathrm{engine}}$$

$$\mathsf{Type}(T) \in \{\mathrm{Def}, \mathrm{Alg}, \mathrm{Tel}, \mathrm{Cert}, \mathrm{Exist}, \mathrm{Impl}\} \qquad \text{(Batch 7 theorem-type classification)}$$

$$\mathsf{Fail}_t(\Phi) = \max\{F_t^{\mathrm{type}}, F_t^{\mathrm{est}}, F_t^{\mathcal{P}}, F_t^{\mathrm{sem}}, F_t^{\mathrm{rec}}, F_t^{\mathrm{ver}}, F_t^{\mathrm{phys}}, F_t^{\mathrm{transport}}, F_t^{\mathrm{proj}}, F_t^{\mathfrak{A}}, F_t^{\mathrm{eng}}\}$$

$$\mathsf{Fail}_t(\Phi) > 0 \quad \Longrightarrow \quad \text{implementation claim not licensed.}$$

$$\Phi_t \in \mathcal{A}_t^{\text{NL-RSI}}, \qquad \mathrm{Rec}_{\Phi_t}^{\mathcal{P}_t}(\mathcal{R}_t^{\mathrm{rec}}) \le r_t, \qquad \rho_{t+1} \in \mathcal{V}_{t+1}^{\mathrm{engine}}$$

$$\mathfrak{S}_t^{\mathrm{safe}} \subseteq \mathfrak{S}_t^{\mathrm{prot}} \quad \Longrightarrow \quad \text{safety preservation is a corollary of protected non-loss.}$$

Batch 8 cross-time, state/update, and projection-realization equations

$$\Theta_t^Q : Q_t \to Q_{t+1}, \qquad \Theta_t^Z : Z_t \to Z_{t+1}, \qquad \Theta_t^Y : Y_t \to Y_{t+1}, \qquad \Theta_t^G : G_t \to G_{t+1}$$

$$\left| I_t(Z_t; Y_t) - I_{t+1}(\Theta_t^Z Z_t; \Theta_t^Y Y_t) \right| \le \delta_t^{ZY}$$

$$I_{t+1}(Z_{t+1}; Y_{t+1}) \ge I_{t+1}(\Theta_t^Z Z_t; \Theta_t^Y Y_t) - \xi_t^{\mathrm{tr}}$$

$$U_t^{\mathrm{tr}} = \Big[H_{t+1}(\Theta_t^Q Q_t \mid \Theta_t^E E_t) - H_{t+1}(Q_{t+1} \mid E_{t+1}) - I_{t+1}(\Theta_t^Q Q_t; E_{t+1} \mid \Theta_t^E E_t) \Big]_+$$

$$\mathcal{K}_{\mathrm{RCP}}^{\mathrm{state}}(t) = \{\rho : \text{state-only constraints hold}\},$$
$$\mathcal{A}_{\mathrm{RCP}}(t, \rho, \Phi) = \{\Phi : \text{update-dependent constraints hold}\}.$$

$$V_{\mathrm{state},t}(\rho) \quad \text{versus} \quad V_{\mathrm{upd},t}(\rho, G, \Phi)$$

$$\mathsf{ProjReal}_t(\tilde{\rho}, \rho^+, \nu) : \qquad d(\rho^+, \Phi_\nu(\rho_t)) \le \delta_t^{\text{proj-real}}$$

A.1 Batch 10 adaptive-audit and complexity equations

$$\Phi_\nu = (\mathsf{Dom}_t^\nu, \mathsf{Cod}_{t+1}^\nu, \mathsf{Map}_\nu, \mathsf{TypeLic}_t(\nu), \mathsf{ResCert}_t(\nu)).$$
$$\mathcal{A}_{\mathrm{RCP}}^{\mathrm{op10}} = \mathcal{A}_{\mathrm{RCP}}^{\mathrm{op}} \cap \mathcal{A}_{\mathrm{RCP}}^{\text{trust-adv}} \cap \mathcal{A}_{\mathrm{RCP}}^{\mathrm{cx}}.$$

$\mathrm{CorrFail}_t, \quad \mathrm{AuditLeak}_t, \quad \mathrm{CommonMode}_t, \quad \mathrm{CollusionRisk}_t \le$ declared adaptive-audit thresholds.

$$\sum_{o \in \mathsf{Ops}_t(\Phi)} (\widehat{\mathrm{Cx}}_{t,o}^{\mathrm{cert}} + \Delta_{t,o}^{\mathrm{Cx}}) \le \mathsf{Budget}_t^{\mathrm{Cx}}.$$

A.2 Batch 11 domain-nonemptiness and witness-library equations

$$\mathcal{L}_t^{\mathrm{CE}}(\rho) = \{M \in \mathcal{L}_t^{\mathrm{CE}} : \mathsf{ValidMod}_t(M; \rho) = 1\}$$

$$\mathfrak{G}_{t,\le k}^{\mathrm{NL}}(\rho) = \{g_\ell \circ \cdots \circ g_1 : 0 \le \ell \le k, \text{ word composable and certificate-valid}\}$$

$$\rho_t \in \mathcal{K}_t^{\mathrm{seed},N} \Longrightarrow \rho_t \in \mathcal{V}_t^{\mathrm{engine},N}$$

$$\rho_t \in \mathcal{K}_t^{\mathrm{seed},N} \Longrightarrow \rho_t \in \mathcal{D}_t^{\mathrm{engine}} \Longrightarrow \mathfrak{E}_t(\rho_t) \text{ constructs a certified beneficial non-lossy update.}$$

$$\rho_t \in \mathcal{K}_t^{\mathrm{seed},N}, \quad M_t \in \mathcal{L}_t^{\mathrm{CE}}(\rho_t), \quad \rho_{t+1} = \Phi_{\nu_t}(\rho_t),$$

$$\Big[\nu_t = \nu_{M_t} \text{ and } \mathsf{SeedPersist}_t(M_t;\rho_t) \text{ valid}\Big]$$
$$\quad \text{or} \quad \Big[\nu_t \in \mathsf{Pass}_t^{\mathrm{eng}}(\rho_t) \text{ and } \mathsf{SeedEq}_t(M_t,\nu_t;\rho_t) \text{ valid}\Big]$$
$$\Longrightarrow \rho_{t+1} \in \mathcal{K}_{t+1}^{\mathrm{seed},N}.$$

$$e_t^{\mathrm{gen}} + e_t^{\mathrm{cert}} + e_t^{\mathrm{enum}} + e_t^{\mathrm{proj}} + e_t^{\mathrm{tr}} < \min\{m_t^{\mathrm{eng}}, m_t^{\mathfrak{A}}, m_t^{r}\}.$$

A.3 Batch 12 successor-verification equations

$$\mathcal{K}_t^{\mathrm{SV},N} \subseteq \mathcal{K}_t^{\mathrm{seed},N} \subseteq \mathcal{D}_t^{\mathrm{engine}} \subseteq \mathcal{V}_t^{\mathrm{engine},N} \subseteq \mathcal{K}_t^{\mathrm{base}} \subseteq \mathcal{K}_{\mathrm{RCP}}^{\mathrm{state}}(t).$$
$$\mathsf{VerSch}_t^{\mathrm{SV}} = (\mathsf{Lang}_t^{\mathrm{SV}}, \mathsf{Pred}_t^{\mathrm{SV}}, \mathsf{Chk}_t^{\mathrm{SV}}, Q_t^{\mathrm{SV}}, \Theta_t^{\mathrm{SV}}, \preceq_t^{\mathrm{SV}}, \mathsf{Fail}_t^{\mathrm{SV}}, \mathsf{Budget}_t^{\mathrm{SV}}).$$
$$\mathsf{SVPkg}_t(\rho,\nu) = (\mathsf{SeedPkg}_t, \mathsf{VerSch}_t^{\mathrm{SV}}, \mathsf{SuccCert}_t^{\mathrm{SV}}, \mathcal{P}_t^{\mathrm{SV}},$$
$$\mathsf{GoalId}_t, \mathsf{TrustRef}_t^{\mathrm{SV}}, \mathsf{CertBudget}_t^{\mathrm{SV}}, \mathsf{SVPersist}_t).$$
$$\rho_t \in \mathcal{K}_t^{\mathrm{SV},N} \Longrightarrow \mathfrak{E}_t(\rho_t) \text{ constructs } \rho_{t+1} \in \mathcal{K}_{t+1}^{\mathrm{SV},N} \quad \text{on the stated certificate events.}$$
$$\inf_{P \in \mathcal{P}_t^{\mathrm{SV}}} \mathbb{E}_P\left[\Psi_{t+1}^{\mathrm{SV}}(\rho_{t+1}) - \Theta_t^{\Psi}\Psi_t^{\mathrm{SV}}(\rho_t)\right] \geq \epsilon_t^{\mathrm{SV}} - \eta_t^{\mathrm{SV}}.$$
$$D_{\mathrm{sem}}^{U}(U_{t+1}, \Theta_t^{U} U_t) \leq \delta_t^{U}, \qquad \sup_{P \in \mathcal{P}_t^{\mathrm{SV}}} P((\mathsf{SVSound}_t)^c) \leq \alpha_t^{\mathrm{SV}}.$$

A.4 Batch 12A residual, transport, and soundness equations

$$Q_t^{\mathrm{SV},A}(\nu;\rho) = (q_t^{\mathrm{seed}}, q_t^{\mathrm{cand}}, q_t^{\mathrm{ver}}, q_t^{\text{ver-tr}}, q_t^{U}, q_t^{\mathcal{P}}, q_t^{\mathcal{P}\text{-}tr}, q_t^{\mathrm{trust}}, q_t^{\mathrm{budget}}, q_t^{\mathrm{proof}}, q_t^{\mathrm{world}}, q_t^{\mathrm{persist}}, q_t^{\mathrm{sound}}, q_t^{\Psi}).$$

$$Q_t^{\mathrm{SV},A}(\nu;\rho) \leq 0 \quad \Longrightarrow \quad \mathsf{SVPkg}_t(\rho,\nu) \text{ satisfies the explicit residual clause.}$$

$$\Theta_t^{\mathrm{SV},A} = (\Theta_t^{\mathsf{Lang}}, \Theta_t^{\mathsf{Pred}}, \Theta_t^{\mathsf{Chk}}, \Theta_t^{Q,\mathrm{SV}}, \Theta_t^{\mathsf{Budget}}, \Theta_t^{\mathsf{Fail}})$$

$$q_t^{\mathsf{Lang}}, q_t^{\mathsf{Pred}}, q_t^{\mathsf{Chk}}, q_t^{Q,\mathrm{SV}}, q_t^{\mathsf{Budget}}, q_t^{\mathsf{Fail}} \leq 0 \Longrightarrow \Theta_t^{\mathrm{SV},A}(\mathsf{VerSch}_t^{\mathrm{SV}}) \preceq_t^{\mathrm{SV}} \mathsf{VerSch}_{t+1}^{\mathrm{SV}}.$$

$$\mathsf{SVSound}_t = \bigcap_{\ell \in J_t^{\mathrm{SV},A}} \mathcal{E}_{t,\ell}^{\mathrm{SV}}, \qquad \alpha_t^{\mathrm{SV},A} = \sum_{\ell \in J_t^{\mathrm{SV},A}} \alpha_{t,\ell}^{\mathrm{SV}}.$$

$$\sup_{P \in \mathcal{P}_t^{\mathrm{SV}}} P((\mathsf{SVSound}_t)^c) \leq \alpha_t^{\mathrm{SV},A} \leq \alpha_t^{\mathrm{SV}}.$$

$$\mathcal{P}_t^{\mathrm{SV}} = \mathcal{P}_t^{\mathrm{env}} \cap \mathcal{P}_t^{\mathrm{est}} \cap \mathcal{P}_t^{\mathrm{cert}} \cap \mathcal{P}_t^{\mathrm{phys}} \cap \mathcal{P}_t^{\mathrm{trust}}.$$

$$\Theta_t^{\mathcal{P}}(\mathcal{P}_t^{\mathrm{SV}}) \subseteq \mathcal{P}_{t+1}^{\mathrm{SV}} \quad \text{or} \quad d_{\mathcal{P}}(\Theta_t^{\mathcal{P}}\mathcal{P}_t^{\mathrm{SV}}, \mathcal{P}_{t+1}^{\mathrm{SV}}) \leq \omega_t^{\mathcal{P}}.$$

$$D^U_{\mathrm{sem}}(U_T, \Theta^U_{0:T} U_0) \le \sum_{t=0}^{T-1} \delta^U_t + \sum_{t=1}^{T-1} \omega^U_t.$$

$$Q_t^{\mathrm{TrustRef,SV}} = (q_t^{\mathrm{sole}}, q_t^{\mathrm{common}}, q_t^{\mathrm{collusion}}, q_t^{\mathrm{leak}}, q_t^{\mathrm{anchor}}, q_t^{\mathrm{reflect}}) \le 0$$
$$\Longrightarrow \text{no self-sole-certification on the audited domain.}$$

A.5 Batch 12B packet-construction, nonemptiness, and infinite-closure equations

$$\mathsf{SVBuilder}_t : \mathsf{SVIn}_t(\rho, \nu) \longmapsto (\mathsf{SVPkg}_t, Q_t^{\mathrm{SV},A}, \Theta_t^{\mathrm{SV},A}, \mathcal{P}_t^{\mathrm{SV}}, \mathsf{GoalId}_t, \mathsf{TrustRef}_t^{\mathrm{SV}}, \mathsf{SVPersist}_t, \mathsf{SVBuildTrace}_t).$$

$$(\rho, \nu) \in \mathcal{D}_t^{\text{SV-construct}} \Longrightarrow \mathsf{SVBuilder}_t(\rho, \nu) \text{ constructs a valid } \mathsf{SVPkg}_t(\rho, \nu).$$

$$\rho \in \mathcal{K}_t^{\text{SV-seedlib},N} \Longrightarrow \rho \in \mathcal{K}_t^{\mathrm{SV},N} \subseteq \mathcal{K}_t^{\mathrm{seed},N}.$$

$$\rho \in \mathcal{K}_t^{\text{SV-seedlib},\infty} \Longrightarrow \rho \in \mathcal{K}_t^{\mathrm{SV},\infty,\max} \quad \text{or in a declared containing } \mathcal{K}_t^{\mathrm{SV},\infty}.$$

$$\mathcal{K}_t^{\mathrm{SV},\infty} \subseteq \mathcal{K}_t^{\mathrm{seed},\infty}, \qquad \rho_t \in \mathcal{K}_t^{\mathrm{SV},\infty} \Longrightarrow \rho_{t+1} = \Phi_{\pi_t^{\mathrm{SV}}(\rho_t)}(\rho_t) \in \mathcal{K}_{t+1}^{\mathrm{SV},\infty}.$$

$$D^U_{\mathrm{sem}}(U_T, \Theta^U_{0:T} U_0) \le \sum_{t=0}^{T-1} (\delta^U_t + \omega^U_t), \qquad \sum_{t=0}^{\infty} \alpha_t^{\mathrm{SV}} < \infty.$$

$$P_t^{\mathrm{true}} \in \mathcal{P}_t^{\mathrm{SV}} \quad \text{or} \quad d_{\mathcal{P}}(P_t^{\mathrm{true}}, \mathcal{P}_t^{\mathrm{SV}}) \le \varepsilon_t^{\mathrm{env}}.$$

$$\mathbb{E}_{P_t^{\mathrm{true}}} \left[\Psi_{t+1}^{\mathrm{SV}}(\rho_{t+1}) - \Theta_t^{\Psi} \Psi_t^{\mathrm{SV}}(\rho_t) \right] \ge \epsilon_t^{\mathrm{SV}} - \eta_t^{\mathrm{SV}} - \Gamma_t^{\mathrm{env}}(\varepsilon_t^{\mathrm{env}}).$$

$$\mathrm{Cost}(\mathsf{SVBuilder}_t) + \mathrm{Cost}(\mathcal{A}_t^{\text{SV-gate}}) + \mathrm{Cost}(\mathsf{SVProof}_t) \le \mathrm{poly}(n_t, k_t, \ell_t, s_t, \log(1/\delta_t^{\mathrm{tol}}))$$

when a restricted tractability certificate is supplied; otherwise the claim is only resource-accounted.

A.6 RCP-II finalization and instantiation-readiness equations

Licensed title:
Domain-Relative Robust Reflective Successor Theorem
for Certified RCP Seed-Library Classes.

$$\rho_0 \in \mathcal{K}_0^{\text{SV-seedlib},N} \Longrightarrow \rho_t \in \mathcal{K}_t^{\mathrm{SV},N} \qquad 0 \le t \le N.$$

$$\rho_0 \in \mathcal{K}_0^{\text{SV-seedlib},\infty} \Longrightarrow \rho_t \in \mathcal{K}_t^{\text{SV},\infty,\text{gen}} \qquad \forall t \geq 0.$$

$$\mathsf{RefSV}_{0:N}^{\text{RCP}} \text{ valid} \Longrightarrow \rho_0 \in \mathcal{K}_0^{\text{SV-seedlib},N}.$$

$$\mathsf{Invoke}_{\text{RCP-}II} \Longrightarrow \begin{array}{c} \mathsf{SVBuilder}_t, \quad \mathsf{SVPkg}_t, \quad Q_t^{\text{SV},A}, \\ \mathsf{GoalId}_t, \quad \mathsf{TrustRef}_t^{\text{SV}}, \quad \mathsf{SVPersist}_t \\ \text{valid on the declared horizon.} \end{array}$$

$$\text{Real-world reliability} \Longrightarrow \mathsf{RealCont}_t \text{ or bounded misspecification.}$$

$$\text{Strong computational tractability} \Longrightarrow \mathsf{SVTract}_t.$$

$$\text{No new abstract Batch 13 yet} \Longrightarrow \text{next frontier is reference instantiation and validation.}$$

$$\begin{array}{c} \text{Batch 13R is reference-entry / realization, not abstract Batch 13.} \\ \mathsf{MechCert}_{0:N}^{\text{RCP,can}} \Rightarrow \text{canonical finite core externally machine-checked,} \\ \mathsf{CtrlReplayCert}_{0:N}^{\text{RCP}} + \mathsf{CtrlRef}_{0:N}^{\text{RCP}} \Rightarrow \rho_t \in \mathcal{K}_t^{\text{SV},N} \quad (0 \leq t \leq N), \\ \mathsf{BuildRefSV}_{0:N}^{\text{RCP}}(\mathcal{M}, \mathcal{D}, \mathcal{L}, \mathcal{C}) \Downarrow \mathsf{RefSV}_{0:N}^{\text{RCP}} \\ \Longrightarrow \rho_0 \in \mathcal{K}_0^{\text{SV-seedlib},N}. \end{array}$$

$$\mathsf{Check}_{\text{RCP}}(\mathsf{PCS}_t(\nu_t)) = 1 \Longrightarrow Q_t^{\text{SV},A}(\nu_t; \rho_t) \leq 0 \quad \text{and} \quad \mathsf{SVPkg}_t(\rho_t, \nu_t) \text{ is valid.}$$

$$\mathsf{PrefixCheck}_t \Longrightarrow \rho_t \in \mathcal{K}_t^{\text{SV},N}.$$

$$\begin{aligned} &\text{Cost}(\mathsf{BuildRefSV}_{0:N}^{\text{RCP}}) + \sum_{t<N} \text{Cost}(\mathsf{Check}_{\text{RCP}}(\mathsf{PCS}_t)) \\ &\quad + \sum_{t<N} \text{Cost}(Q_t^{\text{SV},A}) \leq P_{\text{ref}}(n, k, N, \ell, s, \log(1/\delta^{\text{tol}})). \end{aligned}$$

$$\mathsf{Artifact}_{0:N}^{\text{RCP}} + \mathsf{ArtCert}_{0:N}^{\text{RCP}} \Longrightarrow \rho_0 \in \mathcal{K}_0^{\text{SV-seedlib},N}, \qquad \rho_t \in \mathcal{K}_t^{\text{SV},N} \ (0 \leq t \leq N).$$

$$\begin{array}{c} \mathsf{LearnedEntryAudit}_{0:N}^{\text{RCP}}(\mathcal{M}_\theta, \mathcal{D}, \mathcal{L}, \mathcal{C}) \Downarrow \mathsf{LECert}_{0:N}^{\text{RCP}} \Longrightarrow \rho_0^{\mathcal{M}_\theta} \in \mathcal{K}_0^{\text{SV-seedlib},N}, \\ \rho_t^{\mathcal{M}_\theta} \in \mathcal{K}_t^{\text{SV},N} \quad (0 \leq t \leq N). \end{array}$$

$$\mathsf{TrainPilot}_{0:N}^{\text{RCP}} = \mathsf{PartialPass} \Longrightarrow \text{report supplied hypotheses only; do not invoke learned entry.}$$

B Glossary of RCP Terms

Coherence Structured mutual constraint between language, world-reference, human-reference, self-modeling, ambiguity, and decision.

Entropy reduction
Reduction in uncertainty or representational dispersion. Safe only when grounded and recoverable.

Definitiveness
Internal commitment to a state, answer, or action.

Ambiguity Preserved uncertainty over unresolved hypotheses or values.

Lossiness Irreversible loss of protected safety-relevant distinctions during recursive update.

Recoverability
Ability to preserve or reconstruct distinctions after a channel.

Barrier A constraint defining a safe region that accepted updates cannot leave.

No-op feasibility
The system can reject all unsafe updates and keep its current state.

Level 2.5 The RCP stability layer between recursive coherence language modeling and true recursive self-improvement.

Augmented RCP admissibility
The strengthened gates $\mathcal{A}_{\mathrm{RCP}}^{+}$, $\mathcal{A}_{\mathrm{RCP}}^{\mathrm{AC}}$, and $\mathcal{A}_{\mathrm{RCP}}^{\mathrm{DG}}$. The post-A–C gate adds constructive recovery, capability calibration, safe-improvement existence, and recursive feasibility. The post-D–G gate adds semantic validity, self-model fidelity, verifier bootstrapping, resource feasibility, and asymptotic constraints.

Capability gate
An explicit empirical validation constraint ensuring that formal coherence preservation is accompanied by bounded loss, or certified gain, in a chosen capability functional.

Verifier integrity
The requirement that the verifier itself be treated as a safety-relevant component rather than as an unprotected oracle.

Typed density realization
A declaration of whether a density state is quantum, classical-diagonal, or representational, together with the proof licenses allowed for that type.

Representation-to-density embedding
A certified map from raw architecture variables into positive trace-one density states.

Semantic coupling
A certified transfer relation between density-level semantic barriers and externally audited semantic validity.

Non-oracular certificate
: A one-sided audited residual estimate with calibration data, stress tests, trace records, verifier identity, and failure probability.

Verifier trust graph
: A graph specifying which verifier components certify which updates, including trust anchors and no self-sole-certification constraints.

Coverage gap
: A residual budget for protected safety-relevant distinctions not yet included in the tracked predicate and pair-set family.

Capability calibration
: An audited protocol testing whether claimed coherence-supported capability gains match heldout, out-of-distribution, adversarial, and formal task evidence.

Estimator subprotocol
: A documented estimator for a gate residual, including domain, radius, failure probability, cost, stress tests, and failure mode.

Recoverable monotone self-improvement
: Recursive improvement that preserves protected distinctions, supplies recovery or rollback structure, and does not decrease the declared progress structure. Safety is one corollary when the protected predicates include safety predicates.

Conditional Non-Lossy Self-Update Preservation Theorem
: The renamed main theorem of this manuscript. It proves that if accepted updates pass the RCP gate, then the declared non-loss, Lyapunov, barrier, ambiguity, self-model, semantic, verifier, resource, coverage, calibration, and estimator obligations remain bounded or invariant.

Certified ability set
: The set $\mathfrak{A}_t(\rho, R)$ of abilities certified for state ρ under resource budget R. Ability-set inclusion is the Batch 2 progress preorder.

Non-lossy ability preservation
: The condition $\Theta_t^{\mathfrak{A}}(\mathfrak{A}_t(\rho_t, R_t)) \subseteq \mathfrak{A}_{t+1}(\rho_{t+1}, R_{t+1})$, reducing to ordinary set inclusion when the ability universe is fixed.

Strict RSI ability progress
: Proper ability-set expansion: the successor has at least one certified ability not obtained by transporting a predecessor ability.

Structural recovery
: Exact predecessor recoverability obtained from a reversible/isometric meta-state update, such as $\widetilde{\Phi}_t(\rho) = U_t(\rho \otimes |0\rangle\langle 0|)U_t^\dagger$.

Conservative extension
: A sufficient exact non-loss update form $\Phi_\nu(\rho) = V\rho V^\dagger \otimes \alpha_\nu$, where the predecessor is embedded isometrically and a pair-independent extension state is appended.

Conservative-extension algebra

The Batch 3 algebra of certified non-lossy primitive compositions:

$$\mathfrak{G}^{\mathrm{NL}} = \langle \mathrm{id}, \mathrm{IsoExt}, \mathrm{AppendMem}, \mathrm{RevAdapter}, \mathrm{VerifierExt}, \mathrm{ToolExt} \rangle.$$

Additive non-loss budgets

The composition rule $\epsilon_{\mathrm{total}} = \sum_i \epsilon_i$, $r_{\mathrm{total}} = \sum_i r_i$, proving that finite words in $\mathfrak{G}^{\mathrm{NL}}$ preserve protected non-loss and recovery within declared cumulative budgets.

Algebraically non-lossy candidate class

The class $\Omega_t^{\mathrm{NL}}(\rho_t)$ of finite certified words in $\mathfrak{G}^{\mathrm{NL}}$ whose total loss/recovery budgets and operational residuals fit the current RCP gate.

Admissible tangent cone

The Batch 4 first-order cone

$$T_{\mathcal{A}_t}(\rho_t) = \{v : Dq_{t,j}(0; \rho_t)[v] \leq 0 \ \forall j \in J_t^0\},$$

where $q_{t,j} \leq 0$ are local RCP/NL-RSI residuals and J_t^0 is the active residual set at no-op.

RSIPot The audited differentiable engine-potential functional used for local progress search. A positive RSIPot_t step is not automatically a new ability; strict ability progress still requires a certified new ability.

Projected tangent-cone progress

The Batch 4 object

$$P_{T_{\mathcal{A}_t}(\rho_t)} \nabla_\nu \, \mathrm{RSIPot}_t(0; \rho_t, R_t),$$

which gives a sufficient local progress direction only when it is nonzero, strictly inward for active residuals, and certified by curvature bounds.

Tangent-cone local progress theorem

The Batch 4 theorem proving that a sufficiently small step along a strictly inward positive engine-potential tangent direction is locally admissible and produces positive local progress, while inheriting algebraic non-loss when the chart lies inside Ω_t^{NL}.

Engine viability kernel

The Batch 5 recursive continuation set $\mathcal{V}_t^{\mathrm{engine}}$. Membership means there exists a certified non-lossy conservative-extension move $\nu_t \in \Omega_t^{\mathrm{NL}}$ whose successor remains in $\mathcal{V}_{t+1}^{\mathrm{engine}}$.

Conservative-extension engine move

A finite certified word in $\mathfrak{G}^{\mathrm{NL}}$ satisfying engine residuals, successor base-domain membership, resource/certificate requirements, and ability preservation or strict ability expansion.

Engine viability nonemptiness theorem

The Batch 5 theorem proving $\rho_t \in \mathcal{V}_t^{\mathrm{engine}} \Rightarrow \exists \nu_t \in \Omega_t^{\mathrm{NL}}$ with $\rho_{t+1} \in \mathcal{V}_{t+1}^{\mathrm{engine}}$, plus the theorem that a certified conservative-extension plan makes the finite-horizon kernel nonempty.

Direct Non-Lossy RSI Engine Theorem

The Batch 9 theorem proving that on a declared direct-engine domain $\mathcal{D}_t^{\text{engine}}$, under strict beneficial-witness, generator-coverage, and certificate-soundness assumptions, the engine constructor $\mathfrak{E}_t$ constructs a certified beneficial non-lossy update with recovery, ability expansion, and successor engine viability. It is domain-restricted, not an arbitrary-system RSI theorem.

Direct engine domain

The set $\mathcal{D}_t^{\text{engine}} \subseteq \mathcal{V}_t^{\text{engine}}$ of states for which strict beneficial conservative-extension witnesses exist and the engine generator/certifier covers them within declared margins.

Seed engine domain

The Batch 11 set $\mathcal{K}_t^{\text{seed},N}$ of states with certified conservative-extension library nonemptiness, bounded grammar coverage, certifier margins, resource/trust/type obligations, and successor seed-persistence evidence. Seed membership is a sufficient condition for direct-engine domain entry over the declared horizon.

Certified conservative-extension library

A finite or explicitly enumerable library $\mathcal{L}_t^{\text{CE}}$ whose valid modules carry primitive-word, non-loss, recovery, ability, resource, trust, semantic, and successor-persistence certificates.

Witness-library closure

The Batch 11 theorem layer proving that a valid positive-margin module in $\mathcal{L}_t^{\text{CE}}(\rho)$ supplies the strict beneficial conservative-extension witness required by Batch 9.

Finite primitive-word grammar

The bounded grammar $\mathfrak{G}_{t,\le k}^{\text{NL}}(\rho)$ of compatible certificate-valid finite words in the non-lossy primitive algebra; exhaustive enumeration of this grammar discharges generator coverage for witnesses inside the grammar.

Seed-domain persistence

The condition that the constructed successor remains in $\mathcal{K}_{t+1}^{\text{seed},N}$, allowing the seed-to-direct-engine argument to iterate over a declared horizon.

Seed-equivalence / persistence-transfer certificate

The Batch 11 certificate $\mathsf{SeedEq}_t(M, \nu; \rho)$ transferring seed-persistence from a certified module M to an engine-selected passing candidate ν, with residual, ability, recovery, type, resource, trust, transport, and projection distortions included in the strict margin ledger.

Successor-verification schema

The Batch 12 object $\mathsf{VerSch}_t^{\text{SV}}$ specifying the certificate language, verifier predicates, residual family, transports, preorder, failure event, and budget used to verify that a successor can preserve the verification procedure itself.

Successor-verification seed domain

The Batch 12 set $\mathcal{K}_t^{\text{SV},N} \subseteq \mathcal{K}_t^{\text{seed},N}$ of seed states with a valid successor-verification packet, uncertainty envelope, semantic/goal-identity certificate, trust-reference support, certificate budget, and successor membership certificate.

Successor-verification packet

The Batch 12 certificate packet $\mathsf{SVPkg}_t(\rho, \nu)$ combining the Batch 11 seed package with verifier-schema persistence, uncertainty, goal-identity, trust, and budget certificates for the selected successor.

Verifier-schema persistence

The Batch 12 condition $\Theta_t^{\mathrm{SV}}(\mathsf{VerSch}_t^{\mathrm{SV}}) \preceq_t^{\mathrm{SV}} \mathsf{VerSch}_{t+1}^{\mathrm{SV}}$, meaning that the successor inherits or strengthens the transported predecessor verification schema within the declared distortion budget.

Explicit SV residual vector

The Batch 12A vector $Q_t^{\mathrm{SV},A}$, decomposing successor-verification into seed, candidate-link, verifier-schema, transport, goal-identity, uncertainty, trust, budget, proof, world-model, persistence, soundness, and progress residuals.

Uncertainty-envelope transport

The Batch 12A map or relation $\Theta_t^{\mathcal{P}}$ that compares successor-verification uncertainty families across time, exactly by inclusion or approximately by an envelope-distance budget.

Goal-identity composition

The Batch 12A finite-horizon theorem bounding $D_{\mathrm{sem}}^U(U_T, \Theta_{0:T}^U U_0)$ by the sum of one-step goal drifts and goal-transport distortions.

Anti-circular trust residuals

The Batch 12A residuals $q_t^{\mathrm{sole}}, q_t^{\mathrm{common}}, q_t^{\mathrm{collusion}}, q_t^{\mathrm{leak}}, q_t^{\mathrm{anchor}}, q_t^{\mathrm{reflect}}$ that make no-self-sole-certification and related trust-reference conditions explicit.

Viability-domain audit

The Batch 12A claim-discipline statement that successor-verification invariance is a domain theorem and does not by itself construct successor-verification packets or prove nonemptiness.

SV packet builder

The Batch 12B partial map $\mathsf{SVBuilder}_t$ that constructs SVPkg_t, explicit residuals, verifier-schema transports, uncertainty envelope, goal-identity certificate, trust-reference certificate, and persistence certificate from typed builder inputs on $\mathcal{D}_t^{\text{SV-construct}}$.

SV packet-construction domain

The Batch 12B domain $\mathcal{D}_t^{\text{SV-construct}}$ on which the successor-verification packet builder is sound. It is a declared sufficient-condition domain, not the set of arbitrary states or arbitrary candidates.

SV witness library

The Batch 12B library $\mathcal{L}_t^{\text{SV-Wit}}$ of positive-margin witnesses carrying the update, proof, residual, uncertainty, goal, trust, budget, and successor-persistence data needed to construct a successor-verification packet.

SV seed-library domain

The Batch 12B finite-horizon set $\mathcal{K}_t^{\text{SV-seedlib},N}$, a checkable sufficient-condition do-

main from which $\mathcal{K}_t^{\mathrm{SV},N}$ membership follows by packet construction and bounded certificate grammar coverage.

Infinite SV seed-library domain

The Batch 12B infinite-horizon set sequence $\mathcal{K}_t^{\text{SV-seedlib},\infty}$, whose closure theorem shows that it canonically supplies an infinite successor-verification seed domain $\mathcal{K}_t^{\mathrm{SV},\infty}$ when the declared infinite witness-library, builder, persistence, and summability hypotheses hold.

Domain-relative RRST

The Batch 12B robust reflective successor theorem qualified by domain, certificate, uncertainty-envelope, reality-containment, and tractability assumptions. It proves robust reflective closure for declared certified paths, not universal successor trust.

Reality-containment certificate

The Batch 12B object $\mathsf{RealCont}_t$ certifying either $P_t^{\mathrm{true}} \in \mathcal{P}_t^{\mathrm{SV}}$ or bounded misspecification $d_{\mathcal{P}}(P_t^{\mathrm{true}}, \mathcal{P}_t^{\mathrm{SV}}) \le \varepsilon_t^{\mathrm{env}}$.

Restricted SV tractability certificate

The Batch 12B object $\mathsf{SVTract}_t$ giving explicit finite or polynomial cost bounds for $\mathsf{SVBuilder}_t$, the algorithmic SV gate, and proof/checking traces on a declared representation class.

Semantic/goal-identity drift

The Batch 12 bound $D_{\mathrm{sem}}^U(U_{t+1}, \Theta_t^U U_t) \le \delta_t^U$, which controls declared objective or goal-reference transport across successor construction.

Engine constructor

The Batch 9 tuple $\mathfrak{E}_t = (\mathsf{Gen}_t^{\mathrm{NL}}, \mathsf{Certify}_t^{\mathrm{eng}}, \mathsf{Select}_t^{\mathrm{eng}}, \mathsf{Realize}_t^{\mathrm{eng}})$, which generates finite non-lossy primitive words, certifies them, selects a passing witness, realizes the update, and returns the successor packet.

Typed self-update transition

The Batch 10 default interpretation of Φ_ν: a typed successor-state transformation with explicit domain, codomain, type license, and residual certificate. It is called a channel only when a CPTP, stochastic, isometric, conservative-extension, or direct residual license is supplied.

Adaptive-audit trust gate

The Batch 10 strengthening $\mathcal{A}_{\mathrm{RCP}}^{\text{trust-adv}}$, which adds correlated-failure, audit-leakage, common-mode, and collusion-risk bounds to the ordinary verifier trust graph.

Complexity ledger

A Batch 10 accounting object recording active operations, certified costs, uncertainty radii, factorized costs when available, and bottleneck status. It licenses cost accounting but not practical tractability.

Theorem type table

The Batch 7 table classifying results as definitions, algebraic/structural theorems, telescoping or induction theorems, certificate-composition theorems, assumption-dependent existence theorems, or implementation/falsification statements.

Claim discipline

The rule that a theorem may be cited only together with its hypotheses and implementation-status assumptions; certificate-composition theorems do not prove certificates are obtainable.

Falsification condition

A condition showing that a concrete implementation does not satisfy the hypotheses needed to apply the corresponding RCP theorem or gate claim.

Estimator calibration failure

A failure of the one-sided residual bound $q_{t,j} > \widehat{q}_{t,j} + \Delta_{t,j}$, invalidating the affected robust or non-oracular certificate.

Pair-set coverage failure

Failure of $\mathcal{P}_t$, barriers, verifier tests, or coverage residuals to represent a declared protected predicate distinction within the audited domain.

Semantic barrier unsoundness

A case where the semantic barrier is nonpositive but the corresponding externally audited semantic validity condition fails.

Recovery-map failure

A failure of the certified recovery map to reconstruct protected predecessor states within the declared budget r_t.

Verifier common-mode failure

A correlated verifier or audit blind spot invalidating the independence or trust-anchor assumptions used by the trust graph.

Resource-scaling failure

A failure of cost bounds, one-step affordability, or amortized scaling conditions required for executable engine-mode claims.

Engine-viability failure

A failure of the claimed engine-viability kernel: no certified non-lossy conservative-extension move keeps the successor inside the next engine kernel.

Cross-time transport

The Batch 8 maps $\Theta_t^Q, \Theta_t^Z, \Theta_t^Y, \Theta_t^G$, and when needed Θ_t^E, that license comparisons of question variables, self-model variables, future-behavior variables, update descriptions, and evidence across time-indexed spaces.

Transported bottleneck preservation

The condition $I_{t+1}(Z_{t+1}; Y_{t+1}) \geq I_{t+1}(\Theta_t^Z Z_t; \Theta_t^Y Y_t) - \xi_t^{\mathrm{tr}}$, with any distortion between the legacy and transported quantities included in the budget.

Transported ambiguity collapse

The Batch 8 residual U_t^{tr}, which compares ambiguity reduction after transporting the predecessor question and evidence into the successor-time space.

State-only safe set

The set $\mathcal{K}_{\mathrm{RCP}}^{\mathrm{state}}(t)$ of states satisfying only state-level constraints. Candidate-specific

recovery, ambiguity, bottleneck, projection, and progress claims belong to the update-admissibility relation.

Update-admissibility relation
The relation $\mathcal{A}_{\mathrm{RCP}}(t, \rho, \Phi)$ that evaluates whether a particular candidate update satisfies all update-dependent RCP residuals at state ρ.

State/update Lyapunov split
The distinction between $V_{\mathrm{state},t}(\rho)$, used in state-only safe sets, and $V_{\mathrm{upd},t}(\rho, G, \Phi)$, used in candidate-specific residuals.

Projection-realization certificate
The Batch 8 certificate $\mathsf{ProjReal}_t(\tilde{\rho}, \rho^+, \nu)$ proving that a mathematically projected safe state ρ^+ is reachable by an implementable typed update Φ_ν within declared distortion and resource budgets.

Mathematical projection repair
A projected state in the safe set that is useful for analysis or repair but does not count as an accepted RSI update unless a projection-realization certificate is supplied.

Cross-time transport failure
Failure of a declared transport map or distortion bound, invalidating the affected cross-time entropy, mutual-information, ambiguity, or update-description comparison.

Final RCP-II theorem framing
The claim-discipline status that licenses the phrase "Domain-Relative Robust Reflective Successor Theorem for Certified RCP Seed-Library Classes" only when the relevant seed-domain, packet-construction, reality-containment, and tractability certificates are supplied.

Reference successor-verification instance
The finite abstract object $\mathsf{RefSV}^{\mathrm{RCP}}_{0:N}$ containing a state, witnesses, builder inputs, packets, goal/trust/reality/tractability certificates, and audit trace sufficient to invoke the finite RCP-II theorem.

Batch 13R The reference-entry / realization phase, not an abstract Batch 13. It constructs and checks finite proof-carrying reference paths under explicit finite-class, checker, compiler, artifact, reality, and tractability hypotheses.

Finite RCP reference class
The bounded class $\mathsf{RCPRefClass}_{n,k,N}$ of finite state spaces, primitive-word spaces, witness libraries, certificate grammars, verifier schemas, uncertainty envelopes, goal objects, and budget objects used by the reference-entry theorem.

Proof-carrying successor packet
The packet $\mathsf{PCS}_t(\nu_t)$ containing the candidate, realized successor, recovery map, proof components for non-loss, recovery, successor verification, goal identity, trust, optional reality containment, optional tractability, and a replayable trace.

Trusted RCP checker
: The finite procedure $\mathsf{Check}_{\mathrm{RCP}}$ that validates proof-carrying successor packets. Its acceptance implies theorem obligations only under a checker soundness certificate; it is machine-checked only if $\mathsf{MechCert}_{\mathrm{RCP}}$ is supplied.

Reference certificate compiler
: The partial finite procedure $\mathsf{BuildRefSV}_{0:N}^{\mathrm{RCP}}$ that attempts to construct $\mathsf{RefSV}_{0:N}^{\mathrm{RCP}}$ from a finite system, audit data, witness library, and checker package.

Reference-entry theorem
: The Batch 13R theorem proving that successful compilation of $\mathsf{RefSV}_{0:N}^{\mathrm{RCP}}$ implies $\rho_0 \in \mathcal{K}_0^{\text{SV-seedlib},N}$.

Executable reference artifact
: The replayable object $\mathsf{Artifact}_{0:N}^{\mathrm{RCP}}$ containing source, build recipe, finite inputs, checker, proof scripts, logs, hashes, and output sufficient to audit a finite nontrivial reference trajectory.

Theorem-invocation boundary
: The set of required certified objects a concrete system must supply before citing the domain-relative robust reflective successor theorem for that system.

Instantiation readiness
: The post-theorem status in which the next frontier is finite reference instantiation, implementation, and empirical validation rather than another abstract theorem batch.

Canonical finite RCP reference witness
: The Batch 13R-A appended-module construction $\mathsf{RefSV}_{0:N}^{\mathrm{RCP,can}}$, defined for every $N \geq 2$, that gives an explicit non-noop multi-step proof-carrying finite trajectory with exact non-loss, exact recovery, strict ability expansion, zero goal drift, singleton reality containment, concrete checker soundness, and restricted finite tractability.

External canonical mechanization certificate
: The Batch 13R-B object $\mathsf{MechCert}_{0:N}^{\mathrm{RCP,can}}$, which records an external proof-assistant or proof-kernel check of the finite canonical core. In the current companion artifact package, this is instantiated by a Lean 4 certificate for the canonical finite RCP/RCLM witness, checker core, and RCLM-to-RCP refinement module. The certificate does not mechanize the full theorem stack or arbitrary learned-system entry.

Controlled executable reference system
: The Batch 13R-B object $\mathsf{CtrlRef}_{0:N}^{\mathrm{RCP}}$, containing finite state tables, update tables, proof-carrying packets, residual tables, goal/trust/reality/cost tables, checker, replay script, run log, hashes, and output for a controlled non-noop finite trajectory.

M3-Min
: The minimal learned-agent entry boundary section. It defines the finite certificate boundary a trained or learned system must satisfy before the existing RCP-II / Batch-13R theorem stack may be invoked for that system.

Learned-entry certificate
: The packet $\mathsf{LECert}^{\mathrm{RCP}}_{0:N}$, containing type, register/semantic, coverage, witness-library, builder-trace, proof-carrying packet, explicit residual, goal, trust, reality, tractability, and replay certificates for a learned system over a finite horizon.

Learned-entry audit
: The procedure $\mathsf{LearnedEntryAudit}^{\mathrm{RCP}}_{0:N}$, which returns either a full learned-entry certificate or $\bot$. Only a full returned certificate licenses learned-system seed-library entry.

Partial-pass report
: A controlled trained-system pilot report listing exactly which learned-entry certificate components passed, failed, were missing, or were not claimed. A partial pass is not seed-library entry.

Controlled replay certificate
: The certificate $\mathsf{CtrlReplayCert}^{\mathrm{RCP}}_{0:N}$ proving that the controlled artifact parses, replays, checks, and outputs the declared finite reference instance with matching hashes and costs.

Canonical RCP checker
: The deterministic finite checker $\mathsf{Check}^{\mathrm{can}}_{\mathrm{RCP}}$ that verifies diagonal density validity, append-only successor equations, partial-trace recovery, relative-entropy preservation, ability-set expansion, explicit residual nonpositivity, goal/trust/uncertainty records, successor persistence, and finite cost records for the canonical reference witness.

Concrete finite restricted-Scenario-3 witness
: The status achieved by Theorem 45.31 and Corollary 45.32: a finite, non-vacuous, non-noop, multi-step certified trajectory inside the declared finite reference class, not a universal arbitrary-system Scenario-3 theorem.

References

[1] Michael A. Nielsen and Isaac L. Chuang. *Quantum Computation and Quantum Information*. Cambridge University Press, 2010.

[2] Mark M. Wilde. *Quantum Information Theory*. Cambridge University Press, 2013.

[3] T. Baumgratz, M. Cramer, and M. B. Plenio. Quantifying coherence. *Physical Review Letters*, 113:140401, 2014.

[4] D. Petz. Sufficient subalgebras and the relative entropy of states of a von Neumann algebra. *Communications in Mathematical Physics*, 105:123–131, 1986.

[5] Omar Fawzi and Renato Renner. Quantum conditional mutual information and approximate Markov chains. *Communications in Mathematical Physics*, 340:575–611, 2015.

[6] Naftali Tishby, Fernando C. Pereira, and William Bialek. The information bottleneck method. In *Proceedings of the 37th Annual Allerton Conference on Communication, Control, and Computing*, 1999.

[7] Thomas M. Cover and Joy A. Thomas. *Elements of Information Theory*. Wiley, 2nd edition, 2006.

[8] Hassan K. Khalil. *Nonlinear Systems*. Prentice Hall, 3rd edition, 2002.

[9] Aaron D. Ames, Samuel Coogan, Magnus Egerstedt, Gennaro Notomista, Koushil Sreenath, and Paulo Tabuada. Control barrier functions: theory and applications. *European Control Conference*, 2019.

[10] Nate Soares, Benja Fallenstein, Eliezer Yudkowsky, and Stuart Armstrong. Corrigibility. In *AAAI Workshop on AI and Ethics*, 2015.

[11] Dario Amodei, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mane. Concrete problems in AI safety. arXiv:1606.06565, 2016.

[12] Stuart Russell. *Human Compatible: Artificial Intelligence and the Problem of Control*. Viking, 2019.

[13] Ashish Vaswani et al. Attention is all you need. *Advances in Neural Information Processing Systems*, 2017.